로컬비서 (LocalBiseo) v0.1.0 소스코드 출력물

- 대상: 직접 작성한 소스 파일 81개, 9,894줄
- 제외: 오픈소스 라이브러리, AI 모델, 글꼴, 자동 생성 파일, 테스트 코드
- 각 쪽 위에 파일 경로, 각 줄 앞에 줄 번호를 표시했다.

```rust
//! 화면 캡처와 클립보드 감시.

use std::path::PathBuf;
use std::time::{Duration, SystemTime, UNIX_EPOCH};

use tauri::{AppHandle, Emitter, Manager};

/// 주 모니터를 캡처해 임시 폴더에 PNG로 저장하고 경로를 돌려준다.
/// 우리 창이 찍히지 않도록 잠깐 숨겼다가 다시 띄운다.
pub fn capture_primary(app: &AppHandle) -> Result<String, String> {
    let window = app.get_webview_window("main");
    if let Some(w) = &window {
        let _ = w.hide();
    }
    std::thread::sleep(Duration::from_millis(250));
    let result = (|| {
        let monitors = xcap::Monitor::all().map_err(|e| format!("화면을 찾지 못했어요: {e}"))?;
        let monitor = monitors
            .iter()
            .find(|m| m.is_primary().unwrap_or(false))
            .or_else(|| monitors.first())
            .ok_or("화면을 찾지 못했어요.")?;
        let image = monitor.capture_image().map_err(|e| format!("화면을 캡처하지 못했어요: {e}"))?;
        let stamp = SystemTime::now().duration_since(UNIX_EPOCH).map(|d| d.as_millis()).unwrap_or(0);
        let path: PathBuf = std::env::temp_dir().join(format!("localbiseo-capture-{stamp}.png"));
        image.save(&path).map_err(|e| format!("캡처를 저장하지 못했어요: {e}"))?;
        Ok(path.to_string_lossy().into_owned())
    })();
    if let Some(w) = &window {
        let _ = w.show();
        let _ = w.set_focus();
    }
    result
}

/// 클립보드 글자가 바뀌면 화면에 `clipboard-text` 이벤트로 알린다.
/// 저장할지 말지(설정, 민감정보 처리)는 엔진이 정한다.
pub fn watch_clipboard(app: AppHandle) {
    std::thread::spawn(move || {
        let mut board = match arboard::Clipboard::new() {
            Ok(b) => b,
            Err(_) => return, // 클립보드를 쓸 수 없는 환경
        };
        let mut last = board.get_text().unwrap_or_default();
        loop {
            std::thread::sleep(Duration::from_millis(800));
            if let Ok(text) = board.get_text() {
                if text != last {
                    last = text.clone();
                    if !text.trim().is_empty() {
                        let _ = app.emit("clipboard-text", text);
                    }
                }
            }
        }
    });
}
```

```rust
//! Python 엔진을 자식 프로세스로 띄우고 주소 · 토큰을 화면에 알려준다.
//!
//! 엔진은 준비되면 표준출력에 `LOCALBISEO_READY <port>`를 찍는다.
//! 표준입력을 열어 둔 채로 두면, 앱이 죽어도 파이프가 닫혀 엔진이 같이 끝난다.

use std::io::{BufRead, BufReader};
use std::path::PathBuf;
use std::process::{Child, Command, Stdio};
use std::sync::{mpsc, Arc, Condvar, Mutex};
use std::time::Duration;

use serde::Serialize;
use tauri::{AppHandle, Manager};

#[derive(Clone, Serialize)]
pub struct EngineInfo {
    pub url: String,
    pub token: String,
}

#[derive(Default)]
pub struct EngineState {
    ready: Mutex<Option<Result<EngineInfo, String>>>,
    signal: Condvar,
    child: Mutex<Option<Child>>,
}

pub type SharedEngine = Arc<EngineState>;

impl EngineState {
    pub fn wait(&self, timeout: Duration) -> Result<EngineInfo, String> {
        let guard = self.ready.lock().unwrap();
        let (guard, _) = self
            .signal
            .wait_timeout_while(guard, timeout, |r| r.is_none())
            .unwrap();
        guard
            .clone()
            .unwrap_or_else(|| Err("엔진이 시간 안에 준비되지 않았어요.".into()))
    }

    fn finish(&self, result: Result<EngineInfo, String>) {
        *self.ready.lock().unwrap() = Some(result);
        self.signal.notify_all();
    }

    pub fn kill(&self) {
        if let Some(mut child) = self.child.lock().unwrap().take() {
            let _ = child.kill();
            let _ = child.wait();
        }
    }
}

/// 엔진 실행 명령. 배포판은 리소스의 `engine/localbiseo-engine`, 개발 중에는 `python -m localbiseo`.
fn engine_command(app: &AppHandle) -> Command {
    let exe_name = if cfg!(windows) { "localbiseo-engine.exe" } else { "localbiseo-engine" };
    let mut candidates: Vec<PathBuf> = Vec::new();
    if let Ok(res) = app.path().resource_dir() {
        candidates.push(res.join("engine").join(exe_name));
    }
    if let Ok(exe) = std::env::current_exe() {
```

```
        if let Some(dir) = exe.parent() {
            candidates.push(dir.join(exe_name));
        }
    }
    if let Some(bin) = candidates.into_iter().find(|p| p.exists()) {
        return Command::new(bin);
    }
    // 개발 모드: 저장소의 engine 폴더에서 파이썬으로 실행
    let python = std::env::var("LOCALBISEO_PYTHON").unwrap_or_else(|_| {
        if cfg!(windows) { "python".into() } else { "python3".into() }
    });
    let engine_dir = PathBuf::from(env!("CARGO_MANIFEST_DIR")).join("../../engine");
    let mut cmd = Command::new(python);
    cmd.current_dir(engine_dir).args(["-m", "localbiseo"]);
    cmd
}

pub fn start(app: &AppHandle, state: SharedEngine) {
    let token = uuid::Uuid::new_v4().simple().to_string();
    let mut cmd = engine_command(app);
    // 배포판에 같이 넣은 ffmpeg · tesseract 위치 (tauri.release.conf.json의 resources)
    if let Ok(res) = app.path().resource_dir() {
        cmd.env("LOCALBISEO_RESOURCES", res.join("tools"));
    }
    cmd.args(["--port", "0", "--token", &token, "--watch-stdin"])
        .stdin(Stdio::piped())
        .stdout(Stdio::piped())
        .stderr(Stdio::inherit());
    #[cfg(windows)]
    {
        use std::os::windows::process::CommandExt;
        const CREATE_NO_WINDOW: u32 = 0x0800_0000;
        cmd.creation_flags(CREATE_NO_WINDOW);
    }
    let mut child = match cmd.spawn() {
        Ok(c) => c,
        Err(e) => {
            state.finish(Err(format!("엔진을 실행하지 못했어요: {e}")));
            return;
        }
    };
    let stdout = child.stdout.take();
    *state.child.lock().unwrap() = Some(child);

    let (tx, rx) = mpsc::channel::<u16>();
    if let Some(out) = stdout {
        std::thread::spawn(move || {
            let mut sent = false;
            for line in BufReader::new(out).lines().map_while(Result::ok) {
                if !sent {
                    if let Some(port) = line.strip_prefix("LOCALBISEO_READY ") {
                        if let Ok(p) = port.trim().parse() {
                            let _ = tx.send(p);
                            sent = true;
                        }
                    }
                }
                // 이후 출력은 그냥 흘려보낸다 (파이프가 차서 엔진이 멈추지 않도록).
            }
        });
    }
    std::thread::spawn(move || {
```

```
125         // 처음 실행할 때는 백신 검사 등으로 느릴 수 있어 넉넉히 기다린다.
126         let result = match rx.recv_timeout(Duration::from_secs(90)) {
127             Ok(port) => Ok(EngineInfo { url: format!("http://127.0.0.1:{port}"), token }),
128             Err(_) => Err("엔진이 시작되지 않았어요. 앱을 다시 실행해 주세요.".into()),
129         };
130         state.finish(result);
131     });
132 }
```

```rust
mod desktop;
mod engine;

use std::sync::Arc;
use std::time::Duration;

use tauri::menu::{Menu, MenuItem};
use tauri::tray::{MouseButton, MouseButtonState, TrayIconBuilder, TrayIconEvent};
use tauri::{AppHandle, Emitter, Manager, RunEvent, WindowEvent};
use tauri_plugin_global_shortcut::{Code, GlobalShortcutExt, Modifiers, Shortcut, ShortcutState};

use engine::{EngineInfo, EngineState, SharedEngine};

#[tauri::command]
async fn engine_info(state: tauri::State<'_, SharedEngine>) -> Result<EngineInfo, String> {
    let engine = state.inner().clone();
    tauri::async_runtime::spawn_blocking(move || engine.wait(Duration::from_secs(120)))
        .await
        .map_err(|e| e.to_string())?
}

#[tauri::command]
async fn capture_screen(app: AppHandle) -> Result<String, String> {
    tauri::async_runtime::spawn_blocking(move || desktop::capture_primary(&app))
        .await
        .map_err(|e| e.to_string())?
}

fn show_main(app: &AppHandle) {
    if let Some(w) = app.get_webview_window("main") {
        let _ = w.unminimize();
        let _ = w.show();
        let _ = w.set_focus();
    }
}

fn setup_shortcuts(app: &AppHandle) -> tauri::Result<()> {
    let palette = Shortcut::new(Some(Modifiers::CONTROL | Modifiers::SHIFT), Code::Space);
    let ocr = Shortcut::new(Some(Modifiers::CONTROL | Modifiers::SHIFT), Code::KeyO);
    app.plugin(
        tauri_plugin_global_shortcut::Builder::new()
            .with_handler(move |app, shortcut, event| {
                if event.state() != ShortcutState::Pressed {
                    return;
                }
                if shortcut == &palette {
                    show_main(app);
                    let _ = app.emit("open-palette", ());
                } else if shortcut == &ocr {
                    let app = app.clone();
                    std::thread::spawn(move || {
                        if let Ok(path) = desktop::capture_primary(&app) {
                            let _ = app.emit("screen-captured", path);
                        }
                    });
                }
            })
            .build(),
    )?;
    // 다른 프로그램이 이미 쓰는 단축키면 등록이 실패할 수 있다. 앱은 그대로 동작한다.
    let _ = app.global_shortcut().register(palette);
    let _ = app.global_shortcut().register(ocr);
```

```
    Ok(())
}

fn setup_tray(app: &AppHandle) -> tauri::Result<()> {
    let open = MenuItem::with_id(app, "open", "로컬비서 열기", true, None::<&str>)?;
    let capture = MenuItem::with_id(app, "capture", "화면 글자 읽기", true, None::<&str>)?;
    let quit = MenuItem::with_id(app, "quit", "끝내기", true, None::<&str>)?;
    let menu = Menu::with_items(app, &[&open, &capture, &quit])?;
    let mut tray = TrayIconBuilder::with_id("main")
        .tooltip("로컬비서")
        .menu(&menu)
        .show_menu_on_left_click(false)
        .on_menu_event(|app, event| match event.id().as_ref() {
            "open" => show_main(app),
            "capture" => {
                let app = app.clone();
                std::thread::spawn(move || {
                    if let Ok(path) = desktop::capture_primary(&app) {
                        let _ = app.emit("screen-captured", path);
                    }
                });
            }
            "quit" => {
                app.state::<SharedEngine>().kill();
                app.exit(0);
            }
            _ => {}
        })
        .on_tray_icon_event(|tray, event| {
            if let TrayIconEvent::Click { button: MouseButton::Left, button_state: MouseButtonState::Up, .. } = event {
                show_main(tray.app_handle());
            }
        });
    if let Some(icon) = app.default_window_icon() {
        tray = tray.icon(icon.clone());
    }
    tray.build(app)?;
    Ok(())
}

#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    let engine: SharedEngine = Arc::new(EngineState::default());
    let app = tauri::Builder::default()
        .plugin(tauri_plugin_dialog::init())
        .plugin(tauri_plugin_opener::init())
        .manage(engine.clone())
        .invoke_handler(tauri::generate_handler![engine_info, capture_screen])
        .setup(move |app| {
            let handle = app.handle().clone();
            engine::start(&handle, engine.clone());
            setup_shortcuts(&handle)?;
            setup_tray(&handle)?;
            desktop::watch_clipboard(handle);
            Ok(())
        })
        .on_window_event(|window, event| {
            // 창을 닫아도 트레이에 남아 클립보드 기록과 단축키가 계속 동작한다.
            if let WindowEvent::CloseRequested { api, .. } = event {
                if window.label() == "main" {
                    api.prevent_close();
                    let _ = window.hide();
```

```
                }
            }
        })
        .build(tauri::generate_context!())
        .expect("로컬비서를 시작하지 못했어요");

    app.run(|app, event| {
        if let RunEvent::Exit = event {
            app.state::<SharedEngine>().kill();
        }
    });
}
```

```
1  // 릴리스 빌드에서 Windows 콘솔 창이 뜨지 않게 한다.
2  #![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]
3
4  fn main() {
5      localbiseo_lib::run()
6  }
```

```tsx
import { Command, Loader2, Search as SearchIcon } from "lucide-react";
import { useCallback, useEffect, useMemo, useRef, useState, type ComponentType } from "react";
import { SystemProvider, useSystem } from "./components/system";
import { ToastProvider, useToast } from "./components/ui";
import { api, errorText, initEngine } from "./lib/api";
import { runFocusWatcher } from "./lib/focusTimer";
import { installDropListener, listen } from "./lib/platform";
import { ALL_ITEMS, NAV } from "./nav";
import Actions from "./pages/Actions";
import Clipboard from "./pages/Clipboard";
import Dedupe from "./pages/Dedupe";
import Focus from "./pages/Focus";
import Home from "./pages/Home";
import Mask from "./pages/Mask";
import Media from "./pages/Media";
import Notes from "./pages/Notes";
import Ocr from "./pages/Ocr";
import Papers from "./pages/Papers";
import Pdf from "./pages/Pdf";
import Rename from "./pages/Rename";
import Schedule from "./pages/Schedule";
import Search from "./pages/Search";
import Settings from "./pages/Settings";
import Transcribe from "./pages/Transcribe";
import Translate from "./pages/Translate";
import Writing from "./pages/Writing";

const PAGES: Record<string, ComponentType> = {
  home: Home, transcribe: Transcribe, mask: Mask, media: Media, actions: Actions, schedule: Schedule, notes: Notes,
  clipboard: Clipboard, ocr: Ocr, papers: Papers, translate: Translate, writing: Writing, rename: Rename, dedupe: Dedupe,
  pdf: Pdf, focus: Focus, search: Search, settings: Settings,
};

function pageFromHash(): string {
  const key = window.location.hash.replace(/^#\/?/, "").split("?")[0];
  return PAGES[key] ? key : "home";
}

export default function App() {
  const [ready, setReady] = useState(false);
  const [bootError, setBootError] = useState<string | null>(null);
  const [page, setPage] = useState(pageFromHash);

  useEffect(() => {
    initEngine()
      .then(() => setReady(true))
      .catch((e) => setBootError(errorText(e)));
    const onHash = () => setPage(pageFromHash());
    window.addEventListener("hashchange", onHash);
    return () => window.removeEventListener("hashchange", onHash);
  }, []);

  const navigate = useCallback((key: string) => {
    window.location.hash = `/${key}`;
    setPage(key);
  }, []);

  if (!ready) {
    return (
      <div className="splash">
        <div className="logo">로컬비서</div>
        {bootError ? <p className="error-text">{bootError}</p> : <p className="muted"><Loader2 size={16} className="spin" /> 엔진을 깨우는 중…</p>}
```

```
      </div>
    );
  }
  return (
    <ToastProvider>
      <SystemProvider navigate={navigate}>
        <Shell page={page} navigate={navigate} />
      </SystemProvider>
    </ToastProvider>
  );
}

function Shell({ page, navigate }: { page: string; navigate: (k: string) => void }) {
  const [palette, setPalette] = useState(false);
  const [dropping, setDropping] = useState(false);
  const { status } = useSystem();
  const toast = useToast();
  const Current = PAGES[page];

  useEffect(() => {
    runFocusWatcher((r) => toast(r.kind === "focus" ? `${r.minutes}분 집중 끝! 잠깐 쉬어요.` : "쉬는 시간 끝! 다시 집중해 볼까요?"));
  }, [toast]);

  useEffect(() => {
    const onKey = (e: KeyboardEvent) => {
      if ((e.ctrlKey || e.metaKey) && e.key.toLowerCase() === "k") {
        e.preventDefault();
        setPalette((v) => !v);
      }
      if (e.key === "Escape") setPalette(false);
    };
    window.addEventListener("keydown", onKey);
    installDropListener(setDropping);
    const offs = [
      listen("open-palette", () => setPalette(true)),
      listen<string>("screen-captured", (path) => {
        sessionStorage.setItem("lb-captured", path);
        navigate("ocr");
        window.dispatchEvent(new CustomEvent("lb-captured", { detail: path }));
      }),
      // 앱 셸이 복사 내용을 알려주면 기록한다 (설정에서 끌 수 있다).
      listen<string>("clipboard-text", (text) => {
        api.post("/clipboard", { text }).catch(() => {});
        window.dispatchEvent(new CustomEvent("lb-clipboard"));
      }),
    ];
    return () => {
      window.removeEventListener("keydown", onKey);
      offs.forEach((p) => p.then((off) => off()));
    };
  }, [navigate]);

  const lic = status?.license;
  return (
    <div className="shell">
      <aside className="sidebar">
        <div className="brand">
          <span className="brand-mark">비</span>
          <span>로컬비서</span>
        </div>
        <button className="palette-trigger" onClick={() => setPalette(true)}>
          <SearchIcon size={15} />
```

```
          <span>찾기 · 이동</span>
          <kbd>Ctrl K</kbd>
        </button>
        <nav>
          {NAV.map((g, gi) => (
            <div key={gi} className="nav-group">
              {g.title && <div className="nav-title">{g.title}</div>}
              {g.items.map((it) => (
                <button key={it.key} className={`nav-item ${page === it.key ? "active" : ""}`} onClick={() => navigate(it.key)}>
                  <it.icon size={17} />
                  <span>{it.label}</span>
                </button>
              ))}
            </div>
          ))}
        </nav>
        <div className="sidebar-foot">
          {lic?.status === "licensed" && <span className="muted small">정품 · {lic.holder ¦¦ lic.id}</span>}
          {lic?.status === "trial" && (
            <button className="linklike small" onClick={() => navigate("settings")}>
              체험판 {lic.days_left}일 남음
            </button>
          )}
          {lic?.status === "expired" && (
            <button className="linklike small danger" onClick={() => navigate("settings")}>
              체험 기간 끝 · 키 입력
            </button>
          )}
          <span className="muted small">v{status?.version}</span>
        </div>
      </aside>
      <main className="main">
        <Current key={page} />
      </main>
      {palette && <Palette onClose={() => setPalette(false)} navigate={navigate} />}
      {dropping && <div className="drop-overlay">여기에 놓으세요</div>}
    </div>
  );
}

type SearchHit = { kind: string; kind_label: string; ref: string; title: string; snippet: string };
const KIND_PAGE: Record<string, string> = { note: "notes", todo: "notes", clip: "clipboard", transcript: "transcribe", file: "search", contact:

function Palette({ onClose, navigate }: { onClose: () => void; navigate: (k: string) => void }) {
  const [q, setQ] = useState("");
  const [hits, setHits] = useState<SearchHit[]>([]);
  const [sel, setSel] = useState(0);
  const input = useRef<HTMLInputElement>(null);
  useEffect(() => input.current?.focus(), []);
  const pages = useMemo(() => {
    const s = q.trim().toLowerCase();
    return ALL_ITEMS.filter((it) => !s ¦¦ `${it.label} ${it.desc} ${it.keywords ?? ""}`.toLowerCase().includes(s));
  }, [q]);
  useEffect(() => {
    if (q.trim().length < 2) {
      setHits([]);
      return;
    }
    const t = setTimeout(() => {
      api
        .get<{ results: SearchHit[] }>("/search", { q, limit: 8 })
        .then((r) => setHits(r.results))
```

```
        .catch(() => setHits([]));
    }, 180);
    return () => clearTimeout(t);
  }, [q]);
  const rows = [
    ...pages.map((p) => ({ id: `p-${p.key}`, label: p.label, sub: p.desc, icon: p.icon, go: () => navigate(p.key) })),
    ...hits.map((h) => ({
      id: `h-${h.kind}-${h.ref}`,
      label: h.title,
      sub: `${h.kind_label} · ${h.snippet}`,
      icon: SearchIcon,
      go: () => {
        sessionStorage.setItem("lb-open", JSON.stringify(h));
        navigate(KIND_PAGE[h.kind] ?? "search");
      },
    })),
  ];
  const choose = (i: number) => {
    rows[i]?.go();
    onClose();
  };
  return (
    <div className="palette-backdrop" onMouseDown={onClose}>
      <div className="palette" onMouseDown={(e) => e.stopPropagation()} role="dialog" aria-label="찾기">
        <div className="palette-input">
          <Command size={16} />
          <input
            ref={input}
            value={q}
            placeholder="기능 이름이나 찾을 내용을 입력하세요"
            onChange={(e) => {
              setQ(e.target.value);
              setSel(0);
            }}
            onKeyDown={(e) => {
              if (e.key === "ArrowDown") (e.preventDefault(), setSel((s) => Math.min(rows.length - 1, s + 1)));
              if (e.key === "ArrowUp") (e.preventDefault(), setSel((s) => Math.max(0, s - 1)));
              if (e.key === "Enter") choose(sel);
            }}
          />
        </div>
        <div className="palette-list">
          {rows.length === 0 && <div className="muted pad">결과가 없어요</div>}
          {rows.map((r, i) => (
            <button key={r.id} className={`palette-row ${i === sel ? "sel" : ""}`} onMouseEnter={() => setSel(i)} onClick={() => choose(i)}>
              <r.icon size={16} />
              <span className="palette-label">{r.label}</span>
              <span className="muted truncate small">{r.sub}</span>
            </button>
          ))}
        </div>
      </div>
    </div>
  );
}
```

```tsx
import { createContext, useContext, type ReactNode } from "react";
import { api } from "../lib/api";
import { useData } from "../lib/hooks";
import { Notice } from "./ui";

export type ComponentStatus = { key: string; label: string; available: boolean; used_by: string; extra: string };
export type License = { status: "trial" | "licensed" | "expired"; days_left?: number; trial_ends?: string; holder?: string; id?: string };
export type SystemStatus = {
  version: string;
  os: string;
  data_dir: string;
  components: ComponentStatus[];
  ffmpeg: boolean;
  ocr_backend: string | null;
  korean_font: string | null;
  llm_model: string | null;
  license: License;
};

type Ctx = {
  status: SystemStatus | null;
  reload: () => void;
  navigate: (page: string) => void;
  has: (key: string) => boolean;
};

const SystemCtx = createContext<Ctx>({ status: null, reload: () => {}, navigate: () => {}, has: () => false });

export function SystemProvider({ children, navigate }: { children: ReactNode; navigate: (p: string) => void }) {
  const { data, reload } = useData(() => api.get<SystemStatus>("/system/status"));
  const has = (key: string) => (key === "ffmpeg" ? !!data?.ffmpeg : !!data?.components.find((c) => c.key === key)?.available);
  return <SystemCtx.Provider value={{ status: data, reload, navigate, has }}>{children}</SystemCtx.Provider>;
}

export const useSystem = () => useContext(SystemCtx);

/** 선택 구성요소가 없을 때 안내. */
export function NeedsComponent({ keys, children }: { keys: string[]; children?: ReactNode }) {
  const { status, has, navigate } = useSystem();
  if (!status) return null;
  const missing = keys.filter((k) => !has(k));
  if (!missing.length) return null;
  const labels = missing.map((k) => (k === "ffmpeg" ? "ffmpeg" : status.components.find((c) => c.key === k)?.label ?? k));
  return (
    <Notice tone="warn">
      <span>
        <b>{labels.join(", ")}</b> 구성요소가 아직 없어요. {children}
        <button className="linklike" onClick={() => navigate("settings")}>
          설정 &gt; 구성요소
        </button>
        에서 확인해 주세요.
      </span>
    </Notice>
  );
}
```

```tsx
import { CircleAlert, FolderOpen, Copy, Loader2, Plus, Upload, X } from "lucide-react";
import { createContext, useCallback, useContext, useEffect, useRef, useState, type ReactNode } from "react";
import { api, type Job } from "../lib/api";
import { baseName } from "../lib/format";
import { copyText, isTauri, onFileDrop, openPath, pickFiles, reveal, type Filter } from "../lib/platform";

export function Page({ title, desc, actions, children }: { title: string; desc?: string; actions?: ReactNode; children: ReactNode }) {
  return (
    <div className="page">
      <header className="page-head">
        <div>
          <h1>{title}</h1>
          {desc && <p className="muted">{desc}</p>}
        </div>
        {actions && <div className="row">{actions}</div>}
      </header>
      {children}
    </div>
  );
}

export function Card({ title, right, children, className = "" }: { title?: ReactNode; right?: ReactNode; children: ReactNode; className?: strin
  return (
    <section className={`card ${className}`}>
      {(title || right) && (
        <div className="card-head">
          {title && <h2>{title}</h2>}
          {right}
        </div>
      )}
      {children}
    </section>
  );
}

type BtnProps = React.ButtonHTMLAttributes<HTMLButtonElement> & { variant?: "primary" | "ghost" | "danger" | "default"; busy?: boolean; icon?:

export function Button({ variant = "default", busy, icon, small, children, className = "", disabled, ...rest }: BtnProps) {
  return (
    <button className={`btn btn-${variant} ${small ? "btn-sm" : ""} ${className}`} disabled={disabled || busy} {...rest}>
      {busy ? <Loader2 size={16} className="spin" /> : icon}
      {children && <span>{children}</span>}
    </button>
  );
}

export function Field({ label, hint, children, inline }: { label: string; hint?: ReactNode; children: ReactNode; inline?: boolean }) {
  return (
    <label className={`field ${inline ? "field-inline" : ""}`}>
      <span className="field-label">{label}</span>
      {children}
      {hint && <span className="field-hint">{hint}</span>}
    </label>
  );
}

export function Toggle({ checked, onChange, label }: { checked: boolean; onChange: (v: boolean) => void; label: string }) {
  return (
    <label className="toggle">
      <input type="checkbox" checked={checked} onChange={(e) => onChange(e.target.checked)} />
      <span className="toggle-track" aria-hidden />
      <span>{label}</span>
```

```
    </label>
  );
}

export function Tabs<K extends string>({ tabs, value, onChange }: { tabs: { key: K; label: string }[]; value: K; onChange: (k: K) => void }) {
  return (
    <div className="tabs" role="tablist">
      {tabs.map((t) => (
        <button key={t.key} role="tab" aria-selected={value === t.key} className={value === t.key ? "active" : ""} onClick={() => onChange(t.ke
          {t.label}
        </button>
      ))}
    </div>
  );
}

export function ErrorBox({ error }: { error: string | null | undefined }) {
  if (!error) return null;
  return (
    <div className="alert alert-error" role="alert">
      <CircleAlert size={16} />
      <span>{error}</span>
    </div>
  );
}

export function Notice({ children, tone = "info" }: { children: ReactNode; tone?: "info" | "warn" | "ok" }) {
  return <div className={`alert alert-${tone}`}>{children}</div>;
}

export function Empty({ icon, children }: { icon?: ReactNode; children: ReactNode }) {
  return (
    <div className="empty">
      {icon}
      <div>{children}</div>
    </div>
  );
}

export function Badge({ children, tone = "" }: { children: ReactNode; tone?: string }) {
  return <span className={`badge ${tone ? `badge-${tone}` : ""}`}>{children}</span>;
}

export function JobBar({ job, error, onCancel }: { job: Job | null; error?: string | null; onCancel?: () => void }) {
  if (error && (!job || job.status !== "running")) return <ErrorBox error={error} />;
  if (!job || job.status === "done") return null;
  if (job.status === "cancelled") return <Notice tone="warn">작업을 취소했어요.</Notice>;
  return (
    <div className="jobbar">
      <div className="jobbar-top">
        <span>
          <Loader2 size={14} className="spin" /> {job.message || job.title}
        </span>
        <span className="muted">{Math.round(job.progress * 100)}%</span>
        {onCancel && (
          <Button small variant="ghost" onClick={onCancel}>
            취소
          </Button>
        )}
      </div>
      <div className="progress">
        <div style={{ width: `${Math.max(3, job.progress * 100)}%` }} />
```

```
      </div>
    </div>
  );
}

/** 엔진이 만든 이미지 파일의 작은 미리보기. */
export function Thumb({ path, size = 220 }: { path: string; size?: number }) {
  const [src, setSrc] = useState<string | null>(null);
  useEffect(() => {
    let alive = true;
    api
      .post<{ data_url: string }>("/ocr/preview", { path, max_size: size * 2 })
      .then((r) => alive && setSrc(r.data_url))
      .catch(() => {});
    return () => {
      alive = false;
    };
  }, [path, size]);
  return src ? <img className="thumb" src={src} alt={baseName(path)} style={{ maxWidth: size, maxHeight: size }} /> : <div className="thumb thu
}

export function PathLink({ path, label }: { path: string; label?: string }) {
  return (
    <span className="pathlink">
      <button className="linklike" title={path} onClick={() => openPath(path)}>
        {label ?? baseName(path)}
      </button>
      <button className="iconbtn" title={isTauri ? "폴더에서 보기" : "경로 복사"} onClick={() => reveal(path)}>
        {isTauri ? <FolderOpen size={14} /> : <Copy size={14} />}
      </button>
    </span>
  );
}

/** 파일/폴더 고르기 + 끌어다 놓기. */
export function PathPicker({
  value,
  onChange,
  multiple,
  directory,
  filters,
  placeholder,
}: {
  value: string[];
  onChange: (v: string[]) => void;
  multiple?: boolean;
  directory?: boolean;
  filters?: Filter[];
  placeholder?: string;
}) {
  const ref = useRef(value);
  ref.current = value;
  useEffect(
    () =>
      onFileDrop((paths) => {
        const next = multiple ? Array.from(new Set([...ref.current, ...paths])) : paths.slice(0, 1);
        onChange(next);
      }),
    [multiple, onChange],
  );
  const add = async () => {
    const picked = await pickFiles({ multiple, directory, filters });
```

```
    if (!picked.length) return;
    onChange(multiple ? Array.from(new Set([...value, ...picked])) : picked.slice(0, 1));
  };
  return (
    <div className="picker">
      {value.length === 0 ? (
        <button className="dropzone" onClick={add}>
          <Upload size={22} />
          <span>{placeholder ?? (directory ? "폴더를 고르거나 끌어다 놓으세요" : "파일을 고르거나 여기로 끌어다 놓으세요")}</span>
        </button>
      ) : (
        <div className="picker-list">
          {value.map((p) => (
            <div key={p} className="picker-item" title={p}>
              <span className="truncate">{baseName(p)}</span>
              <span className="muted truncate small">{p}</span>
              <button className="iconbtn" onClick={() => onChange(value.filter((x) => x !== p))} aria-label="빼기">
                <X size={14} />
              </button>
            </div>
          ))}
          <div className="row">
            {(multiple || value.length === 0) && (
              <Button small icon={<Plus size={14} />} onClick={add}>
                더 넣기
              </Button>
            )}
            <Button small variant="ghost" onClick={() => onChange([])}>
              모두 빼기
            </Button>
          </div>
        </div>
      )}
    </div>
  );
}

export function CopyButton({ text, label = "복사" }: { text: string; label?: string }) {
  const [done, setDone] = useState(false);
  return (
    <Button
      small
      icon={<Copy size={14} />}
      onClick={async () => {
        await copyText(text);
        setDone(true);
        setTimeout(() => setDone(false), 1200);
      }}
    >
      {done ? "복사했어요" : label}
    </Button>
  );
}

// --- 토스트 ---
type Toast = { id: number; text: string; tone: "ok" | "error" };
const ToastCtx = createContext<(text: string, tone?: "ok" | "error") => void>(() => {});

export function ToastProvider({ children }: { children: ReactNode }) {
  const [items, setItems] = useState<Toast[]>([]);
  const push = useCallback((text: string, tone: "ok" | "error" = "ok") => {
    const id = Date.now() + Math.random();
```

```
    setItems((xs) => [...xs, { id, text, tone }]);
    setTimeout(() => setItems((xs) => xs.filter((x) => x.id !== id)), 3200);
  }, []);
  return (
    <ToastCtx.Provider value={push}>
      {children}
      <div className="toasts" aria-live="polite">
        {items.map((t) => (
          <div key={t.id} className={`toast toast-${t.tone}`}>
            {t.text}
          </div>
        ))}
      </div>
    </ToastCtx.Provider>
  );
}

export const useToast = () => useContext(ToastCtx);
```

```ts
import { engineInfo, isTauri } from "./platform";

let base = (import.meta.env.VITE_ENGINE_URL as string | undefined) ?? "http://127.0.0.1:8765";
let token = (import.meta.env.VITE_ENGINE_TOKEN as string | undefined) ?? "";

export class ApiError extends Error {
  status: number;
  data: Record<string, unknown>;
  constructor(message: string, status: number, data: Record<string, unknown> = {}) {
    super(message);
    this.status = status;
    this.data = data;
  }
  get component(): string | undefined {
    return this.data.component as string | undefined;
  }
}

export async function initEngine(): Promise<void> {
  if (isTauri) {
    const info = await engineInfo();
    base = info.url;
    token = info.token;
  }
  // 엔진이 막 떠서 아직 준비 중일 수 있으니 잠깐 기다린다.
  for (let i = 0; i < 100; i++) {
    try {
      const r = await fetch(`${base}/health`);
      if (r.ok) return;
    } catch {
      /* 아직 안 떴음 */
    }
    await new Promise((res) => setTimeout(res, 200));
  }
  throw new Error("엔진에 연결하지 못했어요. 앱을 다시 실행해 주세요.");
}

async function request<T>(method: string, path: string, body?: unknown): Promise<T> {
  let res: Response;
  try {
    res = await fetch(`${base}${path}`, {
      method,
      headers: { "Content-Type": "application/json", "X-Biseo-Token": token },
      body: body === undefined ? undefined : JSON.stringify(body),
    });
  } catch {
    throw new ApiError("엔진과 연결이 끊겼어요.", 0);
  }
  const text = await res.text();
  const data = text ? JSON.parse(text) : null;
  if (!res.ok) {
    throw new ApiError((data && data.error) || `오류 (${res.status})`, res.status, data || {});
  }
  return data as T;
}

export const api = {
  get: <T,>(path: string, params?: Record<string, string | number | undefined>) => {
    const q = params
      ? "?" +
        Object.entries(params)
          .filter(([, v]) => v !== undefined && v !== "")
```

```
          .map(([k, v]) => `${encodeURIComponent(k)}=${encodeURIComponent(String(v))}`)
          .join("&")
      : "";
    return request<T>("GET", path + q);
  },
  post: <T,>(path: string, body: unknown = {}) => request<T>("POST", path, body),
  put: <T,>(path: string, body: unknown = {}) => request<T>("PUT", path, body),
  patch: <T,>(path: string, body: unknown = {}) => request<T>("PATCH", path, body),
  del: <T,>(path: string) => request<T>("DELETE", path),
};

export type Job<R = any> = {
  id: string;
  kind: string;
  title: string;
  status: "queued" | "running" | "done" | "error" | "cancelled";
  progress: number;
  message: string;
  result: R | null;
  error: string | null;
};

export type JobRef = { job_id: string; job: Job };

export function errorText(e: unknown): string {
  if (e instanceof Error) return e.message;
  return String(e);
}
```

```ts
// 집중 타이머 상태. 페이지를 옮겨도 계속 돌도록 화면 밖(모듈)에서 관리한다.
import { api } from "./api";

export type Running = { id: number; kind: "focus" | "break"; label: string; endsAt: number; minutes: number };
const KEY = "lb-focus-running";
const listeners = new Set<() => void>();

function read(): Running | null {
  try {
    return JSON.parse(localStorage.getItem(KEY) || "null");
  } catch {
    return null;
  }
}

function write(r: Running | null) {
  try {
    if (r) localStorage.setItem(KEY, JSON.stringify(r));
    else localStorage.removeItem(KEY);
  } catch {
    /* 저장소를 못 쓰면 이번 실행 동안만 유지 */
  }
  memo = r;
  listeners.forEach((l) => l());
}

let memo: Running | null = read();

export const focusTimer = {
  get: () => memo,
  subscribe(l: () => void) {
    listeners.add(l);
    return () => listeners.delete(l);
  },
  async start(kind: "focus" | "break", minutes: number, label: string) {
    const s = await api.post<{ id: number }>("/focus/start", { kind, planned_minutes: minutes, label });
    write({ id: s.id, kind, label, minutes, endsAt: Date.now() + minutes * 60_000 });
  },
  async stop(completed: boolean) {
    const r = memo;
    write(null);
    if (r) await api.post(`/focus/${r.id}/stop`, { completed }).catch(() => {});
  },
};

function chime() {
  try {
    const ctx = new AudioContext();
    [0, 0.25, 0.5].forEach((t, i) => {
      const o = ctx.createOscillator();
      const g = ctx.createGain();
      o.frequency.value = [660, 880, 990][i];
      g.gain.setValueAtTime(0.0001, ctx.currentTime + t);
      g.gain.exponentialRampToValueAtTime(0.25, ctx.currentTime + t + 0.02);
      g.gain.exponentialRampToValueAtTime(0.0001, ctx.currentTime + t + 0.22);
      o.connect(g).connect(ctx.destination);
      o.start(ctx.currentTime + t);
      o.stop(ctx.currentTime + t + 0.25);
    });
  } catch {
    /* 소리를 못 내도 괜찮다 */
  }
```

```
63 }
64
65 let started = false;
66 export function runFocusWatcher(onFinish: (r: Running) => void) {
67   if (started) return;
68   started = true;
69   window.setInterval(() => {
70     const r = memo;
71     if (r && Date.now() >= r.endsAt) {
72       focusTimer.stop(true);
73       chime();
74       onFinish(r);
75     }
76     listeners.forEach((l) => l());
77   }, 1000);
78 }
```

```ts
export function bytes(n: number): string {
  if (n < 1024) return `${n} B`;
  const units = ["KB", "MB", "GB", "TB"];
  let v = n / 1024;
  let i = 0;
  while (v >= 1024 && i < units.length - 1) {
    v /= 1024;
    i++;
  }
  return `${v.toFixed(v < 10 ? 1 : 0)} ${units[i]}`;
}

export function won(n: number | null | undefined): string {
  return n == null ? "-" : `${n.toLocaleString("ko-KR")}원`;
}

export function clock(sec: number): string {
  const s = Math.max(0, Math.floor(sec));
  const h = Math.floor(s / 3600);
  const m = Math.floor((s % 3600) / 60);
  const r = s % 60;
  const mm = String(m).padStart(2, "0");
  const ss = String(r).padStart(2, "0");
  return h ? `${h}:${mm}:${ss}` : `${mm}:${ss}`;
}

export function dateTime(iso: string | null | undefined): string {
  if (!iso) return "";
  const d = new Date(iso);
  if (Number.isNaN(d.getTime())) return iso;
  const sameYear = d.getFullYear() === new Date().getFullYear();
  return d.toLocaleString("ko-KR", {
    year: sameYear ? undefined : "numeric",
    month: "short",
    day: "numeric",
    weekday: "short",
    hour: iso.length > 10 ? "2-digit" : undefined,
    minute: iso.length > 10 ? "2-digit" : undefined,
  });
}

export function baseName(path: string): string {
  return path.split(/[\\/]/).pop() || path;
}

export function dueLabel(due: string | null | undefined): { text: string; tone: "late" | "today" | "soon" | "" } {
  if (!due) return { text: "", tone: "" };
  const today = new Date();
  today.setHours(0, 0, 0, 0);
  const d = new Date(due.slice(0, 10) + "T00:00:00");
  const diff = Math.round((d.getTime() - today.getTime()) / 86400000);
  if (diff < 0) return { text: `${-diff}일 지남`, tone: "late" };
  if (diff === 0) return { text: "오늘", tone: "today" };
  if (diff === 1) return { text: "내일", tone: "soon" };
  if (diff < 7) return { text: `${diff}일 뒤`, tone: "soon" };
  return { text: dateTime(due), tone: "" };
}

/** 브라우저 현지 시간 기준 YYYY-MM-DD (toISOString은 UTC라 아침에 하루가 밀린다). */
export function localDate(d: Date = new Date()): string {
  const p = (n: number) => String(n).padStart(2, "0");
  return `${d.getFullYear()}-${p(d.getMonth() + 1)}-${p(d.getDate())}`;
```

```
63  }
```

```ts
import { useCallback, useEffect, useRef, useState } from "react";
import { api, errorText, type Job, type JobRef } from "./api";

/** 긴 작업(job)을 시작하고 끝날 때까지 상태를 따라간다. */
export function useJob<R = any>() {
  const [job, setJob] = useState<Job<R> | null>(null);
  const [error, setError] = useState<string | null>(null);
  const timer = useRef<number | null>(null);
  const alive = useRef(true);

  useEffect(() => {
    alive.current = true;
    return () => {
      alive.current = false;
      if (timer.current) window.clearTimeout(timer.current);
    };
  }, []);

  const poll = useCallback((id: string, resolve: (j: Job<R>) => void) => {
    const tick = async () => {
      try {
        const j = await api.get<Job<R>>(`/jobs/${id}`);
        if (!alive.current) return;
        setJob(j);
        if (j.status === "done" || j.status === "error" || j.status === "cancelled") {
          if (j.status === "error") setError(j.error);
          resolve(j);
          return;
        }
      } catch (e) {
        setError(errorText(e));
        return;
      }
      timer.current = window.setTimeout(tick, 400);
    };
    tick();
  }, []);

  const start = useCallback(
    (run: () => Promise<JobRef>) =>
      new Promise<Job<R> | null>(async (resolve) => {
        setError(null);
        setJob(null);
        try {
          const ref = await run();
          setJob(ref.job as Job<R>);
          poll(ref.job_id, resolve);
        } catch (e) {
          setError(errorText(e));
          resolve(null);
        }
      }),
    [poll],
  );

  const cancel = useCallback(async () => {
    if (job) await api.post(`/jobs/${job.id}/cancel`);
  }, [job]);

  const running = !!job && (job.status === "queued" || job.status === "running");
  return { job, error, running, start, cancel, reset: () => (setJob(null), setError(null)) };
}
```

```ts

/** 한 번 불러오고 reload()로 다시 불러오는 데이터. */
export function useData<T>(load: () => Promise<T>, deps: unknown[] = []) {
  const [data, setData] = useState<T | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [loading, setLoading] = useState(true);
  const reload = useCallback(async () => {
    setLoading(true);
    try {
      setData(await load());
      setError(null);
    } catch (e) {
      setError(errorText(e));
    } finally {
      setLoading(false);
    }
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, deps);
  useEffect(() => {
    reload();
  }, [reload]);
  return { data, error, loading, reload, setData };
}

/** 비동기 버튼 동작: 실행 중 표시와 오류 메시지. */
export function useAction() {
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const run = useCallback(async <T,>(fn: () => Promise<T>): Promise<T | undefined> => {
    setBusy(true);
    setError(null);
    try {
      return await fn();
    } catch (e) {
      setError(errorText(e));
      return undefined;
    } finally {
      setBusy(false);
    }
  }, []);
  return { busy, error, run, setError };
}
```

```ts
// 데스크톱(Tauri)과 브라우저 개발 모드 사이의 차이를 감춘다.
import { invoke } from "@tauri-apps/api/core";

export const isTauri = typeof window !== "undefined" && "__TAURI_INTERNALS__" in window;

export type Filter = { name: string; extensions: string[] };

function promptPaths(message: string, multiple: boolean): string[] {
  const raw = window.prompt(`${message}\n(브라우저 개발 모드: 전체 경로를 입력${multiple ? ", 여러 개는 ; 로 구분" : ""})`);
  if (!raw) return [];
  return raw
    .split(/[;\n]/)
    .map((s) => s.trim())
    .filter(Boolean);
}

export async function pickFiles(opts: { multiple?: boolean; filters?: Filter[]; directory?: boolean; title?: string } = {}): Promise<string[]>
  if (!isTauri) return promptPaths(opts.directory ? "폴더 경로" : "파일 경로", !!opts.multiple);
  const { open } = await import("@tauri-apps/plugin-dialog");
  const res = await open({ multiple: !!opts.multiple, directory: !!opts.directory, filters: opts.filters, title: opts.title });
  if (!res) return [];
  return Array.isArray(res) ? res : [res];
}

export async function pickSavePath(defaultPath: string, filters?: Filter[]): Promise<string | null> {
  if (!isTauri) return promptPaths(`저장할 경로 (기본: ${defaultPath})`, false)[0] ?? null;
  const { save } = await import("@tauri-apps/plugin-dialog");
  return save({ defaultPath, filters });
}

export async function reveal(path: string): Promise<void> {
  if (!isTauri) {
    await copyText(path);
    return;
  }
  const { revealItemInDir } = await import("@tauri-apps/plugin-opener");
  await revealItemInDir(path);
}

export async function openPath(path: string): Promise<void> {
  if (!isTauri) {
    await copyText(path);
    return;
  }
  const { openPath: op } = await import("@tauri-apps/plugin-opener");
  await op(path);
}

export async function copyText(text: string): Promise<void> {
  try {
    await navigator.clipboard.writeText(text);
  } catch {
    const ta = document.createElement("textarea");
    ta.value = text;
    document.body.appendChild(ta);
    ta.select();
    document.execCommand("copy");
    ta.remove();
  }
}

export async function captureScreen(): Promise<string> {
```

```ts
  if (!isTauri) throw new Error("화면 캡처는 데스크톱 앱에서만 돼요. 이미지 파일을 골라 주세요.");
  return invoke<string>("capture_screen");
}

export async function engineInfo(): Promise<{ url: string; token: string }> {
  return invoke("engine_info");
}

type DropHandler = (paths: string[]) => void;
const dropHandlers: DropHandler[] = [];

/** 창에 파일을 끌어다 놓으면 가장 최근에 등록한 받는 곳으로 전달한다. */
export function onFileDrop(handler: DropHandler): () => void {
  dropHandlers.push(handler);
  return () => {
    const i = dropHandlers.lastIndexOf(handler);
    if (i >= 0) dropHandlers.splice(i, 1);
  };
}

export async function installDropListener(onHover: (hovering: boolean) => void): Promise<void> {
  if (!isTauri) return;
  const { getCurrentWebview } = await import("@tauri-apps/api/webview");
  await getCurrentWebview().onDragDropEvent((event) => {
    const p = event.payload;
    if (p.type === "enter" || p.type === "over") onHover(true);
    else if (p.type === "leave") onHover(false);
    else if (p.type === "drop") {
      onHover(false);
      const h = dropHandlers[dropHandlers.length - 1];
      if (h && p.paths.length) h(p.paths);
    }
  });
}

export async function listen<T>(event: string, cb: (payload: T) => void): Promise<() => void> {
  if (!isTauri) return () => {};
  const { listen: l } = await import("@tauri-apps/api/event");
  return l<T>(event, (e) => cb(e.payload));
}
```

로컬비서 (LocalBiseo) v0.1.0 소스코드 — app/src/main.tsx

```tsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App";
import "./styles.css";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

```ts
import {
  CalendarPlus, ClipboardList, Copy, FilePen, FileText, House, Images, Languages, ListChecks, Mic, NotebookPen, Receipt,
  ScanText, Search, Settings, ShieldCheck, SpellCheck, Timer, type LucideIcon,
} from "lucide-react";

export type NavItem = { key: string; label: string; icon: LucideIcon; desc: string; keywords?: string };
export type NavGroup = { title: string | null; items: NavItem[] };

export const NAV: NavGroup[] = [
  { title: null, items: [{ key: "home", label: "홈", icon: House, desc: "오늘 할 일과 최근 기록" }] },
  {
    title: "핵심 기능",
    items: [
      { key: "transcribe", label: "회의록", icon: Mic, desc: "녹음을 글로 바꾸고 요약 · 할 일 · 일정까지", keywords: "녹음 받아쓰기 STT 강의 요약" },
      { key: "mask", label: "개인정보 가리기", icon: ShieldCheck, desc: "문서 · 이미지 속 주민번호 · 전화번호 자동 가림", keywords: "마스킹 비식별 주민
      { key: "media", label: "이미지 · 영상", icon: Images, desc: "상품 이미지 규격 · 워터마크, 영상 무음 컷 · 자막", keywords: "스마트스토어 쿠팡 썸네
    ],
  },
  {
    title: "비서",
    items: [
      { key: "actions", label: "할 일 뽑기", icon: ListChecks, desc: "글에서 해야 할 일과 담당 · 기한 찾기", keywords: "액션아이템" },
      { key: "schedule", label: "일정 뽑기", icon: CalendarPlus, desc: "'다음 주 화요일 3시' → 캘린더 파일", keywords: "ics 캘린더" },
      { key: "notes", label: "메모 · 할 일", icon: NotebookPen, desc: "메모장과 할 일 목록", keywords: "todo" },
      { key: "clipboard", label: "클립보드", icon: ClipboardList, desc: "복사한 내용 기록 (민감정보 자동 가림)", keywords: "복사 붙여넣기" },
      { key: "ocr", label: "화면 글자 인식", icon: ScanText, desc: "캡처한 화면 · 사진에서 글자 뽑기", keywords: "OCR 캡처" },
      { key: "papers", label: "영수증 · 명함", icon: Receipt, desc: "영수증은 경비 엑셀로, 명함은 연락처로", keywords: "경비 가계부 vcf" },
      { key: "translate", label: "번역", icon: Languages, desc: "인터넷 없이 한 · 영 · 일 · 중 번역", keywords: "영어 일본어" },
      { key: "writing", label: "글다듬기", icon: SpellCheck, desc: "맞춤법, 문체 바꾸기, 이메일 틀", keywords: "맞춤법 공문 존댓말" },
      { key: "rename", label: "이름 일괄 변경", icon: FilePen, desc: "파일 이름을 규칙대로 한 번에", keywords: "파일명 리네임" },
      { key: "dedupe", label: "중복 정리", icon: Copy, desc: "똑같은 파일 · 비슷한 사진 찾기", keywords: "용량 정리" },
      { key: "pdf", label: "PDF 도구", icon: FileText, desc: "합치기, 나누기, 압축, 회전, 이미지 변환", keywords: "병합 분할" },
      { key: "focus", label: "집중 타이머", icon: Timer, desc: "뽀모도로와 주간 리포트", keywords: "뽀모도로 시간관리" },
      { key: "search", label: "통합 검색", icon: Search, desc: "메모 · 회의록 · 클립보드 · 문서를 한 번에", keywords: "찾기" },
    ],
  },
  { title: null, items: [{ key: "settings", label: "설정", icon: Settings, desc: "구성요소, 라이선스, 기본값" }] },
];

export const ALL_ITEMS = NAV.flatMap((g) => g.items);
```

```tsx
import { ListChecks, X } from "lucide-react";
import { useState } from "react";
import { useSystem } from "../components/system";
import { Button, Card, Empty, ErrorBox, Field, Notice, Page } from "../components/ui";
import { api } from "../lib/api";
import { useAction } from "../lib/hooks";

type Item = { text: string; assignee: string | null; due: string | null };

const SAMPLE = `김민수 대리가 금요일까지 상세페이지 시안을 공유하기로 했습니다.
박지영님은 다음 주 화요일까지 견적서를 보내 주세요.
제가 내일 거래처에 연락하겠습니다.
TODO: 서버 비용 정리`;

export default function Actions() {
  const [text, setText] = useState("");
  const [items, setItems] = useState<Item[] | null>(null);
  const [saved, setSaved] = useState(0);
  const act = useAction();
  const { navigate } = useSystem();
  const extract = async () => {
    setSaved(0);
    const r = await act.run(() => api.post<{ items: Item[] }>("/actions/extract", { text }));
    if (r) setItems(r.items);
  };
  const save = async () => {
    if (!items?.length) return;
    const r = await act.run(() => api.post<{ ids: number[] }>("/actions/save", { items, source: "actions" }));
    if (r) setSaved(r.ids.length);
  };
  const update = (i: number, patch: Partial<Item>) => setItems((xs) => xs!.map((x, j) => (j === i ? { ...x, ...patch } : x)));
  return (
    <Page title="할 일 뽑기" desc="회의 메모, 메일, 메신저 대화를 붙여넣으면 해야 할 일과 담당자, 기한을 찾아요.">
      <div className="grid-2">
        <Card>
          <Field label="글">
            <textarea rows={14} value={text} onChange={(e) => setText(e.target.value)} placeholder={SAMPLE} />
          </Field>
          <div className="row">
            <Button variant="primary" icon={<ListChecks size={16} />} busy={act.busy} disabled={!text.trim()} onClick={extract}>
              할 일 찾기
            </Button>
            {!text && <Button variant="ghost" onClick={() => setText(SAMPLE)}>예시 넣기</Button>}
          </div>
          <ErrorBox error={act.error} />
        </Card>
        <Card title={items ? `찾은 할 일 ${items.length}개` : "결과"} right={items?.length ? <Button small variant="primary" onClick={save}>할 일 목
          {!items && <Empty>왼쪽에 글을 넣고 '할 일 찾기'를 눌러 주세요.</Empty>}
          {items && !items.length && <Empty>해야 할 일로 보이는 문장이 없어요.</Empty>}
          {saved > 0 && (
            <Notice tone="ok">
              {saved}개를 넣었어요. <button className="linklike" onClick={() => navigate("notes")}>할 일 보기</button>
            </Notice>
          )}
          <div className="stack">
            {items?.map((it, i) => (
              <div key={i} className="edit-row">
                <input className="grow" value={it.text} onChange={(e) => update(i, { text: e.target.value })} />
                <input className="w-sm" value={it.assignee ?? ""} placeholder="담당" onChange={(e) => update(i, { assignee: e.target.value || nul
                <input className="w-md" type="date" value={it.due ?? ""} onChange={(e) => update(i, { due: e.target.value || null })} />
                <button className="iconbtn" aria-label="빼기" onClick={() => setItems(items.filter((_, j) => j !== i))}>
                  <X size={14} />
```

```
                </button>
              </div>
            ))}
          </div>
        </Card>
      </div>
    </Page>
  );
}
```

```tsx
import { Copy, Pin, Trash2 } from "lucide-react";
import { useEffect, useState } from "react";
import { Badge, Button, Card, Empty, Field, Page, Toggle, useToast } from "../components/ui";
import { api } from "../lib/api";
import { dateTime } from "../lib/format";
import { useData } from "../lib/hooks";
import { copyText, isTauri } from "../lib/platform";

type Clip = { id: number; text: string; sensitive: number; kinds: string; pinned: number; created_at: string };
type Policy = { policy: "mask" | "skip" | "raw"; enabled: boolean };

const KIND_LABEL: Record<string, string> = { rrn: "주민번호", phone: "전화", card: "카드", account: "계좌", email: "이메일", passport: "여권", drive

export default function Clipboard() {
  const [q, setQ] = useState("");
  const clips = useData(() => api.get<Clip[]>("/clipboard", { q }), [q]);
  const policy = useData(() => api.get<Policy>("/clipboard/policy"));
  const [manual, setManual] = useState("");
  const toast = useToast();
  useEffect(() => {
    const on = () => clips.reload();
    window.addEventListener("lb-clipboard", on);
    return () => window.removeEventListener("lb-clipboard", on);
  }, [clips]);
  return (
    <Page
      title="클립보드"
      desc="복사한 글을 기억해 뒀다가 다시 꺼내 써요. 주민번호 · 카드번호 같은 민감정보는 가려서 저장해요."
      actions={<Button variant="ghost" onClick={async () => { if (confirm("고정하지 않은 기록을 모두 지울까요?")) { await api.del("/clipboard"); cli
    >
      <div className="grid-2 wide-left">
        <Card>
          <input value={q} onChange={(e) => setQ(e.target.value)} placeholder="기록에서 찾기" />
          {clips.data && !clips.data.length && <Empty>{isTauri ? "복사하면 여기에 쌓여요." : "데스크톱 앱에서는 복사할 때마다 자동으로 쌓여요."}</Em
          <ul className="list">
            {clips.data?.map((c) => (
              <li key={c.id} className="list-row clip">
                <div className="grow">
                  <div className="clip-text">{c.text}</div>
                  <div className="row small muted">
                    {dateTime(c.created_at)}
                    {c.sensitive ? c.kinds.split(",").filter(Boolean).map((k) => <Badge key={k} tone="late">{KIND_LABEL[k] ?? k} 가림</Badge>) :
                  </div>
                </div>
                <button className="iconbtn" aria-label="복사" onClick={async () => (await copyText(c.text), toast("복사했어요"))}>
                  <Copy size={14} />
                </button>
                <button className={`iconbtn ${c.pinned ? "accent" : ""}`} aria-label="고정" onClick={async () => (await api.post(`/clipboard/${c.
                  <Pin size={14} />
                </button>
                <button className="iconbtn" aria-label="삭제" onClick={async () => (await api.del(`/clipboard/${c.id}`), clips.reload())}>
                  <Trash2 size={14} />
                </button>
              </li>
            ))}
          </ul>
        </Card>
        <Card title="설정">
          {policy.data && (
            <>
              <Toggle
                checked={policy.data.enabled}
```

```
63                 onChange={async (v) => (await api.put("/clipboard/enabled", { enabled: v }), policy.reload())}
64                 label="복사한 내용 기록하기"
65               />
66               <Field label="민감정보가 들어 있으면">
67                 <select value={policy.data.policy} onChange={async (e) => (await api.put("/clipboard/policy", { policy: e.target.value }), poli
68                   <option value="mask">가려서 저장 (권장)</option>
69                   <option value="skip">저장하지 않기</option>
70                   <option value="raw">그대로 저장</option>
71                 </select>
72               </Field>
73             </>
74           )}
75           <p className="muted small">기록은 이 PC 안에만 저장되고 최근 500개까지 남아요. 고정한 항목은 지워지지 않아요.</p>
76           {!isTauri && (
77             <Field label="직접 넣기 (개발 모드)">
78               <textarea rows={3} value={manual} onChange={(e) => setManual(e.target.value)} />
79               <Button small onClick={async () => (await api.post("/clipboard", { text: manual }), setManual(""), clips.reload())}>넣기</Button>
80             </Field>
81           )}
82         </Card>
83       </div>
84     </Page>
85   );
86 }
```

```tsx
import { Copy, Undo2 } from "lucide-react";
import { useMemo, useState } from "react";
import { Button, Card, Empty, ErrorBox, JobBar, Notice, Page, PathPicker, Tabs } from "../components/ui";
import { api } from "../lib/api";
import { baseName, bytes, dateTime } from "../lib/format";
import { useAction, useData, useJob } from "../lib/hooks";
import { reveal } from "../lib/platform";

type FileInfo = { path: string; size: number; mtime: string };
type Group = { files: FileInfo[]; size: number; wasted: number };
type Result = { mode: string; scanned: number; groups: Group[]; wasted: number };
type Hist = { id: number; created_at: string; undone: boolean; count: number };

export default function Dedupe() {
  const [dirs, setDirs] = useState<string[]>([]);
  const [mode, setMode] = useState<"exact" | "similar">("exact");
  const [threshold, setThreshold] = useState(6);
  const job = useJob<Result>();
  const act = useAction();
  const hist = useData(() => api.get<Hist[]>("/dedupe/history"));
  const [checked, setChecked] = useState<Set<string>>(new Set());
  const [moved, setMoved] = useState<string | null>(null);
  const result = job.job?.status === "done" ? job.job.result : null;

  const scan = async () => {
    setMoved(null);
    const j = await job.start(() => api.post("/dedupe/scan", { dirs, mode, threshold }));
    // 각 묶음에서 첫 번째(가장 오래된/큰 것)만 남기고 나머지를 미리 골라 둔다.
    if (j?.result) setChecked(new Set(j.result.groups.flatMap((g) => g.files.slice(1).map((f) => f.path))));
  };
  const selectedSize = useMemo(
    () => (result?.groups ?? []).flatMap((g) => g.files).filter((f) => checked.has(f.path)).reduce((s, f) => s + f.size, 0),
    [result, checked],
  );
  const quarantine = async () => {
    const r = await act.run(() => api.post<{ moved: number; folder: string }>("/dedupe/quarantine", { paths: Array.from(checked) }));
    if (r) {
      setMoved(`${r.moved}개를 보관함으로 옮겼어요. 지운 게 아니라서 언제든 되돌릴 수 있어요.`);
      job.reset();
      setChecked(new Set());
      hist.reload();
    }
  };
  const toggle = (p: string) => setChecked((s) => {
    const n = new Set(s);
    if (n.has(p)) n.delete(p);
    else n.add(p);
    return n;
  });
  return (
    <Page title="중복 정리" desc="내용이 똑같은 파일이나 거의 같은 사진을 찾아요. 지우지 않고 보관함으로 옮겨서 되돌릴 수 있어요.">
      <div className="grid-2 wide-left">
        <Card>
          <PathPicker value={dirs} onChange={setDirs} multiple directory placeholder="찾을 폴더를 고르거나 끌어다 놓으세요" />
          <Tabs tabs={[{ key: "exact", label: "똑같은 파일" }, { key: "similar", label: "비슷한 사진" }]} value={mode} onChange={setMode} />
          {mode === "similar" && (
            <label className="field">
              <span className="field-label">비슷한 정도: {threshold <= 3 ? "거의 같아야" : threshold <= 8 ? "보통" : "느슨하게"}</span>
              <input type="range" min={1} max={14} value={threshold} onChange={(e) => setThreshold(+e.target.value)} />
            </label>
          )}
          <Button variant="primary" icon={<Copy size={16} />} busy={job.running} disabled={!dirs.length} onClick={scan}>찾기</Button>
```

```
63            <JobBar job={job.job} error={job.error} onCancel={job.cancel} />
64            <ErrorBox error={act.error} />
65            {moved && <Notice tone="ok">{moved}</Notice>}
66          </Card>
67          <Card title="보관함 기록">
68            {hist.data?.length ? (
69              <ul className="list compact">
70                {hist.data.map((h) => (
71                  <li key={h.id} className="list-row">
72                    <span className="grow">{dateTime(h.created_at)} · {h.count}개</span>
73                    {h.undone ? <span className="muted small">되돌림</span> : (
74                      <Button small icon={<Undo2 size={14} />} onClick={async () => { await act.run(() => api.post(`/dedupe/restore/${h.id}`)); h
75                    )}
76                  </li>
77                ))}
78              </ul>
79            ) : <p className="muted small">아직 옮긴 기록이 없어요.</p>}
80          </Card>
81        </div>
82        {result && (
83          <Card
84            title={`${result.scanned.toLocaleString()}개 중 ${result.groups.length}묶음 · 아낄 수 있는 용량 ${bytes(result.wasted)}`}
85            right={<Button variant="danger" disabled={!checked.size} busy={act.busy} onClick={quarantine}>고른 {checked.size}개 보관함으로 ({bytes(sel
86          >
87            {!result.groups.length && <Empty>중복이 없어요 </Empty>}
88            <div className="stack">
89              {result.groups.map((g, gi) => (
90                <div key={gi} className="dup-group">
91                  {g.files.map((f) => (
92                    <label key={f.path} className="dup-row">
93                      <input type="checkbox" checked={checked.has(f.path)} onChange={() => toggle(f.path)} />
94                      <span className="grow truncate" title={f.path}>{baseName(f.path)} <span className="muted small">{f.path}</span></span>
95                      <span className="muted small">{bytes(f.size)}</span>
96                      <span className="muted small">{dateTime(f.mtime)}</span>
97                      <button className="iconbtn" onClick={(e) => (e.preventDefault(), reveal(f.path))} aria-label="폴더에서 보기">↗</button>
98                    </label>
99                  ))}
100               </div>
101             ))}
102           </div>
103         </Card>
104       )}
105     </Page>
106   );
107 }
```

```tsx
import { Coffee, Play, Square } from "lucide-react";
import { useState, useSyncExternalStore } from "react";
import { Button, Card, CopyButton, Field, Page } from "../components/ui";
import { api } from "../lib/api";
import { focusTimer } from "../lib/focusTimer";
import { localDate } from "../lib/format";
import { useData } from "../lib/hooks";

type Report = { week_start: string; days: { date: string; label: string; minutes: number; sessions: number }[]; labels: { label: string; minute

function mondayOf(d: Date) {
  const x = new Date(d);
  x.setDate(x.getDate() - ((x.getDay() + 6) % 7));
  return localDate(x);
}

export default function Focus() {
  const running = useSyncExternalStore(focusTimer.subscribe, focusTimer.get);
  const [, force] = useState(0);
  useSyncExternalStore(focusTimer.subscribe, () => Math.floor(Date.now() / 1000));
  const settings = useData(() => api.get<Record<string, number>>("/system/settings"));
  const [label, setLabel] = useState("");
  const focusMin = settings.data?.focus_minutes ?? 25;
  const breakMin = settings.data?.break_minutes ?? 5;
  const [week, setWeek] = useState(mondayOf(new Date()));
  const report = useData(() => api.get<Report>("/focus/report", { week }), [week, running?.id]);
  const today = useData(() => api.get<{ minutes: number; completed: number }>("/focus/today"), [running?.id]);

  const left = running ? Math.max(0, Math.round((running.endsAt - Date.now()) / 1000)) : focusMin * 60;
  const total = running ? running.minutes * 60 : focusMin * 60;
  const pct = 1 - left / total;
  const mm = String(Math.floor(left / 60)).padStart(2, "0");
  const ss = String(left % 60).padStart(2, "0");
  const max = Math.max(60, ...(report.data?.days.map((d) => d.minutes) ?? [0]));
  const shift = (n: number) => {
    const d = new Date(week + "T00:00:00");
    d.setDate(d.getDate() + n * 7);
    setWeek(localDate(d));
  };

  return (
    <Page title="집중 타이머" desc={`${focusMin}분 집중하고 ${breakMin}분 쉬어요. 다른 화면으로 가도 계속 돌아가고, 끝나면 알림 소리가 나요.`}>
      <div className="grid-2">
        <Card>
          <div className={`timer ${running?.kind === "break" ? "break" : ""}`} style={{ ["--pct" as string]: pct }}>
            <div className="timer-ring">
              <div className="timer-text">
                <b>{mm}:{ss}</b>
                <span className="muted small">{running ? (running.kind === "focus" ? running.label || "집중 중" : "쉬는 중") : "준비"}</span>
              </div>
            </div>
          </div>
          {!running ? (
            <>
              <Field label="지금 할 일">
                <input value={label} onChange={(e) => setLabel(e.target.value)} placeholder="예: 기획서 쓰기" />
              </Field>
              <div className="row">
                <Button variant="primary" icon={<Play size={16} />} onClick={async () => (await focusTimer.start("focus", focusMin, label), for
                  {focusMin}분 집중
                </Button>
                <Button icon={<Coffee size={16} />} onClick={() => focusTimer.start("break", breakMin, "")}>
```

```
                  {breakMin}분 쉬기
                </Button>
              </div>
            </>
          ) : (
            <Button variant="danger" icon={<Square size={16} />} onClick={() => focusTimer.stop(false)}>
              그만하기
            </Button>
          )}
          <p className="muted small">오늘 {today.data?.minutes ?? 0}분 집중 · 끝까지 마친 세션 {today.data?.completed ?? 0}회</p>
        </Card>
        <Card
          title={`주간 리포트 · ${Math.floor((report.data?.total_minutes ?? 0) / 60)}시간 ${(report.data?.total_minutes ?? 0) % 60}분`}
          right={
            <div className="row">
              <Button small variant="ghost" onClick={() => shift(-1)}>◀</Button>
              <span className="small">{week.slice(5)} 주</span>
              <Button small variant="ghost" onClick={() => shift(1)}>▶</Button>
              {report.data && <CopyButton text={report.data.markdown} label="리포트 복사" />}
            </div>
          }
        >
          <div className="bars">
            {report.data?.days.map((d) => (
              <div key={d.date} className="bar-col" title={`${d.minutes}분`}>
                <div className="bar" style={{ height: `${(d.minutes / max) * 100}%` }} />
                <span className="small">{d.label}</span>
                <span className="muted small">{d.minutes ? `${d.minutes}분` : ""}</span>
              </div>
            ))}
          </div>
          {report.data?.labels.length ? (
            <ul className="list compact">
              {report.data.labels.slice(0, 6).map((l) => (
                <li key={l.label} className="list-row">
                  <span className="grow">{l.label}</span>
                  <span className="muted">{l.minutes}분</span>
                </li>
              ))}
            </ul>
          ) : null}
          <p className="muted small">이번 주에 끝낸 할 일 {report.data?.done_todos ?? 0}개</p>
        </Card>
      </div>
    </Page>
  );
}
```

```tsx
import { ArrowRight, Check } from "lucide-react";
import { useSystem } from "../components/system";
import { Badge, Card, Empty, Page } from "../components/ui";
import { api } from "../lib/api";
import { dateTime, dueLabel } from "../lib/format";
import { useData } from "../lib/hooks";
import { NAV } from "../nav";

type Todo = { id: number; text: string; due: string | null; assignee: string | null };
type Dash = {
  todos_open: number;
  todos_due_today: number;
  notes: number;
  clips: number;
  transcripts: number;
  contacts: number;
  focus_today: number;
  recent_transcripts: { id: number; title: string; created_at: string }[];
  upcoming_todos: Todo[];
};

export default function Home() {
  const { navigate } = useSystem();
  const { data, reload } = useData(() => api.get<Dash>("/dashboard"));
  const hour = new Date().getHours();
  const greet = hour < 12 ? "좋은 아침이에요" : hour < 18 ? "좋은 오후예요" : "오늘도 수고했어요";
  const done = async (id: number) => {
    await api.patch(`/todos/${id}`, { done: true });
    reload();
  };
  return (
    <Page title={greet} desc="모든 처리는 이 PC 안에서만 이뤄져요. 파일이 밖으로 나가지 않아요.">
      <div className="stats">
        <Stat label="남은 할 일" value={data?.todos_open} sub={data?.todos_due_today ? `오늘까지 ${data.todos_due_today}개` : undefined} onClick={()
        <Stat label="오늘 집중" value={data ? `${data.focus_today}분` : undefined} onClick={() => navigate("focus")} />
        <Stat label="회의록" value={data?.transcripts} onClick={() => navigate("transcribe")} />
        <Stat label="메모" value={data?.notes} onClick={() => navigate("notes")} />
      </div>
      <div className="grid-2">
        <Card title="다가오는 할 일" right={<button className="linklike" onClick={() => navigate("notes")}>전체 보기</button>}>
          {data && data.upcoming_todos.length === 0 && <Empty>할 일이 없어요. 회의록이나 글에서 자동으로 뽑아 보세요.</Empty>}
          <ul className="list">
            {data?.upcoming_todos.map((t) => {
              const d = dueLabel(t.due);
              return (
                <li key={t.id} className="list-row">
                  <button className="check" onClick={() => done(t.id)} aria-label="완료">
                    <Check size={13} />
                  </button>
                  <span className="grow">{t.text}</span>
                  {t.assignee && <Badge>{t.assignee}</Badge>}
                  {d.text && <Badge tone={d.tone}>{d.text}</Badge>}
                </li>
              );
            })}
          </ul>
        </Card>
        <Card title="최근 회의록" right={<button className="linklike" onClick={() => navigate("transcribe")}>새로 만들기</button>}>
          {data && data.recent_transcripts.length === 0 && <Empty>녹음 파일을 넣으면 회의록 · 요약 · 할 일이 한 번에 만들어져요.</Empty>}
          <ul className="list">
            {data?.recent_transcripts.map((t) => (
              <li key={t.id} className="list-row clickable" onClick={() => (sessionStorage.setItem("lb-open-transcript", String(t.id)), navigat
```

```
                <span className="grow">{t.title}</span>
                <span className="muted small">{dateTime(t.created_at)}</span>
              </li>
            ))}
          </ul>
        </Card>
      </div>
      <h3 className="section-title">모든 기능</h3>
      <div className="feature-grid">
        {NAV.slice(1, 3).flatMap((g) =>
          g.items.map((it) => (
            <button key={it.key} className="feature" onClick={() => navigate(it.key)}>
              <it.icon size={20} />
              <div>
                <b>{it.label}</b>
                <span className="muted small">{it.desc}</span>
              </div>
              <ArrowRight size={15} className="muted" />
            </button>
          )),
        )}
      </div>
    </Page>
  );
}

function Stat({ label, value, sub, onClick }: { label: string; value: number | string | undefined; sub?: string; onClick?: () => void }) {
  return (
    <button className="stat" onClick={onClick}>
      <span className="muted small">{label}</span>
      <b>{value ?? " - "}</b>
      {sub && <span className="small accent">{sub}</span>}
    </button>
  );
}
```

```tsx
import { ShieldCheck } from "lucide-react";
import { useMemo, useState } from "react";
import { NeedsComponent } from "../components/system";
import { Badge, Button, Card, CopyButton, ErrorBox, Field, JobBar, Notice, Page, PathLink, PathPicker, Tabs, Toggle } from "../components/ui";
import { api } from "../lib/api";
import { baseName } from "../lib/format";
import { useAction, useData, useJob } from "../lib/hooks";
import { pickFiles } from "../lib/platform";

type Kind = { key: string; label: string; default: boolean };
type FileResult = { src: string; dst?: string; count?: number; by_kind?: Record<string, number>; note?: string; error?: string };
type Finding = { kind: string; label: string; start: number; end: number; value: string };

const MODES = [
  { key: "partial", label: "일부만 가리기 (010-****-5678)" },
  { key: "full", label: "전부 가리기 (***-****-****)" },
  { key: "tag", label: "항목 이름으로 ([전화번호])" },
];
const DOCS = [{ name: "문서 · 이미지", extensions: ["hwpx", "hwp", "docx", "pdf", "txt", "md", "csv", "png", "jpg", "jpeg", "webp", "bmp", "tif", "ti

export default function Mask() {
  const [tab, setTab] = useState<"files" | "text">("files");
  const kinds = useData(() => api.get<{ kinds: Kind[] }>("/mask/kinds"));
  const [selected, setSelected] = useState<string[] | null>(null);
  const [mode, setMode] = useState("partial");
  const active = selected ?? kinds.data?.kinds.filter((k) => k.default).map((k) => k.key) ?? [];
  const labels = useMemo(() => Object.fromEntries((kinds.data?.kinds ?? []).map((k) => [k.key, k.label])), [kinds.data]);
  return (
    <Page title="개인정보 가리기" desc="주민번호, 전화번호, 계좌, 카드, 주소, 이름, 얼굴까지 자동으로 찾아 가려요. 원본은 그대로 두고 '_마스킹' 사본
      <Tabs tabs={[{ key: "files", label: "파일" }, { key: "text", label: "글 바로 가리기" }]} value={tab} onChange={setTab} />
      <div className="grid-2 wide-left">
        {tab === "files" ? <Files kinds={active} mode={mode} labels={labels} /> : <TextMask kinds={active} mode={mode} />}
        <Card title="찾을 항목">
          <div className="checks">
            {kinds.data?.kinds.map((k) => (
              <label key={k.key} className="check-label">
                <input
                  type="checkbox"
                  checked={active.includes(k.key)}
                  onChange={(e) => setSelected(e.target.checked ? [...active, k.key] : active.filter((x) => x !== k.key))}
                />
                {k.label}
              </label>
            ))}
          </div>
          <Field label="가리는 방식">
            <select value={mode} onChange={(e) => setMode(e.target.value)}>
              {MODES.map((m) => <option key={m.key} value={m.key}>{m.label}</option>)}
            </select>
          </Field>
          <p className="muted small">자동 탐지는 사람의 최종 확인을 대신하지 못해요. 결과 파일을 꼭 한 번 열어 보세요.</p>
        </Card>
      </div>
    </Page>
  );
}

function Files({ kinds, mode, labels }: { kinds: string[]; mode: string; labels: Record<string, string> }) {
  const [paths, setPaths] = useState<string[]>([]);
  const [outDir, setOutDir] = useState("");
  const [faces, setFaces] = useState(true);
  const job = useJob<{ results: FileResult[]; total: number }>();
```

```
  const hasImage = paths.some((p) => /\.(png¦jpe?g¦webp¦bmp¦tiff?)$/i.test(p));
  return (
    <Card>
      <PathPicker value={paths} onChange={setPaths} multiple filters={DOCS} placeholder="HWPX · HWP · DOCX · PDF · 이미지 · 텍스트 파일을 고르거나 끌어다
      {paths.some((p) => /\.hwp$/i.test(p)) && <Notice tone="warn">HWP(구형)는 본문 글자만 가린 .txt로 저장돼요. 서식을 지키려면 한글에서 HWPX로 저장
      {hasImage && <NeedsComponent keys={["ocr"]}>이미지 속 글자를 찾으려면 OCR이 필요해요.</NeedsComponent>}
      <div className="row wrap">
        <Field label="저장 폴더 (비우면 원본 옆)" inline>
          <div className="row">
            <input value={outDir} onChange={(e) => setOutDir(e.target.value)} placeholder="원본과 같은 폴더" />
            <Button small onClick={async () => setOutDir((await pickFiles({ directory: true }))[0] ?? outDir)}>고르기</Button>
          </div>
        </Field>
        {hasImage && <Toggle checked={faces} onChange={setFaces} label="얼굴도 가리기" />}
      </div>
      <Button
        variant="primary"
        icon={<ShieldCheck size={16} />}
        busy={job.running}
        disabled={!paths.length ¦¦ !kinds.length}
        onClick={() => job.start(() => api.post("/mask/files", { paths, kinds, mode, out_dir: outDir ¦¦ null, faces }))}
      >
        {paths.length ? `${paths.length}개 파일 가리기` : "가리기"}
      </Button>
      <JobBar job={job.job} error={job.error} onCancel={job.cancel} />
      {job.job?.status === "done" && job.job.result && (
        <div className="results">
          <Notice tone="ok">모두 {job.job.result.total}곳을 가렸어요.</Notice>
          <table className="table">
            <thead>
              <tr><th>파일</th><th>찾은 항목</th><th>결과</th></tr>
            </thead>
            <tbody>
              {job.job.result.results.map((r) => (
                <tr key={r.src}>
                  <td title={r.src}>{baseName(r.src)}</td>
                  <td>
                    {r.error ? <span className="error-text">{r.error}</span> : Object.entries(r.by_kind ?? {}).map(([k, n]) => <Badge key={k}>{
                    {r.count === 0 && <span className="muted">없음</span>}
                    {r.note && <div className="muted small">{r.note}</div>}
                  </td>
                  <td>{r.dst && <PathLink path={r.dst} />}</td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      )}
    </Card>
  );
}

function TextMask({ kinds, mode }: { kinds: string[]; mode: string }) {
  const [text, setText] = useState("");
  const [res, setRes] = useState<{ masked: string; findings: Finding[] } ¦ null>(null);
  const act = useAction();
  return (
    <Card>
      <Field label="원문">
        <textarea rows={8} value={text} onChange={(e) => setText(e.target.value)} placeholder="개인정보가 섞인 글을 붙여넣으세요. 예: 홍길동 고객님
      </Field>
      <Button variant="primary" busy={act.busy} disabled={!text.trim()} onClick={async () => setRes((await act.run(() => api.post("/mask/text",
```

```
        가리기
      </Button>
      <ErrorBox error={act.error} />
      {res && (
        <>
          <Field label={`결과 · ${res.findings.length}곳`}>
            <textarea rows={8} readOnly value={res.masked} />
          </Field>
          <div className="row wrap">
            <CopyButton text={res.masked} />
            {res.findings.map((f, i) => <Badge key={i}>{f.label}</Badge>)}
          </div>
        </>
      )}
    </Card>
  );
}
```

```tsx
import { Images, Scissors, Subtitles } from "lucide-react";
import { useState } from "react";
import { NeedsComponent, useSystem } from "../components/system";
import { Button, Card, ErrorBox, Field, JobBar, Notice, Page, PathLink, PathPicker, Tabs, Thumb, Toggle } from "../components/ui";
import { api } from "../lib/api";
import { baseName, clock } from "../lib/format";
import { useAction, useData, useJob } from "../lib/hooks";

type Preset = { key: string; label: string; width: number; height: number | null; mode: string };
type PresetsResp = { presets: Preset[]; rembg: boolean; ffmpeg: boolean; stt: boolean; font: boolean };
const IMAGES = [{ name: "이미지", extensions: ["png", "jpg", "jpeg", "webp", "bmp", "gif", "tif", "tiff"] }];
const VIDEO = [{ name: "영상 · 녹음", extensions: ["mp4", "mov", "mkv", "webm", "avi", "mp3", "m4a", "wav"] }];

export default function Media() {
  const [tab, setTab] = useState<"image" | "video">("image");
  const info = useData(() => api.get<PresetsResp>("/media/presets"));
  return (
    <Page title="이미지 · 영상" desc="상품 이미지를 마켓 규격에 맞추고 워터마크를 넣어요. 영상은 말 없는 구간을 잘라내고 자막을 만들어요.">
      <Tabs tabs={[{ key: "image", label: "이미지 일괄 처리" }, { key: "video", label: "영상 무음 컷 · 자막" }]} value={tab} onChange={setTab} />
      {tab === "image" ? <ImageBatch info={info.data} /> : <Video />}
    </Page>
  );
}

function ImageBatch({ info }: { info: PresetsResp | null }) {
  const [paths, setPaths] = useState<string[]>([]);
  const [preset, setPreset] = useState("smartstore_main");
  const [custom, setCustom] = useState({ width: 1000, height: 1000, mode: "pad" });
  const [format, setFormat] = useState("jpg");
  const [quality, setQuality] = useState(90);
  const [bg, setBg] = useState("#ffffff");
  const [removeBg, setRemoveBg] = useState(false);
  const [wm, setWm] = useState({ text: "", position: "bottom-right", opacity: 0.35, size: 0.05, color: "#ffffff" });
  const job = useJob<{ results: { src: string; dst?: string; error?: string }[] }>();
  const run = () =>
    job.start(() =>
      api.post("/media/images", {
        paths,
        preset: preset === "custom" ? null : preset,
        ...(preset === "custom" ? custom : {}),
        format,
        quality,
        background: bg,
        remove_bg: removeBg,
        watermark: wm.text ? wm : null,
      }),
    );
  return (
    <div className="grid-2 wide-left">
      <Card>
        <PathPicker value={paths} onChange={setPaths} multiple filters={IMAGES} placeholder="이미지 파일이나 폴더를 고르거나 끌어다 놓으세요" />
        <Button variant="primary" icon={<Images size={16} />} busy={job.running} disabled={!paths.length} onClick={run}>
          처리하기
        </Button>
        <JobBar job={job.job} error={job.error} onCancel={job.cancel} />
        {job.job?.status === "done" && job.job.result && (
          <div className="results">
            <Notice tone="ok">{job.job.result.results.filter((r) => r.dst).length}장을 저장했어요.</Notice>
            <div className="thumbs">
              {job.job.result.results.map((r) => (
                <figure key={r.src}>
                  {r.dst ? <Thumb path={r.dst} /> : <span className="error-text small">{r.error}</span>}
```

```
                <figcaption className="small">{r.dst ? <PathLink path={r.dst} /> : baseName(r.src)}</figcaption>
              </figure>
            ))}
          </div>
        </div>
      )}
    </Card>
    <Card title="설정">
      <Field label="규격">
        <select value={preset} onChange={(e) => setPreset(e.target.value)}>
          {info?.presets.map((p) => <option key={p.key} value={p.key}>{p.label}</option>)}
          <option value="custom">직접 입력</option>
        </select>
      </Field>
      {preset === "custom" && (
        <div className="row">
          <Field label="가로"><input type="number" value={custom.width} onChange={(e) => setCustom({ ...custom, width: +e.target.value })} /></
          <Field label="세로"><input type="number" value={custom.height} onChange={(e) => setCustom({ ...custom, height: +e.target.value })} />
          <Field label="맞추기">
            <select value={custom.mode} onChange={(e) => setCustom({ ...custom, mode: e.target.value })}>
              <option value="pad">여백 넣기</option>
              <option value="crop">잘라 채우기</option>
              <option value="fit">안에 맞추기</option>
            </select>
          </Field>
        </div>
      )}
      <div className="row">
        <Field label="형식">
          <select value={format} onChange={(e) => setFormat(e.target.value)}>
            <option value="jpg">JPG</option>
            <option value="png">PNG</option>
            <option value="webp">WEBP</option>
            <option value="same">원본과 같게</option>
          </select>
        </Field>
        <Field label={`화질 ${quality}`}>
          <input type="range" min={50} max={100} value={quality} onChange={(e) => setQuality(+e.target.value)} />
        </Field>
        <Field label="여백 색">
          <input type="color" value={bg} onChange={(e) => setBg(e.target.value)} />
        </Field>
      </div>
      <Toggle checked={removeBg} onChange={setRemoveBg} label={`배경 지우기${info && !info.rembg ? " (구성요소 필요)" : ""}`} />
      {removeBg && <NeedsComponent keys={["rembg"]} />}
      <Field label="워터마크 글자">
        <input value={wm.text} onChange={(e) => setWm({ ...wm, text: e.target.value })} placeholder="예: @내가게" />
      </Field>
      {wm.text && (
        <div className="row">
          <Field label="위치">
            <select value={wm.position} onChange={(e) => setWm({ ...wm, position: e.target.value })}>
              <option value="bottom-right">오른쪽 아래</option>
              <option value="bottom-left">왼쪽 아래</option>
              <option value="top-right">오른쪽 위</option>
              <option value="top-left">왼쪽 위</option>
              <option value="center">가운데</option>
              <option value="tile">바둑판</option>
            </select>
          </Field>
          <Field label={`진하기 ${Math.round(wm.opacity * 100)}%`}>
            <input type="range" min={10} max={100} value={wm.opacity * 100} onChange={(e) => setWm({ ...wm, opacity: +e.target.value / 100 })
```

```
            </Field>
            <Field label="색"><input type="color" value={wm.color} onChange={(e) => setWm({ ...wm, color: e.target.value })} /></Field>
          </div>
        )}
        {info && !info.font && <p className="muted small">한글 글꼴을 찾지 못해 워터마크 한글이 깨질 수 있어요.</p>}
      </Card>
    </div>
  );
}

type Detect = { duration: number; silences: [number, number][]; keep: [number, number][]; result: number; removed: number };

function Video() {
  const { has } = useSystem();
  const [paths, setPaths] = useState<string[]>([]);
  const [noise, setNoise] = useState(-35);
  const [minSilence, setMinSilence] = useState(0.6);
  const [pad, setPad] = useState(0.15);
  const [preview, setPreview] = useState<Detect | null>(null);
  const act = useAction();
  const cut = useJob<{ dst: string; original: number; result: number; removed: number }>();
  const sub = useJob<{ dst: string; segments: number }>();
  const path = paths[0];
  const opts = { path, noise_db: noise, min_silence: minSilence, pad };
  return (
    <div className="grid-2 wide-left">
      <Card>
        <NeedsComponent keys={["ffmpeg"]} />
        <PathPicker value={paths} onChange={(p) => (setPaths(p), setPreview(null))} filters={VIDEO} placeholder="영상이나 녹음 파일을 고르거나 끌어다
        <div className="row wrap">
          <Button busy={act.busy} disabled={!path || !has("ffmpeg")} onClick={async () => setPreview((await act.run(() => api.post<Detect>("/me
            미리 계산
          </Button>
          <Button variant="primary" icon={<Scissors size={16} />} busy={cut.running} disabled={!path || !has("ffmpeg")} onClick={() => cut.star
            무음 구간 자르기
          </Button>
          <Button icon={<Subtitles size={16} />} busy={sub.running} disabled={!path || !has("stt")} onClick={() => sub.start(() => api.post("/m
            자막(SRT) 만들기
          </Button>
        </div>
        <ErrorBox error={act.error} />
        {preview && (
          <Notice>
            전체 {clock(preview.duration)} 중 말 없는 구간 {preview.silences.length}곳, {clock(preview.removed)}을 잘라 <b>{clock(preview.result)}</
          </Notice>
        )}
        <JobBar job={cut.job} error={cut.error} onCancel={cut.cancel} />
        {cut.job?.status === "done" && cut.job.result && (
          <Notice tone="ok">
            {clock(cut.job.result.original)} → {clock(cut.job.result.result)} · <PathLink path={cut.job.result.dst} />
          </Notice>
        )}
        <JobBar job={sub.job} error={sub.error} onCancel={sub.cancel} />
        {sub.job?.status === "done" && sub.job.result && (
          <Notice tone="ok">
            자막 {sub.job.result.segments}줄 · <PathLink path={sub.job.result.dst} />
          </Notice>
        )}
      </Card>
      <Card title="자르기 기준">
        <Field label={`무음으로 볼 소리 크기: ${noise}dB`} hint="숫자가 클수록(0에 가까울수록) 작은 소리도 무음으로 봐요.">
          <input type="range" min={-60} max={-15} value={noise} onChange={(e) => setNoise(+e.target.value)} />
```

```
        </Field>
        <Field label={`이 시간 넘게 조용하면 자르기: ${minSilence}초`}>
          <input type="range" min={0.2} max={3} step={0.1} value={minSilence} onChange={(e) => setMinSilence(+e.target.value)} />
        </Field>
        <Field label={`앞뒤 여유: ${pad}초`} hint="말이 뚝 끊기지 않게 남겨 둘 시간이에요.">
          <input type="range" min={0} max={0.5} step={0.05} value={pad} onChange={(e) => setPad(+e.target.value)} />
        </Field>
      </Card>
    </div>
  );
}
```

```tsx
import { Check, Pin, Plus, Trash2 } from "lucide-react";
import { useEffect, useState } from "react";
import { Badge, Button, Card, Empty, Page, Tabs } from "../components/ui";
import { api } from "../lib/api";
import { dateTime, dueLabel } from "../lib/format";
import { useData } from "../lib/hooks";

type Todo = { id: number; text: string; done: number; due: string | null; assignee: string | null; source: string | null };
type Note = { id: number; title: string; body: string; tags: string; pinned: number; updated_at: string };

export default function Notes() {
  const [tab, setTab] = useState<"todo" | "note">(() => {
    const hit = sessionStorage.getItem("lb-open");
    return hit && JSON.parse(hit).kind === "note" ? "note" : "todo";
  });
  return (
    <Page title="메모 · 할 일" desc="할 일은 회의록이나 글에서 자동으로 들어오기도 해요.">
      <Tabs tabs={[{ key: "todo", label: "할 일" }, { key: "note", label: "메모" }]} value={tab} onChange={setTab} />
      {tab === "todo" ? <Todos /> : <NotesView />}
    </Page>
  );
}

function Todos() {
  const [status, setStatus] = useState<"open" | "done">("open");
  const { data, reload } = useData(() => api.get<Todo[]>("/todos", { status }), [status]);
  const [text, setText] = useState("");
  const [due, setDue] = useState("");
  const add = async () => {
    if (!text.trim()) return;
    await api.post("/todos", { text, due: due || null });
    setText("");
    setDue("");
    reload();
  };
  const toggle = async (t: Todo) => {
    await api.patch(`/todos/${t.id}`, { done: !t.done });
    reload();
  };
  return (
    <Card>
      <div className="edit-row">
        <input className="grow" value={text} onChange={(e) => setText(e.target.value)} onKeyDown={(e) => e.key === "Enter" && add()} placeholde
        <input className="w-md" type="date" value={due} onChange={(e) => setDue(e.target.value)} />
        <Button variant="primary" icon={<Plus size={15} />} onClick={add}>추가</Button>
      </div>
      <div className="row">
        <Tabs tabs={[{ key: "open", label: "남은 일" }, { key: "done", label: "끝낸 일" }]} value={status} onChange={setStatus} />
      </div>
      {data && !data.length && <Empty>{status === "open" ? "남은 일이 없어요 " : "아직 끝낸 일이 없어요."}</Empty>}
      <ul className="list">
        {data?.map((t) => {
          const d = dueLabel(t.due);
          return (
            <li key={t.id} className={`list-row ${t.done ? "done" : ""}`}>
              <button className={`check ${t.done ? "on" : ""}`} onClick={() => toggle(t)} aria-label="완료 표시">
                <Check size={13} />
              </button>
              <span className="grow">{t.text}</span>
              {t.source?.startsWith("transcript:") && <Badge>회의록</Badge>}
              {t.assignee && <Badge>{t.assignee}</Badge>}
              {d.text && !t.done && <Badge tone={d.tone}>{d.text}</Badge>}
```

```
              <button className="iconbtn" aria-label="삭제" onClick={async () => (await api.del(`/todos/${t.id}`), reload())}>
                <Trash2 size={14} />
              </button>
            </li>
          );
        })}
      </ul>
    </Card>
  );
}

function NotesView() {
  const [q, setQ] = useState("");
  const { data, reload } = useData(() => api.get<Note[]>("/notes", { q }), [q]);
  const [sel, setSel] = useState<Note | null>(null);
  const [draft, setDraft] = useState({ title: "", body: "", tags: "", pinned: false });
  const [dirty, setDirty] = useState(false);

  useEffect(() => {
    const hit = sessionStorage.getItem("lb-open");
    if (hit && data) {
      const h = JSON.parse(hit);
      if (h.kind === "note") {
        sessionStorage.removeItem("lb-open");
        const n = data.find((x) => String(x.id) === h.ref);
        if (n) open(n);
      }
    }
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [data]);

  const open = (n: Note | null) => {
    setSel(n);
    setDraft(n ? { title: n.title, body: n.body, tags: n.tags, pinned: !!n.pinned } : { title: "", body: "", tags: "", pinned: false });
    setDirty(false);
  };
  const save = async () => {
    const saved = sel ? await api.put<Note>(`/notes/${sel.id}`, draft) : await api.post<Note>("/notes", draft);
    setSel(saved);
    setDirty(false);
    reload();
  };
  return (
    <div className="grid-2 notes">
      <Card>
        <div className="edit-row">
          <input className="grow" value={q} onChange={(e) => setQ(e.target.value)} placeholder="메모 찾기" />
          <Button icon={<Plus size={15} />} onClick={() => open(null)}>새 메모</Button>
        </div>
        {data && !data.length && <Empty>메모가 없어요.</Empty>}
        <ul className="list">
          {data?.map((n) => (
            <li key={n.id} className={`list-row clickable ${sel?.id === n.id ? "selected" : ""}`} onClick={() => open(n)}>
              {n.pinned ? <Pin size={13} className="accent" /> : null}
              <span className="grow truncate">{n.title || n.body.slice(0, 30) || "(빈 메모)"}</span>
              <span className="muted small">{dateTime(n.updated_at)}</span>
            </li>
          ))}
        </ul>
      </Card>
      <Card>
        <input className="title-input" value={draft.title} placeholder="제목" onChange={(e) => (setDraft({ ...draft, title: e.target.value }), se
```

```
        <textarea rows={16} value={draft.body} placeholder="내용" onChange={(e) => (setDraft({ ...draft, body: e.target.value }), setDirty(true))
        <div className="edit-row">
          <input className="grow" value={draft.tags} placeholder="태그 (쉼표로 구분)" onChange={(e) => (setDraft({ ...draft, tags: e.target.value })
          <Button small icon={<Pin size={14} />} variant={draft.pinned ? "primary" : "default"} onClick={() => (setDraft({ ...draft, pinned: !d
            고정
          </Button>
        </div>
        <div className="row">
          <Button variant="primary" disabled={!dirty} onClick={save}>저장</Button>
          {sel && (
            <Button variant="danger" onClick={async () => { if (confirm("이 메모를 지울까요?")) { await api.del(`/notes/${sel.id}`); open(null); rel
              삭제
            </Button>
          )}
        </div>
      </Card>
    </div>
  );
}
```

```tsx
import { Camera, ImageUp, ScanText } from "lucide-react";
import { useCallback, useEffect, useRef, useState } from "react";
import { NeedsComponent, useSystem } from "../components/system";
import { Button, Card, CopyButton, Empty, ErrorBox, Page } from "../components/ui";
import { api } from "../lib/api";
import { useAction } from "../lib/hooks";
import { captureScreen, isTauri, onFileDrop, pickFiles } from "../lib/platform";

type Preview = { data_url: string; width: number; height: number };
type Rect = { x: number; y: number; w: number; h: number };

export default function Ocr() {
  const [path, setPath] = useState<string | null>(() => sessionStorage.getItem("lb-captured"));
  const [preview, setPreview] = useState<Preview | null>(null);
  const [sel, setSel] = useState<Rect | null>(null);
  const [text, setText] = useState("");
  const act = useAction();
  const { has, navigate } = useSystem();

  const load = useCallback(
    async (p: string) => {
      sessionStorage.removeItem("lb-captured");
      setPath(p);
      setSel(null);
      setText("");
      setPreview((await act.run(() => api.post<Preview>("/ocr/preview", { path: p }))) ?? null);
    },
    [act],
  );
  useEffect(() => {
    if (path) load(path);
    const on = (e: Event) => load((e as CustomEvent<string>).detail);
    window.addEventListener("lb-captured", on);
    const off = onFileDrop((ps) => load(ps[0]));
    return () => (window.removeEventListener("lb-captured", on), off());
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, []);

  const read = async () => {
    if (!path || !preview) return;
    const crop = sel && sel.w > 4 && sel.h > 4 ? [sel.x, sel.y, sel.w, sel.h].map(Math.round) : null;
    const r = await act.run(() => api.post<{ text: string }>("/ocr/image", { path, crop }));
    if (r) setText(r.text);
  };

  return (
    <Page title="화면 글자 인식" desc={`화면을 캡처하거나 사진을 넣고, 읽을 부분을 끌어서 고르세요.${isTauri ? " 어디서든 Ctrl+Shift+O로 바로 캡처해
      <NeedsComponent keys={["ocr"]} />
      <div className="row">
        {isTauri && (
          <Button icon={<Camera size={16} />} onClick={async () => { const p = await act.run(captureScreen); if (p) load(p); }}>
            화면 캡처
          </Button>
        )}
        <Button icon={<ImageUp size={16} />} onClick={async () => { const p = (await pickFiles({ filters: [{ name: "이미지", extensions: ["png", "
          이미지 열기
        </Button>
        <Button variant="primary" icon={<ScanText size={16} />} busy={act.busy} disabled={!preview || !has("ocr")} onClick={read}>
          {sel ? "고른 부분 읽기" : "전체 읽기"}
        </Button>
      </div>
      <ErrorBox error={act.error} />
```

```
      <div className="grid-2 wide-left">
        <Card>
          {preview ? <Selector preview={preview} sel={sel} onSel={setSel} /> : <Empty icon={<ScanText size={28} />}>캡처하거나 이미지를 넣어 주세요.
        </Card>
        <Card title="읽은 글자" right={text ? <CopyButton text={text} /> : null}>
          <textarea rows={16} value={text} onChange={(e) => setText(e.target.value)} placeholder="여기에 결과가 나와요." />
          {text && (
            <div className="row wrap">
              <Button small onClick={() => (sessionStorage.setItem("lb-paste", text), navigate("translate"))}>번역하기</Button>
              <Button small onClick={() => (sessionStorage.setItem("lb-paste", text), navigate("papers"))}>영수증 · 명함으로</Button>
            </div>
          )}
        </Card>
      </div>
    </Page>
  );
}

function Selector({ preview, sel, onSel }: { preview: Preview; sel: Rect | null; onSel: (r: Rect | null) => void }) {
  const box = useRef<HTMLDivElement>(null);
  const start = useRef<{ x: number; y: number } | null>(null);
  const [live, setLive] = useState<Rect | null>(null);
  const toImage = (e: React.PointerEvent) => {
    const r = box.current!.getBoundingClientRect();
    const sx = preview.width / r.width;
    const sy = preview.height / r.height;
    return { x: Math.max(0, Math.min(r.width, e.clientX - r.left)) * sx, y: Math.max(0, Math.min(r.height, e.clientY - r.top)) * sy };
  };
  const rect = (a: { x: number; y: number }, b: { x: number; y: number }): Rect => ({ x: Math.min(a.x, b.x), y: Math.min(a.y, b.y), w: Math.abs
  const shown = live ?? sel;
  return (
    <div
      ref={box}
      className="selector"
      onPointerDown={(e) => {
        (e.target as HTMLElement).setPointerCapture(e.pointerId);
        start.current = toImage(e);
        setLive(null);
      }}
      onPointerMove={(e) => start.current && setLive(rect(start.current, toImage(e)))}
      onPointerUp={(e) => {
        if (start.current) {
          const r = rect(start.current, toImage(e));
          onSel(r.w > 4 && r.h > 4 ? r : null);
        }
        start.current = null;
        setLive(null);
      }}
    >
      <img src={preview.data_url} alt="미리보기" draggable={false} />
      {shown && (
        <div
          className="selection"
          style={{ left: `${(shown.x / preview.width) * 100}%`, top: `${(shown.y / preview.height) * 100}%`, width: `${(shown.w / preview.width
        />
      )}
    </div>
  );
}
```

```tsx
import { Contact, FileSpreadsheet, ImageUp, Receipt, Trash2 } from "lucide-react";
import { useState } from "react";
import { useSystem } from "../components/system";
import { Button, Card, Empty, ErrorBox, Field, Notice, Page, PathLink, Tabs } from "../components/ui";
import { api } from "../lib/api";
import { localDate, won } from "../lib/format";
import { useAction, useData } from "../lib/hooks";
import { pickFiles, pickSavePath } from "../lib/platform";

type Expense = { id: number; date: string | null; merchant: string; amount: number | null; category: string; memo: string; biz_no: string };
type ContactT = { id: number; name: string; company: string; title: string; mobile: string; phone: string; fax: string; email: string; address:
const IMG = [{ name: "이미지", extensions: ["png", "jpg", "jpeg", "webp", "bmp", "heic", "tif", "tiff"] }];
const CATEGORIES = ["식비", "교통", "사무용품", "통신", "숙박", "마트", "기타"];

export default function Papers() {
  const [tab, setTab] = useState<"receipt" | "card">(() => {
    const hit = sessionStorage.getItem("lb-open");
    return hit && JSON.parse(hit).kind === "contact" ? (sessionStorage.removeItem("lb-open"), "card") : "receipt";
  });
  return (
    <Page title="영수증 · 명함" desc="사진을 넣으면 글자를 읽어 칸을 채워요. 영수증은 경비 엑셀로, 명함은 휴대폰 연락처(.vcf)로 내보내요.">
      <Tabs tabs={[{ key: "receipt", label: "영수증 → 경비" }, { key: "card", label: "명함 → 연락처" }]} value={tab} onChange={setTab} />
      {tab === "receipt" ? <Receipts /> : <Cards />}
    </Page>
  );
}

function useParse<T>(endpoint: string, empty: T) {
  const [fields, setFields] = useState<T>(empty);
  const [text, setText] = useState(() => {
    const v = sessionStorage.getItem("lb-paste") ?? "";
    sessionStorage.removeItem("lb-paste");
    return v;
  });
  const act = useAction();
  const { has } = useSystem();
  const fromImage = async () => {
    const p = (await pickFiles({ filters: IMG }))[0];
    if (!p) return;
    const r = await act.run(() => api.post<{ text: string; fields: T }>(endpoint, { path: p }));
    if (r) (setText(r.text), setFields(r.fields));
  };
  const fromText = async () => {
    const r = await act.run(() => api.post<{ text: string; fields: T }>(endpoint, { text }));
    if (r) setFields(r.fields);
  };
  return { fields, setFields, text, setText, act, fromImage, fromText, ocr: has("ocr") };
}

function Receipts() {
  const p = useParse<Omit<Expense, "id">>("/receipts/parse", { date: "", merchant: "", amount: null, category: "", memo: "", biz_no: "" });
  const month = localDate().slice(0, 7);
  const [m, setM] = useState(month);
  const list = useData(() => api.get<Expense[]>("/expenses", { month: m }), [m]);
  const [saved, setSaved] = useState<string | null>(null);
  const f = p.fields;
  const add = async () => {
    await p.act.run(() => api.post("/expenses", { ...f, date: f.date || null }));
    p.setFields({ date: "", merchant: "", amount: null, category: "", memo: "", biz_no: "" });
    p.setText("");
    list.reload();
  };
```

```
  const exportXlsx = async () => {
    const out = await pickSavePath(`경비_${m}.xlsx`, [{ name: "엑셀", extensions: ["xlsx"] }]);
    const r = await p.act.run(() => api.post<{ path: string }>("/expenses/export", { month: m, out_path: out }));
    if (r) setSaved(r.path);
  };
  const total = (list.data ?? []).reduce((s, e) => s + (e.amount ?? 0), 0);
  return (
    <div className="grid-2">
      <Card title="영수증 읽기">
        <div className="row">
          <Button icon={<ImageUp size={16} />} disabled={!p.ocr} busy={p.act.busy} onClick={p.fromImage}>영수증 사진</Button>
          {!p.ocr && <span className="muted small">사진 읽기는 OCR 구성요소가 필요해요</span>}
        </div>
        <Field label="또는 글자 붙여넣기">
          <textarea rows={4} value={p.text} onChange={(e) => p.setText(e.target.value)} />
        </Field>
        <Button small onClick={p.fromText} disabled={!p.text.trim()}>글에서 찾기</Button>
        <div className="form-grid">
          <Field label="날짜"><input type="date" value={f.date ?? ""} onChange={(e) => p.setFields({ ...f, date: e.target.value })} /></Field>
          <Field label="가맹점"><input value={f.merchant} onChange={(e) => p.setFields({ ...f, merchant: e.target.value })} /></Field>
          <Field label="금액"><input type="number" value={f.amount ?? ""} onChange={(e) => p.setFields({ ...f, amount: e.target.value ? +e.target
          <Field label="분류">
            <select value={f.category} onChange={(e) => p.setFields({ ...f, category: e.target.value })}>
              <option value="">선택</option>
              {CATEGORIES.map((c) => <option key={c}>{c}</option>)}
            </select>
          </Field>
          <Field label="사업자번호"><input value={f.biz_no} onChange={(e) => p.setFields({ ...f, biz_no: e.target.value })} /></Field>
          <Field label="메모"><input value={f.memo} onChange={(e) => p.setFields({ ...f, memo: e.target.value })} /></Field>
        </div>
        <Button variant="primary" icon={<Receipt size={16} />} disabled={!f.amount} onClick={add}>장부에 넣기</Button>
        <ErrorBox error={p.act.error} />
      </Card>
      <Card title={`경비 장부 · ${won(total)}`} right={<div className="row"><input type="month" value={m} onChange={(e) => setM(e.target.value)} />
        {saved && <Notice tone="ok">저장했어요: <PathLink path={saved} /></Notice>}
        {list.data && !list.data.length && <Empty>이번 달 경비가 없어요.</Empty>}
        <table className="table">
          <tbody>
            {list.data?.map((e) => (
              <tr key={e.id}>
                <td className="muted">{e.date?.slice(5)}</td>
                <td>{e.merchant}</td>
                <td className="muted">{e.category}</td>
                <td className="num">{won(e.amount)}</td>
                <td><button className="iconbtn" aria-label="삭제" onClick={async () => (await api.del(`/expenses/${e.id}`), list.reload())}><Tras
              </tr>
            ))}
          </tbody>
        </table>
      </Card>
    </div>
  );
}

const EMPTY_CARD = { name: "", company: "", title: "", mobile: "", phone: "", fax: "", email: "", address: "", website: "" };
const CARD_FIELDS: [keyof typeof EMPTY_CARD, string][] = [
  ["name", "이름"], ["company", "회사"], ["title", "직함"], ["mobile", "휴대폰"], ["phone", "전화"], ["fax", "팩스"], ["email", "이메일"], ["website
];

function Cards() {
  const p = useParse<typeof EMPTY_CARD>("/cards/parse", EMPTY_CARD);
  const [q, setQ] = useState("");
```

```
  const list = useData(() => api.get<ContactT[]>("/contacts", { q }), [q]);
  const [saved, setSaved] = useState<string | null>(null);
  const add = async () => {
    await p.act.run(() => api.post("/contacts", p.fields));
    p.setFields(EMPTY_CARD);
    p.setText("");
    list.reload();
  };
  const exportVcf = async () => {
    const out = await pickSavePath("연락처.vcf", [{ name: "연락처", extensions: ["vcf"] }]);
    const r = await p.act.run(() => api.post<{ path: string }>("/contacts/export", { out_path: out }));
    if (r) setSaved(r.path);
  };
  return (
    <div className="grid-2">
      <Card title="명함 읽기">
        <div className="row">
          <Button icon={<ImageUp size={16} />} disabled={!p.ocr} busy={p.act.busy} onClick={p.fromImage}>명함 사진</Button>
          {!p.ocr && <span className="muted small">사진 읽기는 OCR 구성요소가 필요해요</span>}
        </div>
        <Field label="또는 글자 붙여넣기">
          <textarea rows={4} value={p.text} onChange={(e) => p.setText(e.target.value)} />
        </Field>
        <Button small onClick={p.fromText} disabled={!p.text.trim()}>글에서 찾기</Button>
        <div className="form-grid">
          {CARD_FIELDS.map(([k, label]) => (
            <Field key={k} label={label}>
              <input value={p.fields[k]} onChange={(e) => p.setFields({ ...p.fields, [k]: e.target.value })} />
            </Field>
          ))}
        </div>
        <Button variant="primary" icon={<Contact size={16} />} disabled={!p.fields.name && !p.fields.company} onClick={add}>연락처에 넣기</Button>
        <ErrorBox error={p.act.error} />
      </Card>
      <Card title="연락처" right={<Button small onClick={exportVcf}>vCard로 내보내기</Button>}>
        <input value={q} onChange={(e) => setQ(e.target.value)} placeholder="이름 · 회사 · 이메일로 찾기" />
        {saved && <Notice tone="ok">저장했어요: <PathLink path={saved} /> · 휴대폰으로 보내 열면 연락처에 들어가요.</Notice>}
        {list.data && !list.data.length && <Empty>저장된 연락처가 없어요.</Empty>}
        <ul className="list">
          {list.data?.map((c) => (
            <li key={c.id} className="list-row">
              <div className="grow">
                <b>{c.name}</b> <span className="muted">{[c.title, c.company].filter(Boolean).join(" · ")}</span>
                <div className="muted small">{[c.mobile || c.phone, c.email].filter(Boolean).join(" · ")}</div>
              </div>
              <button className="iconbtn" aria-label="삭제" onClick={async () => (await api.del(`/contacts/${c.id}`), list.reload())}><Trash2 siz
            </li>
          ))}
        </ul>
      </Card>
    </div>
  );
}
```

```tsx
import { useState } from "react";
import { Button, Card, ErrorBox, Field, Notice, Page, PathLink, PathPicker, Tabs } from "../components/ui";
import { api } from "../lib/api";
import { bytes } from "../lib/format";
import { useAction } from "../lib/hooks";

const PDF = [{ name: "PDF", extensions: ["pdf"] }];
const IMG = [{ name: "이미지", extensions: ["png", "jpg", "jpeg", "webp", "bmp", "tif", "tiff"] }];
type Tab = "merge" | "split" | "compress" | "rotate" | "images" | "toimages";

export default function Pdf() {
  const [tab, setTab] = useState<Tab>("merge");
  const [paths, setPaths] = useState<string[]>([]);
  const [ranges, setRanges] = useState("");
  const [every, setEvery] = useState(1);
  const [splitMode, setSplitMode] = useState<"ranges" | "every">("ranges");
  const [quality, setQuality] = useState(60);
  const [degrees, setDegrees] = useState(90);
  const [pages, setPages] = useState("");
  const [a4, setA4] = useState(true);
  const [out, setOut] = useState<{ files: string[]; note?: string } | null>(null);
  const act = useAction();

  const switchTab = (t: Tab) => (setTab(t), setPaths([]), setOut(null), act.setError(null));
  const run = async () => {
    setOut(null);
    const p = paths[0];
    const r = await act.run(async () => {
      switch (tab) {
        case "merge": {
          const x = await api.post<{ dst: string; pages: number }>("/pdf/merge", { paths });
          return { files: [x.dst], note: `${x.pages}쪽으로 합쳤어요.` };
        }
        case "split": {
          const x = await api.post<{ outputs: string[] }>("/pdf/split", splitMode === "every" ? { path: p, every } : { path: p, ranges });
          return { files: x.outputs, note: `${x.outputs.length}개로 나눴어요.` };
        }
        case "compress": {
          const x = await api.post<{ dst: string; before: number; after: number; saved_ratio: number }>("/pdf/compress", { path: p, quality });
          return { files: [x.dst], note: `${bytes(x.before)} → ${bytes(x.after)} (${Math.round(x.saved_ratio * 100)}% 줄임)` };
        }
        case "rotate": {
          const x = await api.post<{ dst: string; rotated: number }>("/pdf/rotate", { path: p, degrees, pages: pages || null });
          return { files: [x.dst], note: `${x.rotated}쪽을 돌렸어요.` };
        }
        case "images": {
          const x = await api.post<{ dst: string; pages: number }>("/pdf/from-images", { paths, page: a4 ? "a4" : "fit" });
          return { files: [x.dst], note: `이미지 ${x.pages}장을 PDF로 묶었어요.` };
        }
        case "toimages": {
          const x = await api.post<{ outputs: string[] }>("/pdf/to-images", { path: p, dpi: 150, format: "png" });
          return { files: x.outputs, note: `${x.outputs.length}장으로 바꿨어요.` };
        }
      }
    });
    if (r) setOut(r);
  };
  const multi = tab === "merge" || tab === "images";
  const need = tab === "merge" ? 2 : 1;
  return (
    <Page title="PDF 도구" desc="합치기, 나누기, 용량 줄이기, 돌리기, 이미지↔PDF. 파일은 PC 밖으로 나가지 않아요.">
      <Tabs
```

```
      tabs={[
        { key: "merge", label: "합치기" },
        { key: "split", label: "나누기" },
        { key: "compress", label: "용량 줄이기" },
        { key: "rotate", label: "돌리기" },
        { key: "images", label: "이미지 → PDF" },
        { key: "toimages", label: "PDF → 이미지" },
      ]}
      value={tab}
      onChange={switchTab}
    />
    <div className="grid-2 wide-left">
      <Card>
        <PathPicker value={paths} onChange={setPaths} multiple={multi} filters={tab === "images" ? IMG : PDF} placeholder={multi ? "파일을 순서대로
        {multi && paths.length > 1 && <p className="muted small">넣은 순서대로 합쳐요. 순서를 바꾸려면 빼고 다시 넣어 주세요.</p>}
        <Button variant="primary" busy={act.busy} disabled={paths.length < need || (tab === "split" && splitMode === "ranges" && !ranges.trim
          실행
        </Button>
        <ErrorBox error={act.error} />
        {out && (
          <Notice tone="ok">
            <div>
              {out.note}
              <ul className="list compact">
                {out.files.slice(0, 20).map((f) => <li key={f}><PathLink path={f} /></li>)}
              </ul>
            </div>
          </Notice>
        )}
      </Card>
      <Card title="옵션">
        {tab === "split" && (
          <>
            <Tabs tabs={[{ key: "ranges", label: "범위로" }, { key: "every", label: "몇 쪽씩" }]} value={splitMode} onChange={setSplitMode} />
            {splitMode === "ranges" ? (
              <Field label="쪽 범위" hint="예: 1-3, 5, 7- (7쪽부터 끝까지). 범위마다 파일 하나가 생겨요.">
                <input value={ranges} onChange={(e) => setRanges(e.target.value)} placeholder="1-3, 5, 7-" />
              </Field>
            ) : (
              <Field label="몇 쪽씩"><input type="number" min={1} value={every} onChange={(e) => setEvery(+e.target.value)} /></Field>
            )}
          </>
        )}
        {tab === "compress" && (
          <Field label={`이미지 화질 ${quality}`} hint="낮을수록 용량이 줄고 그림이 흐려져요. 글자는 그대로예요.">
            <input type="range" min={20} max={90} value={quality} onChange={(e) => setQuality(+e.target.value)} />
          </Field>
        )}
        {tab === "rotate" && (
          <>
            <Field label="방향">
              <select value={degrees} onChange={(e) => setDegrees(+e.target.value)}>
                <option value={90}>오른쪽으로 90°</option>
                <option value={180}>180°</option>
                <option value={270}>왼쪽으로 90°</option>
              </select>
            </Field>
            <Field label="쪽 (비우면 전체)"><input value={pages} onChange={(e) => setPages(e.target.value)} placeholder="예: 2, 4-5" /></Field>
          </>
        )}
        {tab === "images" && (
          <label className="check-label"><input type="checkbox" checked={a4} onChange={(e) => setA4(e.target.checked)} /> A4 크기에 맞추기</label>
```

```
125            )}
126            {(tab === "merge" || tab === "toimages") && <p className="muted small">따로 정할 옵션이 없어요.</p>}
127          </Card>
128        </div>
129      </Page>
130    );
131  }
```

```tsx
import { FilePen, Undo2 } from "lucide-react";
import { useEffect, useState } from "react";
import { Button, Card, ErrorBox, Field, Notice, Page, PathPicker } from "../components/ui";
import { api } from "../lib/api";
import { baseName, dateTime } from "../lib/format";
import { useAction, useData } from "../lib/hooks";

type Item = { src: string; dst: string; changed: boolean; conflict: string | null };
type Hist = { id: number; created_at: string; undone: boolean; count: number };

const TOKENS = [
  ["{name}", "원래 이름"],
  ["{n:03}", "번호 001"],
  ["{date:%Y%m%d}", "수정한 날"],
  ["{exif:%Y-%m-%d}", "사진 찍은 날"],
  ["{today:%Y%m%d}", "오늘"],
  ["{parent}", "폴더 이름"],
];

export default function Rename() {
  const [paths, setPaths] = useState<string[]>([]);
  const [rules, setRules] = useState({ template: "{name}", find: "", replace: "", regex: false, case: "none", spaces: "keep", start: 1, step: 1
  const [items, setItems] = useState<Item[]>([]);
  const [done, setDone] = useState<string | null>(null);
  const act = useAction();
  const hist = useData(() => api.get<Hist[]>("/rename/history"));

  useEffect(() => {
    if (!paths.length) return setItems([]);
    const t = setTimeout(async () => {
      const r = await act.run(() => api.post<{ items: Item[] }>("/rename/plan", { paths, rules }));
      if (r) setItems(r.items);
    }, 200);
    return () => clearTimeout(t);
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [paths, rules]);

  const apply = async () => {
    const r = await act.run(() => api.post<{ renamed: number }>("/rename/apply", { items }));
    if (r) {
      setDone(`${r.renamed}개 이름을 바꿨어요.`);
      setPaths(items.map((i) => i.dst));
      hist.reload();
    }
  };
  const conflicts = items.filter((i) => i.conflict).length;
  const changed = items.filter((i) => i.changed).length;
  return (
    <Page title="이름 일괄 변경" desc="규칙을 정하면 바뀔 이름을 먼저 보여줘요. 잘못 바꿔도 되돌릴 수 있어요.">
      <div className="grid-2 wide-left">
        <Card>
          <PathPicker value={paths} onChange={(p) => (setPaths(p), setDone(null))} multiple placeholder="파일을 고르거나 끌어다 놓으세요" />
          {items.length > 0 && (
            <table className="table">
              <thead>
                <tr><th>지금 이름</th><th>바뀔 이름</th></tr>
              </thead>
              <tbody>
                {items.map((i) => (
                  <tr key={i.src} className={i.conflict ? "row-error" : ""}>
                    <td className="muted">{baseName(i.src)}</td>
                    <td>{baseName(i.dst)} {i.conflict && <span className="error-text small">· {i.conflict}</span>}</td>
```

```
                </tr>
              ))}
            </tbody>
          </table>
        )}
        <div className="row">
          <Button variant="primary" icon={<FilePen size={16} />} busy={act.busy} disabled={!changed || conflicts > 0} onClick={apply}>
            {changed}개 바꾸기
          </Button>
          {conflicts > 0 && <span className="error-text small">겹치는 이름 {conflicts}개를 먼저 해결해 주세요.</span>}
        </div>
        <ErrorBox error={act.error} />
        {done && <Notice tone="ok">{done}</Notice>}
      </Card>
      <div className="stack">
        <Card title="규칙">
          <Field label="새 이름 틀">
            <input value={rules.template} onChange={(e) => setRules({ ...rules, template: e.target.value })} />
          </Field>
          <div className="row wrap">
            {TOKENS.map(([t, label]) => (
              <button key={t} className="chip" onClick={() => setRules({ ...rules, template: rules.template + t })} title={t}>
                + {label}
              </button>
            ))}
          </div>
          <div className="form-grid">
            <Field label="찾을 말"><input value={rules.find} onChange={(e) => setRules({ ...rules, find: e.target.value })} /></Field>
            <Field label="바꿀 말"><input value={rules.replace} onChange={(e) => setRules({ ...rules, replace: e.target.value })} /></Field>
            <Field label="빈칸">
              <select value={rules.spaces} onChange={(e) => setRules({ ...rules, spaces: e.target.value })}>
                <option value="keep">그대로</option>
                <option value="underscore">_ 로</option>
                <option value="dash">- 로</option>
                <option value="remove">없애기</option>
              </select>
            </Field>
            <Field label="대소문자">
              <select value={rules.case} onChange={(e) => setRules({ ...rules, case: e.target.value })}>
                <option value="none">그대로</option>
                <option value="lower">소문자</option>
                <option value="upper">대문자</option>
              </select>
            </Field>
            <Field label="번호 순서">
              <select value={rules.sort} onChange={(e) => setRules({ ...rules, sort: e.target.value })}>
                <option value="name">이름순</option>
                <option value="mtime">수정한 날순</option>
                <option value="size">크기순</option>
              </select>
            </Field>
            <Field label="시작 번호"><input type="number" value={rules.start} onChange={(e) => setRules({ ...rules, start: +e.target.value })} />
          </div>
          <label className="check-label">
            <input type="checkbox" checked={rules.regex} onChange={(e) => setRules({ ...rules, regex: e.target.checked })} /> 정규식으로 찾기
          </label>
        </Card>
        <Card title="되돌리기">
          {hist.data?.length ? (
            <ul className="list compact">
              {hist.data.map((h) => (
                <li key={h.id} className="list-row">
```

```
                    <span className="grow">{dateTime(h.created_at)} · {h.count}개</span>
                    {h.undone ? (
                      <span className="muted small">되돌림</span>
                    ) : (
                      <Button small icon={<Undo2 size={14} />} onClick={async () => { await act.run(() => api.post(`/rename/undo/${h.id}`)); hi
                        되돌리기
                      </Button>
                    )}
                  </li>
                ))}
              </ul>
            ) : (
              <p className="muted small">아직 바꾼 기록이 없어요.</p>
            )}
          </Card>
        </div>
      </div>
    </Page>
  );
}
```

```tsx
import { CalendarPlus, X } from "lucide-react";
import { useState } from "react";
import { Button, Card, Empty, ErrorBox, Field, Notice, Page, PathLink } from "../components/ui";
import { api } from "../lib/api";
import { useAction } from "../lib/hooks";
import { isTauri, pickSavePath } from "../lib/platform";

type Ev = { title: string; start: string; end: string | null; all_day: boolean; source: string };

const SAMPLE = `내일 오후 3시에 디자인 회의
다음 주 화요일 10시 반 고객 미팅
11월 3일(화) 2시~4시 워크숍
금요일까지 보고서 제출`;

export default function Schedule() {
  const [text, setText] = useState("");
  const [events, setEvents] = useState<Ev[] | null>(null);
  const [saved, setSaved] = useState<string | null>(null);
  const act = useAction();
  const parse = async () => {
    setSaved(null);
    const r = await act.run(() => api.post<{ events: Ev[] }>("/schedule/parse", { text }));
    if (r) setEvents(r.events);
  };
  const exportIcs = async () => {
    const out = await pickSavePath("일정.ics", [{ name: "캘린더", extensions: ["ics"] }]);
    if (isTauri && !out) return;
    const r = await act.run(() => api.post<{ path: string }>("/schedule/ics", { events, out_path: out }));
    if (r) setSaved(r.path);
  };
  const update = (i: number, patch: Partial<Ev>) => setEvents((xs) => xs!.map((x, j) => (j === i ? { ...x, ...patch } : x)));
  return (
    <Page title="일정 뽑기" desc="'다음 주 화요일 3시', '11월 3일 2시~4시' 같은 말을 찾아 캘린더 파일(.ics)로 만들어요. 구글 · 아웃룩 · 애플 캘린더에
      <div className="grid-2">
        <Card>
          <Field label="글">
            <textarea rows={12} value={text} onChange={(e) => setText(e.target.value)} placeholder={SAMPLE} />
          </Field>
          <div className="row">
            <Button variant="primary" icon={<CalendarPlus size={16} />} busy={act.busy} disabled={!text.trim()} onClick={parse}>
              일정 찾기
            </Button>
            {!text && <Button variant="ghost" onClick={() => setText(SAMPLE)}>예시 넣기</Button>}
          </div>
          <ErrorBox error={act.error} />
        </Card>
        <Card title={events ? `찾은 일정 ${events.length}개` : "결과"} right={events?.length ? <Button small variant="primary" onClick={exportIcs}>.
          {!events && <Empty>왼쪽에 글을 넣고 '일정 찾기'를 눌러 주세요.</Empty>}
          {events && !events.length && <Empty>날짜나 시간이 들어간 문장이 없어요.</Empty>}
          {saved && <Notice tone="ok">저장했어요: <PathLink path={saved} /> · 파일을 열면 캘린더 앱에 추가돼요.</Notice>}
          <div className="stack">
            {events?.map((e, i) => (
              <div key={i} className="edit-row">
                <input className="grow" value={e.title} onChange={(x) => update(i, { title: x.target.value })} />
                {e.all_day ? (
                  <input className="w-md" type="date" value={e.start.slice(0, 10)} onChange={(x) => update(i, { start: x.target.value })} />
                ) : (
                  <input className="w-lg" type="datetime-local" value={e.start.slice(0, 16)} onChange={(x) => update(i, { start: x.target.value
                )}
                <button className="iconbtn" aria-label="빼기" onClick={() => setEvents(events.filter((_, j) => j !== i))}>
                  <X size={14} />
                </button>
```

```
              </div>
            ))}
          </div>
        </Card>
      </div>
    </Page>
  );
}
```

```tsx
import { FolderSearch, RotateCw, Search as SearchIcon } from "lucide-react";
import { useEffect, useState } from "react";
import { useSystem } from "../components/system";
import { Badge, Button, Card, Empty, JobBar, Page, PathLink, PathPicker, Tabs } from "../components/ui";
import { api } from "../lib/api";
import { dateTime } from "../lib/format";
import { useData, useJob } from "../lib/hooks";

type Hit = { kind: string; kind_label: string; ref: string; title: string; snippet: string; marks: [number, number][]; updated: string };
const KIND_PAGE: Record<string, string> = { note: "notes", todo: "notes", clip: "clipboard", transcript: "transcribe", contact: "papers" };
const KINDS = [
  { key: "", label: "전체" },
  { key: "transcript", label: "회의록" },
  { key: "note", label: "메모" },
  { key: "todo", label: "할 일" },
  { key: "clip", label: "클립보드" },
  { key: "file", label: "문서" },
  { key: "contact", label: "연락처" },
];

function Highlight({ text, marks }: { text: string; marks: [number, number][] }) {
  const out = [];
  let pos = 0;
  for (const [s, e] of marks) {
    if (s < pos) continue;
    out.push(text.slice(pos, s));
    out.push(<mark key={s}>{text.slice(s, e)}</mark>);
    pos = e;
  }
  out.push(text.slice(pos));
  return <>{out}</>;
}

export default function Search() {
  const [q, setQ] = useState("");
  const [kind, setKind] = useState("");
  const [hits, setHits] = useState<Hit[] | null>(null);
  const { navigate } = useSystem();
  const folders = useData(() => api.get<{ folders: string[]; files: number }>("/search/folders"));
  const [dir, setDir] = useState<string[]>([]);
  const job = useJob<{ indexed: number; failed: number; total: number }>();

  useEffect(() => {
    if (!q.trim()) return setHits(null);
    const t = setTimeout(() => api.get<{ results: Hit[] }>("/search", { q, kinds: kind }).then((r) => setHits(r.results)).catch(() => setHits([
    return () => clearTimeout(t);
  }, [q, kind]);

  const open = (h: Hit) => {
    if (h.kind === "file") return;
    sessionStorage.setItem("lb-open", JSON.stringify(h));
    navigate(KIND_PAGE[h.kind]);
  };
  return (
    <Page title="통합 검색" desc="회의록, 메모, 할 일, 클립보드, 연락처, 그리고 등록한 폴더의 문서(HWP · HWPX · DOCX · PDF · TXT) 속 글자까지 한 번에 찾
      <div className="search-box">
        <SearchIcon size={18} />
        <input autoFocus value={q} onChange={(e) => setQ(e.target.value)} placeholder="찾을 말 (두 글자 이상)" />
      </div>
      <Tabs tabs={KINDS} value={kind} onChange={setKind} />
      <div className="grid-2 wide-left">
        <Card>
```

```
63          {!hits && <Empty icon={<SearchIcon size={28} />}>무엇이든 찾아보세요. 어디서든 Ctrl+K로 바로 찾을 수 있어요.</Empty>}
64          {hits && !hits.length && <Empty>결과가 없어요.</Empty>}
65          <ul className="list">
66            {hits?.map((h) => (
67              <li key={`${h.kind}-${h.ref}`} className={`list-row ${h.kind !== "file" ? "clickable" : ""}`} onClick={() => open(h)}>
68                <div className="grow">
69                  <div className="row">
70                    <Badge>{h.kind_label}</Badge>
71                    {h.kind === "file" ? <PathLink path={h.ref} label={h.title} /> : <b>{h.title}</b>}
72                  </div>
73                  <div className="muted small snippet"><Highlight text={h.snippet} marks={h.marks} /></div>
74                </div>
75                <span className="muted small">{dateTime(h.updated)}</span>
76              </li>
77            ))}
78          </ul>
79        </Card>
80        <Card title="문서 폴더 등록" right={folders.data?.folders.length ? <Button small icon={<RotateCw size={14} />} onClick={async () => { await
81          <p className="muted small">폴더를 등록하면 안에 있는 문서의 글자를 읽어 두었다가 검색에 같이 보여줘요.</p>
82          <PathPicker value={dir} onChange={setDir} directory placeholder="문서 폴더를 고르세요" />
83          <Button icon={<FolderSearch size={16} />} busy={job.running} disabled={!dir.length} onClick={async () => { await job.start(() => api.
84            등록하고 읽기
85          </Button>
86          <JobBar job={job.job} error={job.error} onCancel={job.cancel} />
87          {job.job?.status === "done" && job.job.result && <p className="small">문서 {job.job.result.indexed}개를 읽었어요{job.job.result.failed ? `
88          <ul className="list compact">
89            {folders.data?.folders.map((f) => <li key={f} className="list-row"><PathLink path={f} label={f} /></li>)}
90          </ul>
91          {folders.data ? <p className="muted small">색인한 문서 {folders.data.files}개</p> : null}
92        </Card>
93      </div>
94    </Page>
95  );
96 }
```

```tsx
import { CircleCheck, CircleDashed, KeyRound } from "lucide-react";
import { useEffect, useState } from "react";
import { useSystem } from "../components/system";
import { Badge, Button, Card, ErrorBox, Field, Notice, Page } from "../components/ui";
import { api } from "../lib/api";
import { useAction, useData } from "../lib/hooks";
import { isTauri, openPath } from "../lib/platform";

async function openNotices() {
  const { resourceDir, join } = await import("@tauri-apps/api/path");
  await openPath(await join(await resourceDir(), "THIRD_PARTY_NOTICES.txt"));
}

export default function Settings() {
  const { status, reload } = useSystem();
  const settings = useData(() => api.get<Record<string, unknown>>("/system/settings"));
  const [form, setForm] = useState({ user_name: "", company: "", focus_minutes: 25, break_minutes: 5 });
  const [key, setKey] = useState("");
  const [agree, setAgree] = useState(false);
  const act = useAction();
  const [saved, setSaved] = useState(false);
  useEffect(() => {
    if (settings.data) setForm((f) => ({ ...f, ...(settings.data as Partial<typeof f>) }));
  }, [settings.data]);
  const lic = status?.license;

  const activate = async () => {
    const r = await act.run(() => api.post("/license", { key }));
    if (r) {
      setKey("");
      reload();
    }
  };
  return (
    <Page title="설정">
      <div className="grid-2">
        <div className="stack">
          <Card title={<><KeyRound size={16} /> 라이선스</>}>
            {lic?.status === "licensed" && <Notice tone="ok">정품 인증됨 · {lic.holder} ({lic.id})</Notice>}
            {lic?.status === "trial" && <Notice>체험판이에요. {lic.days_left}일 남았어요 ({lic.trial_ends}까지). 체험 기간에는 모든 기능을 쓸 수 있어
            {lic?.status === "expired" && <Notice tone="warn">체험 기간이 끝났어요. 회의록 · 파일 마스킹 · 이미지/영상은 라이선스가 필요해요. 나머지 ㅂ
            {lic?.status !== "licensed" && (
              <>
                <Field label="라이선스 키" hint="인터넷 없이 입력만으로 인증돼요.">
                  <textarea rows={3} value={key} onChange={(e) => setKey(e.target.value)} placeholder="LB1.xxxxx.xxxxx" />
                </Field>
                <label className="check-label small">
                  <input type="checkbox" checked={agree} onChange={(e) => setAgree(e.target.checked)} />
                  키를 등록하면 디지털 콘텐츠 제공이 시작되어 청약철회가 제한된다는 점을 확인했어요 (전자상거래법 제17조 제2항 제5호).
                </label>
                <Button variant="primary" busy={act.busy} disabled={!key.trim() || !agree} onClick={activate}>인증하기</Button>
                <ErrorBox error={act.error} />
              </>
            )}
          </Card>
          <Card title="내 정보와 기본값">
            <div className="form-grid">
              <Field label="이름 (메일 서명)"><input value={form.user_name} onChange={(e) => setForm({ ...form, user_name: e.target.value })} /></Fi
              <Field label="회사"><input value={form.company} onChange={(e) => setForm({ ...form, company: e.target.value })} /></Field>
              <Field label="집중 시간(분)"><input type="number" value={form.focus_minutes} onChange={(e) => setForm({ ...form, focus_minutes: +e.tar
              <Field label="쉬는 시간(분)"><input type="number" value={form.break_minutes} onChange={(e) => setForm({ ...form, break_minutes: +e.tar
            </div>
```

```
            <div className="row">
              <Button variant="primary" onClick={async () => { await api.put("/system/settings", form); setSaved(true); setTimeout(() => setSav
              {saved && <span className="small accent">저장했어요</span>}
            </div>
          </Card>
          {isTauri && (
            <Card title="단축키">
              <ul className="list compact">
                <li className="list-row"><kbd>Ctrl Shift Space</kbd><span className="grow">어디서든 로컬비서 열고 찾기</span></li>
                <li className="list-row"><kbd>Ctrl Shift O</kbd><span className="grow">화면 캡처해서 글자 읽기</span></li>
                <li className="list-row"><kbd>Ctrl K</kbd><span className="grow">앱 안에서 기능 · 내용 찾기</span></li>
              </ul>
            </Card>
          )}
        </div>
        <div className="stack">
          <Card title="구성요소" right={<Button small variant="ghost" onClick={reload}>새로 확인</Button>}>
            <p className="muted small">무거운 AI 기능은 따로 설치돼요. 설치하지 않은 기능만 꺼져 있고 나머지는 그대로 써요.</p>
            <ul className="list compact">
              {status?.components.map((c) => (
                <li key={c.key} className="list-row">
                  {c.available ? <CircleCheck size={16} className="ok" /> : <CircleDashed size={16} className="muted" />}
                  <span className="grow">
                    <b>{c.label}</b> <span className="muted small">{c.used_by}</span>
                  </span>
                  {c.available ? <Badge tone="ok">사용 가능</Badge> : <code className="small">{`[${c.extra}]`}</code>}
                </li>
              ))}
              <li className="list-row">
                {status?.ffmpeg ? <CircleCheck size={16} className="ok" /> : <CircleDashed size={16} className="muted" />}
                <span className="grow"><b>ffmpeg</b> <span className="muted small">영상 무음 컷</span></span>
                {status?.ffmpeg && <Badge tone="ok">사용 가능</Badge>}
              </li>
            </ul>
            {status?.ocr_backend && <p className="muted small">OCR 엔진: {status.ocr_backend}</p>}
          </Card>
          <Card title="정보">
            <p className="small">로컬비서 v{status?.version} · {status?.os}</p>
            <p className="small muted">
              데이터 폴더: {status?.data_dir}{" "}
              {isTauri && status && <button className="linklike" onClick={() => openPath(status.data_dir)}>열기</button>}
            </p>
            <p className="small muted">모든 기록(메모, 회의록, 클립보드 등)은 이 폴더 안의 파일 하나에 저장돼요. 백업하려면 폴더째 복사하면 돼요.</p
            {isTauri && (
              <button className="linklike small" onClick={openNotices}>
                오픈소스 라이선스 고지
              </button>
            )}
          </Card>
        </div>
      </div>
    </Page>
  );
}
```

```tsx
import { CalendarPlus, Download, ListChecks, Mic, Sparkles, Trash2 } from "lucide-react";
import { useEffect, useState } from "react";
import { NeedsComponent, useSystem } from "../components/system";
import { Badge, Button, Card, Empty, ErrorBox, Field, JobBar, Page, PathLink, PathPicker, Tabs, Toggle, useToast } from "../components/ui";
import { api } from "../lib/api";
import { clock, dateTime } from "../lib/format";
import { useAction, useData, useJob } from "../lib/hooks";

type Segment = { start: number; end: number; text: string; speaker: number | null };
type Transcript = { id: number; title: string; source_path: string | null; duration: number | null; segments: Segment[]; speakers: Record<strin
type Item = { id: number; title: string; duration: number | null; created_at: string };
type ActionItem = { text: string; assignee: string | null; due: string | null };
type Ev = { title: string; start: string; end: string | null; all_day: boolean; source: string };
type Insights = { actions: ActionItem[]; events: Ev[] };
type Models = { available: boolean; diarize_available: boolean; sizes: { key: string; label: string }[]; installed: string[] };

const AUDIO = [{ name: "녹음 · 영상", extensions: ["mp3", "m4a", "wav", "aac", "flac", "ogg", "wma", "mp4", "mov", "mkv", "webm", "avi"] }];

export default function Transcribe() {
  const [tab, setTab] = useState<"new" | "list">("new");
  const [openId, setOpenId] = useState<number | null>(() => {
    const v = sessionStorage.getItem("lb-open-transcript");
    sessionStorage.removeItem("lb-open-transcript");
    const hit = sessionStorage.getItem("lb-open");
    if (hit && JSON.parse(hit).kind === "transcript") {
      sessionStorage.removeItem("lb-open");
      return Number(JSON.parse(hit).ref);
    }
    return v ? Number(v) : null;
  });
  if (openId) return <Detail id={openId} onBack={() => (setOpenId(null), setTab("list"))} />;
  return (
    <Page title="회의록" desc="녹음 파일을 넣으면 받아쓰기 → 요약 → 할 일 · 일정까지 한 번에 만들어요. 인터넷 없이 이 PC에서만 처리해요.">
      <Tabs tabs={[{ key: "new", label: "새로 만들기" }, { key: "list", label: "저장된 회의록" }]} value={tab} onChange={setTab} />
      {tab === "new" ? <NewTranscript onDone={setOpenId} /> : <List onOpen={setOpenId} />}
    </Page>
  );
}

function NewTranscript({ onDone }: { onDone: (id: number) => void }) {
  const [mode, setMode] = useState<"audio" | "text">("audio");
  const [paths, setPaths] = useState<string[]>([]);
  const [title, setTitle] = useState("");
  const [text, setText] = useState("");
  const [model, setModel] = useState("small");
  const [diarize, setDiarize] = useState(false);
  const [autoTodos, setAutoTodos] = useState(true);
  const models = useData(() => api.get<Models>("/transcribe/models"));
  const job = useJob<{ transcript_id: number; todos_added: number }>();
  const dl = useJob();
  const act = useAction();
  const toast = useToast();
  const { has } = useSystem();
  const needsDownload = models.data?.available && !models.data.installed.includes(model);

  const run = async () => {
    if (mode === "text") {
      const r = await act.run(() => api.post<{ transcript_id: number; todos_added: number }>("/transcribe/text", { title, text, auto_todos: aut
      if (r) {
        if (r.todos_added) toast(`할 일 ${r.todos_added}개를 등록했어요`);
        onDone(r.transcript_id);
      }
```

```
63      return;
64    }
65    const j = await job.start(() => api.post("/transcribe", { path: paths[0], title, model, diarize, auto_todos: autoTodos }));
66    if (j?.status === "done" && j.result) {
67      if (j.result.todos_added) toast(`할 일 ${j.result.todos_added}개를 등록했어요`);
68      onDone(j.result.transcript_id);
69    }
70  };
71
72  return (
73    <div className="grid-2 wide-left">
74      <Card>
75        <Tabs tabs={[{ key: "audio", label: "녹음 파일" }, { key: "text", label: "글 붙여넣기" }]} value={mode} onChange={setMode} />
76        {mode === "audio" ? (
77          <>
78            <NeedsComponent keys={["stt"]}>녹음 받아쓰기에는 음성 인식 구성요소가 필요해요. 글 붙여넣기는 지금도 돼요.</NeedsComponent>
79            <PathPicker value={paths} onChange={setPaths} filters={AUDIO} placeholder="녹음 · 영상 파일을 고르거나 끌어다 놓으세요 (mp3, m4a, wav, mp
80          </>
81        ) : (
82          <Field label="회의 내용">
83            <textarea rows={10} value={text} onChange={(e) => setText(e.target.value)} placeholder="회의 메모나 다른 곳에서 받아쓴 글을 붙여넣으면 요
84          </Field>
85        )}
86        <Field label="제목 (비우면 파일 이름)">
87          <input value={title} onChange={(e) => setTitle(e.target.value)} placeholder="예: 10월 마케팅 회의" />
88        </Field>
89        <div className="row">
90          <Button variant="primary" icon={<Mic size={16} />} busy={job.running || act.busy} disabled={mode === "audio" ? !paths.length || !has(
91            회의록 만들기
92          </Button>
93        </div>
94        <JobBar job={job.job} error={job.error} onCancel={job.cancel} />
95        <ErrorBox error={act.error} />
96      </Card>
97      <Card title="옵션">
98        {mode === "audio" && (
99          <>
100           <Field label="정확도" hint={needsDownload ? "처음 쓰는 모델은 한 번 내려받아요 (인터넷 필요)." : undefined}>
101             <select value={model} onChange={(e) => setModel(e.target.value)}>
102               {models.data?.sizes.map((s) => (
103                 <option key={s.key} value={s.key}>
104                   {s.label}
105                   {models.data?.installed.includes(s.key) ? " " : ""}
106                 </option>
107               ))}
108             </select>
109           </Field>
110           {needsDownload && (
111             <>
112               <Button small icon={<Download size={14} />} busy={dl.running} onClick={async () => (await dl.start(() => api.post("/transcribe/
113                 지금 내려받기
114               </Button>
115               <JobBar job={dl.job} error={dl.error} />
116             </>
117           )}
118           <Toggle checked={diarize} onChange={setDiarize} label={`화자 구분 (실험 기능${models.data?.diarize_available ? "" : " · 구성요소 필요"}
119         </>
120       )}
121       <Toggle checked={autoTodos} onChange={setAutoTodos} label="할 일 자동 등록" />
122       <p className="muted small">끝나면 요약, 할 일, 일정이 자동으로 정리돼요. 내보낼 때 개인정보를 가린 사본도 만들 수 있어요.</p>
123     </Card>
124   </div>
```

```
  );
}

function List({ onOpen }: { onOpen: (id: number) => void }) {
  const { data, reload } = useData(() => api.get<Item[]>("/transcripts"));
  if (data && !data.length) return <Empty icon={<Mic size={28} />}>아직 회의록이 없어요.</Empty>;
  return (
    <Card>
      <ul className="list">
        {data?.map((t) => (
          <li key={t.id} className="list-row clickable" onClick={() => onOpen(t.id)}>
            <span className="grow">{t.title}</span>
            {t.duration ? <Badge>{clock(t.duration)}</Badge> : <Badge>글</Badge>}
            <span className="muted small">{dateTime(t.created_at)}</span>
            <button
              className="iconbtn"
              aria-label="삭제"
              onClick={async (e) => {
                e.stopPropagation();
                if (confirm(`'${t.title}'을(를) 지울까요?`)) {
                  await api.del(`/transcripts/${t.id}`);
                  reload();
                }
              }}
            >
              <Trash2 size={14} />
            </button>
          </li>
        ))}
      </ul>
    </Card>
  );
}

function Detail({ id, onBack }: { id: number; onBack: () => void }) {
  const t = useData(() => api.get<Transcript>(`/transcripts/${id}`), [id]);
  const ins = useData(() => api.get<Insights>(`/transcripts/${id}/insights`), [id]);
  const [masked, setMasked] = useState(false);
  const [exported, setExported] = useState<string | null>(null);
  const act = useAction();
  const toast = useToast();
  const [title, setTitle] = useState("");
  useEffect(() => setTitle(t.data?.title ?? ""), [t.data?.title]);
  if (!t.data) return <Page title="회의록">{t.error ? <ErrorBox error={t.error} /> : null}</Page>;
  const tr = t.data;
  const speakerIds = Array.from(new Set(tr.segments.map((s) => s.speaker).filter((s): s is number => s !== null)));

  const save = async (patch: Partial<Transcript>) => {
    const r = await act.run(() => api.put<Transcript>(`/transcripts/${id}`, patch));
    if (r) t.setData(r);
  };
  const doExport = async (format: string) => {
    const r = await act.run(() => api.post<{ path: string }>(`/transcripts/${id}/export`, { format, masked }));
    if (r) setExported(r.path);
  };
  const addTodos = async () => {
    if (!ins.data?.actions.length) return;
    await act.run(() => api.post("/actions/save", { items: ins.data!.actions, source: `transcript:${id}` }));
    toast(`할 일 ${ins.data.actions.length}개를 등록했어요`);
  };
  const saveIcs = async () => {
    const r = await act.run(() => api.post<{ path: string }>("/schedule/ics", { events: ins.data?.events ?? [] }));
```

```
187     if (r) setExported(r.path);
188   };
189
190   return (
191     <Page
192       title={tr.title || "회의록"}
193       desc={`${dateTime(tr.created_at)}${tr.duration ? ` · ${clock(tr.duration)}` : ""}`}
194       actions={<Button variant="ghost" onClick={onBack}>← 목록</Button>}
195     >
196       <ErrorBox error={act.error} />
197       <div className="grid-2 wide-left">
198         <div className="stack">
199           <Card
200             title="요약"
201             right={
202               <Button small icon={<Sparkles size={14} />} onClick={async () => { await act.run(() => api.post(`/transcripts/${id}/summarize`));
203                 다시 요약
204               </Button>
205             }
206           >
207             {tr.summary.length ? (
208               <ul className="bullets">
209                 {tr.summary.map((p, i) => <li key={i}>{p}</li>)}
210               </ul>
211             ) : (
212               <p className="muted">요약할 내용이 짧아요.</p>
213             )}
214           </Card>
215           <Card title="전체 내용">
216             <div className="segments">
217               {tr.segments.map((s, i) => (
218                 <div key={i} className="segment">
219                   {tr.duration ? <span className="muted small mono">{clock(s.start)}</span> : null}
220                   {s.speaker !== null && <Badge tone="accent">{tr.speakers[String(s.speaker)] || `화자${s.speaker + 1}`}</Badge>}
221                   <span
222                     contentEditable
223                     suppressContentEditableWarning
224                     className="segment-text"
225                     onBlur={(e) => {
226                       const text = e.currentTarget.textContent ?? "";
227                       if (text !== s.text) save({ segments: tr.segments.map((x, j) => (j === i ? { ...x, text } : x)) });
228                     }}
229                   >
230                     {s.text}
231                   </span>
232                 </div>
233               ))}
234             </div>
235             <p className="muted small">글자를 눌러 바로 고칠 수 있어요.</p>
236           </Card>
237         </div>
238         <div className="stack">
239           <Card title="정보">
240             <Field label="제목">
241               <input value={title} onChange={(e) => setTitle(e.target.value)} onBlur={() => title !== tr.title && save({ title })} />
242             </Field>
243             {speakerIds.length > 0 && (
244               <Field label="화자 이름">
245                 {speakerIds.map((sp) => (
246                   <input
247                     key={sp}
248                     defaultValue={tr.speakers[String(sp)] ?? ""}
```

```
                    placeholder={`화자${sp + 1}`}
                    onBlur={(e) => save({ speakers: { ...tr.speakers, [String(sp)]: e.target.value } })}
                  />
                ))}
              </Field>
            )}
          </Card>
          <Card title={<><ListChecks size={16} /> 할 일 {ins.data?.actions.length ?? 0}</>} right={ins.data?.actions.length ? <Button small onCli
            <ul className="list compact">
              {ins.data?.actions.map((a, i) => (
                <li key={i} className="list-row">
                  <span className="grow">{a.text}</span>
                  {a.assignee && <Badge>{a.assignee}</Badge>}
                  {a.due && <Badge tone="soon">{a.due.slice(5)}</Badge>}
                </li>
              ))}
            </ul>
            {ins.data && !ins.data.actions.length && <p className="muted small">찾은 할 일이 없어요.</p>}
          </Card>
          <Card title={<><CalendarPlus size={16} /> 일정 {ins.data?.events.length ?? 0}</>} right={ins.data?.events.length ? <Button small onClic
            <ul className="list compact">
              {ins.data?.events.map((e, i) => (
                <li key={i} className="list-row">
                  <span className="grow">{e.title}</span>
                  <span className="muted small">{dateTime(e.start)}</span>
                </li>
              ))}
            </ul>
          </Card>
          <Card title="내보내기">
            <Toggle checked={masked} onChange={setMasked} label="개인정보 가린 사본으로" />
            <div className="row wrap">
              {["docx", "txt", "md", "srt", "vtt"].map((f) => (
                <Button key={f} small busy={act.busy} onClick={() => doExport(f)}>
                  {f === "docx" ? "워드(.docx)" : f.toUpperCase()}
                </Button>
              ))}
            </div>
            {exported && <p className="small">저장했어요: <PathLink path={exported} /></p>}
          </Card>
        </div>
      </div>
    </Page>
  );
}
```

```tsx
import { ArrowLeftRight, Download, Languages } from "lucide-react";
import { useState } from "react";
import { NeedsComponent } from "../components/system";
import { Button, Card, CopyButton, ErrorBox, JobBar, Notice, Page } from "../components/ui";
import { ApiError, api } from "../lib/api";
import { useAction, useData, useJob } from "../lib/hooks";

type Status = { available: boolean; llm: boolean; languages: { code: string; label: string }[]; installed: string[] };

export default function Translate() {
  const status = useData(() => api.get<Status>("/translate/status"));
  const [src, setSrc] = useState("ko");
  const [dst, setDst] = useState("en");
  const [text, setText] = useState(() => {
    const v = sessionStorage.getItem("lb-paste") ?? "";
    sessionStorage.removeItem("lb-paste");
    return v;
  });
  const [out, setOut] = useState("");
  const [needPack, setNeedPack] = useState(false);
  const act = useAction();
  const install = useJob();
  const run = async () => {
    setNeedPack(false);
    try {
      const r = await api.post<{ text: string }>("/translate", { text, source: src, target: dst });
      setOut(r.text);
      act.setError(null);
    } catch (e) {
      if (e instanceof ApiError && e.component === "translate_pack") setNeedPack(true);
      act.setError(e instanceof Error ? e.message : String(e));
    }
  };
  const langs = status.data?.languages ?? [];
  return (
    <Page title="번역" desc="인터넷 없이 이 PC에서 번역해요. 언어팩은 처음 한 번만 내려받아요.">
      {status.data && !status.data.available && !status.data.llm && <NeedsComponent keys={["translate"]} />}
      <div className="row">
        <select value={src} onChange={(e) => setSrc(e.target.value)}>
          {langs.map((l) => <option key={l.code} value={l.code}>{l.label}</option>)}
        </select>
        <button className="iconbtn" aria-label="언어 바꾸기" onClick={() => (setSrc(dst), setDst(src), setText(out), setOut(text))}>
          <ArrowLeftRight size={16} />
        </button>
        <select value={dst} onChange={(e) => setDst(e.target.value)}>
          {langs.map((l) => <option key={l.code} value={l.code}>{l.label}</option>)}
        </select>
        <Button variant="primary" icon={<Languages size={16} />} busy={act.busy} disabled={!text.trim()} onClick={() => act.run(run)}>
          번역
        </Button>
      </div>
      <ErrorBox error={act.error} />
      {needPack && (
        <Notice tone="warn">
          <span>
            이 언어 조합의 언어팩이 없어요.{" "}
            <Button small icon={<Download size={14} />} busy={install.running} onClick={async () => { await install.start(() => api.post("/tran
              언어팩 받기 (인터넷 필요)
            </Button>
          </span>
        </Notice>
      )}
```

```
      <JobBar job={install.job} error={install.error} />
      <div className="grid-2">
        <Card>
          <textarea rows={16} value={text} onChange={(e) => setText(e.target.value)} placeholder="번역할 글" />
        </Card>
        <Card right={out ? <CopyButton text={out} /> : null}>
          <textarea rows={16} value={out} readOnly placeholder="번역 결과" />
        </Card>
      </div>
      {status.data?.installed.length ? <p className="muted small">설치된 언어팩: {status.data.installed.join(", ")}</p> : null}
    </Page>
  );
}
```

```tsx
import { Mail, SpellCheck, WandSparkles } from "lucide-react";
import { useState } from "react";
import { Badge, Button, Card, CopyButton, ErrorBox, Field, Page, Tabs } from "../components/ui";
import { api } from "../lib/api";
import { useAction, useData } from "../lib/hooks";

type Change = { before: string; after: string; reason: string; count: number };

export default function Writing() {
  const [tab, setTab] = useState<"spell" | "tone" | "email">("spell");
  return (
    <Page title="글다듬기" desc="자주 틀리는 맞춤법을 고치고, 해요체 · 합니다체 · 공문체로 바꾸고, 업무 메일 틀을 만들어요.">
      <Tabs tabs={[{ key: "spell", label: "맞춤법" }, { key: "tone", label: "문체 바꾸기" }, { key: "email", label: "이메일 틀" }]} value={tab} onCh
      {tab === "spell" && <Spell />}
      {tab === "tone" && <Tone />}
      {tab === "email" && <Email />}
    </Page>
  );
}

function Spell() {
  const [text, setText] = useState("");
  const [res, setRes] = useState<{ text: string; changes: Change[] } | null>(null);
  const act = useAction();
  return (
    <div className="grid-2">
      <Card>
        <textarea rows={14} value={text} onChange={(e) => setText(e.target.value)} placeholder="예: 몇일 뒤에 연락드릴께요. 그건 안되요." />
        <Button variant="primary" icon={<SpellCheck size={16} />} busy={act.busy} disabled={!text.trim()} onClick={async () => setRes((await ac
          고치기
        </Button>
        <ErrorBox error={act.error} />
      </Card>
      <Card title={res ? `고친 곳 ${res.changes.reduce((s, c) => s + c.count, 0)}` : "결과"} right={res ? <div className="row"><CopyButton text={res
        {res && (
          <>
            <textarea rows={8} readOnly value={res.text} />
            <ul className="list compact">
              {res.changes.map((c, i) => (
                <li key={i} className="list-row">
                  <span className="strike">{c.before}</span> → <b>{c.after}</b>
                  <span className="muted small grow">{c.reason}</span>
                  {c.count > 1 && <Badge>{c.count}곳</Badge>}
                </li>
              ))}
            </ul>
            {!res.changes.length && <p className="muted">고칠 곳을 찾지 못했어요.</p>}
          </>
        )}
      </Card>
    </div>
  );
}

function Tone() {
  const [text, setText] = useState("");
  const [style, setStyle] = useState("formal");
  const [useAi, setUseAi] = useState(false);
  const [res, setRes] = useState<{ text: string; changed: number } | null>(null);
  const llm = useData(() => api.get<{ llm: boolean }>("/translate/status"));
  const act = useAction();
  return (
```

```
    <div className="grid-2">
      <Card>
        <textarea rows={14} value={text} onChange={(e) => setText(e.target.value)} placeholder="예: 내일 회의 자료 보내드려요. 확인해 주세요." />
        <div className="row wrap">
          <select value={style} onChange={(e) => setStyle(e.target.value)}>
            <option value="formal">합니다체 (보고 · 안내)</option>
            <option value="polite">해요체 (부드럽게)</option>
            <option value="official">공문체 (~함, ~임)</option>
          </select>
          {llm.data?.llm && (
            <label className="check-label">
              <input type="checkbox" checked={useAi} onChange={(e) => setUseAi(e.target.checked)} /> 로컬 AI로 자연스럽게
            </label>
          )}
          <Button variant="primary" icon={<WandSparkles size={16} />} busy={act.busy} disabled={!text.trim()} onClick={async () => setRes((awai
            바꾸기
          </Button>
        </div>
        <ErrorBox error={act.error} />
      </Card>
      <Card title="결과" right={res ? <CopyButton text={res.text} /> : null}>
        <textarea rows={14} readOnly value={res?.text ?? ""} />
        {res && res.changed >= 0 && <p className="muted small">문장 끝 {res.changed}곳을 바꿨어요. 어색한 곳은 직접 다듬어 주세요.</p>}
      </Card>
    </div>
  );
}

function Email() {
  const settings = useData(() => api.get<Record<string, string>>("/system/settings"));
  const [form, setForm] = useState({ recipient: "", subject: "", body: "", style: "formal" });
  const [res, setRes] = useState<{ subject: string; text: string } | null>(null);
  const act = useAction();
  const sender = settings.data?.user_name ?? "";
  const company = settings.data?.company ?? "";
  return (
    <div className="grid-2">
      <Card>
        <div className="form-grid">
          <Field label="받는 분"><input value={form.recipient} onChange={(e) => setForm({ ...form, recipient: e.target.value })} placeholder="김민수
          <Field label="제목"><input value={form.subject} onChange={(e) => setForm({ ...form, subject: e.target.value })} /></Field>
        </div>
        <Field label="하고 싶은 말" hint={sender ? `보내는 사람: ${company} ${sender}` : "설정에서 내 이름 · 회사를 넣으면 서명이 들어가요."}>
          <textarea rows={8} value={form.body} onChange={(e) => setForm({ ...form, body: e.target.value })} placeholder="회의 자료 보내드려요. 금요일
        </Field>
        <Button variant="primary" icon={<Mail size={16} />} busy={act.busy} disabled={!form.body.trim()} onClick={async () => setRes((await act
          메일 만들기
        </Button>
      </Card>
      <Card title={res?.subject ? `제목: ${res.subject}` : "결과"} right={res ? <CopyButton text={res.text} /> : null}>
        <textarea rows={16} readOnly value={res?.text ?? ""} />
      </Card>
    </div>
  );
}
```

```
:root {
  --bg: #f5f6f8;
  --surface: #ffffff;
  --surface-2: #f0f2f5;
  --border: #e3e6eb;
  --text: #1b2230;
  --muted: #687385;
  --accent: #2f6bed;
  --accent-soft: #e8efff;
  --accent-text: #1f55c9;
  --ok: #138a52;
  --ok-soft: #e4f6ec;
  --warn: #9a6200;
  --warn-soft: #fff4dc;
  --danger: #c93030;
  --danger-soft: #fdeaea;
  --shadow: 0 1px 2px rgba(16, 24, 40, 0.05), 0 1px 3px rgba(16, 24, 40, 0.06);
  --radius: 10px;
  --font: "Pretendard", "Apple SD Gothic Neo", "Malgun Gothic", "Noto Sans KR", system-ui, sans-serif;
  color-scheme: light;
}

@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
    --bg: #0e1116;
    --surface: #161a22;
    --surface-2: #1d222c;
    --border: #2a303c;
    --text: #e5e8ee;
    --muted: #8e97a8;
    --accent: #5b8cff;
    --accent-soft: #1c2945;
    --accent-text: #9bb8ff;
    --ok: #4cc38a;
    --ok-soft: #13291e;
    --warn: #e7b04a;
    --warn-soft: #2d2412;
    --danger: #ff6b6b;
    --danger-soft: #331718;
    --shadow: none;
    color-scheme: dark;
  }
}

* { box-sizing: border-box; }
html, body, #root { height: 100%; margin: 0; }
body { background: var(--bg); color: var(--text); font-family: var(--font); font-size: 14px; line-height: 1.55; -webkit-font-smoothing: antiali
button, input, select, textarea { font: inherit; color: inherit; }
h1, h2, h3 { margin: 0; letter-spacing: -0.01em; }
p { margin: 0; }
code, .mono { font-family: ui-monospace, "Cascadia Mono", Consolas, monospace; }
kbd { font-family: inherit; font-size: 11px; padding: 1px 6px; border: 1px solid var(--border); border-bottom-width: 2px; border-radius: 5px; b
mark { background: #ffe58a; color: inherit; border-radius: 2px; padding: 0 1px; }

/* 레이아웃 */
.shell { display: grid; grid-template-columns: 232px 1fr; height: 100%; }
.sidebar { display: flex; flex-direction: column; gap: 10px; padding: 16px 12px; border-right: 1px solid var(--border); background: var(--surfa
.brand { display: flex; align-items: center; gap: 9px; font-weight: 700; font-size: 16px; padding: 2px 8px 6px; }
.brand-mark { display: grid; place-items: center; width: 28px; height: 28px; border-radius: 8px; background: var(--accent); color: #fff; font-s
.palette-trigger { display: flex; align-items: center; gap: 8px; padding: 7px 10px; border: 1px solid var(--border); border-radius: 8px; backgr
.palette-trigger span { flex: 1; }
.nav-group { display: flex; flex-direction: column; gap: 1px; margin-bottom: 8px; }
```

```
.nav-title { font-size: 11.5px; font-weight: 600; color: var(--muted); padding: 8px 10px 4px; }
.nav-item { display: flex; align-items: center; gap: 10px; padding: 7px 10px; border: 0; border-radius: 7px; background: none; cursor: pointer;
.nav-item:hover { background: var(--surface-2); }
.nav-item.active { background: var(--accent-soft); color: var(--accent-text); font-weight: 600; }
.sidebar-foot { margin-top: auto; display: flex; flex-direction: column; gap: 2px; padding: 8px 10px 0; border-top: 1px solid var(--border); }
.main { overflow-y: auto; }
.page { max-width: 1180px; margin: 0 auto; padding: 28px 32px 48px; display: flex; flex-direction: column; gap: 16px; }
.page-head { display: flex; align-items: flex-start; justify-content: space-between; gap: 16px; }
.page-head h1 { font-size: 22px; margin-bottom: 4px; }
.grid-2 { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; align-items: start; }
.grid-2.wide-left { grid-template-columns: minmax(0, 1.6fr) minmax(280px, 1fr); }
.grid-2.notes { grid-template-columns: minmax(240px, 0.8fr) minmax(0, 1.6fr); }
.stack { display: flex; flex-direction: column; gap: 12px; }
.row { display: flex; align-items: center; gap: 8px; }
.row.wrap { flex-wrap: wrap; }
.grow { flex: 1; min-width: 0; }
.truncate { overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
.muted { color: var(--muted); }
.small { font-size: 12.5px; }
.accent { color: var(--accent-text); }
.ok { color: var(--ok); }
.danger, .error-text { color: var(--danger); }
.pad { padding: 14px; }
.section-title { font-size: 15px; margin-top: 8px; }

/* 카드 */
.card { background: var(--surface); border: 1px solid var(--border); border-radius: var(--radius); box-shadow: var(--shadow); padding: 18px; di
.card-head { display: flex; align-items: center; justify-content: space-between; gap: 8px; }
.card-head h2 { font-size: 15px; display: flex; align-items: center; gap: 6px; }

/* 입력 */
input:not([type="checkbox"]):not([type="range"]):not([type="color"]), select, textarea {
  width: 100%; padding: 8px 10px; border: 1px solid var(--border); border-radius: 8px; background: var(--surface); outline: none; transition: b
}
input:focus, select:focus, textarea:focus { border-color: var(--accent); box-shadow: 0 0 0 3px var(--accent-soft); }
textarea { resize: vertical; min-height: 80px; line-height: 1.6; }
input[type="color"] { width: 44px; height: 34px; border: 1px solid var(--border); border-radius: 8px; padding: 2px; background: var(--surface);
input[type="range"] { width: 100%; accent-color: var(--accent); }
input[type="checkbox"] { accent-color: var(--accent); width: 15px; height: 15px; }
.field { display: flex; flex-direction: column; gap: 5px; min-width: 0; }
.field-inline { flex: 1; }
.field-label { font-size: 12.5px; font-weight: 600; color: var(--muted); }
.field-hint { font-size: 12px; color: var(--muted); }
.form-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; }
.checks { display: grid; grid-template-columns: 1fr 1fr; gap: 6px 12px; }
.check-label { display: flex; align-items: center; gap: 7px; cursor: pointer; }
.edit-row { display: flex; align-items: center; gap: 8px; }
.w-sm { width: 110px !important; flex: none; }
.w-md { width: 150px !important; flex: none; }
.w-lg { width: 210px !important; flex: none; }
.title-input { font-size: 17px; font-weight: 600; border: 0 !important; padding-left: 2px !important; box-shadow: none !important; }
.chip { border: 1px dashed var(--border); background: var(--surface-2); border-radius: 999px; padding: 3px 10px; font-size: 12.5px; cursor: poi
.chip:hover { border-color: var(--accent); color: var(--accent-text); }

.toggle { display: inline-flex; align-items: center; gap: 9px; cursor: pointer; user-select: none; }
.toggle input { position: absolute; opacity: 0; pointer-events: none; }
.toggle-track { width: 34px; height: 20px; border-radius: 999px; background: var(--border); position: relative; transition: background 0.15s; f
.toggle-track::after { content: ""; position: absolute; top: 2px; left: 2px; width: 16px; height: 16px; border-radius: 50%; background: #fff; t
.toggle input:checked + .toggle-track { background: var(--accent); }
.toggle input:checked + .toggle-track::after { transform: translateX(14px); }
.toggle input:focus-visible + .toggle-track { box-shadow: 0 0 0 3px var(--accent-soft); }

```

```
/* 버튼 */
.btn { display: inline-flex; align-items: center; justify-content: center; gap: 6px; padding: 8px 14px; border-radius: 8px; border: 1px solid v
.btn:hover:not(:disabled) { background: var(--surface-2); }
.btn:disabled { opacity: 0.5; cursor: not-allowed; }
.btn-primary { background: var(--accent); border-color: var(--accent); color: #fff; }
.btn-primary:hover:not(:disabled) { background: var(--accent); filter: brightness(1.07); }
.btn-ghost { background: transparent; border-color: transparent; color: var(--muted); }
.btn-danger { color: var(--danger); border-color: var(--danger-soft); background: var(--danger-soft); }
.btn-sm { padding: 5px 10px; font-size: 12.5px; }
.card > .btn, .card > .row > .btn { align-self: flex-start; }
.iconbtn { display: inline-grid; place-items: center; width: 28px; height: 28px; border: 0; border-radius: 6px; background: none; color: var(--
.iconbtn:hover { background: var(--surface-2); color: var(--text); }
.iconbtn.accent { color: var(--accent-text); }
.linklike { border: 0; background: none; padding: 0; color: var(--accent-text); cursor: pointer; text-decoration: underline; text-underline-off
.linklike.danger { color: var(--danger); }
.pathlink { display: inline-flex; align-items: center; gap: 2px; max-width: 100%; }
.pathlink .linklike { overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }

/* 탭 */
.tabs { display: flex; gap: 2px; padding: 3px; background: var(--surface-2); border-radius: 9px; align-self: flex-start; flex-wrap: wrap; }
.tabs button { border: 0; background: none; padding: 6px 13px; border-radius: 7px; cursor: pointer; color: var(--muted); font-weight: 600; }
.tabs button.active { background: var(--surface); color: var(--text); box-shadow: var(--shadow); }

/* 알림 */
.alert { display: flex; align-items: flex-start; gap: 8px; padding: 10px 12px; border-radius: 8px; font-size: 13.5px; }
.alert svg { flex: none; margin-top: 2px; }
.alert-error { background: var(--danger-soft); color: var(--danger); }
.alert-info { background: var(--accent-soft); color: var(--accent-text); }
.alert-warn { background: var(--warn-soft); color: var(--warn); }
.alert-ok { background: var(--ok-soft); color: var(--ok); }
.alert .list { color: var(--text); }
.empty { display: flex; flex-direction: column; align-items: center; gap: 8px; padding: 28px 12px; color: var(--muted); text-align: center; }
.badge { display: inline-flex; align-items: center; gap: 3px; padding: 1px 8px; border-radius: 999px; background: var(--surface-2); color: var(
.badge-late { background: var(--danger-soft); color: var(--danger); }
.badge-today { background: var(--warn-soft); color: var(--warn); }
.badge-soon, .badge-accent { background: var(--accent-soft); color: var(--accent-text); }
.badge-ok { background: var(--ok-soft); color: var(--ok); }

/* 진행률 */
.jobbar { display: flex; flex-direction: column; gap: 6px; padding: 10px 12px; border: 1px solid var(--border); border-radius: 8px; background:
.jobbar-top { display: flex; align-items: center; gap: 10px; font-size: 13px; }
.jobbar-top > span:first-child { flex: 1; display: flex; align-items: center; gap: 6px; }
.progress { height: 6px; background: var(--border); border-radius: 999px; overflow: hidden; }
.progress > div { height: 100%; background: var(--accent); border-radius: 999px; transition: width 0.3s; }
.spin { animation: spin 0.9s linear infinite; }
@keyframes spin { to { transform: rotate(360deg); } }

/* 파일 고르기 */
.picker { display: flex; flex-direction: column; gap: 8px; }
.dropzone { display: flex; flex-direction: column; align-items: center; justify-content: center; gap: 8px; min-height: 120px; padding: 18px; bo
.dropzone:hover { border-color: var(--accent); color: var(--accent-text); }
.picker-list { display: flex; flex-direction: column; gap: 4px; }
.picker-item { display: grid; grid-template-columns: minmax(0, auto) minmax(0, 1fr) auto; align-items: center; gap: 10px; padding: 6px 6px 6px
.drop-overlay { position: fixed; inset: 10px; border: 2.5px dashed var(--accent); border-radius: 14px; background: color-mix(in srgb, var(--acc

/* 목록 · 표 */
.list { list-style: none; margin: 0; padding: 0; display: flex; flex-direction: column; }
.list-row { display: flex; align-items: center; gap: 10px; padding: 9px 4px; border-bottom: 1px solid var(--border); min-width: 0; }
.list-row:last-child { border-bottom: 0; }
.list.compact .list-row { padding: 6px 2px; }
.list-row.clickable { cursor: pointer; border-radius: 6px; }
.list-row.clickable:hover, .list-row.selected { background: var(--surface-2); }
```

```
.list-row.done .grow { color: var(--muted); text-decoration: line-through; }
.list-row.clip { align-items: flex-start; }
.clip-text { white-space: pre-wrap; word-break: break-word; max-height: 4.8em; overflow: hidden; }
.check { display: grid; place-items: center; width: 20px; height: 20px; flex: none; border: 1.5px solid var(--border); border-radius: 6px; back
.check:hover { border-color: var(--ok); color: var(--ok); }
.check.on { background: var(--ok); border-color: var(--ok); color: #fff; }
.table { width: 100%; border-collapse: collapse; font-size: 13.5px; }
.table th { text-align: left; font-size: 12px; color: var(--muted); font-weight: 600; padding: 6px 8px; border-bottom: 1px solid var(--border);
.table td { padding: 7px 8px; border-bottom: 1px solid var(--border); vertical-align: top; }
.table .num { text-align: right; font-variant-numeric: tabular-nums; }
.row-error td { background: var(--danger-soft); }
.results { display: flex; flex-direction: column; gap: 10px; }
.bullets { margin: 0; padding-left: 18px; display: flex; flex-direction: column; gap: 4px; }
.strike { text-decoration: line-through; color: var(--muted); }

/* 홈 */
.stats { display: grid; grid-template-columns: repeat(4, 1fr); gap: 12px; }
.stat { display: flex; flex-direction: column; gap: 2px; padding: 14px 16px; background: var(--surface); border: 1px solid var(--border); borde
.stat b { font-size: 24px; font-variant-numeric: tabular-nums; }
.stat:hover { border-color: var(--accent); }
.feature-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(250px, 1fr)); gap: 10px; }
.feature { display: flex; align-items: center; gap: 12px; padding: 12px 14px; background: var(--surface); border: 1px solid var(--border); bord
.feature > svg:first-child { color: var(--accent-text); flex: none; }
.feature div { flex: 1; display: flex; flex-direction: column; min-width: 0; }
.feature:hover { border-color: var(--accent); }

/* 결과 미리보기 */
.thumbs { display: flex; flex-wrap: wrap; gap: 14px; }
.thumbs figure { margin: 0; display: flex; flex-direction: column; gap: 6px; max-width: 220px; }
.thumb { display: block; border-radius: 8px; border: 1px solid var(--border); background: var(--surface-2); object-fit: contain; }
.thumb-empty { animation: pulse 1.2s ease-in-out infinite; }
@keyframes pulse { 50% { opacity: 0.5; } }

/* 회의록 */
.segments { display: flex; flex-direction: column; gap: 8px; max-height: 560px; overflow-y: auto; }
.segment { display: flex; gap: 8px; align-items: baseline; }
.segment-text { flex: 1; padding: 1px 4px; border-radius: 4px; outline: none; }
.segment-text:hover { background: var(--surface-2); }
.segment-text:focus { background: var(--accent-soft); }

/* OCR 선택 */
.selector { position: relative; user-select: none; cursor: crosshair; touch-action: none; line-height: 0; }
.selector img { width: 100%; border-radius: 6px; border: 1px solid var(--border); }
.selection { position: absolute; border: 2px solid var(--accent); background: color-mix(in srgb, var(--accent) 15%, transparent); border-radius

/* 중복 */
.dup-group { border: 1px solid var(--border); border-radius: 8px; overflow: hidden; }
.dup-row { display: flex; align-items: center; gap: 10px; padding: 7px 10px; border-bottom: 1px solid var(--border); cursor: pointer; }
.dup-row:last-child { border-bottom: 0; }
.dup-row:hover { background: var(--surface-2); }

/* 타이머 */
.timer { display: grid; place-items: center; padding: 8px 0 4px; --c: var(--accent); }
.timer.break { --c: var(--ok); }
.timer-ring { width: 210px; height: 210px; border-radius: 50%; display: grid; place-items: center; background: conic-gradient(var(--c) calc(var
.timer-text { width: 180px; height: 180px; border-radius: 50%; background: var(--surface); display: flex; flex-direction: column; align-items:
.timer-text b { font-size: 44px; font-variant-numeric: tabular-nums; letter-spacing: -0.02em; }
.bars { display: grid; grid-template-columns: repeat(7, 1fr); gap: 8px; height: 170px; align-items: end; }
.bar-col { display: flex; flex-direction: column; align-items: center; justify-content: flex-end; gap: 2px; height: 100%; }
.bar { width: 100%; max-width: 34px; background: var(--accent); border-radius: 5px 5px 2px 2px; min-height: 2px; }

/* 검색 */
```

```
.search-box { display: flex; align-items: center; gap: 10px; padding: 4px 14px; background: var(--surface); border: 1px solid var(--border); bo
.search-box input { border: 0 !important; box-shadow: none !important; font-size: 16px; padding: 10px 0 !important; }
.snippet { margin-top: 2px; }

/* 팔레트 */
.palette-backdrop { position: fixed; inset: 0; background: rgba(10, 14, 22, 0.35); display: flex; justify-content: center; align-items: flex-st
.palette { width: min(620px, calc(100vw - 32px)); background: var(--surface); border: 1px solid var(--border); border-radius: 14px; box-shadow:
.palette-input { display: flex; align-items: center; gap: 10px; padding: 4px 16px; border-bottom: 1px solid var(--border); color: var(--muted);
.palette-input input { border: 0 !important; box-shadow: none !important; padding: 13px 0 !important; font-size: 15px; color: var(--text); }
.palette-list { max-height: 400px; overflow-y: auto; padding: 6px; }
.palette-row { display: flex; align-items: center; gap: 10px; width: 100%; padding: 8px 10px; border: 0; border-radius: 8px; background: none;
.palette-row.sel { background: var(--accent-soft); }
.palette-label { font-weight: 600; white-space: nowrap; }

/* 토스트 */
.toasts { position: fixed; bottom: 20px; right: 20px; display: flex; flex-direction: column; gap: 8px; z-index: 60; }
.toast { padding: 10px 16px; border-radius: 9px; background: var(--text); color: var(--surface); box-shadow: 0 8px 24px rgba(0, 0, 0, 0.2); fon
.toast-error { background: var(--danger); color: #fff; }
@keyframes rise { from { transform: translateY(8px); opacity: 0; } }

/* 시작 화면 */
.splash { height: 100%; display: flex; flex-direction: column; align-items: center; justify-content: center; gap: 12px; }
.splash .logo { font-size: 28px; font-weight: 800; letter-spacing: -0.02em; }
.splash p { display: flex; align-items: center; gap: 6px; }

@media (max-width: 980px) {
  .grid-2, .grid-2.wide-left, .grid-2.notes { grid-template-columns: 1fr; }
  .stats { grid-template-columns: repeat(2, 1fr); }
}
@media (max-width: 720px) {
  .shell { grid-template-columns: 1fr; grid-template-rows: auto 1fr; }
  .sidebar { flex-direction: row; overflow-x: auto; border-right: 0; border-bottom: 1px solid var(--border); padding: 8px; }
  .sidebar nav { display: flex; gap: 4px; }
  .nav-group { flex-direction: row; margin: 0; }
  .nav-title, .sidebar-foot, .palette-trigger span, .palette-trigger kbd { display: none; }
  .nav-item span { white-space: nowrap; }
  .page { padding: 18px 16px 40px; }
  .form-grid, .checks { grid-template-columns: 1fr; }
}
```

로컬비서 (LocalBiseo) v0.1.0 소스코드 — engine/localbiseo/__init__.py

```
"""로컬비서 엔진. 모든 처리는 이 PC 안에서만 이뤄진다."""

__version__ = "0.1.0"
```

```python
"""엔진 실행: python -m localbiseo --port 0 --token XXX

앱 셸(Tauri)이 자식 프로세스로 띄운다. 준비되면 표준출력에
'LOCALBISEO_READY <port>' 한 줄을 찍고, 표준입력이 닫히면(앱 종료) 같이 끝난다.
"""
from __future__ import annotations

import argparse
import os
import socket
import sys
import threading
import time
from pathlib import Path


def _free_port() -> int:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.bind(("127.0.0.1", 0))
        return s.getsockname()[1]


def _watch_stdin() -> None:
    try:
        while sys.stdin.read(1024):
            pass
    except Exception:  # noqa: BLE001
        pass
    os._exit(0)


def main() -> None:
    ap = argparse.ArgumentParser(prog="localbiseo-engine")
    ap.add_argument("--port", type=int, default=0)
    ap.add_argument("--token", default=os.environ.get("LOCALBISEO_TOKEN"))
    ap.add_argument("--data-dir", default=None)
    ap.add_argument("--dev-origin", action="append", default=[], help="개발용으로 허용할 추가 Origin")
    ap.add_argument("--watch-stdin", action="store_true", help="표준입력이 닫히면 종료")
    args = ap.parse_args()

    if sys.stdout and hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(encoding="utf-8")
    if args.data_dir:
        os.environ["LOCALBISEO_HOME"] = args.data_dir

    import uvicorn

    from .app import create_app

    port = args.port or _free_port()
    app = create_app(token=args.token, data_dir=Path(args.data_dir) if args.data_dir else None, extra_origins=args.dev_origin)
    server = uvicorn.Server(uvicorn.Config(app, host="127.0.0.1", port=port, log_level="warning", access_log=False))

    def announce() -> None:
        while not server.started:
            time.sleep(0.05)
        print(f"LOCALBISEO_READY {port}", flush=True)

    threading.Thread(target=announce, daemon=True).start()
    if args.watch_stdin:
        threading.Thread(target=_watch_stdin, daemon=True).start()
    server.run()
```

```
63
64
65  if __name__ == "__main__":
66      main()
```

```python
from __future__ import annotations

import secrets
from contextlib import asynccontextmanager
from pathlib import Path

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse

from . import __version__
from .core.config import data_dir as default_data_dir
from .core.context import AppContext
from .core.db import Database
from .core.errors import AppError
from .core.jobs import JobManager
from .modules import (
    actions, clipboard, dedupe, focus, media, notes, papers, pdf_tools, rename, schedule, screen_ocr, search_api, system,
    translate, writing,
)
from .modules.masking import api as masking
from .modules.transcribe import api as transcribe

ALLOWED_ORIGINS = [
    "tauri://localhost",
    "http://tauri.localhost",
    "https://tauri.localhost",
    "http://localhost:1420",
    "http://127.0.0.1:1420",
]
OPEN_PATHS = {"/health"}

ROUTERS = [
    system.router, masking.router, transcribe.router, media.router, actions.router, schedule.router, notes.router,
    clipboard.router, screen_ocr.router, papers.router, translate.router, writing.router, rename.router, dedupe.router,
    pdf_tools.router, focus.router, search_api.router,
]


def create_app(token: str | None = None, data_dir: Path | None = None, extra_origins: list[str] | None = None) -> FastAPI:
    root = data_dir or default_data_dir()
    root.mkdir(parents=True, exist_ok=True)
    ctx = AppContext(db=Database(root / "localbiseo.db"), jobs=JobManager(), data_dir=root)

    @asynccontextmanager
    async def lifespan(_: FastAPI):
        yield
        ctx.jobs.shutdown()
        ctx.db.close()

    app = FastAPI(title="로컬비서 엔진", version=__version__, docs_url="/docs" if token is None else None, redoc_url=None, lifespan=lifespan)
    app.state.ctx = ctx
    app.state.token = token

    @app.middleware("http")
    async def auth(request: Request, call_next):
        # 다른 웹사이트가 localhost 엔진을 부르지 못하도록 앱 셸이 준 토큰을 확인한다.
        if token and request.method != "OPTIONS" and request.url.path not in OPEN_PATHS:
            given = request.headers.get("x-biseo-token", "")
            if not secrets.compare_digest(given, token):
                return JSONResponse({"error": "인증 실패"}, status_code=401)
```

```
        return await call_next(request)

    app.add_middleware(
        CORSMiddleware,
        allow_origins=ALLOWED_ORIGINS + (extra_origins or []),
        allow_methods=["*"],
        allow_headers=["*"],
    )

    @app.exception_handler(AppError)
    async def app_error(_: Request, exc: AppError):
        return JSONResponse(exc.to_dict(), status_code=exc.status)

    @app.exception_handler(RequestValidationError)
    async def validation_error(_: Request, exc: RequestValidationError):
        first = exc.errors()[0] if exc.errors() else {}
        where = ".".join(str(x) for x in first.get("loc", [])[1:])
        return JSONResponse({"error": f"입력값을 확인해 주세요: {where} ({first.get('msg', '')})"}, status_code=422)

    for router in ROUTERS:
        app.include_router(router)
    return app
```

```
1
```

```python
from __future__ import annotations

import os
import sys
from pathlib import Path

APP_NAME = "LocalBiseo"


def data_dir() -> Path:
    env = os.environ.get("LOCALBISEO_HOME")
    if env:
        path = Path(env)
    elif sys.platform == "win32":
        path = Path(os.environ.get("APPDATA", str(Path.home()))) / APP_NAME
    elif sys.platform == "darwin":
        path = Path.home() / "Library" / "Application Support" / APP_NAME
    else:
        path = Path(os.environ.get("XDG_DATA_HOME", str(Path.home() / ".local" / "share"))) / "localbiseo"
    path.mkdir(parents=True, exist_ok=True)
    return path


def sub_dir(name: str, base: Path | None = None) -> Path:
    path = (base or data_dir()) / name
    path.mkdir(parents=True, exist_ok=True)
    return path


def output_path(src: Path, suffix: str, ext: str | None = None, out_dir: str | Path | None = None) -> Path:
    """원본 옆(또는 out_dir)에 '<이름><suffix>.<ext>' 경로를 만들고, 이미 있으면 번호를 붙인다."""
    folder = Path(out_dir) if out_dir else src.parent
    folder.mkdir(parents=True, exist_ok=True)
    ext = ext if ext is not None else src.suffix.lstrip(".")
    dot_ext = f".{ext}" if ext else ""
    candidate = folder / f"{src.stem}{suffix}{dot_ext}"
    n = 2
    while candidate.exists():
        candidate = folder / f"{src.stem}{suffix}({n}){dot_ext}"
        n += 1
    return candidate
```

```python
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Annotated

from fastapi import Depends, Request

from .db import Database
from .jobs import JobManager


@dataclass
class AppContext:
    db: Database
    jobs: JobManager
    data_dir: Path


def get_ctx(request: Request) -> AppContext:
    return request.app.state.ctx


Ctx = Annotated[AppContext, Depends(get_ctx)]


def job_ref(job) -> dict:
    return {"job_id": job.id, "job": job.to_dict()}
```

```python
from __future__ import annotations

import json
import sqlite3
import threading
from contextlib import contextmanager
from datetime import datetime
from pathlib import Path
from typing import Any, Iterator

SCHEMA_VERSION = 1

SCHEMA = """
CREATE TABLE IF NOT EXISTS settings (
  key TEXT PRIMARY KEY,
  value TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS notes (
  id INTEGER PRIMARY KEY,
  title TEXT NOT NULL DEFAULT '',
  body TEXT NOT NULL DEFAULT '',
  tags TEXT NOT NULL DEFAULT '',
  pinned INTEGER NOT NULL DEFAULT 0,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS todos (
  id INTEGER PRIMARY KEY,
  text TEXT NOT NULL,
  done INTEGER NOT NULL DEFAULT 0,
  due TEXT,
  assignee TEXT,
  source TEXT,
  created_at TEXT NOT NULL,
  done_at TEXT
);
CREATE TABLE IF NOT EXISTS clips (
  id INTEGER PRIMARY KEY,
  text TEXT NOT NULL,
  sensitive INTEGER NOT NULL DEFAULT 0,
  kinds TEXT NOT NULL DEFAULT '',
  pinned INTEGER NOT NULL DEFAULT 0,
  created_at TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS transcripts (
  id INTEGER PRIMARY KEY,
  title TEXT NOT NULL DEFAULT '',
  source_path TEXT,
  language TEXT,
  duration REAL,
  segments TEXT NOT NULL,
  speakers TEXT NOT NULL DEFAULT '{}',
  summary TEXT NOT NULL DEFAULT '',
  created_at TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS focus_sessions (
  id INTEGER PRIMARY KEY,
  label TEXT NOT NULL DEFAULT '',
  kind TEXT NOT NULL DEFAULT 'focus',
  planned_minutes INTEGER,
  started_at TEXT NOT NULL,
  ended_at TEXT,
```

```
  completed INTEGER NOT NULL DEFAULT 0
);
CREATE TABLE IF NOT EXISTS expenses (
  id INTEGER PRIMARY KEY,
  date TEXT,
  merchant TEXT NOT NULL DEFAULT '',
  amount INTEGER,
  category TEXT NOT NULL DEFAULT '',
  memo TEXT NOT NULL DEFAULT '',
  biz_no TEXT NOT NULL DEFAULT '',
  created_at TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS contacts (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL DEFAULT '',
  company TEXT NOT NULL DEFAULT '',
  title TEXT NOT NULL DEFAULT '',
  mobile TEXT NOT NULL DEFAULT '',
  phone TEXT NOT NULL DEFAULT '',
  fax TEXT NOT NULL DEFAULT '',
  email TEXT NOT NULL DEFAULT '',
  address TEXT NOT NULL DEFAULT '',
  website TEXT NOT NULL DEFAULT '',
  created_at TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS file_ops (
  id INTEGER PRIMARY KEY,
  kind TEXT NOT NULL,
  payload TEXT NOT NULL,
  created_at TEXT NOT NULL,
  undone INTEGER NOT NULL DEFAULT 0
);
"""


def now_iso() -> str:
    return datetime.now().replace(microsecond=0).isoformat()


class Database:
    """스레드 하나짜리 SQLite 연결을 락으로 감싼 얇은 래퍼."""

    def __init__(self, path: Path | str):
        self.path = str(path)
        self.lock = threading.RLock()
        self.conn = sqlite3.connect(self.path, check_same_thread=False, isolation_level=None)
        self.conn.row_factory = sqlite3.Row
        if self.path != ":memory:":
            self.conn.execute("PRAGMA journal_mode=WAL")
        self.fts_tokenizer = "trigram"
        self._migrate()

    def _migrate(self) -> None:
        with self.lock:
            self.conn.executescript(SCHEMA)
            try:
                self.conn.execute(
                    "CREATE VIRTUAL TABLE IF NOT EXISTS search_index USING fts5("
                    "kind UNINDEXED, ref UNINDEXED, title, body, updated UNINDEXED, tokenize='trigram')"
                )
            except sqlite3.OperationalError:
                # 오래된 SQLite(3.34 미만)는 trigram 토크나이저가 없다. 검색은 LIKE로 대체된다.
```

```
                self.fts_tokenizer = "unicode61"
                self.conn.execute(
                    "CREATE VIRTUAL TABLE IF NOT EXISTS search_index USING fts5("
                    "kind UNINDEXED, ref UNINDEXED, title, body, updated UNINDEXED)"
                )
            self.conn.execute(f"PRAGMA user_version={SCHEMA_VERSION}")

    def execute(self, sql: str, params: tuple | dict = ()) -> sqlite3.Cursor:
        with self.lock:
            return self.conn.execute(sql, params)

    def insert(self, sql: str, params: tuple | dict = ()) -> int:
        with self.lock:
            return int(self.conn.execute(sql, params).lastrowid)

    def query(self, sql: str, params: tuple | dict = ()) -> list[dict]:
        with self.lock:
            return [dict(r) for r in self.conn.execute(sql, params).fetchall()]

    def one(self, sql: str, params: tuple | dict = ()) -> dict | None:
        with self.lock:
            row = self.conn.execute(sql, params).fetchone()
            return dict(row) if row else None

    def scalar(self, sql: str, params: tuple | dict = ()) -> Any:
        with self.lock:
            row = self.conn.execute(sql, params).fetchone()
            return row[0] if row else None

    @contextmanager
    def transaction(self) -> Iterator[sqlite3.Connection]:
        with self.lock:
            self.conn.execute("BEGIN")
            try:
                yield self.conn
            except BaseException:
                self.conn.execute("ROLLBACK")
                raise
            else:
                self.conn.execute("COMMIT")

    # --- 설정 ---
    def get_setting(self, key: str, default: Any = None) -> Any:
        row = self.one("SELECT value FROM settings WHERE key=?", (key,))
        return json.loads(row["value"]) if row else default

    def set_setting(self, key: str, value: Any) -> None:
        self.execute(
            "INSERT INTO settings(key, value) VALUES(?, ?) ON CONFLICT(key) DO UPDATE SET value=excluded.value",
            (key, json.dumps(value, ensure_ascii=False)),
        )

    def all_settings(self) -> dict:
        return {r["key"]: json.loads(r["value"]) for r in self.query("SELECT key, value FROM settings")}

    def close(self) -> None:
        with self.lock:
            self.conn.close()
```

```python
from __future__ import annotations


class AppError(Exception):
    """사용자에게 그대로 보여줄 수 있는 오류. message는 한국어로 쓴다."""

    status = 400

    def __init__(self, message: str, **extra):
        super().__init__(message)
        self.message = message
        self.extra = extra

    def to_dict(self) -> dict:
        return {"error": self.message, **self.extra}


class NotFound(AppError):
    status = 404


class FeatureUnavailable(AppError):
    """선택 구성요소(모델, 라이브러리)가 설치되지 않아 쓸 수 없는 기능."""

    status = 424


class LicenseRequired(AppError):
    status = 402


class Cancelled(Exception):
    pass
```

```python
"""문서에서 텍스트 뽑기: txt/md/csv, docx, hwpx, hwp(5.0), pdf."""
from __future__ import annotations

import html
import re
import struct
import zipfile
import zlib
from pathlib import Path

from .errors import AppError

TEXT_EXTS = {".txt", ".md", ".csv", ".tsv", ".log", ".json", ".srt", ".vtt", ".html", ".xml"}
DOC_EXTS = TEXT_EXTS | {".docx", ".hwpx", ".hwp", ".pdf"}


def read_text_file(path: Path) -> tuple[str, str]:
    """(텍스트, 인코딩). 한국 문서는 UTF-8 아니면 CP949인 경우가 대부분이다."""
    raw = path.read_bytes()
    for enc in ("utf-8-sig", "cp949", "utf-16"):
        try:
            return raw.decode(enc), enc
        except UnicodeDecodeError:
            continue
    return raw.decode("utf-8", errors="replace"), "utf-8"


def extract_text(path: str | Path) -> str:
    path = Path(path)
    if not path.exists():
        raise AppError(f"파일을 찾을 수 없어요: {path}")
    ext = path.suffix.lower()
    if ext in TEXT_EXTS:
        return read_text_file(path)[0]
    if ext == ".docx":
        return docx_text(path)
    if ext == ".hwpx":
        return hwpx_text(path)
    if ext == ".hwp":
        return hwp_text(path)
    if ext == ".pdf":
        return pdf_text(path)
    raise AppError(f"지원하지 않는 문서 형식이에요: {ext}")


def docx_text(path: Path) -> str:
    import docx

    doc = docx.Document(str(path))
    lines = []
    for p in doc.element.body.iter("{http://schemas.openxmlformats.org/wordprocessingml/2006/main}p"):
        texts = [t.text or "" for t in p.iter("{http://schemas.openxmlformats.org/wordprocessingml/2006/main}t")]
        lines.append("".join(texts))
    return "\n".join(lines)


HWPX_SECTION_RE = re.compile(r"^Contents/section\d+\.xml$")
HWPX_TOKEN_RE = re.compile(r"<hp:t(?:\s[^>]*)?>(.*?)</hp:t>|<hp:t\s*/>|<hp:p\b[^>]*>|</hp:p>", re.S)
TAG_RE = re.compile(r"<[^>]+>")


def hwpx_sections(zf: zipfile.ZipFile) -> list[str]:
```

```
    names = [n for n in zf.namelist() if HWPX_SECTION_RE.match(n)]
    return sorted(names, key=lambda n: int(re.search(r"(\d+)", n).group(1)))


def hwpx_text(path: Path) -> str:
    with zipfile.ZipFile(path) as zf:
        parts = []
        for name in hwpx_sections(zf):
            xml = zf.read(name).decode("utf-8")
            buf: list[str] = []
            for m in HWPX_TOKEN_RE.finditer(xml):
                tok = m.group(0)
                if tok.startswith("<hp:p") or tok == "</hp:p>":
                    if buf and buf[-1] != "\n":
                        buf.append("\n")
                elif m.group(1) is not None:
                    buf.append(html.unescape(TAG_RE.sub("", m.group(1))))
            parts.append("".join(buf).strip("\n"))
        return "\n".join(parts)


# --- HWP 5.0 바이너리 ---
HWPTAG_PARA_TEXT = 67
# 8 WCHAR(16바이트)를 차지하는 제어 문자 (인라인/확장 컨트롤)
_HWP_WIDE_CTRL = {1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23}


def _hwp_para_text(data: bytes) -> str:
    out = []
    i = 0
    n = len(data) // 2
    while i < n:
        code = struct.unpack_from("<H", data, i * 2)[0]
        if code < 32:
            if code == 9:
                out.append("\t")
            elif code in (10, 13):
                out.append("\n")
            i += 8 if code in _HWP_WIDE_CTRL else 1
            continue
        out.append(chr(code))
        i += 1
    return "".join(out)


def _hwp_records(stream: bytes):
    pos = 0
    while pos + 4 <= len(stream):
        header = struct.unpack_from("<I", stream, pos)[0]
        tag = header & 0x3FF
        size = (header >> 20) & 0xFFF
        pos += 4
        if size == 0xFFF:
            size = struct.unpack_from("<I", stream, pos)[0]
            pos += 4
        yield tag, stream[pos : pos + size]
        pos += size


def hwp_text(path: Path) -> str:
    import olefile

```

```
    if not olefile.isOleFile(str(path)):
        raise AppError("HWP 5.0 형식이 아니에요. (HWP 3.0 이하는 지원하지 않아요)")
    with olefile.OleFileIO(str(path)) as ole:
        header = ole.openstream("FileHeader").read()
        flags = struct.unpack_from("<I", header, 36)[0]
        compressed = bool(flags & 0x1)
        if flags & 0x2:
            raise AppError("암호가 걸린 HWP 문서는 읽을 수 없어요.")
        if flags & 0x4:
            raise AppError("배포용 HWP 문서는 읽을 수 없어요. 한글에서 HWPX로 저장해 주세요.")
        sections = sorted(
            (e for e in ole.listdir() if len(e) == 2 and e[0] == "BodyText" and e[1].startswith("Section")),
            key=lambda e: int(e[1][7:]),
        )
        paras = []
        for entry in sections:
            raw = ole.openstream(entry).read()
            stream = zlib.decompress(raw, -15) if compressed else raw
            for tag, payload in _hwp_records(stream):
                if tag == HWPTAG_PARA_TEXT:
                    paras.append(_hwp_para_text(payload).rstrip("\n"))
        return "\n".join(paras)


def pdf_page_chars(page) -> tuple[str, object]:
    """페이지 글자를 글자 인덱스와 1:1로 맞춘 문자열과 텍스트 페이지를 돌려준다."""
    import pypdfium2.raw as raw

    tp = page.get_textpage()
    n = tp.count_chars()
    chars = []
    for i in range(n):
        code = raw.FPDFText_GetUnicode(tp, i)
        chars.append(chr(code) if 0 < code < 0x110000 else " ")
    return "".join(chars).replace("\r", " "), tp


def pdf_text(path: Path) -> str:
    import pypdfium2 as pdfium

    doc = pdfium.PdfDocument(str(path))
    try:
        return "\n".join(pdf_page_chars(doc[i])[0] for i in range(len(doc)))
    finally:
        doc.close()
```

```python
from __future__ import annotations

import subprocess
import sys

from .errors import FeatureUnavailable
from .tools import find_tool


def find_ffmpeg() -> str | None:
    # 배포판에서는 엔진 옆(또는 앱 리소스 폴더)에 ffmpeg를 같이 넣는다.
    found = find_tool("ffmpeg")
    if found:
        return found
    try:
        import imageio_ffmpeg

        return imageio_ffmpeg.get_ffmpeg_exe()
    except Exception:  # noqa: BLE001
        return None


def require_ffmpeg() -> str:
    exe = find_ffmpeg()
    if not exe:
        raise FeatureUnavailable(
            "영상 처리를 하려면 ffmpeg가 필요해요. 설정 > 구성요소를 확인해 주세요.",
            component="ffmpeg",
            install_hint="pip install 'localbiseo-engine[ffmpeg]'",
        )
    return exe


def run(args: list[str], **kwargs) -> subprocess.CompletedProcess:
    exe = require_ffmpeg()
    flags = 0
    if sys.platform == "win32":
        flags = subprocess.CREATE_NO_WINDOW  # type: ignore[attr-defined]
    return subprocess.run([exe, "-hide_banner", *args], capture_output=True, text=True, encoding="utf-8", errors="replace", creationflags=flags


def popen(args: list[str]) -> subprocess.Popen:
    exe = require_ffmpeg()
    flags = 0
    if sys.platform == "win32":
        flags = subprocess.CREATE_NO_WINDOW  # type: ignore[attr-defined]
    return subprocess.Popen(
        [exe, "-hide_banner", *args],
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True,
        encoding="utf-8",
        errors="replace",
        creationflags=flags,
    )
```

```python
"""한글 글꼴 찾기. Pillow 기본 글꼴에는 한글이 없다."""
from __future__ import annotations

import os
import sys
from functools import lru_cache
from pathlib import Path

from PIL import ImageFont

CANDIDATES = [
    # Windows
    r"C:\Windows\Fonts\malgun.ttf",
    r"C:\Windows\Fonts\malgunbd.ttf",
    r"C:\Windows\Fonts\gulim.ttc",
    # macOS
    "/System/Library/Fonts/AppleSDGothicNeo.ttc",
    "/System/Library/Fonts/Supplemental/AppleGothic.ttf",
    "/Library/Fonts/NanumGothic.ttf",
    # Linux
    "/usr/share/fonts/truetype/nanum/NanumGothic.ttf",
    "/usr/share/fonts/truetype/nanum/NanumGothicBold.ttf",
    "/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc",
    "/usr/share/fonts/noto-cjk/NotoSansCJK-Regular.ttc",
]


@lru_cache(maxsize=1)
def korean_font_path() -> str | None:
    env = os.environ.get("LOCALBISEO_FONT")
    if env and Path(env).exists():
        return env
    bundled = Path(getattr(sys, "_MEIPASS", Path(__file__).parent)) / "fonts" / "NanumGothic.ttf"
    for path in [str(bundled), *CANDIDATES]:
        if Path(path).exists():
            return path
    return None


def load(size: int) -> ImageFont.ImageFont:
    path = korean_font_path()
    if path:
        return ImageFont.truetype(path, size)
    return ImageFont.load_default(size)
```

```python
from __future__ import annotations

import threading
import time
import traceback
import uuid
from concurrent.futures import ThreadPoolExecutor
from dataclasses import dataclass, field
from typing import Any, Callable

from .errors import AppError, Cancelled


@dataclass
class Job:
    id: str
    kind: str
    title: str
    status: str = "queued"  # queued | running | done | error | cancelled
    progress: float = 0.0
    message: str = ""
    result: Any = None
    error: str | None = None
    created_at: float = field(default_factory=time.time)
    finished_at: float | None = None
    _cancel: threading.Event = field(default_factory=threading.Event, repr=False)

    def to_dict(self) -> dict:
        return {
            "id": self.id,
            "kind": self.kind,
            "title": self.title,
            "status": self.status,
            "progress": round(self.progress, 4),
            "message": self.message,
            "result": self.result,
            "error": self.error,
            "created_at": self.created_at,
            "finished_at": self.finished_at,
        }


class JobContext:
    """작업 함수에 넘겨지는 진행률/취소 핸들."""

    def __init__(self, job: Job):
        self._job = job

    def progress(self, value: float, message: str | None = None) -> None:
        self._job.progress = max(0.0, min(1.0, value))
        if message is not None:
            self._job.message = message
        self.check()

    def check(self) -> None:
        if self._job._cancel.is_set():
            raise Cancelled()

    @property
    def cancelled(self) -> bool:
        return self._job._cancel.is_set()

```

```
63
64  class NullContext(JobContext):
65      """작업 큐 밖(테스트, 동기 호출)에서 쓰는 빈 컨텍스트."""
66
67      def __init__(self):
68          super().__init__(Job(id="-", kind="sync", title=""))
69
70
71  class JobManager:
72      def __init__(self, max_workers: int = 2, keep: int = 200):
73          self._pool = ThreadPoolExecutor(max_workers=max_workers, thread_name_prefix="job")
74          self._jobs: dict[str, Job] = {}
75          self._lock = threading.Lock()
76          self._keep = keep
77
78      def submit(self, kind: str, title: str, fn: Callable[..., Any], *args, **kwargs) -> Job:
79          job = Job(id=uuid.uuid4().hex[:12], kind=kind, title=title)
80          with self._lock:
81              self._jobs[job.id] = job
82              self._trim()
83          self._pool.submit(self._run, job, fn, args, kwargs)
84          return job
85
86      def _run(self, job: Job, fn, args, kwargs) -> None:
87          if job._cancel.is_set():
88              job.status = "cancelled"
89              job.finished_at = time.time()
90              return
91          job.status = "running"
92          try:
93              job.result = fn(JobContext(job), *args, **kwargs)
94              job.status = "done"
95              job.progress = 1.0
96          except Cancelled:
97              job.status = "cancelled"
98          except AppError as e:
99              job.status = "error"
100             job.error = e.message
101         except Exception as e:  # noqa: BLE001 - 작업 실패는 사용자에게 보여준다
102             traceback.print_exc()
103             job.status = "error"
104             job.error = f"처리 중 오류가 났어요: {e}"
105         finally:
106             job.finished_at = time.time()
107
108     def _trim(self) -> None:
109         finished = [j for j in self._jobs.values() if j.finished_at]
110         if len(self._jobs) <= self._keep:
111             return
112         for j in sorted(finished, key=lambda j: j.finished_at or 0)[: len(self._jobs) - self._keep]:
113             self._jobs.pop(j.id, None)
114
115     def get(self, job_id: str) -> Job | None:
116         return self._jobs.get(job_id)
117
118     def list(self) -> list[Job]:
119         return sorted(self._jobs.values(), key=lambda j: j.created_at, reverse=True)
120
121     def cancel(self, job_id: str) -> bool:
122         job = self._jobs.get(job_id)
123         if not job or job.finished_at:
124             return False
```

```
        job._cancel.set()
        return True

    def wait(self, job_id: str, timeout: float = 60) -> Job:
        """테스트용: 작업이 끝날 때까지 기다린다."""
        deadline = time.time() + timeout
        job = self._jobs[job_id]
        while not job.finished_at and time.time() < deadline:
            time.sleep(0.02)
        return job

    def shutdown(self) -> None:
        for job in self._jobs.values():
            job._cancel.set()
        self._pool.shutdown(wait=False, cancel_futures=True)
```

```python
"""한글 처리 유틸: 자모 분해/조합, 문장 분리, 조사 떼기, 성씨."""
from __future__ import annotations

import re

HANGUL_BASE = 0xAC00
HANGUL_LAST = 0xD7A3

# 종성 인덱스
JONG_NONE, JONG_M, JONG_B, JONG_SS = 0, 16, 17, 20
# 중성 인덱스
JUNG_A, JUNG_AE, JUNG_EO, JUNG_E, JUNG_YEO, JUNG_O, JUNG_WA, JUNG_U, JUNG_WO, JUNG_EU, JUNG_I = 0, 1, 4, 5, 6, 8, 9, 13, 14, 18, 20

SURNAMES_1 = set(
    "김이박최정강조윤장임한오서신권황안송류유전홍고문양손배백허남심노하곽성차주우구민진나지엄채원천방공현함변염여추도소석선설마길연위표명기반왕금옥육
)
SURNAMES_2 = {"남궁", "황보", "제갈", "선우", "독고", "사공", "서문", "동방", "어금", "망절"}


def is_hangul(ch: str) -> bool:
    return len(ch) == 1 and HANGUL_BASE <= ord(ch) <= HANGUL_LAST


def decompose(ch: str) -> tuple[int, int, int]:
    code = ord(ch) - HANGUL_BASE
    return code // 588, (code % 588) // 28, code % 28


def compose(cho: int, jung: int, jong: int = 0) -> str:
    return chr(HANGUL_BASE + cho * 588 + jung * 28 + jong)


def jong_of(ch: str) -> int:
    return decompose(ch)[2] if is_hangul(ch) else -1


def has_jong(ch: str) -> bool:
    return jong_of(ch) > 0


def with_jong(ch: str, jong: int) -> str:
    cho, jung, _ = decompose(ch)
    return compose(cho, jung, jong)


def with_jung(ch: str, jung: int) -> str:
    cho, _, jong = decompose(ch)
    return compose(cho, jung, jong)


def looks_like_name(word: str) -> bool:
    if not 2 <= len(word) <= 4 or not all(is_hangul(c) for c in word):
        return False
    if len(word) == 4:
        return word[:2] in SURNAMES_2
    return word[0] in SURNAMES_1 or word[:2] in SURNAMES_2


_SENT_RE = re.compile(r"[^\n.!?。]*(?:[.!?。]+|\n|$)")


def split_sentences(text: str) -> list[tuple[int, int, str]]:
```

```
    """(시작, 끝, 문장) 목록. 빈 문장은 뺀다."""
    out = []
    for m in _SENT_RE.finditer(text):
        raw = m.group(0)
        if not raw.strip():
            continue
        lead = len(raw) - len(raw.lstrip())
        s = m.start() + lead
        sentence = raw.strip()
        out.append((s, s + len(sentence), sentence))
    return out


JOSA = sorted(
    [
        "이었습니다", "였습니다", "입니다", "합니다", "했습니다", "됩니다", "에서는", "으로는", "에게서", "한테서",
        "에서", "으로", "에게", "한테", "께서", "부터", "까지", "처럼", "보다", "이라", "라고", "이고", "하고",
        "에는", "와는", "과는", "이나", "든지", "마다", "조차", "밖에", "만큼", "이다", "했다", "한다", "하는",
        "해서", "하면", "하여", "되어", "되는", "된다", "은", "는", "이", "가", "을", "를", "에", "의", "도",
        "만", "와", "과", "로", "고", "요",
    ],
    key=len,
    reverse=True,
)

STOPWORDS = {
    "그리고", "그런데", "그래서", "하지만", "그러나", "그러면", "이제", "일단", "근데", "그냥", "진짜", "정말",
    "약간", "조금", "많이", "너무", "이번", "저희", "우리", "제가", "저는", "그거", "이거", "저거", "그게",
    "이게", "뭔가", "어떤", "이런", "그런", "저런", "다시", "또한", "그래", "네네", "아니", "있는", "없는",
    "있습니다", "없습니다", "그렇죠", "그렇게", "이렇게", "같은", "같습니다", "말씀", "생각", "부분", "경우",
}

_WORD_RE = re.compile(r"[가-힣A-Za-z0-9]+")


def strip_josa(word: str) -> str:
    if len(word) <= 1 or not is_hangul(word[-1]):
        return word
    for j in JOSA:
        if word.endswith(j) and len(word) - len(j) >= 2:
            return word[: -len(j)]
    return word


def content_words(text: str) -> list[str]:
    words = []
    for raw in _WORD_RE.findall(text):
        w = strip_josa(raw)
        if len(w) >= 2 and w not in STOPWORDS and not w.isdigit():
            words.append(w.lower())
    return words
```

```
"""오프라인 라이선스 (Ed25519 서명).

키 형식: LB1.<base64url(payload JSON)>.<base64url(서명)>
발급은 tools/license_gen.py 로 하고, 앱에는 공개키(license_pubkey.pem)만 들어간다.
체험판은 첫 실행부터 TRIAL_DAYS일 동안 모든 기능을 쓸 수 있고,
그 뒤에는 코어 기능(회의록, 파일 마스킹, 이미지 · 영상)만 라이선스가 필요하다.
"""
from __future__ import annotations

import base64
import json
import sys
from datetime import date, datetime, timedelta
from pathlib import Path

from .db import Database
from .errors import AppError, LicenseRequired

TRIAL_DAYS = 14
PREFIX = "LB1"


def _pubkey_path() -> Path:
    base = Path(getattr(sys, "_MEIPASS", Path(__file__).parent.parent))
    bundled = base / "localbiseo" / "core" / "license_pubkey.pem"
    return bundled if bundled.exists() else Path(__file__).parent / "license_pubkey.pem"


def _b64d(s: str) -> bytes:
    return base64.urlsafe_b64decode(s + "=" * (-len(s) % 4))


def _b64e(b: bytes) -> str:
    return base64.urlsafe_b64encode(b).decode().rstrip("=")


def sign(payload: dict, private_key) -> str:
    body = json.dumps(payload, ensure_ascii=False, separators=(",", ":")).encode()
    return f"{PREFIX}.{_b64e(body)}.{_b64e(private_key.sign(body))}"


def verify(key: str, public_key=None) -> dict:
    from cryptography.exceptions import InvalidSignature
    from cryptography.hazmat.primitives.serialization import load_pem_public_key

    key = "".join(key.split())
    parts = key.split(".")
    if len(parts) != 3 or parts[0] != PREFIX:
        raise AppError("라이선스 키 형식이 올바르지 않아요.")
    try:
        body, sig = _b64d(parts[1]), _b64d(parts[2])
    except ValueError as e:
        raise AppError("라이선스 키 형식이 올바르지 않아요.") from e
    if public_key is None:
        public_key = load_pem_public_key(_pubkey_path().read_bytes())
    try:
        public_key.verify(sig, body)
    except InvalidSignature as e:
        raise AppError("유효하지 않은 라이선스 키예요.") from e
    payload = json.loads(body)
    exp = payload.get("exp")
    if exp and date.fromisoformat(exp) < date.today():
```

```
        raise AppError("기간이 끝난 라이선스 키예요.")
    return payload


def state(db: Database) -> dict:
    started = db.get_setting("trial_started")
    if not started:
        started = date.today().isoformat()
        db.set_setting("trial_started", started)
    key = db.get_setting("license_key")
    if key:
        try:
            payload = verify(key)
            return {"status": "licensed", "holder": payload.get("name", ""), "id": payload.get("id", ""), "edition": payload.get("ed", "pro")}
        except AppError:
            pass
    end = date.fromisoformat(started) + timedelta(days=TRIAL_DAYS)
    left = (end - date.today()).days
    return {"status": "trial" if left >= 0 else "expired", "days_left": max(left, 0), "trial_ends": end.isoformat()}


def activate(db: Database, key: str) -> dict:
    payload = verify(key)
    db.set_setting("license_key", "".join(key.split()))
    db.set_setting("license_activated_at", datetime.now().isoformat(timespec="seconds"))
    return {"status": "licensed", "holder": payload.get("name", ""), "id": payload.get("id", ""), "edition": payload.get("ed", "pro")}


def require_pro(db: Database) -> None:
    if state(db)["status"] == "expired":
        raise LicenseRequired("체험 기간이 끝났어요. 설정 > 라이선스에서 키를 입력해 주세요.")
```

```python
"""선택 기능: 로컬 GGUF 모델(llama.cpp)로 문장 생성.

모델 파일이 models/llm/*.gguf 에 있고 llama-cpp-python이 설치돼 있을 때만 켜진다.
없으면 각 기능은 규칙 기반 처리로 동작한다.
"""
from __future__ import annotations

import threading
from pathlib import Path

from . import optional
from .config import sub_dir

_lock = threading.Lock()
_model = None
_model_path: str | None = None


def model_file() -> Path | None:
    files = sorted(sub_dir("llm", sub_dir("models")).glob("*.gguf"))
    return files[0] if files else None


def available() -> bool:
    return optional.has_module("llama_cpp") and model_file() is not None


def complete(prompt: str, max_tokens: int = 512, temperature: float = 0.3) -> str:
    global _model, _model_path
    path = model_file()
    if not optional.has_module("llama_cpp") or path is None:
        raise RuntimeError("로컬 AI 모델이 없어요.")
    with _lock:
        if _model is None or _model_path != str(path):
            from llama_cpp import Llama

            _model = Llama(model_path=str(path), n_ctx=4096, verbose=False)
            _model_path = str(path)
        out = _model.create_chat_completion(
            messages=[{"role": "user", "content": prompt}],
            max_tokens=max_tokens,
            temperature=temperature,
        )
    return out["choices"][0]["message"]["content"].strip()
```

```python
"""OCR 백엔드 묶음. PaddleOCR > Tesseract > RapidOCR 순서로 있는 걸 쓴다."""
from __future__ import annotations

import re
import threading
from dataclasses import dataclass, field
from pathlib import Path

from PIL import Image

from . import optional
from .tools import find_tool

Box = tuple[int, int, int, int]  # left, top, right, bottom


@dataclass
class OcrLine:
    text: str
    box: Box
    conf: float = 1.0
    # 줄 안의 단어 위치: (텍스트 시작, 끝, 박스). 마스킹할 때 글자 범위를 박스로 바꾸는 데 쓴다.
    words: list[tuple[int, int, Box]] = field(default_factory=list)

    def span_box(self, start: int, end: int) -> Box:
        hits = [b for s, e, b in self.words if s < end and e > start]
        if hits:
            return (min(b[0] for b in hits), min(b[1] for b in hits), max(b[2] for b in hits), max(b[3] for b in hits))
        # 단어 정보가 없으면 글자 수 비율로 나눈다.
        left, top, right, bottom = self.box
        n = max(len(self.text), 1)
        width = right - left
        return (left + int(width * start / n), top, left + int(width * end / n + 0.999), bottom)

    def to_dict(self) -> dict:
        return {"text": self.text, "box": list(self.box), "conf": round(self.conf, 3)}


_lock = threading.Lock()
_engines: dict[str, object] = {}


def backend_name() -> str | None:
    for name in ("paddleocr", "pytesseract", "rapidocr_onnxruntime"):
        if optional.has_module(name):
            if name == "pytesseract" and not optional.available("ocr"):
                continue
            return name
    return None


def ocr_image(image: Image.Image | str | Path, block: bool = False) -> list[OcrLine]:
    """block=True: 사용자가 고른 영역처럼 글 덩어리 하나라고 보고 읽는다 (화면 OCR에서 정확도가 높다)."""
    optional.require("ocr")
    img = Image.open(image) if isinstance(image, (str, Path)) else image
    img = img.convert("RGB")
    name = backend_name()
    if name == "paddleocr":
        lines = _paddle(img)
    elif name == "pytesseract":
        lines = _tesseract(img, block)
    else:
```

```
        lines = _rapid(img)
    return [l for l in lines if not _is_noise(l)]


_JAMO = re.compile(r"^[\u3131-\u318E]+$")


def _psm(block: bool) -> str:
    # psm 6: 글 덩어리 하나(고른 영역), psm 4: 크기가 다른 줄이 세로로 이어진 문서.
    # 기본값(psm 3)은 화면 캡처나 영수증에서 줄을 통째로 빠뜨리거나 숫자 · 괄호를 자주 틀린다.
    return "--psm 6" if block else "--psm 4"


def ocr_text(image: Image.Image, block: bool = False) -> str:
    """글자만 필요할 때. Tesseract는 자체 출력이 띄어쓰기를 더 정확하게 살린다."""
    optional.require("ocr")
    img = image.convert("RGB")
    if backend_name() != "pytesseract":
        return "\n".join(l.text for l in ocr_image(img, block))
    import pytesseract

    cmd = find_tool("tesseract")
    if cmd:
        pytesseract.pytesseract.tesseract_cmd = cmd
    lang = "kor+eng" if "kor" in pytesseract.get_languages(config="") else "eng"
    raw = pytesseract.image_to_string(img, lang=lang, config=_psm(block))
    lines = []
    for line in raw.splitlines():
        t = line.strip()
        compact = t.replace(" ", "")
        if not t or _JAMO.match(compact) or (len(compact) <= 2 and not re.search(r"[가-힣0-9]", compact)):
            continue
        lines.append(t)
    return "\n".join(lines)


def _is_noise(line: OcrLine) -> bool:
    # 테두리 · 그림자에서 생기는 낱자모('ㄱ', 'ㅋ')나 신뢰도 낮은 한 글자 줄은 버린다.
    text = line.text.strip()
    return not text or bool(_JAMO.match(text.replace(" ", ""))) or (len(text) == 1 and line.conf < 0.85)


def _tesseract(img: Image.Image, block: bool = False) -> list[OcrLine]:
    import pytesseract

    cmd = find_tool("tesseract")
    if cmd:
        pytesseract.pytesseract.tesseract_cmd = cmd

    langs = pytesseract.get_languages(config="")
    lang = "kor+eng" if "kor" in langs else "eng"
    config = _psm(block)
    data = pytesseract.image_to_data(img, lang=lang, config=config, output_type=pytesseract.Output.DICT)
    groups: dict[tuple, list[int]] = {}
    for i, word in enumerate(data["text"]):
        # 신뢰도가 아주 낮은 조각이나 낱자모는 잡음(테두리, 얼룩)인 경우가 많다.
        if word and word.strip() and float(data["conf"][i]) >= 35 and not _JAMO.match(word.strip()):
            key = (data["block_num"][i], data["par_num"][i], data["line_num"][i])
            groups.setdefault(key, []).append(i)
    lines = []
    for idxs in groups.values():
        text = ""
```

```
        words, confs = [], []
        prev_right = None
        heights = sorted(data["height"][i] for i in idxs)
        height = heights[len(heights) // 2] or 1
        for i in idxs:
            w = data["text"][i].strip()
            box = (data["left"][i], data["top"][i], data["left"][i] + data["width"][i], data["top"][i] + data["height"][i])
            # Tesseract는 한글을 음절마다 띄워서 내놓는다. 실제 간격이 좁으면 붙인다.
            if prev_right is not None and box[0] - prev_right > height * 0.45:
                text += " "
            words.append((len(text), len(text) + len(w), box))
            text += w
            prev_right = box[2]
            confs.append(float(data["conf"][i]) / 100)
        boxes = [b for _, _, b in words]
        line_box = (min(b[0] for b in boxes), min(b[1] for b in boxes), max(b[2] for b in boxes), max(b[3] for b in boxes))
        lines.append(OcrLine(text, line_box, sum(confs) / len(confs), words))
    lines.sort(key=lambda l: (l.box[1] // 10, l.box[0]))
    return lines


def _quad_box(points) -> Box:
    xs = [int(p[0]) for p in points]
    ys = [int(p[1]) for p in points]
    return (min(xs), min(ys), max(xs), max(ys))


def _paddle(img: Image.Image) -> list[OcrLine]:
    import numpy as np
    from paddleocr import PaddleOCR

    with _lock:
        engine = _engines.get("paddle")
        if engine is None:
            engine = PaddleOCR(lang="korean", use_angle_cls=True, show_log=False)
            _engines["paddle"] = engine
    result = engine.ocr(np.array(img), cls=True) or []
    lines = []
    for page in result:
        for points, (text, conf) in page or []:
            lines.append(OcrLine(text, _quad_box(points), float(conf)))
    return lines


def _rapid(img: Image.Image) -> list[OcrLine]:
    import numpy as np
    from rapidocr_onnxruntime import RapidOCR

    with _lock:
        engine = _engines.get("rapid")
        if engine is None:
            engine = RapidOCR()
            _engines["rapid"] = engine
    result, _ = engine(np.array(img))
    return [OcrLine(text, _quad_box(points), float(conf)) for points, text, conf in result or []]
```

```python
"""선택 구성요소(무거운 모델/라이브러리) 확인.

기본 설치에는 가벼운 기능만 들어 있다. STT, OCR, 배경 제거 같은 기능은
구성요소가 있을 때만 켜지고, 없으면 어떤 걸 설치해야 하는지 안내한다.
"""
from __future__ import annotations

import importlib.util
from dataclasses import dataclass

from .errors import FeatureUnavailable
from .tools import find_tool


@dataclass(frozen=True)
class Component:
    key: str
    label: str
    modules: tuple[str, ...]  # 이 중 하나라도 있으면 사용 가능
    extra: str
    used_by: str


COMPONENTS: dict[str, Component] = {
    c.key: c
    for c in [
        Component("stt", "음성 인식 (Whisper)", ("faster_whisper",), "stt", "회의록, 자막"),
        Component("diarize", "화자 분리 (실험적)", ("speechbrain",), "diarize", "회의록 화자 구분"),
        Component("ocr", "문자 인식 (OCR)", ("paddleocr", "pytesseract", "rapidocr_onnxruntime"), "ocr", "화면 OCR, 영수증 · 명함, 이미지 마스킹"),
        Component("rembg", "배경 제거", ("rembg",), "rembg", "이미지 일괄 처리"),
        Component("face", "얼굴 감지", ("cv2",), "face", "이미지 마스킹"),
        Component("translate", "오프라인 번역", ("argostranslate",), "translate", "번역"),
        Component("llm", "로컬 AI 문장 생성", ("llama_cpp",), "llm", "요약 · 문장 다듬기 품질 향상"),
    ]
}


def has_module(name: str) -> bool:
    try:
        return importlib.util.find_spec(name) is not None
    except (ImportError, ValueError):
        return False


def available(key: str) -> bool:
    comp = COMPONENTS[key]
    if key == "ocr" and has_module("pytesseract") and not has_module("paddleocr") and not has_module("rapidocr_onnxruntime"):
        # pytesseract는 tesseract 실행 파일도 있어야 한다.
        return find_tool("tesseract") is not None
    return any(has_module(m) for m in comp.modules)


def require(key: str) -> None:
    if not available(key):
        comp = COMPONENTS[key]
        raise FeatureUnavailable(
            f"'{comp.label}' 구성요소가 설치되어 있지 않아요. 설정 > 구성요소에서 설치해 주세요.",
            component=key,
            install_hint=f"pip install 'localbiseo-engine[{comp.extra}]'",
        )
```

```
63  def status() -> list[dict]:
64      return [
65          {"key": c.key, "label": c.label, "available": available(c.key), "used_by": c.used_by, "extra": c.extra}
66          for c in COMPONENTS.values()
67      ]
```

```python
"""통합 검색 색인 (SQLite FTS5, trigram 토크나이저로 한글 부분 일치 지원)."""
from __future__ import annotations

import re

from .db import Database, now_iso

KIND_LABELS = {
    "note": "메모",
    "todo": "할 일",
    "clip": "클립보드",
    "transcript": "회의록",
    "file": "파일",
    "contact": "연락처",
}


def index(db: Database, kind: str, ref: str | int, title: str, body: str) -> None:
    ref = str(ref)
    with db.transaction() as conn:
        conn.execute("DELETE FROM search_index WHERE kind=? AND ref=?", (kind, ref))
        conn.execute(
            "INSERT INTO search_index(kind, ref, title, body, updated) VALUES(?,?,?,?,?)",
            (kind, ref, title or "", body or "", now_iso()),
        )


def remove(db: Database, kind: str, ref: str | int) -> None:
    db.execute("DELETE FROM search_index WHERE kind=? AND ref=?", (kind, str(ref)))


def _terms(q: str) -> list[str]:
    return [t for t in re.split(r"\s+", q.strip()) if t]


def snippet(text: str, terms: list[str], width: int = 70) -> tuple[str, list[list[int]]]:
    lower = text.lower()
    first = min((i for i in (lower.find(t.lower()) for t in terms) if i >= 0), default=0)
    start = max(0, first - width // 2)
    end = min(len(text), start + width * 2)
    piece = text[start:end].replace("\n", " ")
    prefix = "…" if start > 0 else ""
    suffix = "…" if end < len(text) else ""
    piece = prefix + piece + suffix
    marks = []
    lp = piece.lower()
    for t in terms:
        for m in re.finditer(re.escape(t.lower()), lp):
            marks.append([m.start(), m.end()])
    marks.sort()
    return piece, marks


def query(db: Database, q: str, kinds: list[str] | None = None, limit: int = 40) -> list[dict]:
    terms = _terms(q)
    if not terms:
        return []
    kind_sql = ""
    params: list = []
    if db.fts_tokenizer == "trigram" and all(len(t) >= 3 for t in terms):
        match = " ".join('"' + t.replace('"', '""') + '"' for t in terms)
        where = "search_index MATCH ?"
```

```
        params.append(match)
        order = "ORDER BY bm25(search_index, 5.0, 1.0)"
    else:
        # trigram은 3글자 미만 검색어를 못 찾으니 LIKE로 훑는다.
        clauses = []
        for t in terms:
            clauses.append("(title LIKE ? OR body LIKE ?)")
            like = f"%{t}%"
            params += [like, like]
        where = " AND ".join(clauses)
        order = "ORDER BY updated DESC"
    if kinds:
        kind_sql = f" AND kind IN ({','.join('?' * len(kinds))})"
        params += kinds
    rows = db.query(
        f"SELECT kind, ref, title, body, updated FROM search_index WHERE {where}{kind_sql} {order} LIMIT ?",
        (*params, limit),
    )
    out = []
    for r in rows:
        snip, marks = snippet(r["body"] or r["title"], terms)
        out.append(
            {
                "kind": r["kind"],
                "kind_label": KIND_LABELS.get(r["kind"], r["kind"]),
                "ref": r["ref"],
                "title": r["title"] or (r["body"][:40] if r["body"] else ""),
                "snippet": snip,
                "marks": marks,
                "updated": r["updated"],
            }
        )
    return out
```

```python
"""외부 실행 파일(ffmpeg, tesseract) 찾기.

배포판은 앱이 LOCALBISEO_RESOURCES(설치 폴더의 리소스 경로)를 넘겨준다.
macOS 앱은 셸 PATH를 물려받지 못해서 Homebrew 경로도 직접 살펴본다.
"""
from __future__ import annotations

import os
import shutil
import sys
from functools import lru_cache
from pathlib import Path

EXTRA_DIRS = ["/opt/homebrew/bin", "/usr/local/bin", "/usr/bin", r"C:\Program Files\Tesseract-OCR"]


@lru_cache(maxsize=None)
def find_tool(name: str) -> str | None:
    exe = f"{name}.exe" if sys.platform == "win32" else name
    env = os.environ.get(f"LOCALBISEO_{name.upper()}")
    if env and Path(env).exists():
        return env
    roots = []
    res = os.environ.get("LOCALBISEO_RESOURCES")
    if res:
        roots += [Path(res) / name, Path(res)]
    roots.append(Path(sys.executable).parent)
    for root in roots:
        if (root / exe).exists():
            return str(root / exe)
    found = shutil.which(name)
    if found:
        return found
    for d in EXTRA_DIRS:
        if (Path(d) / exe).exists():
            return str(Path(d) / exe)
    return None
```

```
1
```

```python
"""회의록 · 메모 문장에서 할 일(액션아이템) 뽑기."""
from __future__ import annotations

import re
from datetime import datetime

from fastapi import APIRouter
from pydantic import BaseModel

from ..core.context import Ctx
from ..core.korean import looks_like_name, split_sentences
from .notes import add_todo
from .schedule import find_date

router = APIRouter(prefix="/actions", tags=["액션아이템"])

ACTION_PATTERNS = [re.compile(p) for p in [
    r"기로\s?(?:했|함|하였|합의|결정|정했|정함)",
    r"(?:해야|돼야|되어야|드려야|보내야|받아야|가야|봐야|챙겨야)\s?(?:함|한다|합니다|해요|하고|할|겠|돼|된다|됩니다)?",
    r"(?:할 것|하실 것|해 주세요|해주세요|해 주시기|해주시기|부탁드립니다|부탁드려요|부탁해요|바랍니다|요청드립니다|요청합니다|해 주십시오|해주십시오
    r"(?:까지|전까지)\s?(?:\S+\s?){0,3}(?:제출|보고|공유|전달|완료|마감|회신|보내|올려|업로드|정리|확인|준비|작성|검토)",
    r"(?:하겠습니다|드리겠습니다|할게요|할께요|드릴게요|드릴께요|보겠습니다|진행하겠|챙기겠|맡겠|처리하겠)",
    r"(?:제가|저희가|내가|우리가|제 쪽에서)\s.*(?:하겠|할게|할께|드리겠|드릴게|볼게|보겠|맡을|챙길)",
]]
PREFIX_RE = re.compile(r"^\s*(?:[-*•·]\s*)?(?:\[\s?\]\s*)?(?:TODO|To-?do|할\s?일|액션|Action\s?Item|AI)\s*[:：]\s*", re.I)
CHECKBOX_RE = re.compile(r"^\s*[-*•]?\s*\[\s?\]\s+")
TITLES = r"(?:님|씨|팀장님?|과장님?|대리님?|부장님?|차장님?|매니저님?|선임님?|책임님?|실장님?|이사님?|대표님?|주임님?|사원)"
ASSIGNEE_RE = re.compile(rf"(?<![가-힣])([가-힣]{{2,4}}?)\s?({TITLES})?\s?(?:이|가|은|는|께서|께|에게|한테)\s")
ASSIGNEE_LABEL_RE = re.compile(r"(?:담당|담당자|책임)\s*[:：]\s*([가-힣]{2,4})|@([가-힣A-Za-z]{2,10})")
SELF_RE = re.compile(r"^(?:제가|저희가|내가|제 쪽에서)\s")


def _assignee(sentence: str) -> str | None:
    m = ASSIGNEE_LABEL_RE.search(sentence)
    if m:
        return m.group(1) or m.group(2)
    m = ASSIGNEE_RE.search(sentence[:30])
    if m and looks_like_name(m.group(1)) and (m.group(2) or len(m.group(1)) == 3):
        title = m.group(2) or ""
        # '홍길동님'은 붙이고 '김민수 대리'는 띄운다.
        return m.group(1) + (title if title in ("님", "씨") else f" {title}" if title else "")
    if SELF_RE.search(sentence):
        return "나"
    return None


def _clean(sentence: str) -> str:
    t = PREFIX_RE.sub("", sentence)
    t = CHECKBOX_RE.sub("", t)
    return t.strip(" .。!\t")


def extract_actions(text: str, now: datetime | None = None) -> list[dict]:
    today = (now or datetime.now()).date()
    items = []
    seen = set()
    for _, _, sentence in split_sentences(text):
        explicit = bool(PREFIX_RE.match(sentence) or CHECKBOX_RE.match(sentence))
        if not explicit and not any(p.search(sentence) for p in ACTION_PATTERNS):
            continue
        clean = _clean(sentence)
        if len(clean) < 4 or clean in seen:
```

```
            continue
        seen.add(clean)
        dm = find_date(clean, today)
        items.append({"text": clean, "assignee": _assignee(clean), "due": dm.value.isoformat() if dm else None})
    return items


class ExtractIn(BaseModel):
    text: str
    now: str | None = None


class SaveIn(BaseModel):
    items: list[dict]
    source: str | None = None


@router.post("/extract")
def extract_api(body: ExtractIn):
    now = datetime.fromisoformat(body.now) if body.now else None
    return {"items": extract_actions(body.text, now)}


@router.post("/save")
def save_api(body: SaveIn, ctx: Ctx):
    ids = [add_todo(ctx.db, i["text"], i.get("due"), i.get("assignee"), body.source) for i in body.items if i.get("text")]
    return {"ids": ids}
```

```python
"""클립보드 기록. 앱 셀이 복사 내용을 보내면 저장하고, 민감정보는 가려서 저장한다."""
from __future__ import annotations

from fastapi import APIRouter
from pydantic import BaseModel

from ..core import search
from ..core.context import Ctx
from ..core.db import Database, now_iso
from .masking.detect import detect, mask_text

router = APIRouter(prefix="/clipboard", tags=["클립보드"])

MAX_ITEMS = 500
MAX_CHARS = 20000
POLICIES = ("mask", "skip", "raw")  # 민감정보: 가려서 저장 / 저장 안 함 / 그대로 저장


def add_clip(db: Database, text: str) -> dict | None:
    text = text.strip("\x00")
    if not text.strip() or len(text) > MAX_CHARS or not db.get_setting("clipboard_enabled", True):
        return None
    policy = db.get_setting("clipboard_sensitive", "mask")
    findings = detect(text)
    kinds = sorted({f.kind for f in findings})
    if findings and policy == "skip":
        return {"skipped": True, "kinds": kinds}
    stored = mask_text(text)[0] if findings and policy == "mask" else text
    last = db.one("SELECT id, text FROM clips ORDER BY id DESC LIMIT 1")
    if last and last["text"] == stored:
        return {"id": last["id"], "duplicate": True}
    cid = db.insert(
        "INSERT INTO clips(text, sensitive, kinds, created_at) VALUES(?,?,?,?)",
        (stored, int(bool(findings)), ",".join(kinds), now_iso()),
    )
    search.index(db, "clip", cid, stored[:40], stored)
    _trim(db)
    return {"id": cid, "sensitive": bool(findings), "kinds": kinds}


def _trim(db: Database) -> None:
    old = db.query("SELECT id FROM clips WHERE pinned=0 ORDER BY id DESC LIMIT -1 OFFSET ?", (MAX_ITEMS,))
    for r in old:
        db.execute("DELETE FROM clips WHERE id=?", (r["id"],))
        search.remove(db, "clip", r["id"])


class ClipIn(BaseModel):
    text: str


class PolicyIn(BaseModel):
    policy: str


@router.post("")
def add(body: ClipIn, ctx: Ctx):
    return add_clip(ctx.db, body.text) or {"ignored": True}


@router.get("")
def list_clips(ctx: Ctx, q: str = "", limit: int = 100):
```

```
    if q:
        return ctx.db.query("SELECT * FROM clips WHERE text LIKE ? ORDER BY pinned DESC, id DESC LIMIT ?", (f"%{q}%", limit))
    return ctx.db.query("SELECT * FROM clips ORDER BY pinned DESC, id DESC LIMIT ?", (limit,))


@router.post("/{clip_id}/pin")
def pin(clip_id: int, ctx: Ctx):
    ctx.db.execute("UPDATE clips SET pinned = 1 - pinned WHERE id=?", (clip_id,))
    return ctx.db.one("SELECT * FROM clips WHERE id=?", (clip_id,))


@router.delete("/{clip_id}")
def delete(clip_id: int, ctx: Ctx):
    ctx.db.execute("DELETE FROM clips WHERE id=?", (clip_id,))
    search.remove(ctx.db, "clip", clip_id)
    return {"ok": True}


@router.delete("")
def clear(ctx: Ctx):
    for r in ctx.db.query("SELECT id FROM clips WHERE pinned=0"):
        search.remove(ctx.db, "clip", r["id"])
    ctx.db.execute("DELETE FROM clips WHERE pinned=0")
    return {"ok": True}


@router.get("/policy")
def get_policy(ctx: Ctx):
    return {"policy": ctx.db.get_setting("clipboard_sensitive", "mask"), "enabled": ctx.db.get_setting("clipboard_enabled", True)}


@router.put("/policy")
def set_policy(body: PolicyIn, ctx: Ctx):
    if body.policy not in POLICIES:
        return {"error": "mask, skip, raw 중 하나여야 해요."}
    ctx.db.set_setting("clipboard_sensitive", body.policy)
    return {"policy": body.policy}


@router.put("/enabled")
def set_enabled(body: dict, ctx: Ctx):
    ctx.db.set_setting("clipboard_enabled", bool(body.get("enabled", True)))
    return {"enabled": ctx.db.get_setting("clipboard_enabled", True)}
```

```python
"""중복 파일 · 비슷한 사진 찾기. 지우지 않고 보관함으로 옮겨서 되돌릴 수 있게 한다."""
from __future__ import annotations

import hashlib
import json
import os
import shutil
from datetime import datetime
from pathlib import Path

from fastapi import APIRouter
from PIL import Image
from pydantic import BaseModel, Field

from ..core.config import sub_dir
from ..core.context import Ctx, job_ref
from ..core.db import Database, now_iso
from ..core.errors import AppError, NotFound
from ..core.jobs import JobContext

router = APIRouter(prefix="/dedupe", tags=["중복 정리"])

IMAGE_EXTS = {".jpg", ".jpeg", ".png", ".webp", ".bmp", ".heic", ".gif", ".tif", ".tiff"}
SKIP_DIRS = {".git", "node_modules", "$RECYCLE.BIN", "System Volume Information", ".Trash", "__pycache__"}


def _walk(dirs: list[str], min_size: int, exts: set[str] | None = None) -> list[Path]:
    out = []
    for d in dirs:
        root = Path(d)
        if not root.is_dir():
            raise AppError(f"폴더를 찾을 수 없어요: {d}")
        for cur, subdirs, files in os.walk(root):
            subdirs[:] = [s for s in subdirs if s not in SKIP_DIRS and not s.startswith(".")]
            for name in files:
                p = Path(cur) / name
                if exts and p.suffix.lower() not in exts:
                    continue
                try:
                    if p.is_file() and not p.is_symlink() and p.stat().st_size >= min_size:
                        out.append(p)
                except OSError:
                    continue
    return out


def _hash(path: Path, limit: int | None = None) -> str:
    h = hashlib.blake2b(digest_size=20)
    with path.open("rb") as f:
        if limit:
            h.update(f.read(limit))
        else:
            for chunk in iter(lambda: f.read(1 << 20), b""):
                h.update(chunk)
    return h.hexdigest()


def find_duplicates(job: JobContext, dirs: list[str], min_size: int = 1) -> dict:
    job.progress(0.02, "파일 목록 만드는 중")
    files = _walk(dirs, min_size)
    by_size: dict[int, list[Path]] = {}
    for f in files:
```

```
        by_size.setdefault(f.stat().st_size, []).append(f)
    candidates = [g for g in by_size.values() if len(g) > 1]
    total = sum(len(g) for g in candidates) or 1
    done = 0
    groups = []
    for group in candidates:
        by_head: dict[str, list[Path]] = {}
        for f in group:
            try:
                by_head.setdefault(_hash(f, 65536), []).append(f)
            except OSError:
                pass
        for same_head in by_head.values():
            if len(same_head) < 2:
                continue
            by_full: dict[str, list[Path]] = {}
            for f in same_head:
                try:
                    by_full.setdefault(_hash(f), []).append(f)
                except OSError:
                    pass
            for h, same in by_full.items():
                if len(same) > 1:
                    same.sort(key=lambda p: (p.stat().st_mtime, len(str(p))))
                    size = same[0].stat().st_size
                    groups.append({"hash": h, "size": size, "files": [_info(p) for p in same], "wasted": size * (len(same) - 1)})
        done += len(group)
        job.progress(0.05 + 0.95 * done / total, f"비교 중 {done}/{total}")
    groups.sort(key=lambda g: g["wasted"], reverse=True)
    return {"mode": "exact", "scanned": len(files), "groups": groups, "wasted": sum(g["wasted"] for g in groups)}


def _info(p: Path) -> dict:
    st = p.stat()
    return {"path": str(p), "size": st.st_size, "mtime": datetime.fromtimestamp(st.st_mtime).isoformat(timespec="seconds")}


def dhash(path: Path, size: int = 8) -> int:
    with Image.open(path) as im:
        im = im.convert("L").resize((size + 1, size), Image.Resampling.LANCZOS)
        px = list(im.getdata())
    bits = 0
    for row in range(size):
        for col in range(size):
            left = px[row * (size + 1) + col]
            right = px[row * (size + 1) + col + 1]
            bits = (bits << 1) | (left > right)
    return bits


def find_similar(job: JobContext, dirs: list[str], threshold: int = 6) -> dict:
    job.progress(0.02, "사진 목록 만드는 중")
    files = _walk(dirs, 1, IMAGE_EXTS)
    hashes: list[tuple[Path, int]] = []
    for i, f in enumerate(files):
        try:
            hashes.append((f, dhash(f)))
        except Exception:  # noqa: BLE001 - 열 수 없는 이미지
            pass
        if i % 20 == 0:
            job.progress(0.05 + 0.6 * i / max(len(files), 1), f"사진 분석 {i}/{len(files)}")
    parent = list(range(len(hashes)))
```

```
125
126     def find(x: int) -> int:
127         while parent[x] != x:
128             parent[x] = parent[parent[x]]
129             x = parent[x]
130         return x
131
132     for i in range(len(hashes)):
133         hi = hashes[i][1]
134         for j in range(i + 1, len(hashes)):
135             if (hi ^ hashes[j][1]).bit_count() <= threshold:
136                 parent[find(j)] = find(i)
137         if i % 200 == 0:
138             job.progress(0.65 + 0.35 * i / max(len(hashes), 1), "비슷한 사진 묶는 중")
139     clusters: dict[int, list[Path]] = {}
140     for idx, (p, _) in enumerate(hashes):
141         clusters.setdefault(find(idx), []).append(p)
142     groups = []
143     for members in clusters.values():
144         if len(members) > 1:
145             infos = sorted((_info(p) for p in members), key=lambda x: x["size"], reverse=True)
146             groups.append({"files": infos, "size": infos[0]["size"], "wasted": sum(x["size"] for x in infos[1:])})
147     groups.sort(key=lambda g: g["wasted"], reverse=True)
148     return {"mode": "similar", "scanned": len(files), "groups": groups, "wasted": sum(g["wasted"] for g in groups)}
149
150
151 def quarantine(db: Database, paths: list[str]) -> dict:
152     box = sub_dir(datetime.now().strftime("%Y%m%d_%H%M%S"), sub_dir("보관함"))
153     moved = []
154     for i, p in enumerate(paths):
155         src = Path(p)
156         if not src.is_file():
157             continue
158         dst = box / f"{i:04d}_{src.name}"
159         shutil.move(str(src), dst)
160         moved.append({"src": str(src), "dst": str(dst)})
161     op_id = db.insert("INSERT INTO file_ops(kind, payload, created_at) VALUES('quarantine', ?, ?)", (json.dumps(moved, ensure_ascii=False), now
162     return {"op_id": op_id, "moved": len(moved), "folder": str(box)}
163
164
165 def restore(db: Database, op_id: int) -> dict:
166     op = db.one("SELECT * FROM file_ops WHERE id=? AND kind='quarantine'", (op_id,))
167     if not op:
168         raise NotFound("되돌릴 기록이 없어요.")
169     if op["undone"]:
170         raise AppError("이미 되돌렸어요.")
171     restored, failed = 0, []
172     for m in json.loads(op["payload"]):
173         src, dst = Path(m["src"]), Path(m["dst"])
174         if dst.exists() and not src.exists():
175             src.parent.mkdir(parents=True, exist_ok=True)
176             shutil.move(str(dst), src)
177             restored += 1
178         else:
179             failed.append(m["src"])
180     db.execute("UPDATE file_ops SET undone=1 WHERE id=?", (op_id,))
181     return {"restored": restored, "failed": failed}
182
183
184 class ScanIn(BaseModel):
185     dirs: list[str] = Field(min_length=1)
186     mode: str = "exact"  # exact ¦ similar
```

```
    threshold: int = 6
    min_size: int = 1


class PathsIn(BaseModel):
    paths: list[str] = Field(min_length=1)


@router.post("/scan")
def scan(body: ScanIn, ctx: Ctx):
    if body.mode == "similar":
        job = ctx.jobs.submit("dedupe", "비슷한 사진 찾기", find_similar, body.dirs, body.threshold)
    else:
        job = ctx.jobs.submit("dedupe", "중복 파일 찾기", find_duplicates, body.dirs, body.min_size)
    return job_ref(job)


@router.post("/quarantine")
def quarantine_api(body: PathsIn, ctx: Ctx):
    return quarantine(ctx.db, body.paths)


@router.post("/restore/{op_id}")
def restore_api(op_id: int, ctx: Ctx):
    return restore(ctx.db, op_id)


@router.get("/history")
def history(ctx: Ctx):
    rows = ctx.db.query("SELECT * FROM file_ops WHERE kind='quarantine' ORDER BY id DESC LIMIT 20")
    return [{"id": r["id"], "created_at": r["created_at"], "undone": bool(r["undone"]), "count": len(json.loads(r["payload"]))} for r in rows]
```

```python
"""집중 타이머(뽀모도로) 기록과 주간 리포트. 타이머 자체는 화면에서 돌고, 시작/끝만 기록한다."""
from __future__ import annotations

from datetime import date, datetime, timedelta

from fastapi import APIRouter
from pydantic import BaseModel

from ..core.context import Ctx
from ..core.db import Database, now_iso
from ..core.errors import NotFound

router = APIRouter(prefix="/focus", tags=["집중 타이머"])

WEEKDAYS = "월화수목금토일"


class StartIn(BaseModel):
    label: str = ""
    kind: str = "focus"  # focus | break
    planned_minutes: int = 25


class StopIn(BaseModel):
    completed: bool = True


def _minutes(row: dict) -> float:
    if not row["ended_at"]:
        return 0.0
    return (datetime.fromisoformat(row["ended_at"]) - datetime.fromisoformat(row["started_at"])).total_seconds() / 60


def weekly_report(db: Database, week_of: date) -> dict:
    monday = week_of - timedelta(days=week_of.weekday())
    sunday = monday + timedelta(days=6)
    rows = db.query(
        "SELECT * FROM focus_sessions WHERE kind='focus' AND ended_at IS NOT NULL AND date(started_at) BETWEEN ? AND ?",
        (monday.isoformat(), sunday.isoformat()),
    )
    days = [{"date": (monday + timedelta(days=i)).isoformat(), "label": WEEKDAYS[i], "minutes": 0.0, "sessions": 0} for i in range(7)]
    labels: dict[str, float] = {}
    for r in rows:
        d = datetime.fromisoformat(r["started_at"]).date()
        m = _minutes(r)
        slot = days[(d - monday).days]
        slot["minutes"] += m
        slot["sessions"] += 1 if r["completed"] else 0
        key = r["label"] or "(이름 없음)"
        labels[key] = labels.get(key, 0) + m
    done_todos = db.query("SELECT text FROM todos WHERE done=1 AND date(done_at) BETWEEN ? AND ? ORDER BY done_at", (monday.isoformat(), sunday
    total = sum(d["minutes"] for d in days)
    for d in days:
        d["minutes"] = round(d["minutes"])
    top = sorted(labels.items(), key=lambda x: x[1], reverse=True)
    best = max(days, key=lambda d: d["minutes"])
    md = [f"# 주간 리포트 ({monday:%m/%d} ~ {sunday:%m/%d})", "", f"- 집중 시간: **{int(total // 60)}시간 {int(total % 60)}분**", f"- 완료한 집중 세
    if total:
        md.append(f"- 가장 많이 집중한 날: {best['label']}요일 ({best['minutes']}분)")
    if top:
        md += ["", "## 작업별", *[f"- {k}: {round(v)}분" for k, v in top[:10]]]
    if done_todos:
```

```
        md += ["", f"## 끝낸 일 ({len(done_todos)}개)", *[f"- {t['text']}" for t in done_todos[:30]]]
    return {
        "week_start": monday.isoformat(),
        "days": days,
        "labels": [{"label": k, "minutes": round(v)} for k, v in top],
        "total_minutes": round(total),
        "done_todos": len(done_todos),
        "markdown": "\n".join(md),
    }


@router.post("/start")
def start(body: StartIn, ctx: Ctx):
    # 끝나지 않은 세션이 있으면 먼저 닫는다.
    ctx.db.execute("UPDATE focus_sessions SET ended_at=?, completed=0 WHERE ended_at IS NULL", (now_iso(),))
    sid = ctx.db.insert(
        "INSERT INTO focus_sessions(label, kind, planned_minutes, started_at) VALUES(?,?,?,?)",
        (body.label, body.kind, body.planned_minutes, now_iso()),
    )
    return ctx.db.one("SELECT * FROM focus_sessions WHERE id=?", (sid,))


@router.post("/{sid}/stop")
def stop(sid: int, body: StopIn, ctx: Ctx):
    cur = ctx.db.execute("UPDATE focus_sessions SET ended_at=?, completed=? WHERE id=? AND ended_at IS NULL", (now_iso(), int(body.completed),
    if cur.rowcount == 0:
        raise NotFound("진행 중인 세션이 없어요.")
    return ctx.db.one("SELECT * FROM focus_sessions WHERE id=?", (sid,))


@router.get("/today")
def today(ctx: Ctx):
    rows = ctx.db.query("SELECT * FROM focus_sessions WHERE date(started_at)=date('now','localtime') ORDER BY id DESC")
    focus = [r for r in rows if r["kind"] == "focus"]
    return {"sessions": rows, "minutes": round(sum(_minutes(r) for r in focus)), "completed": sum(1 for r in focus if r["completed"])}


@router.get("/report")
def report(ctx: Ctx, week: str | None = None):
    return weekly_report(ctx.db, date.fromisoformat(week) if week else date.today())
```

```
1
```

```python
from __future__ import annotations

from fastapi import APIRouter
from pydantic import BaseModel, Field

from ...core import license
from ...core.context import Ctx, job_ref
from ...core.errors import AppError
from .detect import DEFAULT_KINDS, KIND_LABELS, MODES, mask_text
from .files import SUPPORTED_EXTS, mask_file

router = APIRouter(prefix="/mask", tags=["개인정보 마스킹"])


class TextIn(BaseModel):
    text: str
    kinds: list[str] | None = None
    mode: str = "partial"


class FilesIn(BaseModel):
    paths: list[str] = Field(min_length=1)
    kinds: list[str] | None = None
    mode: str = "partial"
    out_dir: str | None = None
    faces: bool = True


def _check(kinds: list[str] | None, mode: str) -> None:
    if mode not in MODES:
        raise AppError("가리는 방식은 partial, full, tag 중 하나여야 해요.")
    unknown = set(kinds or []) - set(KIND_LABELS)
    if unknown:
        raise AppError(f"알 수 없는 항목: {', '.join(sorted(unknown))}")


@router.get("/kinds")
def kinds():
    return {
        "kinds": [{"key": k, "label": v, "default": k in DEFAULT_KINDS} for k, v in KIND_LABELS.items()],
        "extensions": sorted(SUPPORTED_EXTS),
    }


@router.post("/text")
def mask_text_api(body: TextIn):
    _check(body.kinds, body.mode)
    masked, findings = mask_text(body.text, body.kinds, body.mode)
    return {"masked": masked, "findings": [f.to_dict() for f in findings]}


def run_files(job, paths, kinds, mode, out_dir, faces):
    results = []
    for i, path in enumerate(paths):
        job.progress(i / len(paths), f"{i + 1}/{len(paths)} 처리 중")
        try:
            results.append(mask_file(path, kinds, mode, out_dir, faces))
        except AppError as e:
            results.append({"src": path, "error": e.message})
    total = sum(r.get("count", 0) for r in results)
    return {"results": results, "total": total}

```

```
63
64  @router.post("/files")
65  def mask_files_api(body: FilesIn, ctx: Ctx):
66      _check(body.kinds, body.mode)
67      license.require_pro(ctx.db)
68      job = ctx.jobs.submit("mask", f"파일 {len(body.paths)}개 마스킹", run_files, body.paths, body.kinds, body.mode, body.out_dir, body.faces)
69      return job_ref(job)
```

```python
"""한국 개인정보 탐지와 가리기 (순수 텍스트 처리)."""
from __future__ import annotations

import re
from dataclasses import dataclass
from typing import Callable, Iterable

from ...core.korean import looks_like_name

KIND_LABELS = {
    "rrn": "주민등록번호",
    "phone": "전화번호",
    "card": "카드번호",
    "account": "계좌번호",
    "email": "이메일",
    "passport": "여권번호",
    "driver": "운전면허번호",
    "plate": "차량번호",
    "name": "이름",
    "address": "주소",
    "bizno": "사업자번호",
    "ip": "IP 주소",
}
DEFAULT_KINDS = [k for k in KIND_LABELS if k not in ("bizno", "ip")]
# 겹칠 때 먼저 차지하는 순서
PRIORITY = ["rrn", "card", "driver", "passport", "bizno", "phone", "account", "email", "plate", "address", "name", "ip"]
MODES = ("partial", "full", "tag")


@dataclass(frozen=True)
class Finding:
    kind: str
    start: int
    end: int
    value: str

    def to_dict(self) -> dict:
        return {"kind": self.kind, "label": KIND_LABELS[self.kind], "start": self.start, "end": self.end, "value": self.value}


ND = r"(?<!\d)"  # 앞이 숫자가 아님 (\b는 한글 옆에서 동작하지 않는다)
NA = r"(?!\d)"

RRN_RE = re.compile(ND + r"(\d{2})(\d{2})(\d{2})(\s?[-‐-―]\s?)([1-8])(\d{6})" + NA)
MOBILE_RE = re.compile(ND + r"(?:(?:\+82|0082)[-.\s]?0?|0)1[016789][-.\s)]?\d{3,4}[-.\s]?\d{4}" + NA)
LANDLINE_RE = re.compile(ND + r"(?:(?:\+82|0082)[-.\s]?0?|0)(?:2|[3-6][1-5]|70|50[2-8])[-.\s)]{0,2}\d{3,4}[-.\s]?\d{4}" + NA)
CARD_RE = re.compile(ND + r"\d{4}[-\s]?\d{4}[-\s]?\d{4}[-\s]?\d{4}" + NA)
AMEX_RE = re.compile(ND + r"3[47]\d{2}[-\s]?\d{6}[-\s]?\d{5}" + NA)
ACCOUNT_HYPHEN_RE = re.compile(r"(?<![\d-])\d{2,6}(?:-\d{2,7}){1,3}(?![\d-])")
ACCOUNT_PLAIN_RE = re.compile(ND + r"\d{10,14}" + NA)
ACCOUNT_CONTEXT_RE = re.compile(
    r"(계좌|입금|송금|이체|예금주|은행|뱅크|농협|국민|신한|우리|하나|기업|카카오|토스|우체국|새마을|수협|신협|씨티|SC제일|케이뱅크|부산|대구|경남|광
)
EMAIL_RE = re.compile(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}")
PASSPORT_RE = re.compile(r"(?<![A-Za-z0-9])[MSRODG](?:\d{8}|\d{3}[A-Z]\d{4})(?![A-Za-z0-9])")
DRIVER_RE = re.compile(ND + r"\d{2}-\d{2}-\d{6}-\d{2}" + NA)
PLATE_RE = re.compile(
    r"(?<![\d가-힣])(?:(?:서울|부산|대구|인천|광주|대전|울산|세종|경기|강원|충북|충남|전북|전남|경북|경남|제주)\s?)?"
    r"\d{2,3}\s?[가나다라마거너더러머버서어저고노도로모보소오조구누두루무부수우주아바사자배하허호]\s?\d{4}" + NA
)
BIZNO_RE = re.compile(ND + r"\d{3}-\d{2}-\d{5}" + NA)
IP_RE = re.compile(r"(?<![\d.])(?:(?:25[0-5]|2[0-4]\d|1?\d?\d)\.){3}(?:25[0-5]|2[0-4]\d|1?\d?\d)(?![\d.])")
```

```
63
64  NAME_LABEL_RE = re.compile(
65      r"(?:성명¦이름¦성함¦담당자¦보호자¦신청인¦대표자¦작성자¦수령인¦받는\s?분¦예금주¦환자명¦고객명¦피보험자¦계약자)\s*[:：]?\s*([가-힣]{2,4})(?![가-힣
66  )
67  NAME_HONORIFIC_RE = re.compile(
68      r"(?<![가-힣])([가-힣]{2,4})\s?(?:님¦씨¦고객님¦환자분¦선생님¦과장님¦대리님¦부장님¦팀장님¦사장님¦대표님¦교수님¦원장님¦실장님¦주임님)"
69  )
70  NAME_STOP = {
71      "선생", "고객", "여러", "환자", "회원", "학생", "손님", "사장", "부장", "과장", "대리", "팀장", "실장", "이사",
72      "대표", "교수", "기사", "주인", "담당", "작가", "의사", "직원", "상담", "관리", "원장", "형님", "누님", "따님",
73      "아드", "부모", "어머", "아버", "할머", "할아", "고모", "이모", "삼촌", "선배", "후배", "동생", "마음", "주임",
74      "전문", "신청", "보호", "수강", "구독", "시청", "청취", "사용", "이용", "방문", "정회", "임원", "조합", "강사",
75  }
76
77  # '담당자 연락처'처럼 라벨 뒤에 이름 대신 오는 낱말
78  LABEL_NOUNS = {
79      "연락처", "전화", "전화번호", "번호", "주소", "이메일", "메일", "확인", "변경", "정보", "없음", "미정", "본인", "대리인",
80      "담당", "부서", "직위", "직급", "서명", "날인", "인적", "기재", "정정", "작성", "제출", "지정", "배정", "선정", "교체",
81  }
82
83  _PROVINCES = (
84      "서울특별시¦부산광역시¦대구광역시¦인천광역시¦광주광역시¦대전광역시¦울산광역시¦세종특별자치시¦경기도¦강원특별자치도¦강원도¦"
85      "충청북도¦충청남도¦전북특별자치도¦전라북도¦전라남도¦경상북도¦경상남도¦제주특별자치도¦"
86      "서울시?¦부산시?¦대구시?¦인천시?¦광주시?¦대전시?¦울산시?¦세종시?¦경기¦강원¦충북¦충남¦전북¦전남¦경북¦경남¦제주"
87  )
88  ADDRESS_RE = re.compile(
89      rf"(?<![가-힣])(?:{_PROVINCES})\s+[가-힣]+(?:시¦군¦구)(?:\s+[가-힣]+(?:구¦읍¦면))?\s+"
90      r"(?:[가-힣0-9]+(?:로¦길)(?:\s?\d+(?:번)?길)?\s?\d+(?:-\d+)?¦[가-힣0-9]+(?:동¦리¦가)\s?\d+(?:-\d+)?(?:번지)?)"
91      r"(?:\s*,?\s*(?:[가-힣A-Za-z]+\s?)?\d+동)?(?:\s*\d+층)?(?:\s*\d+호)?"
92  )
93
94
95  def _digits(s: str) -> str:
96      return re.sub(r"\D", "", s)
97
98
99  def luhn_ok(number: str) -> bool:
100     digits = [int(c) for c in _digits(number)]
101     total = 0
102     for i, d in enumerate(reversed(digits)):
103         if i % 2 == 1:
104             d *= 2
105             if d > 9:
106                 d -= 9
107         total += d
108     return total % 10 == 0
109
110
111 def rrn_checksum_ok(number: str) -> bool:
112     d = [int(c) for c in _digits(number)]
113     if len(d) != 13:
114         return False
115     weights = [2, 3, 4, 5, 6, 7, 8, 9, 2, 3, 4, 5]
116     return (11 - sum(a * b for a, b in zip(d, weights)) % 11) % 10 == d[12]
117
118
119 def bizno_checksum_ok(number: str) -> bool:
120     d = [int(c) for c in _digits(number)]
121     if len(d) != 10:
122         return False
123     w = [1, 3, 7, 1, 3, 7, 1, 3, 5]
124     total = sum(a * b for a, b in zip(d, w)) + (d[8] * 5) // 10
```

```
    return (10 - total % 10) % 10 == d[9]


def _candidates(text: str) -> Iterable[Finding]:
    for m in RRN_RE.finditer(text):
        month, day, sep = int(m.group(2)), int(m.group(3)), m.group(4)
        if not (1 <= month <= 12 and 1 <= day <= 31):
            continue
        # 2020년 10월 이후 번호는 체크섬을 따르지 않아서, 구분자가 있으면 날짜만 본다.
        context = text[max(0, m.start() - 20) : m.start()]
        if sep.strip() or rrn_checksum_ok(m.group(0)) or "주민" in context or "외국인" in context:
            yield Finding("rrn", m.start(), m.end(), m.group(0))
    for rx in (MOBILE_RE, LANDLINE_RE):
        for m in rx.finditer(text):
            yield Finding("phone", m.start(), m.end(), m.group(0))
    for rx in (CARD_RE, AMEX_RE):
        for m in rx.finditer(text):
            if luhn_ok(m.group(0)):
                yield Finding("card", m.start(), m.end(), m.group(0))
    for m in BIZNO_RE.finditer(text):
        yield Finding("bizno", m.start(), m.end(), m.group(0))
    for m in ACCOUNT_HYPHEN_RE.finditer(text):
        value = m.group(0)
        n = len(_digits(value))
        groups = value.count("-") + 1
        if not 10 <= n <= 14 or re.fullmatch(r"\d{3}-\d{2}-\d{5}", value):
            continue
        context = text[max(0, m.start() - 20) : m.start()]
        if groups >= 3 or ACCOUNT_CONTEXT_RE.search(context):
            yield Finding("account", m.start(), m.end(), value)
    for m in ACCOUNT_PLAIN_RE.finditer(text):
        context = text[max(0, m.start() - 20) : m.start()]
        if ACCOUNT_CONTEXT_RE.search(context):
            yield Finding("account", m.start(), m.end(), m.group(0))
    for m in EMAIL_RE.finditer(text):
        yield Finding("email", m.start(), m.end(), m.group(0))
    for m in PASSPORT_RE.finditer(text):
        yield Finding("passport", m.start(), m.end(), m.group(0))
    for m in DRIVER_RE.finditer(text):
        yield Finding("driver", m.start(), m.end(), m.group(0))
    for m in PLATE_RE.finditer(text):
        yield Finding("plate", m.start(), m.end(), m.group(0))
    for m in ADDRESS_RE.finditer(text):
        yield Finding("address", m.start(), m.end(), m.group(0))
    for m in NAME_LABEL_RE.finditer(text):
        if looks_like_name(m.group(1)) and m.group(1) not in LABEL_NOUNS:
            yield Finding("name", m.start(1), m.end(1), m.group(1))
    for m in NAME_HONORIFIC_RE.finditer(text):
        word = m.group(1)
        if word[:2] not in NAME_STOP and looks_like_name(word):
            yield Finding("name", m.start(1), m.end(1), word)
    for m in IP_RE.finditer(text):
        yield Finding("ip", m.start(), m.end(), m.group(0))


def detect(text: str, kinds: Iterable[str] | None = None) -> list[Finding]:
    enabled = set(kinds) if kinds is not None else set(DEFAULT_KINDS)
    rank = {k: i for i, k in enumerate(PRIORITY)}
    cands = sorted(set(_candidates(text)), key=lambda f: (rank[f.kind], -(f.end - f.start), f.start))
    taken: list[Finding] = []
    for f in cands:
        if any(f.start < t.end and t.start < f.end for t in taken):
```

```
            continue
        taken.append(f)
    # 전부 탐지하고 겹침을 정리한 다음에 거른다. (예: 전화번호를 끈 경우 계좌로 오인하지 않도록)
    return sorted((f for f in taken if f.kind in enabled), key=lambda f: f.start)


def _star_range(value: str, keep: Callable[[int, int], bool]) -> str:
    """숫자/글자 순번(i, 전체 n)에 대해 keep이 False인 것만 *로 바꾼다. 구분자는 그대로."""
    idxs = [i for i, c in enumerate(value) if c.isalnum() or "가" <= c <= "힣"]
    n = len(idxs)
    chars = list(value)
    for order, i in enumerate(idxs):
        if not keep(order, n):
            chars[i] = "*"
    return "".join(chars)


def _star_groups(value: str, which: Callable[[int, int], bool]) -> str:
    groups = list(re.finditer(r"\d+", value))
    chars = list(value)
    for gi, g in enumerate(groups):
        if which(gi, len(groups)):
            for i in range(g.start(), g.end()):
                chars[i] = "*"
    return "".join(chars)


def mask_value(f: Finding, mode: str = "partial") -> str:
    v = f.value
    if mode == "tag":
        return f"[{KIND_LABELS[f.kind]}]"
    if mode == "full":
        return re.sub(r"[0-9A-Za-z가-힣]", "*", v)
    k = f.kind
    if k == "rrn":
        return _star_range(v, lambda i, n: i < 7)
    if k == "phone":
        return _star_groups(v, lambda gi, n: n >= 3 and gi == n - 2) if len(re.findall(r"\d+", v)) >= 3 else _star_range(v, lambda i, n: i < 3
    if k == "card":
        return _star_range(v, lambda i, n: i < 4 or i >= n - 4)
    if k in ("account", "bizno"):
        return _star_range(v, lambda i, n: i >= n - 4)
    if k == "email":
        local, _, domain = v.partition("@")
        keep = 0 if len(local) <= 1 else max(1, min(2, len(local) // 3))
        return local[:keep] + "*" * (len(local) - keep) + "@" + domain
    if k in ("passport", "driver"):
        return _star_range(v, lambda i, n: i < 2)
    if k == "plate":
        return _star_range(v, lambda i, n: i < n - 4)
    if k == "name":
        if len(v) == 2:
            return v[0] + "*"
        return v[0] + "*" * (len(v) - 2) + v[-1]
    if k == "address":
        parts = re.split(r"(\s+)", v)
        out, words = [], 0
        for p in parts:
            if p.strip():
                words += 1
                out.append(p if words <= 2 else re.sub(r"[0-9A-Za-z가-힣]", "*", p))
            else:
```

```
                out.append(p)
        return "".join(out)
    if k == "ip":
        return _star_groups(v, lambda gi, n: gi == n - 1)
    return re.sub(r"\S", "*", v)


def mask_text(text: str, kinds: Iterable[str] | None = None, mode: str = "partial") -> tuple[str, list[Finding]]:
    findings = detect(text, kinds)
    out = text
    for f in reversed(findings):
        out = out[: f.start] + mask_value(f, mode) + out[f.end :]
    return out, findings


def remap_runs(runs: list[str], findings: list[Finding], mode: str) -> list[str]:
    """여러 조각(run)으로 나뉜 문단에 마스킹을 적용한다.

    가린 값의 길이가 원래와 같으면 글자 단위로 원래 조각에 돌려놔서 서식이 유지된다.
    길이가 다르면(태그 모드) 시작 조각에 넣고 나머지 조각에서는 지운다.
    """
    owner: list[int] = []
    for ri, r in enumerate(runs):
        owner.extend([ri] * len(r))
    full = "".join(runs)
    out: list[list[str]] = [[] for _ in runs]
    pos = 0
    for f in findings:
        for i in range(pos, f.start):
            out[owner[i]].append(full[i])
        rep = mask_value(f, mode)
        if len(rep) == f.end - f.start:
            for i, ch in enumerate(rep):
                out[owner[f.start + i]].append(ch)
        else:
            out[owner[f.start]].append(rep)
        pos = f.end
    for i in range(pos, len(full)):
        out[owner[i]].append(full[i])
    return ["".join(o) for o in out]


def summarize_findings(findings: Iterable[Finding]) -> dict[str, int]:
    counts: dict[str, int] = {}
    for f in findings:
        counts[f.kind] = counts.get(f.kind, 0) + 1
    return counts
```

```python
"""파일 형식별 마스킹: 텍스트, DOCX, HWPX, HWP(텍스트 추출), PDF, 이미지."""
from __future__ import annotations

import html
import io
import re
import zipfile
from pathlib import Path

from PIL import Image, ImageDraw, ImageFilter

from ...core import ocr, optional
from ...core.config import output_path
from ...core.errors import AppError
from ...core.extract import HWPX_TOKEN_RE, TEXT_EXTS, hwp_text, hwpx_sections, pdf_page_chars, read_text_file
from .detect import Finding, detect, mask_text, remap_runs, summarize_findings

IMAGE_EXTS = {".png", ".jpg", ".jpeg", ".bmp", ".webp", ".tif", ".tiff"}
SUPPORTED_EXTS = TEXT_EXTS | IMAGE_EXTS | {".docx", ".hwpx", ".hwp", ".pdf"}


def mask_file(path: str | Path, kinds: list[str] | None, mode: str, out_dir: str | None = None, faces: bool = True) -> dict:
    src = Path(path)
    if not src.exists():
        raise AppError(f"파일을 찾을 수 없어요: {src}")
    ext = src.suffix.lower()
    if ext in TEXT_EXTS:
        return _mask_textfile(src, kinds, mode, out_dir)
    if ext == ".docx":
        return _mask_docx(src, kinds, mode, out_dir)
    if ext == ".hwpx":
        return _mask_hwpx(src, kinds, mode, out_dir)
    if ext == ".hwp":
        return _mask_hwp(src, kinds, mode, out_dir)
    if ext == ".pdf":
        return _mask_pdf(src, kinds, out_dir)
    if ext in IMAGE_EXTS:
        return _mask_image(src, kinds, out_dir, faces)
    raise AppError(f"지원하지 않는 형식이에요: {ext}")


def _result(src: Path, dst: Path, findings: list[Finding], note: str = "") -> dict:
    return {"src": str(src), "dst": str(dst), "count": len(findings), "by_kind": summarize_findings(findings), "note": note}


def _mask_textfile(src: Path, kinds, mode, out_dir) -> dict:
    text, enc = read_text_file(src)
    masked, findings = mask_text(text, kinds, mode)
    dst = output_path(src, "_마스킹", out_dir=out_dir)
    dst.write_text(masked, encoding="utf-8" if enc.startswith("utf-8") else enc)
    return _result(src, dst, findings)


W_NS = "{http://schemas.openxmlformats.org/wordprocessingml/2006/main}"
XML_SPACE = "{http://www.w3.org/XML/1998/namespace}space"


def _mask_docx(src: Path, kinds, mode, out_dir) -> dict:
    import docx

    doc = docx.Document(str(src))
    roots = [doc.element.body]
```

```
    for section in doc.sections:
        for hf in (section.header, section.footer, section.first_page_header, section.first_page_footer, section.even_page_header, section.even
            try:
                if hf is not None and not hf.is_linked_to_previous:
                    roots.append(hf._element)
            except Exception:  # noqa: BLE001 - 머리글이 없는 구역
                pass
    all_findings: list[Finding] = []
    for root in roots:
        for p in root.iter(f"{W_NS}p"):
            ts = list(p.iter(f"{W_NS}t"))
            if not ts:
                continue
            runs = [t.text or "" for t in ts]
            findings = detect("".join(runs), kinds)
            if not findings:
                continue
            all_findings += findings
            for t, new in zip(ts, remap_runs(runs, findings, mode)):
                if new != (t.text or ""):
                    t.text = new
                    if new != new.strip():
                        t.set(XML_SPACE, "preserve")
    # 문서 속성에 남은 작성자 이름도 지운다.
    props = doc.core_properties
    props.author = ""
    props.last_modified_by = ""
    dst = output_path(src, "_마스킹", out_dir=out_dir)
    doc.save(str(dst))
    return _result(src, dst, all_findings)


def _escape_xml(s: str) -> str:
    return s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")


SEP = "\x00"  # 문단 경계. 정규식의 \s에 걸리지 않아 문단을 넘는 탐지가 생기지 않는다.
_PIECE_RE = re.compile(r"<[^>]+>|[^<]+")


def mask_hwpx_xml(xml: str, kinds, mode) -> tuple[str, list[Finding]]:
    """section XML 안의 <hp:t> 텍스트만 바꾸고 나머지는 바이트 그대로 둔다."""
    pieces: list[tuple[int, int, str]] = []  # (xml 안 시작, 끝, 원문 텍스트)
    runs: list[str] = []
    for m in HWPX_TOKEN_RE.finditer(xml):
        if m.group(1) is None:
            if m.group(0).startswith("<hp:p") or m.group(0) == "</hp:p>":
                runs.append(SEP)
                pieces.append((-1, -1, SEP))
            continue
        base = m.start(1)
        for pm in _PIECE_RE.finditer(m.group(1)):
            if pm.group(0).startswith("<"):
                continue
            text = html.unescape(pm.group(0))
            runs.append(text)
            pieces.append((base + pm.start(), base + pm.end(), text))
    full = "".join(runs)
    findings = detect(full, kinds)
    if not findings:
        return xml, []
    new_runs = remap_runs(runs, findings, mode)
```

```
    out = xml
    for (start, end, old), new in sorted(zip(pieces, new_runs), key=lambda x: x[0][0], reverse=True):
        if start < 0 or new == old:
            continue
        out = out[:start] + _escape_xml(new) + out[end:]
    return out, findings


def _blank_png_like(data: bytes) -> bytes:
    try:
        size = Image.open(io.BytesIO(data)).size
    except Exception:  # noqa: BLE001
        size = (200, 280)
    buf = io.BytesIO()
    Image.new("RGB", size, "white").save(buf, "PNG")
    return buf.getvalue()


def _mask_hwpx(src: Path, kinds, mode, out_dir) -> dict:
    findings: list[Finding] = []
    with zipfile.ZipFile(src) as zin:
        sections = set(hwpx_sections(zin))
        if not sections:
            raise AppError("HWPX 본문을 찾지 못했어요. 파일이 손상됐을 수 있어요.")
        dst = output_path(src, "_마스킹", out_dir=out_dir)
        with zipfile.ZipFile(dst, "w") as zout:
            _copy_hwpx(zin, zout, sections, kinds, mode, findings)
    return _result(src, dst, findings, "미리보기 그림은 흰 화면으로 바뀌어요. 한글에서 저장하면 다시 만들어져요.")


def _copy_hwpx(zin: zipfile.ZipFile, zout: zipfile.ZipFile, sections: set[str], kinds, mode, findings: list[Finding]) -> None:
    for info in zin.infolist():
        data = zin.read(info.filename)
        if info.filename in sections:
            xml, f = mask_hwpx_xml(data.decode("utf-8"), kinds, mode)
            findings += f
            data = xml.encode("utf-8")
        elif info.filename == "Preview/PrvText.txt":
            # 미리보기 텍스트에도 원문이 들어 있다. (utf-16은 거의 모든 바이트열이 디코딩되므로 나중에 시도)
            for enc in ("utf-8", "utf-16"):
                try:
                    data = mask_text(data.decode(enc), kinds, mode)[0].encode(enc)
                    break
                except UnicodeDecodeError:
                    continue
        elif info.filename.startswith("Preview/") and info.filename.lower().endswith(".png"):
            # 첫 장 미리보기 그림은 다시 만들 수 없으니 흰 그림으로 바꾼다.
            data = _blank_png_like(data)
        # mimetype은 압축하지 않은 첫 항목이어야 한다. 원래 순서와 압축 방식을 그대로 따른다.
        zout.writestr(info, data, compress_type=info.compress_type)


def _mask_hwp(src: Path, kinds, mode, out_dir) -> dict:
    # HWP 5.0 바이너리는 압축 스트림 크기가 바뀌어 원본 형식으로 다시 쓰기 어렵다.
    # 본문 텍스트를 마스킹해서 .txt로 내보내고 HWPX 변환을 안내한다.
    masked, findings = mask_text(hwp_text(src), kinds, mode)
    dst = output_path(src, "_마스킹", ext="txt", out_dir=out_dir)
    dst.write_text(masked, encoding="utf-8")
    return _result(src, dst, findings, "HWP는 본문 텍스트만 마스킹해 .txt로 저장했어요. 서식을 유지하려면 한글에서 HWPX로 저장한 뒤 다시 넣어 주세요


PDF_RENDER_SCALE = 200 / 72  # 200dpi
```

```
187
188
189 def _pdf_page_boxes(page, kinds, findings_out: list[Finding]) -> list[tuple[float, float, float, float]]:
190     """가릴 영역(렌더링 픽셀 좌표) 목록."""
191     text, tp = pdf_page_chars(page)
192     _, height = page.get_size()
193     boxes = []
194     for f in detect(text, kinds):
195         findings_out.append(f)
196         for i in range(f.start, f.end):
197             left, bottom, right, top = tp.get_charbox(i)
198             if right - left <= 0 or top - bottom <= 0:
199                 continue
200             boxes.append((left * PDF_RENDER_SCALE, (height - top) * PDF_RENDER_SCALE, right * PDF_RENDER_SCALE, (height - bottom) * PDF_RENDER_
201     return boxes
202
203
204 def _mask_pdf(src: Path, kinds, out_dir) -> dict:
205     """개인정보가 있는 페이지만 그림으로 바꾸고 그 위를 검게 칠한다.
206
207     글자 레이어를 남기지 않으니 복사·검색으로도 원문이 나오지 않는다.
208     (PyMuPDF의 진짜 텍스트 삭제는 AGPL이라 판매용 앱에 넣지 않았다.)
209     """
210     import pypdfium2 as pdfium
211
212     try:
213         doc = pdfium.PdfDocument(str(src))
214     except pdfium.PdfiumError as e:
215         raise AppError("PDF를 열 수 없어요. 암호가 걸렸거나 손상된 파일일 수 있어요.") from e
216     findings: list[Finding] = []
217     replaced, ocr_pages = 0, 0
218     out = pdfium.PdfDocument.new()
219     try:
220         for i in range(len(doc)):
221             page = doc[i]
222             rotation = page.get_rotation()
223             if rotation:
224                 page.set_rotation(0)
225             page_findings: list[Finding] = []
226             boxes = _pdf_page_boxes(page, kinds, page_findings)
227             has_text = page.get_textpage().count_chars() > 0
228             img = None
229             if boxes or (not has_text and optional.available("ocr")):
230                 img = page.render(scale=PDF_RENDER_SCALE).to_pil().convert("RGB")
231                 if not has_text:
232                     ocr_pages += 1
233                     for line in ocr.ocr_image(img):
234                         for f in detect(line.text, kinds):
235                             page_findings.append(f)
236                             boxes.append(line.span_box(f.start, f.end))
237             findings += page_findings
238             if not boxes:
239                 if rotation:
240                     page.set_rotation(rotation)
241                 out.import_pages(doc, [i])
242                 continue
243             draw = ImageDraw.Draw(img)
244             for l, t, r, b in boxes:
245                 draw.rectangle((l - 2, t - 2, r + 2, b + 2), fill="black")
246             width, height = page.get_size()
247             new_page = out.new_page(width, height)
248             buf = io.BytesIO()
```

```
            img.save(buf, "JPEG", quality=88)
            buf.seek(0)
            image = pdfium.PdfImage.new(out)
            image.load_jpeg(buf, inline=False)
            image.set_matrix(pdfium.PdfMatrix().scale(width, height))
            new_page.insert_obj(image)
            new_page.gen_content()
            if rotation:
                new_page.set_rotation(rotation)
            replaced += 1
        dst = output_path(src, "_마스킹", out_dir=out_dir)
        out.save(str(dst))
    finally:
        out.close()
        doc.close()
    note = f"개인정보가 있는 {replaced}페이지는 그림으로 바꿔 가렸어요(글자를 복사 · 검색해도 안 나와요)."
    if ocr_pages:
        note += f" 스캔 페이지 {ocr_pages}장은 OCR로 찾았어요."
    return _result(src, dst, findings, note)


def detect_faces(img: Image.Image) -> list[tuple[int, int, int, int]]:
    import cv2
    import numpy as np

    gray = cv2.cvtColor(np.array(img.convert("RGB")), cv2.COLOR_RGB2GRAY)
    cascade = cv2.CascadeClassifier(cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
    faces = cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))
    return [(int(x), int(y), int(x + w), int(y + h)) for x, y, w, h in faces]


def _pixelate(img: Image.Image, box) -> None:
    region = img.crop(box)
    w, h = region.size
    small = region.resize((max(1, w // 12), max(1, h // 12)), Image.Resampling.BILINEAR)
    img.paste(small.resize((w, h), Image.Resampling.NEAREST).filter(ImageFilter.GaussianBlur(2)), box[:2])


def _mask_image(src: Path, kinds, out_dir, faces: bool) -> dict:
    img = Image.open(src)
    img.load()
    img = img.convert("RGB")
    findings: list[Finding] = []
    notes = []
    draw = ImageDraw.Draw(img)
    if optional.available("ocr"):
        for line in ocr.ocr_image(img):
            for f in detect(line.text, kinds):
                findings.append(f)
                l, t, r, b = line.span_box(f.start, f.end)
                draw.rectangle((l - 2, t - 2, r + 2, b + 2), fill="black")
    else:
        notes.append("OCR 구성요소가 없어 글자는 찾지 못했어요.")
    face_count = 0
    if faces and optional.available("face"):
        for box in detect_faces(img):
            _pixelate(img, box)
            face_count += 1
    elif faces:
        notes.append("얼굴 감지 구성요소가 없어 얼굴은 가리지 못했어요.")
    if not optional.available("ocr") and not (faces and optional.available("face")):
        raise AppError("이미지 마스킹에는 OCR 또는 얼굴 감지 구성요소가 필요해요.", component="ocr")
```

```
    dst = output_path(src, "_마스킹", ext="png" if src.suffix.lower() not in (".jpg", ".jpeg") else "jpg", out_dir=out_dir)
    img.save(dst, quality=92)
    result = _result(src, dst, findings, " ".join(notes))
    if face_count:
        result["by_kind"]["face"] = face_count
        result["count"] += face_count
    return result
```

```python
"""이미지 일괄 처리(규격, 워터마크, 배경 제거)와 영상 무음 컷 · 자막."""
from __future__ import annotations

import re
from pathlib import Path

from fastapi import APIRouter
from PIL import Image, ImageColor, ImageDraw, ImageOps
from pydantic import BaseModel, Field

from ..core import ffmpeg, fonts, license, optional
from ..core.config import output_path
from ..core.context import Ctx, job_ref
from ..core.errors import AppError
from ..core.jobs import JobContext
from .transcribe import export, stt

router = APIRouter(prefix="/media", tags=["이미지 · 영상"])

PRESETS = {
    "smartstore_main": {"label": "스마트스토어 대표 (1000×1000)", "width": 1000, "height": 1000, "mode": "pad"},
    "coupang_main": {"label": "쿠팡 대표 (1000×1000)", "width": 1000, "height": 1000, "mode": "pad"},
    "detail_860": {"label": "상세페이지 (가로 860)", "width": 860, "height": None, "mode": "width"},
    "insta_square": {"label": "인스타 정사각 (1080×1080)", "width": 1080, "height": 1080, "mode": "crop"},
    "insta_portrait": {"label": "인스타 세로 (1080×1350)", "width": 1080, "height": 1350, "mode": "crop"},
    "thumb_500": {"label": "썸네일 (500×500)", "width": 500, "height": 500, "mode": "crop"},
}
IMAGE_EXTS = {".png", ".jpg", ".jpeg", ".webp", ".bmp", ".gif", ".tif", ".tiff"}


class Watermark(BaseModel):
    text: str = ""
    position: str = "bottom-right"  # center ¦ bottom-right ¦ bottom-left ¦ top-left ¦ top-right ¦ tile
    opacity: float = 0.35
    size: float = 0.05  # 이미지 너비 대비 글자 크기
    color: str = "#ffffff"


class ImageOpts(BaseModel):
    paths: list[str] = Field(min_length=1)
    preset: str ¦ None = None
    width: int ¦ None = None
    height: int ¦ None = None
    mode: str = "fit"  # fit ¦ pad ¦ crop ¦ width ¦ none
    background: str = "#ffffff"
    remove_bg: bool = False
    watermark: Watermark ¦ None = None
    format: str = "jpg"  # jpg ¦ png ¦ webp ¦ same
    quality: int = 90
    suffix: str = "_편집"
    out_dir: str ¦ None = None


def _resize(img: Image.Image, w: int ¦ None, h: int ¦ None, mode: str, bg: str) -> Image.Image:
    if mode == "none" or (not w and not h):
        return img
    if mode == "width" or not h:
        ratio = w / img.width
        return img.resize((w, max(1, round(img.height * ratio))), Image.Resampling.LANCZOS)
    if not w:
        ratio = h / img.height
        return img.resize((max(1, round(img.width * ratio)), h), Image.Resampling.LANCZOS)
```

```
    if mode == "crop":
        return ImageOps.fit(img, (w, h), Image.Resampling.LANCZOS)
    fitted = ImageOps.contain(img, (w, h), Image.Resampling.LANCZOS)
    if mode == "fit":
        return fitted
    canvas = Image.new("RGBA", (w, h), ImageColor.getrgb(bg) + (255,))
    canvas.alpha_composite(fitted.convert("RGBA"), ((w - fitted.width) // 2, (h - fitted.height) // 2))
    return canvas


def _watermark(img: Image.Image, wm: Watermark) -> Image.Image:
    if not wm.text.strip():
        return img
    base = img.convert("RGBA")
    layer = Image.new("RGBA", base.size, (0, 0, 0, 0))
    draw = ImageDraw.Draw(layer)
    size = max(12, int(base.width * wm.size))
    font = fonts.load(size)
    rgb = ImageColor.getrgb(wm.color)[:3]
    fill = rgb + (int(255 * max(0.0, min(1.0, wm.opacity))),)
    shadow = (0, 0, 0, int(fill[3] * 0.5))
    l, t, r, b = draw.textbbox((0, 0), wm.text, font=font)
    tw, th = r - l, b - t
    margin = int(base.width * 0.03)
    if wm.position == "tile":
        step_x, step_y = tw + size * 3, th + size * 3
        for y in range(-th, base.height + th, step_y):
            offset = (y // step_y % 2) * step_x // 2
            for x in range(-tw + offset, base.width + tw, step_x):
                draw.text((x, y), wm.text, font=font, fill=fill)
    else:
        positions = {
            "center": ((base.width - tw) // 2, (base.height - th) // 2),
            "bottom-right": (base.width - tw - margin, base.height - th - margin),
            "bottom-left": (margin, base.height - th - margin),
            "top-left": (margin, margin),
            "top-right": (base.width - tw - margin, margin),
        }
        x, y = positions.get(wm.position, positions["bottom-right"])
        x, y = x - l, y - t
        draw.text((x + 2, y + 2), wm.text, font=font, fill=shadow)
        draw.text((x, y), wm.text, font=font, fill=fill)
    return Image.alpha_composite(base, layer)


def _remove_bg(img: Image.Image) -> Image.Image:
    optional.require("rembg")
    from rembg import remove

    return remove(img.convert("RGBA"))


def process_image(src: Path, o: ImageOpts) -> Path:
    img = Image.open(src)
    img = ImageOps.exif_transpose(img)
    img = img.convert("RGBA") if img.mode not in ("RGB", "RGBA") else img
    if o.remove_bg:
        img = _remove_bg(img)
    w, h, mode = o.width, o.height, o.mode
    if o.preset:
        p = PRESETS[o.preset]
        w, h, mode = p["width"], p["height"], p["mode"]
```

```
    img = _resize(img, w, h, mode, o.background)
    if o.watermark:
        img = _watermark(img, o.watermark)
    fmt = src.suffix.lower().lstrip(".") if o.format == "same" else o.format
    fmt = {"jpeg": "jpg", "tif": "png", "tiff": "png", "bmp": "png", "gif": "png"}.get(fmt, fmt)
    if fmt == "jpg" and img.mode == "RGBA":
        bg = Image.new("RGB", img.size, ImageColor.getrgb(o.background))
        bg.paste(img, mask=img.split()[3])
        img = bg
    dst = output_path(src, o.suffix, ext=fmt, out_dir=o.out_dir)
    save_kw = {"quality": o.quality} if fmt in ("jpg", "webp") else {"optimize": True}
    img.save(dst, {"jpg": "JPEG", "png": "PNG", "webp": "WEBP"}[fmt], **save_kw)
    return dst


def run_images(job: JobContext, o: ImageOpts) -> dict:
    files: list[Path] = []
    for p in o.paths:
        path = Path(p)
        if path.is_dir():
            files += sorted(f for f in path.iterdir() if f.suffix.lower() in IMAGE_EXTS)
        elif path.exists():
            files.append(path)
    if not files:
        raise AppError("처리할 이미지가 없어요.")
    results = []
    for i, f in enumerate(files):
        job.progress(i / len(files), f"{i + 1}/{len(files)} {f.name}")
        try:
            results.append({"src": str(f), "dst": str(process_image(f, o))})
        except AppError:
            raise
        except Exception as e:  # noqa: BLE001 - 파일 하나 실패해도 계속
            results.append({"src": str(f), "error": f"처리 실패: {e}"})
    return {"results": results}


# --- 영상 ---
SILENCE_START = re.compile(r"silence_start:\s*(-?[\d.]+)")
SILENCE_END = re.compile(r"silence_end:\s*([\d.]+)")
DURATION_RE = re.compile(r"Duration:\s*(\d+):(\d+):([\d.]+)")


def probe(path: str) -> dict:
    res = ffmpeg.run(["-i", path])
    m = DURATION_RE.search(res.stderr)
    duration = int(m.group(1)) * 3600 + int(m.group(2)) * 60 + float(m.group(3)) if m else 0.0
    return {"duration": duration, "has_video": bool(re.search(r"Stream #.*Video:", res.stderr)), "has_audio": bool(re.search(r"Stream #.*Audio:


def detect_silence(path: str, noise_db: float = -35, min_silence: float = 0.6) -> dict:
    if not Path(path).exists():
        raise AppError(f"파일을 찾을 수 없어요: {path}")
    info = probe(path)
    if not info["has_audio"]:
        raise AppError("소리가 없는 파일이에요.")
    res = ffmpeg.run(["-nostats", "-i", path, "-af", f"silencedetect=noise={noise_db}dB:d={min_silence}", "-vn", "-f", "null", "-"])
    silences = []
    start = None
    for line in res.stderr.splitlines():
        if (m := SILENCE_START.search(line)):
            start = max(0.0, float(m.group(1)))
```

```
        elif (m := SILENCE_END.search(line)) and start is not None:
            silences.append((start, float(m.group(1))))
            start = None
    if start is not None:
        silences.append((start, info["duration"]))
    return {"duration": info["duration"], "silences": silences, "has_video": info["has_video"]}


def keep_ranges(silences: list[tuple[float, float]], duration: float, pad: float = 0.15, min_keep: float = 0.25) -> list[tuple[float, float]]:
    ranges = []
    cur = 0.0
    for s, e in silences:
        if s - cur > 0:
            ranges.append((cur, min(duration, s + pad)))
        cur = max(cur, e - pad)
    if duration - cur > 0:
        ranges.append((cur, duration))
    merged: list[tuple[float, float]] = []
    for s, e in ranges:
        if e - s < min_keep:
            continue
        if merged and s <= merged[-1][1]:
            merged[-1] = (merged[-1][0], max(merged[-1][1], e))
        else:
            merged.append((s, e))
    return merged


def cut_silence(job: JobContext, path: str, noise_db: float, min_silence: float, pad: float, out_dir: str | None) -> dict:
    job.progress(0.02, "무음 구간 찾는 중")
    info = detect_silence(path, noise_db, min_silence)
    ranges = keep_ranges(info["silences"], info["duration"], pad)
    if not ranges:
        raise AppError("남길 구간이 없어요. 무음 기준(dB)을 낮춰 보세요.")
    kept = sum(e - s for s, e in ranges)
    expr = "+".join(f"between(t,{s:.3f},{e:.3f})" for s, e in ranges)
    src = Path(path)
    dst = output_path(src, "_무음컷", out_dir=out_dir)
    args = ["-y", "-i", path]
    if info["has_video"]:
        args += ["-vf", f"select='{expr}',setpts=N/FRAME_RATE/TB"]
    args += ["-af", f"aselect='{expr}',asetpts=N/SR/TB", "-progress", "pipe:1", "-nostats", str(dst)]
    proc = ffmpeg.popen(args)
    assert proc.stdout is not None
    for line in proc.stdout:
        if line.startswith("out_time_us=") or line.startswith("out_time_ms="):
            try:
                t = int(line.split("=")[1]) / 1_000_000
                job.progress(0.05 + 0.9 * min(1.0, t / max(kept, 0.1)), f"자르는 중 {int(t)}초 / {int(kept)}초")
            except ValueError:
                pass
        if job.cancelled:
            proc.kill()
            job.check()
    err = proc.stderr.read() if proc.stderr else ""
    if proc.wait() != 0:
        raise AppError(f"영상 자르기에 실패했어요: {err[-300:]}")
    return {"dst": str(dst), "original": info["duration"], "result": kept, "removed": info["duration"] - kept, "cuts": len(info["silences"])}


def make_subtitles(job: JobContext, path: str, model: str, fmt: str, out_dir: str | None) -> dict:
    res = stt.transcribe(job, path, model, "ko")
```

```
    src = Path(path)
    dst = output_path(src, "", ext=fmt, out_dir=out_dir)
    content = export.to_srt(res["segments"]) if fmt == "srt" else export.to_vtt(res["segments"])
    dst.write_text(content, encoding="utf-8")
    return {"dst": str(dst), "segments": len(res["segments"])}


class SilenceIn(BaseModel):
    path: str
    noise_db: float = -35
    min_silence: float = 0.6
    pad: float = 0.15
    out_dir: str | None = None


class SubtitleIn(BaseModel):
    path: str
    model: str = "small"
    format: str = "srt"
    out_dir: str | None = None


@router.get("/presets")
def presets():
    return {
        "presets": [{"key": k, **v} for k, v in PRESETS.items()],
        "rembg": optional.available("rembg"),
        "ffmpeg": ffmpeg.find_ffmpeg() is not None,
        "stt": optional.available("stt"),
        "font": fonts.korean_font_path() is not None,
    }


@router.post("/images")
def images(body: ImageOpts, ctx: Ctx):
    license.require_pro(ctx.db)
    if body.preset and body.preset not in PRESETS:
        raise AppError("알 수 없는 규격이에요.")
    if body.format not in ("jpg", "png", "webp", "same"):
        raise AppError("저장 형식은 jpg, png, webp, same 중 하나예요.")
    if body.remove_bg:
        optional.require("rembg")
    return job_ref(ctx.jobs.submit("images", f"이미지 {len(body.paths)}개 처리", run_images, body))


@router.post("/silence/detect")
def silence_detect(body: SilenceIn):
    info = detect_silence(body.path, body.noise_db, body.min_silence)
    ranges = keep_ranges(info["silences"], info["duration"], body.pad)
    kept = sum(e - s for s, e in ranges)
    return {**info, "keep": ranges, "result": kept, "removed": info["duration"] - kept}


@router.post("/silence/cut")
def silence_cut(body: SilenceIn, ctx: Ctx):
    license.require_pro(ctx.db)
    ffmpeg.require_ffmpeg()
    return job_ref(ctx.jobs.submit("cut", f"무음 컷: {Path(body.path).name}", cut_silence, body.path, body.noise_db, body.min_silence, body.pad, b


@router.post("/subtitles")
def subtitles(body: SubtitleIn, ctx: Ctx):
```

```
    license.require_pro(ctx.db)
    optional.require("stt")
    if body.format not in ("srt", "vtt"):
        raise AppError("자막 형식은 srt 또는 vtt예요.")
    return job_ref(ctx.jobs.submit("subtitles", f"자막 만들기: {Path(body.path).name}", make_subtitles, body.path, body.model, body.format, body.out
```

```python
"""메모와 할 일."""
from __future__ import annotations

from fastapi import APIRouter
from pydantic import BaseModel

from ..core import search
from ..core.context import Ctx
from ..core.db import Database, now_iso
from ..core.errors import NotFound

router = APIRouter(tags=["메모 · 할 일"])


def _index_note(db: Database, note_id: int) -> None:
    n = db.one("SELECT * FROM notes WHERE id=?", (note_id,))
    if n:
        search.index(db, "note", note_id, n["title"], f"{n['body']}\n{n['tags']}")


def add_note(db: Database, title: str, body: str, tags: str = "", pinned: bool = False) -> int:
    ts = now_iso()
    nid = db.insert(
        "INSERT INTO notes(title, body, tags, pinned, created_at, updated_at) VALUES(?,?,?,?,?,?)",
        (title, body, tags, int(pinned), ts, ts),
    )
    _index_note(db, nid)
    return nid


def add_todo(db: Database, text: str, due: str | None = None, assignee: str | None = None, source: str | None = None) -> int:
    tid = db.insert(
        "INSERT INTO todos(text, due, assignee, source, created_at) VALUES(?,?,?,?,?)",
        (text, due, assignee, source, now_iso()),
    )
    search.index(db, "todo", tid, text[:40], f"{text}\n{assignee or ''}")
    return tid


class NoteIn(BaseModel):
    title: str = ""
    body: str = ""
    tags: str = ""
    pinned: bool = False


class TodoIn(BaseModel):
    text: str
    due: str | None = None
    assignee: str | None = None
    source: str | None = None


class TodoPatch(BaseModel):
    text: str | None = None
    due: str | None = None
    assignee: str | None = None
    done: bool | None = None


@router.get("/notes")
def list_notes(ctx: Ctx, q: str = ""):
```

```
    if q:
        like = f"%{q}%"
        return ctx.db.query(
            "SELECT * FROM notes WHERE title LIKE ? OR body LIKE ? OR tags LIKE ? ORDER BY pinned DESC, updated_at DESC",
            (like, like, like),
        )
    return ctx.db.query("SELECT * FROM notes ORDER BY pinned DESC, updated_at DESC")


@router.post("/notes")
def create_note(body: NoteIn, ctx: Ctx):
    nid = add_note(ctx.db, body.title, body.body, body.tags, body.pinned)
    return ctx.db.one("SELECT * FROM notes WHERE id=?", (nid,))


@router.put("/notes/{note_id}")
def update_note(note_id: int, body: NoteIn, ctx: Ctx):
    cur = ctx.db.execute(
        "UPDATE notes SET title=?, body=?, tags=?, pinned=?, updated_at=? WHERE id=?",
        (body.title, body.body, body.tags, int(body.pinned), now_iso(), note_id),
    )
    if cur.rowcount == 0:
        raise NotFound("메모를 찾을 수 없어요.")
    _index_note(ctx.db, note_id)
    return ctx.db.one("SELECT * FROM notes WHERE id=?", (note_id,))


@router.delete("/notes/{note_id}")
def delete_note(note_id: int, ctx: Ctx):
    ctx.db.execute("DELETE FROM notes WHERE id=?", (note_id,))
    search.remove(ctx.db, "note", note_id)
    return {"ok": True}


@router.get("/todos")
def list_todos(ctx: Ctx, status: str = "open"):
    where = {"open": "WHERE done=0", "done": "WHERE done=1"}.get(status, "")
    order = "ORDER BY done, due IS NULL, due, id DESC"
    return ctx.db.query(f"SELECT * FROM todos {where} {order}")


@router.post("/todos")
def create_todo(body: TodoIn, ctx: Ctx):
    tid = add_todo(ctx.db, body.text, body.due, body.assignee, body.source)
    return ctx.db.one("SELECT * FROM todos WHERE id=?", (tid,))


@router.patch("/todos/{todo_id}")
def patch_todo(todo_id: int, body: TodoPatch, ctx: Ctx):
    cur = ctx.db.one("SELECT * FROM todos WHERE id=?", (todo_id,))
    if not cur:
        raise NotFound("할 일을 찾을 수 없어요.")
    fields = body.model_dump(exclude_unset=True)
    if "done" in fields:
        fields["done_at"] = now_iso() if fields["done"] else None
        fields["done"] = int(fields["done"])
    if fields:
        sets = ", ".join(f"{k}=?" for k in fields)
        ctx.db.execute(f"UPDATE todos SET {sets} WHERE id=?", (*fields.values(), todo_id))
    row = ctx.db.one("SELECT * FROM todos WHERE id=?", (todo_id,))
    search.index(ctx.db, "todo", todo_id, row["text"][:40], f"{row['text']}\n{row['assignee'] or ''}")
    return row
```

```
125
126
127 @router.delete("/todos/{todo_id}")
128 def delete_todo(todo_id: int, ctx: Ctx):
129     ctx.db.execute("DELETE FROM todos WHERE id=?", (todo_id,))
130     search.remove(ctx.db, "todo", todo_id)
131     return {"ok": True}
```

```python
"""영수증 → 경비 장부(엑셀), 명함 → 연락처(vCard)."""
from __future__ import annotations

import re
from datetime import date, datetime
from pathlib import Path

from fastapi import APIRouter
from pydantic import BaseModel

from ..core import search
from ..core.config import sub_dir
from ..core.context import Ctx
from ..core.db import now_iso
from ..core.errors import AppError
from ..core.korean import looks_like_name
from .screen_ocr import read_image

router = APIRouter(tags=["영수증 · 명함"])

# --- 영수증 ---
TOTAL_KEYS = ["총결제금액", "결제금액", "받을금액", "받은금액", "합계금액", "총합계", "총액", "합계", "판매총액", "청구금액", "승인금액", "결제 금액
AMOUNT_RE = re.compile(r"(?<![\d,])(\d{1,3}(?:,\d{3})+|\d{3,9})(?:\s*원)?(?![\d,])")
DATE_RES = [
    re.compile(r"(20\d{2})\s*[-./년]\s*(\d{1,2})\s*[-./월]\s*(\d{1,2})"),
    re.compile(r"(?<!\d)(\d{2})[-./](\d{2})[-./](\d{2})(?!\d)"),
]
BIZNO_RE = re.compile(r"(?<!\d)(\d{3})-?(\d{2})-?(\d{5})(?!\d)")
CATEGORY_HINTS = {
    "식비": ["식당", "카페", "커피", "베이커리", "치킨", "피자", "김밥", "국밥", "스타벅스", "투썸", "이디야", "맥도날드", "버거", "분식", "한식", "
    "교통": ["주유", "택시", "주차", "하이패스", "고속", "철도", "코레일", "KTX", "SRT", "버스", "오일"],
    "사무용품": ["문구", "오피스", "다이소", "알파", "모닝글로리", "인쇄", "복사"],
    "통신": ["통신", "KT", "SKT", "LG U+", "유플러스"],
    "숙박": ["호텔", "모텔", "펜션", "리조트", "숙박"],
    "마트": ["마트", "이마트", "홈플러스", "롯데마트", "편의점", "GS25", "CU", "세븐일레븐", "이마트24"],
}


def _to_int(s: str) -> int:
    return int(s.replace(",", ""))


def parse_receipt(text: str) -> dict:
    lines = [l.strip() for l in text.splitlines() if l.strip()]
    total = None
    for key in TOTAL_KEYS:
        k = key.replace(" ", "")
        for i, line in enumerate(lines):
            if k in line.replace(" ", ""):
                after = line.replace(" ", "").split(k, 1)[1]
                nums = AMOUNT_RE.findall(after) or (AMOUNT_RE.findall(lines[i + 1]) if i + 1 < len(lines) else [])
                if nums:
                    total = max(_to_int(n) for n in nums)
                    break
        if total:
            break
    if total is None:
        nums = [_to_int(n) for n in AMOUNT_RE.findall(text) if "," in n or n.endswith("00")]
        total = max(nums) if nums else None
    day = None
    for rx in DATE_RES:
        m = rx.search(text)
```

```
        if m:
            y, mo, d = m.groups()
            y = int(y) + (2000 if len(y) == 2 else 0)
            try:
                day = date(y, int(mo), int(d)).isoformat()
                break
            except ValueError:
                continue
    biz = BIZNO_RE.search(text)
    merchant = ""
    for line in lines[:5]:
        if re.search(r"[가-힣A-Za-z]{2,}", line) and not re.search(r"영수증|receipt|매출전표|카드|사업자|대표|전화|TEL|주소", line, re.I):
            merchant = re.sub(r"\s{2,}", " ", line)[:40]
            break
    category = ""
    compact = re.sub(r"\s", "", text).lower()
    for cat, words in CATEGORY_HINTS.items():
        if any(w.lower().replace(" ", "") in compact for w in words):
            category = cat
            break
    return {
        "merchant": merchant,
        "date": day,
        "amount": total,
        "biz_no": "-".join(biz.groups()) if biz else "",
        "category": category,
    }


# --- 명함 ---
JOB_TITLES = ["대표이사", "대표", "부사장", "전무", "상무", "이사", "본부장", "실장", "부장", "차장", "과장", "대리", "주임", "사원", "팀장", "센터장
COMPANY_RE = re.compile(r"(?:주식회사|\(주\)|㈜|유한회사|\(유\)|재단법인|사단법인|법무법인|세무법인|회계법인|Co\.,?\s?Ltd|Inc\.?|Corp\.?)", re.I)
EMAIL_RE = re.compile(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}")
WEB_RE = re.compile(r"(?:https?://)?(?:www\.)[A-Za-z0-9.-]+\.[A-Za-z]{2,}(?:/\S*)?|https?://[A-Za-z0-9.-]+\.[A-Za-z]{2,}(?:/\S*)?")
PHONE_RE = re.compile(r"(?:\+82[-.\s]?)?0?\d{1,2}[-.\s)]?\d{3,4}[-.\s]?\d{4}")
ADDR_RE = re.compile(r"(서울|부산|대구|인천|광주|대전|울산|세종|경기|강원|충북|충남|전북|전남|경북|경남|제주|충청|전라|경상)\S*\s.+")


def _norm_phone(s: str) -> str:
    d = re.sub(r"\D", "", s)
    if d.startswith("82"):
        d = "0" + d[2:].lstrip("0")
    if d.startswith("02"):
        return f"02-{d[2:-4]}-{d[-4:]}"
    return f"{d[:3]}-{d[3:-4]}-{d[-4:]}" if len(d) >= 9 else s


def parse_card(text: str) -> dict:
    lines = [l.strip() for l in text.splitlines() if l.strip()]
    out = {k: "" for k in ("name", "company", "title", "mobile", "phone", "fax", "email", "address", "website")}
    for line in lines:
        low = line.lower()
        if not out["email"] and (m := EMAIL_RE.search(line)):
            out["email"] = m.group(0)
            continue
        if not out["website"] and (m := WEB_RE.search(line)) and "@" not in line:
            out["website"] = m.group(0)
            continue
        phones = PHONE_RE.findall(line)
        if phones:
            for p in phones:
                np_ = _norm_phone(p)
```

```
                before = line[: line.find(p)].lower()
                if re.search(r"fax|팩스|f\.", before) and not out["fax"]:
                    out["fax"] = np_
                elif np_.startswith("01") and not out["mobile"]:
                    out["mobile"] = np_
                elif not out["phone"]:
                    out["phone"] = np_
            continue
        if not out["address"] and ADDR_RE.match(line):
            out["address"] = line
            continue
        if not out["company"] and COMPANY_RE.search(line):
            out["company"] = line
            continue
        if "fax" in low or "tel" in low:
            continue
        if not out["title"]:
            for t in JOB_TITLES:
                if t in line:
                    out["title"] = t
                    rest = line.replace(t, "").strip(" /| · ")
                    if not out["name"] and looks_like_name(rest.replace(" ", "")):
                        out["name"] = rest.replace(" ", "")
                    break
            if out["title"]:
                continue
        compact = line.replace(" ", "")
        if not out["name"] and looks_like_name(compact):
            out["name"] = compact
    return out


def to_vcard(c: dict) -> str:
    lines = ["BEGIN:VCARD", "VERSION:3.0", f"FN:{c.get('name', '')}"]
    name = c.get("name", "")
    if name:
        lines.append(f"N:{name[0]};{name[1:]};;;")
    if c.get("company"):
        lines.append(f"ORG:{c['company']}")
    if c.get("title"):
        lines.append(f"TITLE:{c['title']}")
    for key, typ in (("mobile", "CELL"), ("phone", "WORK"), ("fax", "FAX")):
        if c.get(key):
            lines.append(f"TEL;TYPE={typ}:{c[key]}")
    if c.get("email"):
        lines.append(f"EMAIL;TYPE=WORK:{c['email']}")
    if c.get("address"):
        lines.append(f"ADR;TYPE=WORK:;;{c['address']};;;;")
    if c.get("website"):
        lines.append(f"URL:{c['website']}")
    lines.append("END:VCARD")
    return "\r\n".join(lines) + "\r\n"


class ParseIn(BaseModel):
    text: str | None = None
    path: str | None = None


class ExpenseIn(BaseModel):
    date: str | None = None
    merchant: str = ""
```

```
    amount: int | None = None
    category: str = ""
    memo: str = ""
    biz_no: str = ""


class ContactIn(BaseModel):
    name: str = ""
    company: str = ""
    title: str = ""
    mobile: str = ""
    phone: str = ""
    fax: str = ""
    email: str = ""
    address: str = ""
    website: str = ""


class ExportIn(BaseModel):
    month: str | None = None  # YYYY-MM
    out_path: str | None = None


def _source_text(body: ParseIn) -> str:
    if body.text:
        return body.text
    if body.path:
        return read_image(body.path)["text"]
    raise AppError("글자나 이미지 경로를 넣어 주세요.")


@router.post("/receipts/parse")
def receipt_parse(body: ParseIn):
    text = _source_text(body)
    return {"text": text, "fields": parse_receipt(text)}


@router.get("/expenses")
def list_expenses(ctx: Ctx, month: str | None = None):
    if month:
        return ctx.db.query("SELECT * FROM expenses WHERE date LIKE ? ORDER BY date DESC, id DESC", (f"{month}%",))
    return ctx.db.query("SELECT * FROM expenses ORDER BY date DESC, id DESC")


@router.post("/expenses")
def add_expense(body: ExpenseIn, ctx: Ctx):
    eid = ctx.db.insert(
        "INSERT INTO expenses(date, merchant, amount, category, memo, biz_no, created_at) VALUES(?,?,?,?,?,?,?)",
        (body.date, body.merchant, body.amount, body.category, body.memo, body.biz_no, now_iso()),
    )
    return ctx.db.one("SELECT * FROM expenses WHERE id=?", (eid,))


@router.delete("/expenses/{eid}")
def delete_expense(eid: int, ctx: Ctx):
    ctx.db.execute("DELETE FROM expenses WHERE id=?", (eid,))
    return {"ok": True}


@router.post("/expenses/export")
def export_expenses(body: ExportIn, ctx: Ctx):
    from openpyxl import Workbook
```

```
    from openpyxl.styles import Font, PatternFill

    rows = list_expenses(ctx, body.month)
    wb = Workbook()
    ws = wb.active
    ws.title = "경비"
    head = ["날짜", "가맹점", "금액", "분류", "메모", "사업자번호"]
    ws.append(head)
    for c in ws[1]:
        c.font = Font(bold=True)
        c.fill = PatternFill("solid", fgColor="E8EEF9")
    for r in rows:
        ws.append([r["date"], r["merchant"], r["amount"], r["category"], r["memo"], r["biz_no"]])
    total_row = len(rows) + 2
    ws.cell(row=total_row, column=2, value="합계").font = Font(bold=True)
    ws.cell(row=total_row, column=3, value=f"=SUM(C2:C{total_row - 1})").font = Font(bold=True)
    for row in ws.iter_rows(min_row=2, min_col=3, max_col=3):
        for c in row:
            c.number_format = "#,##0"
    for col, width in zip("ABCDEF", (12, 28, 12, 10, 30, 14)):
        ws.column_dimensions[col].width = width
    path = Path(body.out_path) if body.out_path else sub_dir("exports") / f"경비_{body.month or datetime.now().strftime('%Y%m%d')}.xlsx"
    path.parent.mkdir(parents=True, exist_ok=True)
    wb.save(path)
    return {"path": str(path), "count": len(rows), "total": sum(r["amount"] or 0 for r in rows)}


@router.post("/cards/parse")
def card_parse(body: ParseIn):
    text = _source_text(body)
    return {"text": text, "fields": parse_card(text)}


@router.get("/contacts")
def list_contacts(ctx: Ctx, q: str = ""):
    if q:
        like = f"%{q}%"
        return ctx.db.query("SELECT * FROM contacts WHERE name LIKE ? OR company LIKE ? OR email LIKE ? ORDER BY name", (like, like, like))
    return ctx.db.query("SELECT * FROM contacts ORDER BY id DESC")


@router.post("/contacts")
def add_contact(body: ContactIn, ctx: Ctx):
    data = body.model_dump()
    cid = ctx.db.insert(
        f"INSERT INTO contacts({', '.join(data)}, created_at) VALUES({', '.join('?' * len(data))}, ?)",
        (*data.values(), now_iso()),
    )
    search.index(ctx.db, "contact", cid, body.name or body.company, " ".join(v for v in data.values() if v))
    return ctx.db.one("SELECT * FROM contacts WHERE id=?", (cid,))


@router.delete("/contacts/{cid}")
def delete_contact(cid: int, ctx: Ctx):
    ctx.db.execute("DELETE FROM contacts WHERE id=?", (cid,))
    search.remove(ctx.db, "contact", cid)
    return {"ok": True}


@router.post("/contacts/export")
def export_contacts(body: ExportIn, ctx: Ctx):
    rows = ctx.db.query("SELECT * FROM contacts ORDER BY id")
```

```
311      path = Path(body.out_path) if body.out_path else sub_dir("exports") / f"연락처_{datetime.now():%Y%m%d}.vcf"
312      path.parent.mkdir(parents=True, exist_ok=True)
313      path.write_text("".join(to_vcard(r) for r in rows), encoding="utf-8", newline="")
314      return {"path": str(path), "count": len(rows)}
```

```python
"""PDF 합치기, 나누기, 회전, 압축, 이미지↔PDF."""
from __future__ import annotations

import io
import re
from pathlib import Path

from fastapi import APIRouter
from PIL import Image, ImageOps
from pydantic import BaseModel, Field

from ..core.config import output_path
from ..core.errors import AppError

router = APIRouter(prefix="/pdf", tags=["PDF 도구"])


def _reader(path: str):
    from pypdf import PdfReader
    from pypdf.errors import PdfReadError

    p = Path(path)
    if not p.exists():
        raise AppError(f"파일을 찾을 수 없어요: {path}")
    try:
        reader = PdfReader(str(p))
    except PdfReadError as e:
        raise AppError(f"PDF를 읽을 수 없어요: {p.name}") from e
    if reader.is_encrypted:
        try:
            if not reader.decrypt(""):
                raise AppError(f"암호가 걸린 PDF예요: {p.name}")
        except Exception as e:  # noqa: BLE001
            raise AppError(f"암호가 걸린 PDF예요: {p.name}") from e
    return reader


def parse_ranges(spec: str, total: int) -> list[list[int]]:
    """'1-3, 5, 7-' → [[0,1,2],[4],[6..끝]] (0부터 시작하는 페이지 번호 묶음)"""
    groups = []
    for part in re.split(r"[,\s]+", spec.strip()):
        if not part:
            continue
        m = re.fullmatch(r"(\d*)-(\d*)|(\d+)", part)
        if not m:
            raise AppError(f"페이지 범위를 이해하지 못했어요: {part}")
        if m.group(3):
            a = b = int(m.group(3))
        else:
            a = int(m.group(1) or 1)
            b = int(m.group(2) or total)
        if not (1 <= a <= b <= total):
            raise AppError(f"페이지 범위가 문서({total}쪽)를 벗어나요: {part}")
        groups.append(list(range(a - 1, b)))
    if not groups:
        raise AppError("페이지 범위를 넣어 주세요.")
    return groups


def merge(paths: list[str], out: str | None) -> dict:
    from pypdf import PdfWriter

```

```
    writer = PdfWriter()
    pages = 0
    for p in paths:
        reader = _reader(p)
        for page in reader.pages:
            writer.add_page(page)
            pages += 1
    dst = Path(out) if out else output_path(Path(paths[0]), "_합본")
    with dst.open("wb") as f:
        writer.write(f)
    return {"dst": str(dst), "pages": pages}


def split(path: str, ranges: str | None, every: int | None, out_dir: str | None) -> dict:
    from pypdf import PdfWriter

    reader = _reader(path)
    total = len(reader.pages)
    if every:
        groups = [list(range(i, min(i + every, total))) for i in range(0, total, every)]
    else:
        groups = parse_ranges(ranges or "", total)
    src = Path(path)
    outputs = []
    for g in groups:
        writer = PdfWriter()
        for i in g:
            writer.add_page(reader.pages[i])
        label = f"_{g[0] + 1}" if len(g) == 1 else f"_{g[0] + 1}-{g[-1] + 1}"
        dst = output_path(src, label, out_dir=out_dir)
        with dst.open("wb") as f:
            writer.write(f)
        outputs.append(str(dst))
    return {"outputs": outputs}


def rotate(path: str, degrees: int, pages: str | None) -> dict:
    from pypdf import PdfWriter

    if degrees % 90:
        raise AppError("회전은 90도 단위만 돼요.")
    reader = _reader(path)
    targets = {i for g in parse_ranges(pages, len(reader.pages)) for i in g} if pages else set(range(len(reader.pages)))
    writer = PdfWriter()
    for i, page in enumerate(reader.pages):
        if i in targets:
            page.rotate(degrees)
        writer.add_page(page)
    dst = output_path(Path(path), "_회전")
    with dst.open("wb") as f:
        writer.write(f)
    return {"dst": str(dst), "rotated": len(targets)}


def compress(path: str, quality: int = 60, max_px: int = 1600) -> dict:
    from pypdf import PdfWriter

    src = Path(path)
    reader = _reader(path)
    writer = PdfWriter(clone_from=reader)
    images = 0
    for page in writer.pages:
```

```
        for img in page.images:
            try:
                pil = img.image
                if pil is None:
                    continue
                if max(pil.size) > max_px:
                    pil = ImageOps.contain(pil, (max_px, max_px))
                if pil.mode not in ("RGB", "L"):
                    pil = pil.convert("RGB")
                img.replace(pil, quality=quality)
                images += 1
            except Exception:  # noqa: BLE001 - 바꿀 수 없는 이미지 형식은 그대로 둔다
                continue
        page.compress_content_streams()
    writer.compress_identical_objects(remove_identicals=True, remove_orphans=True)
    dst = output_path(src, "_압축")
    with dst.open("wb") as f:
        writer.write(f)
    before, after = src.stat().st_size, dst.stat().st_size
    return {"dst": str(dst), "before": before, "after": after, "images": images, "saved_ratio": round(1 - after / before, 3) if before else 0}


def images_to_pdf(paths: list[str], out: str | None, page: str = "fit") -> dict:
    if not paths:
        raise AppError("이미지를 골라 주세요.")
    pages = []
    for p in paths:
        im = ImageOps.exif_transpose(Image.open(p)).convert("RGB")
        if page == "a4":
            canvas = Image.new("RGB", (1240, 1754), "white")  # A4 150dpi
            fitted = ImageOps.contain(im, (1160, 1674))
            canvas.paste(fitted, ((1240 - fitted.width) // 2, (1754 - fitted.height) // 2))
            im = canvas
        pages.append(im)
    dst = Path(out) if out else output_path(Path(paths[0]), "_모음", ext="pdf")
    pages[0].save(dst, "PDF", save_all=True, append_images=pages[1:], resolution=150)
    return {"dst": str(dst), "pages": len(pages)}


def pdf_to_images(path: str, dpi: int = 150, fmt: str = "png", out_dir: str | None = None) -> dict:
    import pypdfium2 as pdfium

    src = Path(path)
    doc = pdfium.PdfDocument(str(src))
    outputs = []
    try:
        for i in range(len(doc)):
            img = doc[i].render(scale=dpi / 72).to_pil()
            dst = output_path(src, f"_{i + 1:03d}", ext=fmt, out_dir=out_dir)
            img.convert("RGB").save(dst, quality=90) if fmt == "jpg" else img.save(dst)
            outputs.append(str(dst))
    finally:
        doc.close()
    return {"outputs": outputs}


def info(path: str) -> dict:
    reader = _reader(path)
    meta = reader.metadata or {}
    return {
        "pages": len(reader.pages),
        "size": Path(path).stat().st_size,
```

```
        "title": str(meta.get("/Title", "") or ""),
        "author": str(meta.get("/Author", "") or ""),
    }


class MergeIn(BaseModel):
    paths: list[str] = Field(min_length=2)
    out_path: str | None = None


class SplitIn(BaseModel):
    path: str
    ranges: str | None = None
    every: int | None = None
    out_dir: str | None = None


class RotateIn(BaseModel):
    path: str
    degrees: int = 90
    pages: str | None = None


class CompressIn(BaseModel):
    path: str
    quality: int = 60
    max_px: int = 1600


class ImagesIn(BaseModel):
    paths: list[str] = Field(min_length=1)
    out_path: str | None = None
    page: str = "fit"


class ToImagesIn(BaseModel):
    path: str
    dpi: int = 150
    format: str = "png"
    out_dir: str | None = None


class PathIn(BaseModel):
    path: str


@router.post("/merge")
def merge_api(body: MergeIn):
    return merge(body.paths, body.out_path)


@router.post("/split")
def split_api(body: SplitIn):
    if not body.ranges and not body.every:
        raise AppError("나눌 범위나 쪽수를 넣어 주세요.")
    return split(body.path, body.ranges, body.every, body.out_dir)


@router.post("/rotate")
def rotate_api(body: RotateIn):
    return rotate(body.path, body.degrees, body.pages)

```

```
249
250 @router.post("/compress")
251 def compress_api(body: CompressIn):
252     return compress(body.path, body.quality, body.max_px)
253
254
255 @router.post("/from-images")
256 def from_images_api(body: ImagesIn):
257     return images_to_pdf(body.paths, body.out_path, body.page)
258
259
260 @router.post("/to-images")
261 def to_images_api(body: ToImagesIn):
262     if body.format not in ("png", "jpg"):
263         raise AppError("형식은 png 또는 jpg예요.")
264     return pdf_to_images(body.path, body.dpi, body.format, body.out_dir)
265
266
267 @router.post("/info")
268 def info_api(body: PathIn):
269     return info(body.path)
```

```python
"""파일 이름 일괄 변경. 미리보기 → 적용 → 되돌리기."""
from __future__ import annotations

import json
import re
import uuid
from datetime import datetime
from pathlib import Path

from fastapi import APIRouter
from pydantic import BaseModel, Field

from ..core.context import Ctx
from ..core.db import Database, now_iso
from ..core.errors import AppError, NotFound

router = APIRouter(prefix="/rename", tags=["이름 일괄 변경"])

INVALID = re.compile(r'[\\/:*?"<>|\x00-\x1f]')
TOKEN_RE = re.compile(r"\{(name|ext|parent|n|date|today|exif)(?::([^}]*))?\}")


class Rules(BaseModel):
    template: str = "{name}"  # {name} {ext} {parent} {n:03} {date:%Y%m%d} {today:%Y%m%d} {exif:%Y%m%d}
    find: str = ""
    replace: str = ""
    regex: bool = False
    case: str = "none"  # none | lower | upper
    spaces: str = "keep"  # keep | underscore | dash | remove
    start: int = 1
    step: int = 1
    sort: str = "name"  # name | mtime | size


def _exif_date(path: Path) -> datetime | None:
    try:
        from PIL import Image

        with Image.open(path) as im:
            exif = im.getexif()
            raw = exif.get_ifd(0x8769).get(36867) or exif.get(306)  # DateTimeOriginal, DateTime
        return datetime.strptime(str(raw), "%Y:%m:%d %H:%M:%S") if raw else None
    except Exception:  # noqa: BLE001 - 사진이 아니거나 EXIF가 없음
        return None


def _render(template: str, path: Path, n: int) -> str:
    def repl(m: re.Match) -> str:
        key, fmt = m.group(1), m.group(2)
        if key == "name":
            return path.stem
        if key == "ext":
            return path.suffix.lstrip(".")
        if key == "parent":
            return path.parent.name
        if key == "n":
            width = int(fmt) if fmt and fmt.isdigit() else len(fmt or "") or 1
            return str(n).zfill(width)
        if key == "date":
            return datetime.fromtimestamp(path.stat().st_mtime).strftime(fmt or "%Y%m%d")
        if key == "today":
            return datetime.now().strftime(fmt or "%Y%m%d")
```

```
        if key == "exif":
            d = _exif_date(path) or datetime.fromtimestamp(path.stat().st_mtime)
            return d.strftime(fmt or "%Y%m%d")
        return m.group(0)

    return TOKEN_RE.sub(repl, template)


def _sorted(paths: list[Path], key: str) -> list[Path]:
    if key == "mtime":
        return sorted(paths, key=lambda p: p.stat().st_mtime)
    if key == "size":
        return sorted(paths, key=lambda p: p.stat().st_size)
    return sorted(paths, key=lambda p: [int(t) if t.isdigit() else t.lower() for t in re.split(r"(\d+)", p.name)])


def plan(paths: list[str], rules: Rules) -> list[dict]:
    files = [Path(p) for p in paths]
    missing = [str(p) for p in files if not p.is_file()]
    if missing:
        raise AppError(f"파일을 찾을 수 없어요: {missing[0]}")
    out = []
    n = rules.start
    targets: dict[str, int] = {}
    for f in _sorted(files, rules.sort):
        stem = _render(rules.template, f, n)
        if rules.find:
            try:
                stem = re.sub(rules.find, rules.replace, stem) if rules.regex else stem.replace(rules.find, rules.replace)
            except re.error as e:
                raise AppError(f"정규식이 올바르지 않아요: {e}") from e
        if rules.case == "lower":
            stem = stem.lower()
        elif rules.case == "upper":
            stem = stem.upper()
        stem = {"underscore": stem.replace(" ", "_"), "dash": stem.replace(" ", "-"), "remove": stem.replace(" ", "")}.get(rules.spaces, stem)
        stem = INVALID.sub("_", stem).strip(" .") or f.stem
        ext = "" if "{ext}" in rules.template else f.suffix
        dst = f.with_name(stem + ext)
        key = str(dst).lower()
        targets[key] = targets.get(key, 0) + 1
        out.append({"src": str(f), "dst": str(dst), "changed": dst.name != f.name})
        n += rules.step
    sources = {r["src"].lower() for r in out}
    for r in out:
        dup = targets[r["dst"].lower()] > 1
        exists = Path(r["dst"]).exists() and r["dst"].lower() not in sources
        r["conflict"] = "같은 이름이 겹쳐요" if dup else ("이미 있는 파일이에요" if exists else None)
    return out


def apply(db: Database, items: list[dict]) -> dict:
    todo = [i for i in items if i["src"] != i["dst"]]
    if any(i.get("conflict") for i in items):
        raise AppError("이름이 겹치는 항목이 있어요. 미리보기를 확인해 주세요.")
    # 서로 이름을 바꾸는 경우(a→b, b→a)를 위해 임시 이름을 거친다.
    temp = []
    done = []
    try:
        for i in todo:
            src = Path(i["src"])
            tmp = src.with_name(f".lb_{uuid.uuid4().hex[:8]}{src.suffix}")
```

```
            src.rename(tmp)
            temp.append((tmp, i))
        for tmp, i in temp:
            tmp.rename(i["dst"])
            done.append({"src": i["src"], "dst": i["dst"]})
    except OSError as e:
        for tmp, i in temp:
            if tmp.exists():
                tmp.rename(i["src"])
        for d in done:
            if Path(d["dst"]).exists():
                Path(d["dst"]).rename(d["src"])
        raise AppError(f"이름을 바꾸지 못했어요: {e}") from e
    op_id = db.insert("INSERT INTO file_ops(kind, payload, created_at) VALUES('rename', ?, ?)", (json.dumps(done, ensure_ascii=False), now_iso(
    return {"op_id": op_id, "renamed": len(done)}


def undo(db: Database, op_id: int) -> dict:
    op = db.one("SELECT * FROM file_ops WHERE id=? AND kind='rename'", (op_id,))
    if not op:
        raise NotFound("되돌릴 기록이 없어요.")
    if op["undone"]:
        raise AppError("이미 되돌렸어요.")
    items = json.loads(op["payload"])
    restored, failed = 0, []
    for d in reversed(items):
        try:
            if Path(d["src"]).exists():
                failed.append(d["src"])
                continue
            Path(d["dst"]).rename(d["src"])
            restored += 1
        except OSError:
            failed.append(d["dst"])
    db.execute("UPDATE file_ops SET undone=1 WHERE id=?", (op_id,))
    return {"restored": restored, "failed": failed}


class PlanIn(BaseModel):
    paths: list[str] = Field(min_length=1)
    rules: Rules = Rules()


class ApplyIn(BaseModel):
    items: list[dict]


@router.post("/plan")
def plan_api(body: PlanIn):
    return {"items": plan(body.paths, body.rules)}


@router.post("/apply")
def apply_api(body: ApplyIn, ctx: Ctx):
    return apply(ctx.db, body.items)


@router.post("/undo/{op_id}")
def undo_api(op_id: int, ctx: Ctx):
    return undo(ctx.db, op_id)


```

```
@router.get("/history")
def history(ctx: Ctx):
    rows = ctx.db.query("SELECT * FROM file_ops WHERE kind='rename' ORDER BY id DESC LIMIT 20")
    return [{"id": r["id"], "created_at": r["created_at"], "undone": bool(r["undone"]), "count": len(json.loads(r["payload"]))} for r in rows]
```

```python
"""문장 속 한국어 날짜 · 시간 표현을 일정으로 바꾸고 .ics로 내보낸다."""
from __future__ import annotations

import calendar
import re
import uuid
from dataclasses import dataclass
from datetime import date, datetime, time, timedelta
from pathlib import Path

from fastapi import APIRouter
from pydantic import BaseModel

from ..core.config import sub_dir
from ..core.korean import split_sentences

router = APIRouter(prefix="/schedule", tags=["일정 추출"])

WEEKDAY = {"월": 0, "화": 1, "수": 2, "목": 3, "금": 4, "토": 5, "일": 6}
REL_DAY = {"오늘": 0, "금일": 0, "당일": 0, "내일": 1, "명일": 1, "모레": 2, "내일모레": 2, "글피": 3, "어제": -1, "그제": -2, "그저께": -2}
WEEK_OFFSET = {"이번": 0, "이번주": 0, "금주": 0, "다음": 1, "다음주": 1, "차주": 1, "다다음": 2, "다다음주": 2, "지난": -1, "지난주": -1, "저번": -

_DOW = r"(?:\s*\(?\s*[월화수목금토일](?:요일)?\s*\)?)?"
DATE_PATTERNS: list[tuple[str, re.Pattern]] = [
    ("iso", re.compile(r"(?<!\d)(20\d{2})\s*[-./]\s*(\d{1,2})\s*[-./]\s*(\d{1,2})(?!\d)\.?" + _DOW)),
    ("ymd", re.compile(r"(?:(\d{4})\s*년\s*)?(\d{1,2})\s*월\s*(\d{1,2})\s*일" + _DOW)),
    ("md_slash", re.compile(r"(?<![\d/])(\d{1,2})/(\d{1,2})(?![\d/])" + _DOW)),
    ("in_days", re.compile(r"(\d{1,3})\s*(일|주|개월|달)\s*(?:후|뒤)")),
    ("week_dow", re.compile(r"(이번\s?주|금주|다음\s?주|차주|다다음\s?주|지난\s?주|저번\s?주)\s*([월화수목금토일])(?:요일|(?![가-힣]))")),
    ("dow", re.compile(r"(?<![가-힣])([월화수목금토일])요일")),
    ("rel", re.compile(r"(?<![가-힣])(내일모레|오늘|금일|당일|내일|명일|모레|글피|어제|그저께|그제)(?!날)")),
    ("month_day", re.compile(r"(이번\s?달|이달|다음\s?달|담달)\s*(\d{1,2})\s*일")),
    ("day", re.compile(r"(?<![\d월년매])(\d{1,2})\s*일(?!\s*(?:후|뒤|전|간|동안|째|치|분|내|이내|마다))")),
    ("week_end", re.compile(r"(이번\s?주|다음\s?주)?\s*(주말|주중)")),
    ("month_end", re.compile(r"(이번\s?달|다음\s?달)?\s*(월말|월초)")),
    ("week_only", re.compile(r"(이번\s?주|다음\s?주|차주)\s*(?:중|안|내)?(?=\s*(?:까지|에|중|안|내|$))")),
]

_AMPM = r"(오전|오후|아침|점심|낮|저녁|밤|새벽)?\s*"
TIME_RE = re.compile(_AMPM + r"(\d{1,2})\s*시(?!간)\s*(?:(\d{1,2})\s*분|(반))?|" + _AMPM + r"(?<!\d)(\d{1,2}):(\d{2})(?!\d)|(정오|자정)")
RANGE_SEP_RE = re.compile(r"^\s*(?:~|-| - |부터|에서)\s*")
DURATION_RE = re.compile(r"(\d{1,2})\s*시간(?:\s*(\d{1,2})\s*분)?\s*(?:동안|정도|간)?")


@dataclass
class Match:
    start: int
    end: int
    value: object


def _norm(s: str | None) -> str:
    return re.sub(r"\s", "", s or "")


def _safe_date(y: int, m: int, d: int) -> date | None:
    try:
        return date(y, m, d)
    except ValueError:
        return None


```

```
def _future_year(m: int, d: int, today: date) -> date | None:
    cand = _safe_date(today.year, m, d)
    if cand and (today - cand).days > 60:
        cand = _safe_date(today.year + 1, m, d)
    return cand


def _week_monday(today: date, offset: int) -> date:
    return today - timedelta(days=today.weekday()) + timedelta(weeks=offset)


def find_date(text: str, today: date) -> Match | None:
    best: Match | None = None
    for kind, rx in DATE_PATTERNS:
        for m in rx.finditer(text):
            d = _resolve(kind, m, today)
            if d is None:
                continue
            cand = Match(m.start(), m.end(), d)
            # 가장 먼저 나오는 표현, 같으면 더 긴 표현
            if best is None or cand.start < best.start or (cand.start == best.start and cand.end > best.end):
                best = cand
            break
    return best


def _resolve(kind: str, m: re.Match, today: date) -> date | None:
    g = m.groups()
    if kind == "iso":
        return _safe_date(int(g[0]), int(g[1]), int(g[2]))
    if kind == "ymd":
        if g[0]:
            return _safe_date(int(g[0]), int(g[1]), int(g[2]))
        return _future_year(int(g[1]), int(g[2]), today)
    if kind == "md_slash":
        mo, d = int(g[0]), int(g[1])
        return _future_year(mo, d, today) if 1 <= mo <= 12 else None
    if kind == "in_days":
        n, unit = int(g[0]), g[1]
        if unit == "일":
            return today + timedelta(days=n)
        if unit == "주":
            return today + timedelta(weeks=n)
        return _add_months(today, n)
    if kind == "week_dow":
        return _week_monday(today, WEEK_OFFSET.get(_norm(g[0]), 0)) + timedelta(days=WEEKDAY[g[1]])
    if kind == "dow":
        delta = (WEEKDAY[g[0]] - today.weekday()) % 7 or 7
        return today + timedelta(days=delta)
    if kind == "rel":
        return today + timedelta(days=REL_DAY[g[0]])
    if kind == "month_day":
        base = today if _norm(g[0]) in ("이번달", "이달") else _add_months(today.replace(day=1), 1)
        return _safe_date(base.year, base.month, int(g[1]))
    if kind == "day":
        d = int(g[0])
        if not 1 <= d <= 31:
            return None
        cand = _safe_date(today.year, today.month, d)
        if cand and cand < today:
            nxt = _add_months(today.replace(day=1), 1)
            cand = _safe_date(nxt.year, nxt.month, d)
```

```
        return cand
    if kind == "week_end":
        offset = 1 if _norm(g[0]) == "다음주" else 0
        monday = _week_monday(today, offset)
        return monday + timedelta(days=5 if g[1] == "주말" else 2)
    if kind == "month_end":
        base = _add_months(today.replace(day=1), 1) if _norm(g[0]) == "다음달" else today
        if g[1] == "월초":
            return base.replace(day=1) if _norm(g[0]) == "다음달" else _add_months(today.replace(day=1), 1)
        return base.replace(day=calendar.monthrange(base.year, base.month)[1])
    if kind == "week_only":
        # "이번 주까지" → 그 주 금요일
        return _week_monday(today, WEEK_OFFSET.get(_norm(g[0]), 0)) + timedelta(days=4)
    return None


def _add_months(d: date, n: int) -> date:
    y, mo = divmod(d.month - 1 + n, 12)
    y += d.year
    mo += 1
    return date(y, mo, min(d.day, calendar.monthrange(y, mo)[1]))


def _hour(ampm: str | None, h: int) -> int | None:
    if h > 24:
        return None
    if ampm in ("오후", "저녁", "밤"):
        if ampm == "밤" and h == 12:
            return 0
        return h + 12 if h < 12 else h
    if ampm in ("오전", "아침", "새벽"):
        return 0 if h == 12 else h
    if ampm in ("점심", "낮"):
        return h + 12 if h <= 5 else h
    # 오전/오후 표시가 없으면 업무 시간 기준으로 본다: 1~7시는 오후.
    if 1 <= h <= 7:
        return h + 12
    return 0 if h == 24 else h


def find_times(text: str) -> list[Match]:
    out = []
    for m in TIME_RE.finditer(text):
        g = m.groups()
        if g[7]:
            t = time(12, 0) if g[7] == "정오" else time(0, 0)
        elif g[1] is not None:
            h = _hour(g[0], int(g[1]))
            minute = 30 if g[3] else int(g[2] or 0)
            if h is None or minute > 59:
                continue
            t = time(h % 24, minute)
        else:
            h = _hour(g[4], int(g[5]))
            minute = int(g[6])
            if h is None or minute > 59:
                continue
            t = time(h % 24, minute)
        out.append(Match(m.start(), m.end(), t))
    return out


```

```
@dataclass
class Event:
    title: str
    start: datetime | date
    end: datetime | date | None
    all_day: bool
    source: str

    def to_dict(self) -> dict:
        return {
            "title": self.title,
            "start": self.start.isoformat(),
            "end": self.end.isoformat() if self.end else None,
            "all_day": self.all_day,
            "source": self.source,
        }


_PARTICLE_EDGE = re.compile(r"^[\s,.~\-·:에까지부터는은도의]+|[\s,.~\-·:]+$")
_TRAIL_WORDS = re.compile(r"(에|까지|부터|에는|으로|로)\s*$")


def _title(sentence: str, spans: list[tuple[int, int]]) -> str:
    chars = list(sentence)
    for s, e in spans:
        for i in range(s, e):
            chars[i] = " "
    t = re.sub(r"\s+", " ", "".join(chars)).strip()
    t = re.sub(r"^(?:에는|에|까지|부터|은|는)\s+", "", t)
    t = _PARTICLE_EDGE.sub("", t)
    t = _TRAIL_WORDS.sub("", t).strip()
    return (t[:60] or "일정").strip()


_FOLLOW_PARTICLE = re.compile(r"\s*(?:까지|에는|에|부터|중으로|중에|중|안에|안으로|내로|내에|쯤|경)(?![가-힣])")
# 날짜만 있고 시각이 없으면 일정이 아니라 그냥 '오늘은~' 같은 말일 때가 많다.
_NOT_EVENT_DAYS = {"오늘", "금일", "당일", "어제", "그제", "그저께"}


def _span_with_particle(sentence: str, start: int, end: int) -> tuple[int, int]:
    m = _FOLLOW_PARTICLE.match(sentence, end)
    return (start, m.end()) if m else (start, end)


def parse_events(text: str, now: datetime | None = None) -> list[Event]:
    now = now or datetime.now()
    today = now.date()
    events = []
    for _, _, sentence in split_sentences(text):
        dm = find_date(sentence, today)
        times = [t for t in find_times(sentence) if not dm or not (dm.start <= t.start < dm.end)]
        if not dm and not times:
            continue
        day = dm.value if dm else today
        if day < today:
            continue  # 지난 일을 말하는 문장
        spans = [_span_with_particle(sentence, dm.start, dm.end)] if dm else []
        if not times:
            if sentence[dm.start : dm.end].strip() in _NOT_EVENT_DAYS:
                continue
            events.append(Event(_title(sentence, spans), day, None, True, sentence))
            continue
```

```
        start_t = times[0]
        spans.append(_span_with_particle(sentence, start_t.start, start_t.end))
        start = datetime.combine(day, start_t.value)
        end = None
        if len(times) > 1 and RANGE_SEP_RE.match(sentence[start_t.end : times[1].start] + " "):
            end_t = times[1]
            spans.append((start_t.end, end_t.end))
            end = datetime.combine(day, end_t.value)
            # "3시~5시"에서 끝 시각이 오전으로 해석되면 시작과 같은 오후로 맞춘다.
            if end <= start and end_t.value.hour < 12:
                end += timedelta(hours=12)
            if end <= start:
                end += timedelta(days=1)
        dur = DURATION_RE.search(sentence, start_t.end)
        if end is None and dur:
            spans.append((dur.start(), dur.end()))
            end = start + timedelta(hours=int(dur.group(1)), minutes=int(dur.group(2) or 0))
        events.append(Event(_title(sentence, spans), start, end or start + timedelta(hours=1), False, sentence))
    return events


def _ics_escape(s: str) -> str:
    return s.replace("\\", "\\\\").replace(";", "\\;").replace(",", "\\,").replace("\n", "\\n")


def _fold(line: str) -> str:
    # RFC 5545: 한 줄 75바이트 넘으면 접는다.
    out, cur = [], b""
    for ch in line:
        b = ch.encode("utf-8")
        if len(cur) + len(b) > 73:
            out.append(cur.decode("utf-8"))
            cur = b" "
        cur += b
    out.append(cur.decode("utf-8"))
    return "\r\n".join(out)


VTIMEZONE = [
    "BEGIN:VTIMEZONE", "TZID:Asia/Seoul", "BEGIN:STANDARD", "DTSTART:19700101T000000",
    "TZOFFSETFROM:+0900", "TZOFFSETTO:+0900", "TZNAME:KST", "END:STANDARD", "END:VTIMEZONE",
]


def to_ics(events: list[dict]) -> str:
    lines = ["BEGIN:VCALENDAR", "VERSION:2.0", "PRODID:-//LocalBiseo//KO", "CALSCALE:GREGORIAN", *VTIMEZONE]
    stamp = datetime.utcnow().strftime("%Y%m%dT%H%M%SZ")
    for ev in events:
        lines += ["BEGIN:VEVENT", f"UID:{uuid.uuid4()}@localbiseo", f"DTSTAMP:{stamp}"]
        if ev.get("all_day"):
            d = date.fromisoformat(ev["start"][:10])
            lines.append(f"DTSTART;VALUE=DATE:{d:%Y%m%d}")
            lines.append(f"DTEND;VALUE=DATE:{d + timedelta(days=1):%Y%m%d}")
        else:
            s = datetime.fromisoformat(ev["start"])
            e = datetime.fromisoformat(ev["end"]) if ev.get("end") else s + timedelta(hours=1)
            lines.append(f"DTSTART;TZID=Asia/Seoul:{s:%Y%m%dT%H%M%S}")
            lines.append(f"DTEND;TZID=Asia/Seoul:{e:%Y%m%dT%H%M%S}")
        lines.append(f"SUMMARY:{_ics_escape(ev.get('title') or '일정')}")
        if ev.get("source"):
            lines.append(f"DESCRIPTION:{_ics_escape(ev['source'])}")
        lines.append("END:VEVENT")
```

```
    lines.append("END:VCALENDAR")
    return "\r\n".join(_fold(l) for l in lines) + "\r\n"


class ParseIn(BaseModel):
    text: str
    now: str | None = None


class IcsIn(BaseModel):
    events: list[dict]
    out_path: str | None = None


@router.post("/parse")
def parse_api(body: ParseIn):
    now = datetime.fromisoformat(body.now) if body.now else None
    return {"events": [e.to_dict() for e in parse_events(body.text, now)]}


@router.post("/ics")
def ics_api(body: IcsIn):
    path = Path(body.out_path) if body.out_path else sub_dir("exports") / f"일정_{datetime.now():%Y%m%d_%H%M%S}.ics"
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(to_ics(body.events), encoding="utf-8", newline="")
    return {"path": str(path), "count": len(body.events)}
```

```python
"""화면/이미지 글자 인식. 앱 셸이 화면을 캡처해 파일 경로를 넘긴다."""
from __future__ import annotations

from pathlib import Path

from fastapi import APIRouter
from PIL import Image, ImageOps
from pydantic import BaseModel

from ..core import ocr, optional
from ..core.errors import AppError

router = APIRouter(prefix="/ocr", tags=["화면 OCR"])


class OcrIn(BaseModel):
    path: str
    crop: list[int] | None = None  # [x, y, w, h] 원본 픽셀 기준
    upscale: bool = True


def read_image(path: str, crop: list[int] | None = None, upscale: bool = True) -> dict:
    if not Path(path).exists():
        raise AppError(f"파일을 찾을 수 없어요: {path}")
    img = Image.open(path)
    img = ImageOps.exif_transpose(img).convert("RGB")
    if crop:
        x, y, w, h = crop
        if w <= 2 or h <= 2:
            raise AppError("선택한 영역이 너무 작아요.")
        img = img.crop((x, y, x + w, y + h))
    # 화면 글자는 작아서 2배로 키우면 인식률이 확 오른다.
    if upscale and img.width < 1600:
        img = img.resize((img.width * 2, img.height * 2), Image.Resampling.LANCZOS)
    return {"text": ocr.ocr_text(img, block=bool(crop)), "backend": ocr.backend_name()}


@router.get("/status")
def status():
    return {"available": optional.available("ocr"), "backend": ocr.backend_name()}


@router.post("/image")
def image(body: OcrIn):
    return read_image(body.path, body.crop, body.upscale)


class PreviewIn(BaseModel):
    path: str
    max_size: int = 1600


@router.post("/preview")
def preview(body: PreviewIn):
    """화면에 띄울 미리보기(data URL). 원본 크기도 같이 돌려줘서 선택 영역을 원본 좌표로 바꿀 수 있게 한다."""
    import base64
    import io

    if not Path(body.path).exists():
        raise AppError(f"파일을 찾을 수 없어요: {body.path}")
    img = ImageOps.exif_transpose(Image.open(body.path)).convert("RGB")
    width, height = img.size
```

```
63      img.thumbnail((body.max_size, body.max_size))
64      buf = io.BytesIO()
65      img.save(buf, "JPEG", quality=85)
66      return {"data_url": "" + base64.b64encode(buf.getvalue()).decode(), "width": width, "height": height}
```

```python
"""통합 검색: 메모, 할 일, 클립보드, 회의록, 연락처, 색인한 폴더의 문서."""
from __future__ import annotations

import os
from pathlib import Path

from fastapi import APIRouter
from pydantic import BaseModel

from ..core import search
from ..core.context import Ctx, job_ref
from ..core.db import Database
from ..core.errors import AppError
from ..core.extract import DOC_EXTS, extract_text
from ..core.jobs import JobContext

router = APIRouter(prefix="/search", tags=["통합 검색"])

MAX_FILE_BYTES = 30 * 1024 * 1024


def index_folder(job: JobContext, db: Database, folder: str) -> dict:
    root = Path(folder)
    if not root.is_dir():
        raise AppError(f"폴더를 찾을 수 없어요: {folder}")
    files = []
    for cur, dirs, names in os.walk(root):
        dirs[:] = [d for d in dirs if not d.startswith(".") and d not in ("node_modules", "__pycache__")]
        files += [Path(cur) / n for n in names if Path(n).suffix.lower() in DOC_EXTS]
    indexed, failed = 0, 0
    for i, f in enumerate(files):
        if i % 5 == 0:
            job.progress(i / max(len(files), 1), f"{i}/{len(files)} {f.name}")
        try:
            if f.stat().st_size > MAX_FILE_BYTES:
                continue
            text = extract_text(f)
            search.index(db, "file", str(f), f.name, text[:200_000])
            indexed += 1
        except Exception:  # noqa: BLE001 - 읽을 수 없는 파일은 건너뛴다
            failed += 1
    folders = set(db.get_setting("indexed_folders", []))
    folders.add(str(root))
    db.set_setting("indexed_folders", sorted(folders))
    return {"indexed": indexed, "failed": failed, "total": len(files)}


class IndexIn(BaseModel):
    folder: str


@router.get("")
def query(ctx: Ctx, q: str, kinds: str = "", limit: int = 40):
    kind_list = [k for k in kinds.split(",") if k] or None
    return {"results": search.query(ctx.db, q, kind_list, limit)}


@router.post("/index-folder")
def index_api(body: IndexIn, ctx: Ctx):
    return job_ref(ctx.jobs.submit("index", f"폴더 색인: {Path(body.folder).name}", index_folder, ctx.db, body.folder))


```

```
@router.get("/folders")
def folders(ctx: Ctx):
    count = ctx.db.scalar("SELECT count(*) FROM search_index WHERE kind='file'")
    return {"folders": ctx.db.get_setting("indexed_folders", []), "files": count}


@router.post("/reindex")
def reindex(ctx: Ctx):
    folders = ctx.db.get_setting("indexed_folders", [])
    ctx.db.execute("DELETE FROM search_index WHERE kind='file'")
    jobs = [ctx.jobs.submit("index", f"폴더 색인: {Path(f).name}", index_folder, ctx.db, f) for f in folders if Path(f).is_dir()]
    return {"jobs": [j.id for j in jobs]}
```

```python
"""상태, 설정, 라이선스, 작업 목록, 홈 화면 요약."""
from __future__ import annotations

import platform
import sys

from fastapi import APIRouter
from pydantic import BaseModel

from .. import __version__
from ..core import ffmpeg, fonts, license, llm, ocr, optional
from ..core.context import Ctx
from ..core.errors import NotFound

router = APIRouter(tags=["시스템"])

PUBLIC_SETTINGS = {"clipboard_enabled", "clipboard_sensitive", "default_out_dir", "whisper_model", "theme", "focus_minutes", "break_minutes", "


class LicenseIn(BaseModel):
    key: str


@router.get("/health")
def health():
    return {"ok": True, "version": __version__}


@router.get("/system/status")
def status(ctx: Ctx):
    return {
        "version": __version__,
        "python": sys.version.split()[0],
        "os": f"{platform.system()} {platform.release()}",
        "data_dir": str(ctx.data_dir),
        "components": optional.status(),
        "ffmpeg": ffmpeg.find_ffmpeg() is not None,
        "ocr_backend": ocr.backend_name(),
        "korean_font": fonts.korean_font_path(),
        "llm_model": str(llm.model_file()) if llm.model_file() else None,
        "search_tokenizer": ctx.db.fts_tokenizer,
        "license": license.state(ctx.db),
    }


@router.get("/system/settings")
def get_settings(ctx: Ctx):
    return {k: v for k, v in ctx.db.all_settings().items() if k in PUBLIC_SETTINGS}


@router.put("/system/settings")
def put_settings(body: dict, ctx: Ctx):
    for k, v in body.items():
        if k in PUBLIC_SETTINGS:
            ctx.db.set_setting(k, v)
    return get_settings(ctx)


@router.get("/license")
def get_license(ctx: Ctx):
    return license.state(ctx.db)

```

```
63
64 @router.post("/license")
65 def activate(body: LicenseIn, ctx: Ctx):
66     return license.activate(ctx.db, body.key)
67
68
69 @router.get("/dashboard")
70 def dashboard(ctx: Ctx):
71     db = ctx.db
72     return {
73         "todos_open": db.scalar("SELECT count(*) FROM todos WHERE done=0"),
74         "todos_due_today": db.scalar("SELECT count(*) FROM todos WHERE done=0 AND due <= date('now','localtime')"),
75         "notes": db.scalar("SELECT count(*) FROM notes"),
76         "clips": db.scalar("SELECT count(*) FROM clips"),
77         "transcripts": db.scalar("SELECT count(*) FROM transcripts"),
78         "contacts": db.scalar("SELECT count(*) FROM contacts"),
79         "focus_today": round(
80             db.scalar(
81                 "SELECT coalesce(sum((julianday(ended_at)-julianday(started_at))*1440),0) FROM focus_sessions "
82                 "WHERE kind='focus' AND ended_at IS NOT NULL AND date(started_at)=date('now','localtime')"
83             )
84             or 0
85         ),
86         "recent_transcripts": db.query("SELECT id, title, created_at FROM transcripts ORDER BY id DESC LIMIT 5"),
87         "upcoming_todos": db.query("SELECT * FROM todos WHERE done=0 ORDER BY due IS NULL, due, id DESC LIMIT 6"),
88         "license": license.state(db),
89     }
90
91
92 @router.get("/jobs")
93 def jobs(ctx: Ctx):
94     return [j.to_dict() for j in ctx.jobs.list()[:50]]
95
96
97 @router.get("/jobs/{job_id}")
98 def job(job_id: str, ctx: Ctx):
99     j = ctx.jobs.get(job_id)
100     if not j:
101         raise NotFound("작업을 찾을 수 없어요.")
102     return j.to_dict()
103
104
105 @router.post("/jobs/{job_id}/cancel")
106 def cancel(job_id: str, ctx: Ctx):
107     return {"cancelled": ctx.jobs.cancel(job_id)}
```

```
1
```

```python
from __future__ import annotations

import json
from datetime import datetime
from pathlib import Path

from fastapi import APIRouter
from pydantic import BaseModel

from ...core import license, optional, search
from ...core.config import sub_dir
from ...core.context import Ctx, job_ref
from ...core.db import Database, now_iso
from ...core.errors import AppError, NotFound
from ...core.korean import split_sentences
from ..actions import extract_actions
from ..masking.detect import mask_text
from ..notes import add_todo
from ..schedule import parse_events
from . import export, stt
from .summary import summarize

router = APIRouter(tags=["회의록"])


def _row_to_dict(row: dict) -> dict:
    t = dict(row)
    t["segments"] = json.loads(t["segments"])
    t["speakers"] = json.loads(t.get("speakers") or "{}")
    t["summary"] = json.loads(t["summary"]) if t.get("summary") else []
    return t


def get_transcript(db: Database, tid: int) -> dict:
    row = db.one("SELECT * FROM transcripts WHERE id=?", (tid,))
    if not row:
        raise NotFound("회의록을 찾을 수 없어요.")
    return _row_to_dict(row)


def _index(db: Database, tid: int) -> None:
    t = get_transcript(db, tid)
    search.index(db, "transcript", tid, t["title"], export.full_text(t["segments"]) + "\n" + "\n".join(t["summary"]))


def save_transcript(db: Database, title: str, segments: list[dict], source_path: str | None, language: str | None, duration: float | None) -> i
    text = export.full_text(segments)
    points = summarize(text)["points"] if text else []
    tid = db.insert(
        "INSERT INTO transcripts(title, source_path, language, duration, segments, summary, created_at) VALUES(?,?,?,?,?,?,?)",
        (title, source_path, language, duration, json.dumps(segments, ensure_ascii=False), json.dumps(points, ensure_ascii=False), now_iso()),
    )
    _index(db, tid)
    return tid


def _post_process(db: Database, tid: int, auto_todos: bool) -> dict:
    t = get_transcript(db, tid)
    text = "\n".join(s["text"] for s in t["segments"])
    actions = extract_actions(text)
    events = [e.to_dict() for e in parse_events(text)]
    added = []
```

```
    if auto_todos:
        added = [add_todo(db, a["text"], a["due"], a["assignee"], f"transcript:{tid}") for a in actions]
    return {"transcript_id": tid, "actions": actions, "events": events, "todos_added": len(added)}


def run_transcribe(job, db: Database, path: str, title: str, size: str, language: str | None, diarize: bool, max_speakers: int | None, auto_tod
    if not Path(path).exists():
        raise AppError(f"파일을 찾을 수 없어요: {path}")
    res = stt.transcribe(job, path, size, language)
    segments = res["segments"]
    if diarize and segments:
        segments = stt.diarize(job, path, segments, max_speakers)
    tid = save_transcript(db, title or Path(path).stem, segments, path, res["language"], res["duration"])
    return _post_process(db, tid, auto_todos)


class TranscribeIn(BaseModel):
    path: str
    title: str = ""
    model: str = "small"
    language: str | None = "ko"
    diarize: bool = False
    max_speakers: int | None = None
    auto_todos: bool = True


class TextIn(BaseModel):
    title: str = ""
    text: str
    auto_todos: bool = False


class UpdateIn(BaseModel):
    title: str | None = None
    segments: list[dict] | None = None
    speakers: dict | None = None
    summary: list[str] | None = None


class ExportIn(BaseModel):
    format: str = "docx"
    masked: bool = False
    timestamps: bool = True
    include_actions: bool = True
    out_path: str | None = None


class DownloadIn(BaseModel):
    size: str = "small"


@router.get("/transcribe/models")
def models():
    return {
        "available": optional.available("stt"),
        "diarize_available": optional.available("diarize"),
        "sizes": [{"key": k, "label": v} for k, v in stt.MODEL_SIZES.items()],
        "installed": stt.installed_models() if optional.available("stt") else [],
    }


@router.post("/transcribe/models/download")
```

```
def download(body: DownloadIn, ctx: Ctx):
    if body.size not in stt.MODEL_SIZES:
        raise AppError("알 수 없는 모델 크기예요.")
    optional.require("stt")
    return job_ref(ctx.jobs.submit("model", f"음성 모델({body.size}) 내려받기", stt.download_model, body.size))


@router.post("/transcribe")
def transcribe(body: TranscribeIn, ctx: Ctx):
    license.require_pro(ctx.db)
    optional.require("stt")
    if body.diarize:
        optional.require("diarize")
    if body.model not in stt.MODEL_SIZES:
        raise AppError("알 수 없는 모델 크기예요.")
    job = ctx.jobs.submit(
        "transcribe", f"회의록 만들기: {Path(body.path).name}", run_transcribe, ctx.db, body.path, body.title,
        body.model, body.language, body.diarize, body.max_speakers, body.auto_todos,
    )
    return job_ref(job)


@router.post("/transcribe/text")
def from_text(body: TextIn, ctx: Ctx):
    """녹음 없이 붙여넣은 글로 회의록을 만든다 (요약 · 할 일 추출용)."""
    segments = [{"start": 0, "end": 0, "text": s, "speaker": None} for _, _, s in split_sentences(body.text)]
    if not segments:
        raise AppError("내용이 비어 있어요.")
    tid = save_transcript(ctx.db, body.title or f"메모 {datetime.now():%m/%d %H:%M}", segments, None, "ko", None)
    return _post_process(ctx.db, tid, body.auto_todos)


@router.get("/transcripts")
def list_transcripts(ctx: Ctx):
    return ctx.db.query("SELECT id, title, source_path, duration, created_at FROM transcripts ORDER BY id DESC")


@router.get("/transcripts/{tid}")
def read(tid: int, ctx: Ctx):
    return get_transcript(ctx.db, tid)


@router.get("/transcripts/{tid}/insights")
def insights(tid: int, ctx: Ctx):
    t = get_transcript(ctx.db, tid)
    text = "\n".join(s["text"] for s in t["segments"])
    return {"actions": extract_actions(text), "events": [e.to_dict() for e in parse_events(text)]}


@router.put("/transcripts/{tid}")
def update(tid: int, body: UpdateIn, ctx: Ctx):
    get_transcript(ctx.db, tid)
    fields = {}
    if body.title is not None:
        fields["title"] = body.title
    if body.segments is not None:
        fields["segments"] = json.dumps(body.segments, ensure_ascii=False)
    if body.speakers is not None:
        fields["speakers"] = json.dumps(body.speakers, ensure_ascii=False)
    if body.summary is not None:
        fields["summary"] = json.dumps(body.summary, ensure_ascii=False)
    if fields:
```

```
        ctx.db.execute(f"UPDATE transcripts SET {', '.join(f'{k}=?' for k in fields)} WHERE id=?", (*fields.values(), tid))
        _index(ctx.db, tid)
    return get_transcript(ctx.db, tid)


@router.post("/transcripts/{tid}/summarize")
def resummarize(tid: int, ctx: Ctx):
    t = get_transcript(ctx.db, tid)
    res = summarize(export.full_text(t["segments"]))
    ctx.db.execute("UPDATE transcripts SET summary=? WHERE id=?", (json.dumps(res["points"], ensure_ascii=False), tid))
    _index(ctx.db, tid)
    return res


@router.delete("/transcripts/{tid}")
def delete(tid: int, ctx: Ctx):
    ctx.db.execute("DELETE FROM transcripts WHERE id=?", (tid,))
    search.remove(ctx.db, "transcript", tid)
    return {"ok": True}


def _masked_copy(t: dict) -> dict:
    t = dict(t)
    t["segments"] = [{**s, "text": mask_text(s["text"])[0]} for s in t["segments"]]
    t["summary"] = [mask_text(p)[0] for p in t["summary"]]
    t["title"] = mask_text(t["title"])[0]
    return t


@router.post("/transcripts/{tid}/export")
def export_api(tid: int, body: ExportIn, ctx: Ctx):
    if body.format not in export.FORMATS:
        raise AppError(f"형식은 {', '.join(export.FORMATS)} 중 하나예요.")
    t = get_transcript(ctx.db, tid)
    actions = extract_actions("\n".join(s["text"] for s in t["segments"])) if body.include_actions else None
    if body.masked:
        t = _masked_copy(t)
        if actions:
            actions = [{**a, "text": mask_text(a["text"])[0]} for a in actions]
    if body.out_path:
        path = Path(body.out_path)
    else:
        safe = "".join(c for c in (t["title"] or "회의록") if c not in '\\/:*?"<>|').strip() or "회의록"
        path = sub_dir("exports") / f"{safe}{'_마스킹' if body.masked else ''}.{body.format}"
    path.parent.mkdir(parents=True, exist_ok=True)
    if body.format == "docx":
        export.to_docx(t, path, actions)
    else:
        content = {
            "txt": lambda: export.to_txt(t, body.timestamps),
            "md": lambda: export.to_md(t, actions),
            "srt": lambda: export.to_srt(t["segments"], t["speakers"]),
            "vtt": lambda: export.to_vtt(t["segments"], t["speakers"]),
        }[body.format]()
        path.write_text(content, encoding="utf-8")
    return {"path": str(path)}
```

```python
"""회의록 내보내기: txt, md, srt, vtt, docx."""
from __future__ import annotations

from pathlib import Path

FORMATS = ("txt", "md", "srt", "vtt", "docx")


def _ts(sec: float, sep: str = ",") -> str:
    ms = int(round(sec * 1000))
    h, ms = divmod(ms, 3_600_000)
    m, ms = divmod(ms, 60_000)
    s, ms = divmod(ms, 1000)
    return f"{h:02d}:{m:02d}:{s:02d}{sep}{ms:03d}"


def _clock(sec: float) -> str:
    sec = int(sec)
    h, rem = divmod(sec, 3600)
    m, s = divmod(rem, 60)
    return f"{h:d}:{m:02d}:{s:02d}" if h else f"{m:02d}:{s:02d}"


def speaker_name(seg: dict, speakers: dict) -> str:
    sp = seg.get("speaker")
    if sp is None:
        return ""
    return speakers.get(str(sp)) or f"화자{int(sp) + 1}"


def paragraphs(segments: list[dict], speakers: dict, gap: float = 1.5) -> list[dict]:
    """화자가 바뀌거나 오래 쉬면 문단을 나눈다."""
    out: list[dict] = []
    for seg in segments:
        name = speaker_name(seg, speakers)
        if out and out[-1]["speaker"] == name and seg["start"] - out[-1]["end"] < gap:
            out[-1]["text"] += " " + seg["text"].strip()
            out[-1]["end"] = seg["end"]
        else:
            out.append({"speaker": name, "start": seg["start"], "end": seg["end"], "text": seg["text"].strip()})
    return out


def full_text(segments: list[dict]) -> str:
    return " ".join(s["text"].strip() for s in segments)


def to_txt(t: dict, timestamps: bool = True) -> str:
    lines = []
    for p in paragraphs(t["segments"], t.get("speakers", {})):
        head = []
        if timestamps:
            head.append(f"[{_clock(p['start'])}]")
        if p["speaker"]:
            head.append(f"{p['speaker']}:")
        lines.append((" ".join(head) + " " if head else "") + p["text"])
    return "\n\n".join(lines) + "\n"


def to_srt(segments: list[dict], speakers: dict | None = None) -> str:
    blocks = []
    for i, s in enumerate(segments, 1):
```

```
        name = speaker_name(s, speakers or {})
        text = f"{name}: {s['text'].strip()}" if name else s["text"].strip()
        blocks.append(f"{i}\n{_ts(s['start'])} --> {_ts(s['end'])}\n{text}\n")
    return "\n".join(blocks)


def to_vtt(segments: list[dict], speakers: dict | None = None) -> str:
    blocks = ["WEBVTT\n"]
    for s in segments:
        name = speaker_name(s, speakers or {})
        text = f"<v {name}>{s['text'].strip()}" if name else s["text"].strip()
        blocks.append(f"{_ts(s['start'], '.')} --> {_ts(s['end'], '.')}\n{text}\n")
    return "\n".join(blocks)


def to_md(t: dict, actions: list[dict] | None = None) -> str:
    out = [f"# {t.get('title') or '회의록'}", ""]
    if t.get("created_at"):
        out += [f"- 작성: {t['created_at'][:16].replace('T', ' ')}", ""]
    summary = t.get("summary") or []
    if summary:
        out += ["## 요약", *[f"- {p}" for p in summary], ""]
    if actions:
        out.append("## 할 일")
        for a in actions:
            extra = " / ".join(x for x in (a.get("assignee"), a.get("due")) if x)
            out.append(f"- [ ] {a['text']}" + (f" ({extra})" if extra else ""))
        out.append("")
    out += ["## 전체 내용", ""]
    for p in paragraphs(t["segments"], t.get("speakers", {})):
        who = f"**{p['speaker']}** " if p["speaker"] else ""
        out += [f"{who}`{_clock(p['start'])}` {p['text']}", ""]
    return "\n".join(out)


def to_docx(t: dict, path: Path, actions: list[dict] | None = None) -> None:
    import docx
    from docx.shared import Pt, RGBColor

    doc = docx.Document()
    style = doc.styles["Normal"]
    style.font.name = "맑은 고딕"
    style.font.size = Pt(10.5)
    doc.add_heading(t.get("title") or "회의록", level=1)
    if t.get("created_at"):
        doc.add_paragraph(f"작성: {t['created_at'][:16].replace('T', ' ')}")
    summary = t.get("summary") or []
    if summary:
        doc.add_heading("요약", level=2)
        for p in summary:
            doc.add_paragraph(p, style="List Bullet")
    if actions:
        doc.add_heading("할 일", level=2)
        for a in actions:
            extra = " / ".join(x for x in (a.get("assignee"), a.get("due")) if x)
            doc.add_paragraph(f" {a['text']}" + (f" ({extra})" if extra else ""))
    doc.add_heading("전체 내용", level=2)
    for p in paragraphs(t["segments"], t.get("speakers", {})):
        para = doc.add_paragraph()
        stamp = para.add_run(f"[{_clock(p['start'])}] ")
        stamp.font.color.rgb = RGBColor(0x88, 0x88, 0x88)
        if p["speaker"]:
```

```
125                 para.add_run(f"{p['speaker']}: ").bold = True
126             para.add_run(p["text"])
127         doc.save(str(path))
```

```python
"""음성 인식(faster-whisper)과 실험적 화자 분리(speechbrain)."""
from __future__ import annotations

import os
import threading
from pathlib import Path

from ...core import optional
from ...core.config import sub_dir
from ...core.jobs import JobContext

MODEL_SIZES = {
    "base": "빠름 (정확도 보통, 약 150MB)",
    "small": "균형 (권장, 약 500MB)",
    "medium": "정확함 (느림, 약 1.5GB)",
    "large-v3": "가장 정확함 (아주 느림, 약 3GB, 16GB RAM 이상)",
}

_lock = threading.Lock()
_models: dict[str, object] = {}


def whisper_dir() -> Path:
    return sub_dir("whisper", sub_dir("models"))


def installed_models() -> list[str]:
    root = whisper_dir()
    out = []
    for size in MODEL_SIZES:
        # huggingface_hub 캐시 폴더 이름 규칙
        if any(root.glob(f"models--Systran--faster-whisper-{size}/snapshots/*/model.bin")):
            out.append(size)
    return out


def _model(size: str):
    optional.require("stt")
    from faster_whisper import WhisperModel

    with _lock:
        if size not in _models:
            _models.clear()  # 메모리를 아끼려고 한 번에 하나만 올린다
            _models[size] = WhisperModel(
                size,
                device="auto",
                compute_type="int8",
                download_root=str(whisper_dir()),
                cpu_threads=max(1, (os.cpu_count() or 4) - 1),
            )
        return _models[size]


def download_model(job: JobContext, size: str) -> dict:
    job.progress(0.05, f"{size} 모델 내려받는 중… (처음 한 번만)")
    _model(size)
    return {"size": size, "installed": installed_models()}


def transcribe(job: JobContext, path: str, size: str = "small", language: str | None = "ko") -> dict:
    model = _model(size)
    job.progress(0.02, "음성 분석 시작")
```

```
    segments_iter, info = model.transcribe(
        path,
        language=language or None,
        vad_filter=True,
        beam_size=5,
        condition_on_previous_text=False,
    )
    duration = float(info.duration or 0) or 1.0
    segments = []
    for seg in segments_iter:
        segments.append({"start": round(seg.start, 2), "end": round(seg.end, 2), "text": seg.text.strip(), "speaker": None})
        job.progress(min(0.95, seg.end / duration), f"{int(seg.end // 60)}분 {int(seg.end % 60)}초 / {int(duration // 60)}분")
    return {"segments": segments, "duration": duration, "language": info.language}


def diarize(job: JobContext, path: str, segments: list[dict], max_speakers: int | None = None, threshold: float = 0.55) -> list[dict]:
    """구간별 목소리 임베딩을 묶어 화자 번호를 붙인다. (실험 기능)"""
    optional.require("diarize")
    import numpy as np
    import torch
    from faster_whisper.audio import decode_audio
    from speechbrain.inference.speaker import EncoderClassifier

    job.progress(0.96, "화자 구분 중")
    encoder = EncoderClassifier.from_hparams(
        source="speechbrain/spkrec-ecapa-voxceleb", savedir=str(sub_dir("ecapa", sub_dir("models")))
    )
    sr = 16000
    wave = decode_audio(path, sampling_rate=sr)
    embs: list = []
    for seg in segments:
        a, b = int(seg["start"] * sr), int(seg["end"] * sr)
        if b - a < sr // 2:
            embs.append(None)
            continue
        with torch.no_grad():
            e = encoder.encode_batch(torch.from_numpy(wave[a:b]).unsqueeze(0)).squeeze().numpy()
        embs.append(e / (np.linalg.norm(e) + 1e-9))
    centroids: list = []
    counts: list[int] = []
    labels: list[int | None] = []
    for e in embs:
        if e is None:
            labels.append(None)
            continue
        sims = [float(np.dot(e, c / (np.linalg.norm(c) + 1e-9))) for c in centroids]
        best = int(np.argmax(sims)) if sims else -1
        if best >= 0 and sims[best] >= threshold:
            centroids[best] = centroids[best] + e
            counts[best] += 1
            labels.append(best)
        else:
            centroids.append(e.copy())
            counts.append(1)
            labels.append(len(centroids) - 1)
    # 너무 많이 쪼개졌으면 가장 가까운 묶음끼리 합친다.
    mapping = list(range(len(centroids)))
    if max_speakers and len(centroids) > max_speakers:
        alive = list(range(len(centroids)))
        while len(alive) > max_speakers:
            best_pair, best_sim = None, -2.0
            for i in alive:
```

```
                for j in alive:
                    if i < j:
                        ci, cj = centroids[i], centroids[j]
                        s = float(np.dot(ci, cj) / (np.linalg.norm(ci) * np.linalg.norm(cj) + 1e-9))
                        if s > best_sim:
                            best_pair, best_sim = (i, j), s
            i, j = best_pair
            centroids[i] = centroids[i] + centroids[j]
            alive.remove(j)
            mapping = [i if m == j else m for m in mapping]
        order = {c: n for n, c in enumerate(sorted(set(mapping)))}
        mapping = [order[m] for m in mapping]
    # 앞 구간과 같은 화자로 채워 빈 칸을 메운다.
    prev = 0
    out = []
    for seg, lab in zip(segments, labels):
        sp = mapping[lab] if lab is not None else prev
        prev = sp
        out.append({**seg, "speaker": sp})
    return out
```

```python
"""요약: 로컬 AI 모델이 있으면 그걸 쓰고, 없으면 핵심 문장 추출."""
from __future__ import annotations

import math
from collections import Counter

from ...core import llm
from ...core.korean import content_words, split_sentences

KEY_HINTS = ("결정", "합의", "결론", "중요", "요약", "마감", "까지", "확정", "문제", "이슈", "목표", "다음", "예산", "일정")


def extractive(text: str, max_sentences: int = 5) -> list[str]:
    sentences = [s for _, _, s in split_sentences(text) if len(s) >= 8]
    if len(sentences) <= max_sentences:
        return sentences
    freq = Counter(w for s in sentences for w in content_words(s))
    if not freq:
        return sentences[:max_sentences]
    top = freq.most_common(1)[0][1]
    scores = []
    for i, s in enumerate(sentences):
        words = set(content_words(s))
        if not words:
            scores.append(0.0)
            continue
        score = sum(freq[w] / top for w in words) / math.sqrt(len(words) + 1)
        if any(h in s for h in KEY_HINTS):
            score *= 1.3
        if i < len(sentences) * 0.1:
            score *= 1.1
        scores.append(score)
    picked = sorted(sorted(range(len(sentences)), key=lambda i: scores[i], reverse=True)[:max_sentences])
    return [sentences[i] for i in picked]


def summarize(text: str, max_sentences: int = 5) -> dict:
    if llm.available() and len(text) > 200:
        prompt = (
            "다음은 회의/강의 녹취록이야. 핵심 내용을 한국어 글머리표 3~6개로 요약해. "
            "결정된 사항과 해야 할 일이 있으면 꼭 포함해. 녹취록에 없는 내용은 쓰지 마.\n\n" + text[:12000]
        )
        try:
            out = llm.complete(prompt, max_tokens=600)
            lines = [l.strip(" -•*\t") for l in out.splitlines() if l.strip(" -•*\t")]
            if lines:
                return {"method": "llm", "points": lines}
        except Exception:  # noqa: BLE001 - 모델 실패 시 규칙 기반으로 넘어간다
            pass
    return {"method": "extractive", "points": extractive(text, max_sentences)}
```

```python
"""오프라인 번역 (Argos Translate). 언어팩은 처음 한 번 내려받는다."""
from __future__ import annotations

from fastapi import APIRouter
from pydantic import BaseModel

from ..core import llm, optional
from ..core.context import Ctx, job_ref
from ..core.errors import AppError, FeatureUnavailable
from ..core.jobs import JobContext

router = APIRouter(prefix="/translate", tags=["번역"])

LANGS = {"ko": "한국어", "en": "영어", "ja": "일본어", "zh": "중국어"}


def _installed_pairs() -> set[tuple[str, str]]:
    if not optional.available("translate"):
        return set()
    from argostranslate import package

    return {(p.from_code, p.to_code) for p in package.get_installed_packages()}


def _route(src: str, dst: str, pairs: set[tuple[str, str]]) -> list[tuple[str, str]] | None:
    if (src, dst) in pairs:
        return [(src, dst)]
    # 한↔일 같은 조합은 영어를 거쳐 간다.
    if (src, "en") in pairs and ("en", dst) in pairs:
        return [(src, "en"), ("en", dst)]
    return None


def translate(text: str, src: str, dst: str) -> dict:
    if src == dst:
        return {"text": text, "engine": "none"}
    pairs = _installed_pairs()
    route = _route(src, dst, pairs)
    if route:
        from argostranslate import translate as at

        out = text
        for a, b in route:
            out = "\n".join(at.translate(line, a, b) if line.strip() else line for line in out.split("\n"))
        return {"text": out, "engine": "argos", "route": route}
    if llm.available():
        prompt = f"다음 글을 {LANGS.get(dst, dst)}로 자연스럽게 번역해. 번역문만 출력해.\n\n{text}"
        return {"text": llm.complete(prompt, max_tokens=1500), "engine": "llm"}
    if not optional.available("translate"):
        optional.require("translate")
    raise FeatureUnavailable(
        f"{LANGS.get(src, src)}→{LANGS.get(dst, dst)} 언어팩이 없어요. 번역 화면에서 언어팩을 내려받아 주세요.",
        component="translate_pack",
    )


def install_pair(job: JobContext, src: str, dst: str) -> dict:
    optional.require("translate")
    from argostranslate import package

    job.progress(0.05, "언어팩 목록 확인 중")
    package.update_package_index()
```

```
    available = package.get_available_packages()
    needed = [(src, dst)] if any(p.from_code == src and p.to_code == dst for p in available) else [(src, "en"), ("en", dst)]
    have = _installed_pairs()
    for i, (a, b) in enumerate(needed):
        if (a, b) in have:
            continue
        pkg = next((p for p in available if p.from_code == a and p.to_code == b), None)
        if pkg is None:
            raise AppError(f"{a}→{b} 언어팩을 찾지 못했어요.")
        job.progress(0.1 + 0.8 * i / len(needed), f"{LANGS.get(a, a)}→{LANGS.get(b, b)} 내려받는 중")
        package.install_from_path(pkg.download())
    return {"installed": sorted(f"{a}-{b}" for a, b in _installed_pairs())}


class TranslateIn(BaseModel):
    text: str
    source: str = "ko"
    target: str = "en"


class InstallIn(BaseModel):
    source: str
    target: str


@router.get("/status")
def status():
    pairs = _installed_pairs()
    return {
        "available": optional.available("translate"),
        "llm": llm.available(),
        "languages": [{"code": k, "label": v} for k, v in LANGS.items()],
        "installed": sorted(f"{a}-{b}" for a, b in pairs),
    }


@router.post("")
def translate_api(body: TranslateIn):
    if body.source not in LANGS or body.target not in LANGS:
        raise AppError("지원하지 않는 언어예요.")
    return translate(body.text, body.source, body.target)


@router.post("/install")
def install_api(body: InstallIn, ctx: Ctx):
    optional.require("translate")
    return job_ref(ctx.jobs.submit("translate_pack", f"언어팩 {body.source}→{body.target}", install_pair, body.source, body.target))
```

```python
"""맞춤법 교정(자주 틀리는 표현), 문체 바꾸기(해요체/합니다체/공문체), 이메일 틀."""
from __future__ import annotations

import re
from typing import Callable

from fastapi import APIRouter
from pydantic import BaseModel

from ..core import llm
from ..core.errors import AppError
from ..core.korean import (
    JONG_B, JONG_M, JONG_NONE, JONG_SS, JUNG_A, JUNG_AE, JUNG_E, JUNG_EO, JUNG_EU, JUNG_I, JUNG_O, JUNG_U, JUNG_WA, JUNG_WO,
    JUNG_YEO, compose, decompose, has_jong, is_hangul, jong_of, with_jong,
)

router = APIRouter(prefix="/writing", tags=["글다듬기"])

JONG_L = 8
JUNG_WAE, JUNG_OE = 10, 11

# (정규식, 바꿀 말, 이유)
SPELL_RULES: list[tuple[str, str | Callable[[re.Match], str], str]] = [
    (r"몇\s?일", "며칠", "'몇 일'이 아니라 '며칠'이 맞아요."),
    (r"어의\s?없", "어이없", "'어의없다'가 아니라 '어이없다'예요."),
    (r"금새", "금세", "'금시에'의 준말이라 '금세'예요."),
    (r"왠(만|일)", r"웬\1", "'웬만하다 · 웬일'이 맞아요. ('왠지'만 '왠')"),
    (r"왠(?=\s)", "웬", "'어찌 된'의 뜻이면 '웬'이에요."),
    (r"오랫만", "오랜만", "'오래간만'의 준말이라 '오랜만'이에요."),
    (r"(?<=[할갈볼줄올될쓸릴낼칠울])께(?=요|[\s.!?,]|$)", "게", "'-ㄹ게'가 맞아요. ('-ㄹ께' ×)"),
    (r"안\s?되(?=요|[.!?]|$)", "안 돼", "'되어'의 준말이라 '돼'예요."),
    (r"(?<![가-힣])되(?=요|서|야|[.!?]|$)", "돼", "'되어'의 준말이라 '돼'예요."),
    (r"(?<=[가-힣])되(?=요(?:[\s.!?,]|$)|서\s)", "돼", "'되어'의 준말이라 '돼'예요."),
    (r"됬", "됐", "'되었'의 준말이라 '됐'이에요."),
    (r"않\s?(하|되|돼)", r"안 \1", "뒤에 동사가 오면 '안'을 써요. ('않'은 '-지 않다')"),
    (r"일부로", "일부러", "'일부러'가 맞아요."),
    (r"설겆이", "설거지", "'설거지'가 표준어예요."),
    (r"희안", "희한", "'희한하다'가 맞아요."),
    (r"(깨끗|틈틈|일일|곰곰|번번|깊숙|따뜻|느긋)히", r"\1이", "'-이'로 적는 말이에요."),
    (r"(솔직|꼼꼼|열심|조용|급)이(?=[\s.,!?]|$)", r"\1히", "'-히'로 적는 말이에요."),
    (r"역활", "역할", "'역할()'이 맞아요."),
    (r"어떻해", "어떡해", "'어떻게 해'의 준말은 '어떡해'예요."),
    (r"뵈요", "봬요", "'뵈어요'의 준말이라 '봬요'예요."),
    (r"문안하", "무난하", "'무난하다()'가 맞아요."),
    (r"구지(?=\s)", "굳이", "'굳이'가 맞아요."),
    (r"요세(?=\s|[.,!?]|$)", "요새", "'요사이'의 준말이라 '요새'예요."),
    (r"제작년", "재작년", "'재작년()'이 맞아요."),
    (r"내노라", "내로라", "'내로라하다'가 맞아요."),
    (r"짜집기", "짜깁기", "'짜깁기'가 맞아요."),
    (r"얼만큼", "얼마큼", "'얼마만큼'의 준말은 '얼마큼'이에요."),
    (r"넓직", "널찍", "'널찍하다'가 맞아요."),
    (r"(?<=[있었했겠없])슴(?=[\s.!?,]|$)", "음", "'-음'으로 적어요. ('-슴' ×)"),
    (r"바래(요|서|야)", r"바라\1", "'바라다(희망)'의 활용은 '바라요'예요."),
    (r"바랬", "바랐", "'바라다'의 과거는 '바랐다'예요."),
    (r"\s+([.,!?])", r"\1", "문장부호 앞은 띄우지 않아요."),
    (r"[ \t]{2,}", " ", "빈칸이 두 번 이상 들어갔어요."),
]


SPACING_TAILS = ("수", "때", "것", "줄", "뿐", "만큼")


```

```
def _fix_spacing(text: str, changes: list[dict]) -> str:
    """'할수있다' → '할 수 있다'처럼 ㄹ 받침 뒤 의존명사를 띄운다."""
    out = []
    i = 0
    count = 0
    while i < len(text):
        ch = text[i]
        out.append(ch)
        if is_hangul(ch) and jong_of(ch) == JONG_L and i + 1 < len(text):
            for tail in SPACING_TAILS:
                if text.startswith(tail, i + 1):
                    after = text[i + 1 + len(tail) : i + 2 + len(tail)]
                    # '할수록', '갈때기'처럼 붙여 쓰는 말은 빼고, 뒤가 빈칸 · 조사 · 용언일 때만
                    if tail == "수" and text.startswith("수록", i + 1):
                        break
                    if after and after not in " 있없도는가를이에.,!?\n":
                        break
                    out.append(" ")
                    count += 1
                    break
        if ch == "수" and i + 1 < len(text) and text[i + 1] in "있없" and out[-2:-1] == [" "]:
            out.append(" ")
        i += 1
    if count:
        changes.append({"before": "할수", "after": "할 수", "reason": "의존명사(수 · 때 · 것 · 줄)는 띄어 써요.", "count": count})
    return "".join(out)


def spellcheck(text: str) -> dict:
    changes: list[dict] = []
    out = text
    for pattern, repl, reason in SPELL_RULES:
        rx = re.compile(pattern)
        found = list(rx.finditer(out))
        if not found:
            continue
        m0 = found[0]
        context = out[max(0, m0.start() - 2) : m0.end() + 2]
        before = context
        after = rx.sub(repl, context)
        out, n = rx.subn(repl, out)
        if before != after:
            changes.append({"before": before.strip() or "", "after": after.strip() or "", "reason": reason, "count": n})
    # ㄹ 받침 + 려고: 할려고 → 하려고
    rx = re.compile(r"([가-힣])려고")
    def _ryeo(m: re.Match) -> str:
        c = m.group(1)
        return (with_jong(c, JONG_NONE) if jong_of(c) == JONG_L and c not in "갈멀알열걸들살놀만" else c) + "려고"
    new = rx.sub(_ryeo, out)
    if new != out:
        changes.append({"before": "할려고", "after": "하려고", "reason": "'-려고' 앞에 ㄹ을 덧붙이지 않아요.", "count": 1})
        out = new
    out = _fix_spacing(out, changes)
    return {"text": out, "changes": changes}


# ---------------- 문체 ----------------
END_RE = re.compile(r"([가-힣]+)([.!?…~]*)(?=\s|$|[\"')\]])")

POLITE_TO_FORMAL_WORDS = {
    "아니에요": "아닙니다", "아녜요": "아닙니다", "거예요": "것입니다", "거에요": "것입니다", "해요": "합니다", "돼요": "됩니다",
    "아파요": "아픕니다", "바빠요": "바쁩니다", "나빠요": "나쁩니다", "기뻐요": "기쁩니다", "슬퍼요": "슬픕니다", "예뻐요": "예쁩니다",
```

```
    "써요": "씁니다", "꺼요": "끕니다", "떠요": "뜹니다", "커요": "큽니다", "몰라요": "모릅니다", "달라요": "다릅니다", "빨라요": "빠릅니다",
    "골라요": "고릅니다", "불러요": "부릅니다", "도와요": "돕습니다", "들어요": "듣습니다", "걸어요": "걷습니다", "지어요": "짓습니다",
    "나아요": "낫습니다", "그래요": "그렇습니다", "이래요": "이렇습니다", "저래요": "저렇습니다", "어때요": "어떻습니까", "줘요": "줍니다",
}


# 합니다체로 바꾸면 뜻이 달라지는 어미는 그대로 둔다.
KEEP_ENDINGS = ("지요", "죠", "네요", "데요", "군요", "거든요", "까요", "래요", "대요", "잖아요", "던데요", "는걸요")


def _polite_word_to_formal(word: str, question: bool) -> str | None:
    """해요체 어절 하나를 합니다체로. 바꿀 수 없으면 None."""
    tail = "니까" if question else "니다"
    for k, v in POLITE_TO_FORMAL_WORDS.items():
        if word.endswith(k):
            v = v[:-2] + "니까" if question and v.endswith("니다") else v
            return word[: -len(k)] + v
    for k, v in (("이에요", "입"), ("예요", "입"), ("에요", "입")):
        if word.endswith(k):
            return word[: -len(k)] + v + tail
    if word.endswith(("을게요", "을께요")):
        return word[:-3] + "겠습" + tail
    if word.endswith(("게요", "께요")) and len(word) > 2 and jong_of(word[-3]) == JONG_L:
        return word[:-3] + with_jong(word[-3], JONG_NONE) + "겠습" + tail
    if word.endswith("세요"):
        return word[:-2] + ("십니까" if question else "십시오")
    if word.endswith("나요") and len(word) > 2 and is_hangul(word[-3]):
        stem = word[:-2]
        if has_jong(stem[-1]):
            return stem + "습" + tail
        return stem[:-1] + with_jong(stem[-1], JONG_B) + tail
    if word.endswith(KEEP_ENDINGS):
        return None
    if not word.endswith("요") or len(word) < 2:
        return None
    stem = word[:-1]
    last = stem[-1]
    if not is_hangul(last):
        return None
    if stem.endswith(("어", "아")) and len(stem) >= 2 and is_hangul(stem[-2]):
        prev = stem[-2]
        j = jong_of(prev)
        if j == JONG_L:  # 알아요 → 압니다, 만들어요 → 만듭니다
            return stem[:-2] + with_jong(prev, JONG_B) + tail
        if j > 0:  # 있어요 → 있습니다
            return stem[:-1] + "습" + tail
        return None
    cho, jung, jong = decompose(last)
    if jong:
        return None
    if last == "워" and len(stem) >= 2:  # 어려워요 → 어렵습니다
        return stem[:-2] + with_jong(stem[-2], JONG_B) + "습" + tail
    new_jung = {JUNG_YEO: JUNG_I, JUNG_WA: JUNG_O, JUNG_WO: JUNG_U, JUNG_WAE: JUNG_OE}.get(jung, jung)
    if jung in (JUNG_A, JUNG_AE, JUNG_EO, JUNG_E, JUNG_YEO, JUNG_WA, JUNG_WO, JUNG_WAE, JUNG_I, JUNG_O, JUNG_U, JUNG_OE):
        return stem[:-1] + compose(cho, new_jung, JONG_B) + tail
    return None


def _formal_word_to_polite(word: str, question: bool) -> str | None:
    q = "?" if question else ""
    special = {"합니다": "해요", "합니까": "해요", "됩니다": "돼요", "됩니까": "돼요", "아닙니다": "아니에요", "아닙니까": "아니에요",
               "십시오": "세요", "십니다": "세요", "십니까": "세요", "것입니다": "거예요", "것입니까": "거예요",
```

```
                "겁니다": "거예요", "압니다": "알아요", "만듭니다": "만들어요", "엽니다": "열어요", "놉니다": "놀아요"}
    for k, v in special.items():
        if word.endswith(k):
            return word[: -len(k)] + v
    for k in ("입니다", "입니까"):
        if word.endswith(k):
            stem = word[: -len(k)]
            if not stem:
                return None
            return stem + ("이에요" if has_jong(stem[-1]) else "예요")
    for k in ("습니다", "습니까"):
        if word.endswith(k) and len(word) > 3:
            stem = word[: -len(k)]
            last = stem[-1]
            if not is_hangul(last):
                return None
            if stem.endswith(IRREGULAR_B):
                return stem[:-1] + with_jong(last, JONG_NONE) + "워요"
            if stem.endswith("돕"):
                return stem[:-1] + "도와요"
            _, jung, jong = decompose(last)
            if jong == JONG_SS:
                return stem + "어요"
            return stem + ("아요" if jung in (JUNG_A, JUNG_O, 2) else "어요")
    for k in ("니다", "니까"):
        if word.endswith(k) and len(word) > 2:
            stem = word[: -len(k)]
            last = stem[-1]
            if not is_hangul(last) or jong_of(last) != JONG_B:
                return None
            cho, jung, _ = decompose(last)
            base = stem[:-1]
            if jung == JUNG_O:
                return base + compose(cho, JUNG_WA) + "요"
            if jung == JUNG_U:
                return base + compose(cho, JUNG_WO) + "요"
            if jung == JUNG_OE:
                return base + compose(cho, JUNG_WAE) + "요"
            if jung == JUNG_I:
                return base + compose(cho, JUNG_YEO) + "요"
            if jung == JUNG_EU:
                prev_jung = decompose(base[-1])[1] if base and is_hangul(base[-1]) else JUNG_EO
                return base + compose(cho, JUNG_A if prev_jung in (JUNG_A, JUNG_O) else JUNG_EO) + "요"
            return base + compose(cho, jung) + "요"
    return None


IRREGULAR_B = ("어렵", "쉽", "고맙", "반갑", "덥", "춥", "가볍", "무겁", "귀엽", "즐겁", "아름답", "새롭", "어둡", "맵", "가깝", "아깝", "부럽", "그

def _formal_word_to_official(word: str) -> str | None:
    if word.endswith("습니다"):
        stem = word[:-3]
        if stem.endswith(IRREGULAR_B):  # 어렵습니다 → 어려움
            return stem[:-1] + with_jong(stem[-1], JONG_NONE) + "움"
        return stem + "음"
    if word.endswith("니다") and len(word) > 2 and jong_of(word[-3]) == JONG_B:
        return word[:-3] + with_jong(word[-3], JONG_M)
    if word.endswith("십시오"):
        return word[:-3] + "시기 바람"
    return None

```

```python

def _convert(text: str, fn: Callable[[str, bool], str | None]) -> tuple[str, int]:
    count = 0

    def repl(m: re.Match) -> str:
        nonlocal count
        word, punct = m.group(1), m.group(2)
        # 문장 끝 어절만 바꾼다 (문장부호가 있거나 줄 끝).
        end = m.end()
        at_end = bool(punct) or end == len(text) or text[end] == "\n"
        if not at_end:
            return m.group(0)
        new = fn(word, "?" in punct)
        if new and new != word:
            count += 1
            return new + punct
        return m.group(0)

    return END_RE.sub(repl, text), count


STYLES = {"formal": "합니다체", "polite": "해요체", "official": "공문체(~함/~임)"}


def change_tone(text: str, style: str) -> dict:
    if style not in STYLES:
        raise AppError("문체는 formal, polite, official 중 하나예요.")
    if style == "polite":
        out, n = _convert(text, _formal_word_to_polite)
    else:
        out, n = _convert(text, _polite_word_to_formal)
        if style == "official":
            out, n2 = _convert(out, lambda w, q: _formal_word_to_official(w))
            n = max(n, n2)
            # 개조식은 '~함', '~음' 뒤에 마침표를 찍지 않는다.
            out = re.sub(r"([가-힣])\.(?=\s|$)", lambda m: m.group(1) if jong_of(m.group(1)) == JONG_M else m.group(0), out)
    return {"text": out, "changed": n, "style": STYLES[style]}


def email(body: str, recipient: str = "", sender: str = "", company: str = "", subject: str = "", style: str = "formal") -> dict:
    content = change_tone(body, "formal" if style == "formal" else "polite")["text"].strip()
    to = f"{recipient}님" if recipient and not recipient.endswith("님") else recipient
    me = f"{company} {sender}".strip()
    lines = []
    lines.append(f"안녕하세요, {to}." if to else "안녕하세요.")
    if me:
        lines.append(f"{me}입니다.")
    lines += ["", content, ""]
    lines.append("확인 부탁드립니다." if style == "formal" else "확인 부탁드려요.")
    lines.append("감사합니다.")
    if sender:
        lines += ["", f"{me} 드림"]
    return {"subject": subject or (content.splitlines()[0][:30] if content else ""), "text": "\n".join(lines)}


class TextIn(BaseModel):
    text: str


class ToneIn(BaseModel):
    text: str
    style: str = "formal"
```

```
    use_ai: bool = False


class EmailIn(BaseModel):
    body: str
    recipient: str = ""
    sender: str = ""
    company: str = ""
    subject: str = ""
    style: str = "formal"


@router.post("/spell")
def spell_api(body: TextIn):
    return spellcheck(body.text)


@router.post("/tone")
def tone_api(body: ToneIn):
    if body.use_ai and llm.available():
        prompt = f"다음 글의 내용은 그대로 두고 {STYLES.get(body.style, '합니다체')}로 자연스럽게 고쳐 써. 고친 글만 출력해.\n\n{body.text}"
        return {"text": llm.complete(prompt, max_tokens=1500), "changed": -1, "style": STYLES.get(body.style), "engine": "llm"}
    return change_tone(body.text, body.style)


@router.post("/email")
def email_api(body: EmailIn):
    return email(body.body, body.recipient, body.sender, body.company, body.subject, body.style)
```

```
"""엔진을 실행 파일 하나로 묶어 Tauri 사이드카 자리에 둔다.

  python tools/build_engine.py                    # 현재 PC용
  python tools/build_engine.py --ffmpeg C:/ffmpeg/bin/ffmpeg.exe --tesseract "C:/Program Files/Tesseract-OCR"

결과:
  app/src-tauri/resources/engine/localbiseo-engine[.exe] (+ _internal 폴더)
  app/src-tauri/resources/tools/ffmpeg/…, tools/tesseract/… (넘긴 경우)

한 파일(onefile)로 묶으면 실행할 때마다 수백 MB를 임시 폴더에 풀어서 느리다.
그래서 폴더째(onedir) 앱 리소스로 넣는다.

미리 설치할 것: pip install "./engine[stt,ocr,rembg,face,ffmpeg]" pyinstaller
"""
from __future__ import annotations

import argparse
import shutil
import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
ENGINE = ROOT / "engine"
TAURI = ROOT / "app" / "src-tauri"
FONT_CANDIDATES = [
    Path(r"C:\Windows\Fonts\NanumGothic.ttf"),
    Path("/usr/share/fonts/truetype/nanum/NanumGothic.ttf"),
    Path("/Library/Fonts/NanumGothic.ttf"),
]

# 동적으로 불러와서 PyInstaller가 놓치는 모듈
HIDDEN = [
    "uvicorn.logging", "uvicorn.loops.auto", "uvicorn.protocols.http.auto", "uvicorn.protocols.websockets.auto",
    "uvicorn.lifespan.on", "pypdfium2", "pypdfium2_raw", "olefile", "openpyxl", "docx",
]
COLLECT = ["faster_whisper", "ctranslate2", "rembg", "pytesseract", "cv2", "pypdfium2", "pypdfium2_raw", "docx", "imageio_ffmpeg"]


def target_triple() -> str:
    out = subprocess.run(["rustc", "-vV"], capture_output=True, text=True, check=True).stdout
    return next(line.split(": ", 1)[1] for line in out.splitlines() if line.startswith("host: "))


def installed(mod: str) -> bool:
    import importlib.util

    return importlib.util.find_spec(mod) is not None


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--target", help="Rust target triple (기본: 현재 PC)")
    ap.add_argument("--ffmpeg", help="함께 넣을 ffmpeg 실행 파일 (LGPL 빌드 권장)")
    ap.add_argument("--tesseract", help="함께 넣을 Tesseract 설치 폴더 (kor.traineddata 포함)")
    ap.add_argument("--font", help="워터마크용 한글 글꼴 (기본: 나눔고딕을 찾아서 넣음)")
    args = ap.parse_args()

    triple = args.target or target_triple()
    ext = ".exe" if "windows" in triple else ""
    work = ROOT / "engine" / "build"
    sep = ";" if sys.platform == "win32" else ":"
```

```
63
64      cmd = [
65          sys.executable, "-m", "PyInstaller", "--noconfirm", "--clean", "--onedir",
66          "--name", "localbiseo-engine",
67          "--distpath", str(work / "dist"), "--workpath", str(work / "work"), "--specpath", str(work),
68          "--add-data", f"{ENGINE / 'localbiseo' / 'core' / 'license_pubkey.pem'}{sep}localbiseo/core",
69      ]
70      font = Path(args.font) if args.font else next((f for f in FONT_CANDIDATES if f.exists()), None)
71      if font:
72          cmd += ["--add-data", f"{font}{sep}fonts"]
73      for h in HIDDEN:
74          cmd += ["--hidden-import", h]
75      for c in COLLECT:
76          if installed(c):
77              cmd += ["--collect-all", c]
78      if sys.platform != "win32":
79          cmd += ["--strip"]
80      cmd.append(str(ENGINE / "run_engine.py"))
81      print(" ".join(cmd))
82      subprocess.run(cmd, check=True, cwd=ENGINE)
83
84      engine_dst = TAURI / "resources" / "engine"
85      if engine_dst.exists():
86          shutil.rmtree(engine_dst)
87      shutil.copytree(work / "dist" / "localbiseo-engine", engine_dst)
88      print(f"엔진: {engine_dst} ({triple})")
89
90      tools = TAURI / "resources" / "tools"
91      tools.mkdir(parents=True, exist_ok=True)
92      (tools / "README.txt").write_text("로컬비서가 쓰는 외부 도구(ffmpeg, tesseract)가 이 폴더에 들어갑니다.\n", encoding="utf-8")
93      if args.ffmpeg:
94          (tools / "ffmpeg").mkdir(exist_ok=True)
95          shutil.copy2(args.ffmpeg, tools / "ffmpeg" / f"ffmpeg{ext}")
96          print("ffmpeg 포함")
97      if args.tesseract:
98          target = tools / "tesseract"
99          if target.exists():
100             shutil.rmtree(target)
101         shutil.copytree(args.tesseract, target, ignore=shutil.ignore_patterns("*.html", "doc", "uninstall*"))
102         print("tesseract 포함")
103
104
105 if __name__ == "__main__":
106     main()
```

```python
"""시연 영상 · 스크린샷용 예시 파일 만들기 (모두 가상의 인물 · 번호).

  python tools/demo/make_samples.py <폴더> [--audio]

--audio: edge-tts로 회의 녹음(meeting.mp3)도 만든다 (인터넷 필요).
"""
from __future__ import annotations

import asyncio
import sys
import zipfile
from pathlib import Path

from PIL import Image, ImageDraw, ImageFilter, ImageFont

FONT = "/usr/share/fonts/truetype/nanum/NanumGothic.ttf"
FONT_BOLD = "/usr/share/fonts/truetype/nanum/NanumGothicBold.ttf"

MEETING = (
    "오늘 회의 안건은 와디즈 펀딩 준비입니다. 목표 금액은 삼백만 원으로 정했습니다. "
    "김민수 대리가 금요일까지 상세페이지 시안을 공유하기로 했습니다. "
    "박지영 과장은 다음 주 월요일까지 촬영 장소 견적을 받아 주세요. "
    "다음 주 화요일 오후 두 시에 제품 촬영 일정을 잡았습니다. "
    "예산 문제는 다음 회의에서 다시 논의하겠습니다."
)

HWPX_SECTION = (
    '<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>'
    '<hs:sec xmlns:hs="http://www.hancom.co.kr/hwpml/2011/section" xmlns:hp="http://www.hancom.co.kr/hwpml/2011/paragraph">'
    '<hp:p id="1"><hp:run charPrIDRef="0"><hp:t>장학금 신청서</hp:t></hp:run></hp:p>'
    '<hp:p id="2"><hp:run charPrIDRef="0"><hp:t>신청인: 이서연</hp:t></hp:run></hp:p>'
    '<hp:p id="3"><hp:run charPrIDRef="0"><hp:t>연락처: 010-2468-1357</hp:t></hp:run></hp:p>'
    '<hp:p id="4"><hp:run charPrIDRef="0"><hp:t>주민등록번호: 030315-4123456</hp:t></hp:run></hp:p>'
    '<hp:p id="5"><hp:run charPrIDRef="0"><hp:t>이메일: seoyeon.lee@example.com</hp:t></hp:run></hp:p>'
    "</hs:sec>"
)


def product(path: Path, color: tuple[int, int, int], size=(1600, 1000)) -> None:
    w, h = size
    img = Image.new("RGB", size, (226, 221, 214))
    d = ImageDraw.Draw(img)
    d.rectangle((0, int(h * 0.64), w, h), fill=(205, 196, 184))
    sh = Image.new("L", size, 0)
    ImageDraw.Draw(sh).ellipse((w * 0.35, h * 0.61, w * 0.66, h * 0.69), fill=120)
    img.paste((150, 140, 128), mask=sh.filter(ImageFilter.GaussianBlur(18)))
    dark = tuple(int(c * 0.78) for c in color)
    light = tuple(min(255, int(c * 1.35) + 30) for c in color)
    d.rounded_rectangle((w * 0.375, h * 0.3, w * 0.6125, h * 0.66), radius=40, fill=color)
    d.ellipse((w * 0.375, h * 0.27, w * 0.6125, h * 0.33), fill=dark)
    d.ellipse((w * 0.3875, h * 0.28, w * 0.6, h * 0.322), fill=(30, 40, 60))
    d.ellipse((w * 0.6, h * 0.38, w * 0.68, h * 0.56), outline=color, width=34)
    d.rounded_rectangle((w * 0.4, h * 0.34, w * 0.419, h * 0.62), radius=14, fill=light)
    img.save(path, quality=92)


def text_image(path: Path, lines: list[str], size=(900, 300), title=False) -> None:
    img = Image.new("RGB", size, "white")
    d = ImageDraw.Draw(img)
    for i, line in enumerate(lines):
        f = ImageFont.truetype(FONT_BOLD if title and i == 0 else FONT, 34)
        d.text((30, 30 + i * 80), line, font=f, fill="black")
```

```
    img.save(path)


def main() -> None:
    out = Path(sys.argv[1])
    out.mkdir(parents=True, exist_ok=True)
    import docx

    doc = docx.Document()
    doc.add_heading("고객 상담 기록", 1)
    for line in ["상담일: 2026-10-20", "성명: 홍길동", "연락처: 010-1234-5678", "주민등록번호: 900101-1234567",
                 "주소: 서울특별시 강남구 테헤란로 123, 101동 1001호", "환불 계좌: 국민은행 123456-01-234567"]:
        doc.add_paragraph(line)
    doc.save(out / "고객상담기록.docx")

    with zipfile.ZipFile(out / "장학금신청서.hwpx", "w") as z:
        z.writestr(zipfile.ZipInfo("mimetype"), "application/hwp+zip")
        z.writestr("Contents/section0.xml", HWPX_SECTION, compress_type=zipfile.ZIP_DEFLATED)

    from fpdf import FPDF

    pdf = FPDF()
    pdf.add_font("n", fname=FONT)
    pdf.set_font("n", size=13)
    pdf.add_page()
    for line in ["보험금 청구서", "피보험자: 김철수", "주민등록번호: 850505-2345678", "연락처 010-9876-5432", "이메일 chulsoo.kim@example.com"]:
        pdf.cell(0, 10, line, new_x="LMARGIN", new_y="NEXT")
    pdf.output(str(out / "청구서.pdf"))

    text_image(out / "택배송장.png", ["택배 송장", "받는 분 연락처 010-5555-7777", "주소 부산광역시 해운대구 센텀로 45"], title=True)

    products = out / "상품사진"
    products.mkdir(exist_ok=True)
    product(products / "머그컵_블루.jpg", (47, 107, 237))
    product(products / "머그컵_코랄.jpg", (240, 112, 96))
    product(products / "머그컵_그린.jpg", (46, 158, 110), size=(1400, 1050))

    (out / "회의메모.txt").write_text(MEETING, encoding="utf-8")
    if "--audio" in sys.argv:
        import edge_tts

        asyncio.run(edge_tts.Communicate(MEETING, "ko-KR-SunHiNeural", rate="-5%").save(str(out / "회의녹음.mp3")))
    print(f"예시 파일: {out}")


if __name__ == "__main__":
    main()
```

```python
"""라이선스 키 발급 도구 (판매자 전용, 앱에 포함하지 않는다).

  # 1) 서명 키 만들기 (한 번만). 개인키는 절대 저장소에 올리지 말 것.
  python tools/license_gen.py keygen --private ~/.localbiseo-signing/private.pem

  # 2) 키 발급 (와디즈 서포터 1명당 1개)
  python tools/license_gen.py issue --private ~/.localbiseo-signing/private.pem --id WADIZ-0001 --name 홍길동

  # 3) CSV로 한꺼번에 발급 (열: id,name) → 키가 추가된 CSV 출력
  python tools/license_gen.py batch --private ~/.localbiseo-signing/private.pem backers.csv out.csv
"""
from __future__ import annotations

import argparse
import csv
import sys
from datetime import date
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "engine"))

from cryptography.hazmat.primitives import serialization  # noqa: E402
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey  # noqa: E402

from localbiseo.core.license import sign  # noqa: E402

PUBKEY_OUT = Path(__file__).resolve().parent.parent / "engine" / "localbiseo" / "core" / "license_pubkey.pem"


def load_private(path: str) -> Ed25519PrivateKey:
    return serialization.load_pem_private_key(Path(path).expanduser().read_bytes(), password=None)


def cmd_keygen(args) -> None:
    priv_path = Path(args.private).expanduser()
    if priv_path.exists() and not args.force:
        sys.exit(f"이미 있어요: {priv_path} (덮어쓰려면 --force)")
    priv_path.parent.mkdir(parents=True, exist_ok=True)
    key = Ed25519PrivateKey.generate()
    priv_path.write_bytes(
        key.private_bytes(serialization.Encoding.PEM, serialization.PrivateFormat.PKCS8, serialization.NoEncryption())
    )
    priv_path.chmod(0o600)
    PUBKEY_OUT.write_bytes(
        key.public_key().public_bytes(serialization.Encoding.PEM, serialization.PublicFormat.SubjectPublicKeyInfo)
    )
    print(f"개인키: {priv_path}\n공개키(앱에 포함): {PUBKEY_OUT}")


def make_payload(lic_id: str, name: str, edition: str, exp: str | None) -> dict:
    payload = {"id": lic_id, "name": name, "ed": edition, "iat": date.today().isoformat()}
    if exp:
        payload["exp"] = exp
    return payload


def cmd_issue(args) -> None:
    print(sign(make_payload(args.id, args.name, args.edition, args.exp), load_private(args.private)))


def cmd_batch(args) -> None:
    key = load_private(args.private)
```

```
    with open(args.input, newline="", encoding="utf-8-sig") as f, open(args.output, "w", newline="", encoding="utf-8-sig") as out:
        reader = csv.DictReader(f)
        writer = csv.DictWriter(out, fieldnames=[*(reader.fieldnames or []), "license_key"])
        writer.writeheader()
        for row in reader:
            row["license_key"] = sign(make_payload(row["id"], row.get("name", ""), args.edition, args.exp), key)
            writer.writerow(row)
    print(f"완료: {args.output}")


def main() -> None:
    ap = argparse.ArgumentParser(description="로컬비서 라이선스 키 발급")
    sub = ap.add_subparsers(required=True)
    k = sub.add_parser("keygen")
    k.add_argument("--private", required=True)
    k.add_argument("--force", action="store_true")
    k.set_defaults(fn=cmd_keygen)
    for name, fn in (("issue", cmd_issue), ("batch", cmd_batch)):
        p = sub.add_parser(name)
        p.add_argument("--private", required=True)
        p.add_argument("--edition", default="pro")
        p.add_argument("--exp", help="만료일 YYYY-MM-DD (없으면 평생)")
        if name == "issue":
            p.add_argument("--id", required=True)
            p.add_argument("--name", default="")
        else:
            p.add_argument("input")
            p.add_argument("output")
        p.set_defaults(fn=fn)
    args = ap.parse_args()
    args.fn(args)


if __name__ == "__main__":
    main()
```

```
"""설치 파일에 넣을 오픈소스 고지문(THIRD_PARTY_NOTICES.txt)을 만든다.

  python tools/notices.py            # 엔진을 묶은 그 파이썬 환경에서 실행
결과: app/src-tauri/resources/THIRD_PARTY_NOTICES.txt

- Python: 지금 환경에 설치된 패키지 (엔진에 같이 묶이는 것들)
- JavaScript: app의 배포 의존성 (npm ls --omit=dev)
- Rust: 데스크톱 셸 의존성 (cargo metadata)
- 함께 넣는 외부 도구: FFmpeg, Tesseract, 글꼴
"""
from __future__ import annotations

import json
import subprocess
import sys
from importlib import metadata
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
APP = ROOT / "app"
TAURI = APP / "src-tauri"
OUT = TAURI / "resources" / "THIRD_PARTY_NOTICES.txt"
LICENSE_NAMES = ("LICENSE", "LICENCE", "COPYING", "NOTICE", "AUTHORS")
SKIP_PY = {"pip", "setuptools", "wheel", "pyinstaller", "pyinstaller-hooks-contrib", "pytest", "localbiseo-engine", "altgraph", "pefile", "mach

BUNDLED_TOOLS = """\
FFmpeg (Windows 설치 파일에 포함)
  라이선스: GNU LGPL 2.1 이상 (LGPL 빌드, https://github.com/BtbN/FFmpeg-Builds)
  소스 코드: https://ffmpeg.org/download.html
  FFmpeg는 별도 실행 파일로 들어 있으며, 같은 버전의 다른 LGPL 빌드로 바꿔 쓸 수 있습니다.

Tesseract OCR과 한국어 학습 데이터 (Windows 설치 파일에 포함)
  라이선스: Apache License 2.0
  소스 코드: https://github.com/tesseract-ocr/tesseract , https://github.com/tesseract-ocr/tessdata_fast

나눔고딕 (워터마크 글꼴)
  라이선스: SIL Open Font License 1.1 (https://help.naver.com/service/30016/contents/18088)

처음 사용할 때 내려받는 AI 모델 (설치 파일에는 들어 있지 않음)
  Whisper (faster-whisper 변환본, Systran): MIT License
  U²-Net (rembg 배경 제거): Apache License 2.0
"""


def license_texts(paths: list[Path], limit: int = 40_000) -> str:
    chunks = []
    for p in paths:
        try:
            text = p.read_text(encoding="utf-8", errors="replace").strip()
        except OSError:
            continue
        if text and text not in chunks:
            chunks.append(text[:limit])
    return "\n\n".join(chunks)


def python_section() -> list[str]:
    out = []
    seen = set()
    for dist in sorted(metadata.distributions(), key=lambda d: (d.metadata["Name"] or "").lower()):
        name = dist.metadata["Name"] or ""
        if not name or name.lower() in seen or name.lower() in SKIP_PY:
```

```
            continue
        seen.add(name.lower())
        lic = dist.metadata.get("License-Expression") or dist.metadata.get("License") or ""
        if len(lic) > 120 or not lic:
            classifiers = [c.split("::")[-1].strip() for c in dist.metadata.get_all("Classifier") or [] if c.startswith("License ::")]
            lic = ", ".join(classifiers) or lic[:120]
        files = [Path(dist.locate_file(f)) for f in (dist.files or []) if Path(str(f)).name.upper().startswith(LICENSE_NAMES)]
        out.append(f"{name} {dist.version}\n  라이선스: {lic or '패키지 정보 참고'}\n\n{license_texts(files)}")
    return out


def npm_section() -> list[str]:
    try:
        raw = subprocess.run(["npm", "ls", "--omit=dev", "--all", "--json"], cwd=APP, capture_output=True, text=True, shell=sys.platform == "wi
        tree = json.loads(raw or "{}")
    except (OSError, json.JSONDecodeError):
        return []
    found: dict[str, str] = {}

    def walk(deps: dict) -> None:
        for name, info in (deps or {}).items():
            key = f"{name}@{info.get('version', '')}"
            if key not in found:
                found[key] = name
                walk(info.get("dependencies", {}))

    walk(tree.get("dependencies", {}))
    out = []
    for key, name in sorted(found.items()):
        pkg_dir = APP / "node_modules" / name
        try:
            pkg = json.loads((pkg_dir / "package.json").read_text(encoding="utf-8"))
        except (OSError, json.JSONDecodeError):
            continue
        files = [p for p in pkg_dir.iterdir() if p.is_file() and p.name.upper().startswith(LICENSE_NAMES)]
        out.append(f"{key}\n  라이선스: {pkg.get('license', '패키지 정보 참고')}\n\n{license_texts(files)}")
    return out


def cargo_section() -> list[str]:
    try:
        raw = subprocess.run(["cargo", "metadata", "--format-version", "1", "--locked"], cwd=TAURI, capture_output=True, text=True).stdout
        meta = json.loads(raw)
    except (OSError, json.JSONDecodeError):
        return []
    resolved = {n["id"] for n in (meta.get("resolve") or {}).get("nodes", [])}
    out = []
    for pkg in sorted(meta["packages"], key=lambda p: p["name"]):
        if pkg["id"] not in resolved or pkg.get("source") is None:
            continue
        d = Path(pkg["manifest_path"]).parent
        files = [p for p in d.iterdir() if p.is_file() and p.name.upper().startswith(LICENSE_NAMES)]
        # 같은 라이선스 본문이 수백 번 반복되지 않도록 Rust 쪽은 첫 파일만 넣는다.
        out.append(f"{pkg['name']} {pkg['version']}\n  라이선스: {pkg.get('license') or pkg.get('license_file') or '패키지 정보 참고'}\n\n{license_t
    return out


def main() -> None:
    sections = [
        ("함께 들어 있는 외부 도구와 글꼴", [BUNDLED_TOOLS]),
        ("엔진 (Python 패키지)", python_section()),
        ("화면 (JavaScript 패키지)", npm_section()),
```

```
        ("데스크톱 셸 (Rust 패키지)", cargo_section()),
    ]
    rule = "=" * 78
    parts = [
        "로컬비서 오픈소스 고지 (THIRD PARTY NOTICES)",
        "",
        "로컬비서는 아래 오픈소스 소프트웨어를 사용합니다. 각 소프트웨어의 저작권은 원저작자에게 있으며,",
        "각자의 라이선스 조건에 따라 사용합니다.",
        "",
    ]
    for title, items in sections:
        parts += [rule, f"■ {title} ({len(items)}개)", rule, ""]
        for item in items:
            parts += [item.strip(), "", "-" * 78, ""]
    OUT.parent.mkdir(parents=True, exist_ok=True)
    OUT.write_text("\n".join(parts), encoding="utf-8")
    counts = ", ".join(f"{t}: {len(i)}" for t, i in sections)
    print(f"{OUT} ({OUT.stat().st_size // 1024} KB) — {counts}")


if __name__ == "__main__":
    main()
```